台達電子工業股份有限公司

PCI-DMC-A01
PCI-DMC-B01
PCI-DMC-F01

高速通訊型 PCI 介面 12/6 軸運動控制卡

程式開發手冊

版本:15.4.17

（此頁有意留為空白）

關於此手冊...

使用者須知

請將本手冊置於安全的地方。

由於可能因產品種類的新增、技術的改善與異動、或是文案數據和表的變更，敝公司將保留本手冊更新的權利，恕不另行通知。

未經台達電子公司書面同意，請勿複印、複製本手冊的任一章節內文。

商標

Windows NT/2000/XP, Visual Studio, Visual C++, Visual BASIC 的註冊商標屬於Microsoft所有。

BCB (Borland C++ Builder) 的註冊商標屬於Borland公司所有。

其它產品名稱均僅作識別之用，其註冊商標均屬該原公司所有。

技術支持與服務

假若您需要技術支援、服務與相關訊息，或者您對於在使用本產品過程中有任何問題，歡迎您連結至我們的網站(http://www.delta.com.tw/industrialautomation)或與我們聯絡。我們期待能夠依照您的需求，提供我們最的支持與服務給您。可以透過以下方式聯絡我們。

ASIA
DELTA ELECTRONICS, INC.
Taoyuan Plant 1
31-1, XINGBANG ROAD,
GUISHAN INDUSTRIAL ZONE,
TAOYUAN COUNTY 33370, TAIWAN, R.O.C.
TEL: 886-3-362-6301
FAX: 886-3-362-7267

NORTH/SOUTH AMERICA
DELTA PRODUCTS CORPORATION (USA)
Raleigh Office
P.O. BOX 12173
5101 DAVIS DRIVE,
RESEARCH TRIANGLE PARK, NC 27709, U.S.A.
TEL: 1-919-767-3813
FAX: 1-919-767-3969

JAPAN
DELTA ELECTRONICS (JAPAN), INC.
Tokyo Office
DELTA SHIBADAIMON BUILDING
2-1-14 SHIBADAIMON, MINATO-KU,
TOKYO, 105-0012, JAPAN
TEL: 81-3-5733-1111
FAX: 81-3-5733-1211

EUROPE
DELTRONICS (THE NETHERLANDS) B.V.
Eindhoven Office
DE WITBOGT 15, 5652 AG EINDHOVEN,
THE NETHERLANDS
TEL: 31-40-259-2850
FAX: 31-40-259-2851

第一章 API 函式庫介紹 ---- 1
1.1 函式庫的使用 ---- 1
1.2 開始編輯新的專案 ---- 1

第二章 命令傳回值與訊息描述 ---- 1
2.1 錯誤傳回值一覽 ---- 1
2.2 錯誤代碼回傳範例 ---- 8

第三章 操作原理 ---- 1
3.1 介面卡初始化 ---- 1
3.2 讀取/寫入驅動器參數 ---- 5
3.3 CANopen 協定的操作 ---- 8
3.4 原點復歸操作運動控制 ---- 10
3.5 轉矩運動控制 ---- 15
3.6 速度運動控制 ---- 20
3.7 單軸運動控制 ---- 25
3.8 雙軸運動控制 ---- 36
3.9 三軸運動控制 ---- 52
3.10 多軸運動控制 ---- 61
3.11 同步運動控制 ---- 66
3.12 Speed Continue ---- 73
3.13 螺旋補間 Helix 應用 Normal_Follow ---- 79
3.14 電子凸輪 ---- 90
3.15 間隙補償 ---- 97
3.16 背隙補償 ---- 104
3.17 軟體極限操作 ---- 111
3.18 遠端擴充 IO 模組 Input 應用(RM32MN、RM64MN)操作 ---- 118
3.19 遠端擴充 IO 模組 Output 應用(RM32NT、RM64NT)操作 ---- 122
3.20 遠端擴充 IO 模組 In-Out 應用(RM32PT、GA+GE16MN/NT)操作 ---- 125
3.21 遠端擴充 IO 模組應用-手輪操作 ---- 129
3.22 遠端擴充脈波介面模組應用-RM04PI Mode 1 操作 ---- 135
3.23 遠端擴充脈波介面模組應用-RM04PI Mode 2 操作 ---- 144
3.24 中斷操作 Interrupt ---- 151
3.25 主控介面卡安全機制操作 Master Security ---- 158
3.26 遠端擴充類比輸入模組應用-RM04AD ---- 165
3.27 遠端擴充類比輸出模組應用-RM04DA ---- 168
3.28 位置比較 Compare ---- 172
3.29 Axis Group ---- 179
3.30 DDA Axes Move ---- 183
3.99 歷程記錄器 ---- 187

第四章　控制 API---------- 1
4.1　資料型別與範疇---------- 1
4.2　函數描述---------- 2

第五章　硬體初始化 API ---------- 1
5.1　_DMC_01_open ---------- 1
5.2　_DMC_01_close---------- 1
5.3　_DMC_01_get_CardNo_seq---------- 2
5.4　_DMC_01_pci_initial ---------- 3
5.5　_DMC_01_get_card_version ---------- 4

第六章　介面卡操作 API ---------- 1
6.1　_DMC_01_initial_bus ---------- 1
6.2　_DMC_01_start_ring ---------- 2
6.3　_DMC_01_get_device_table---------- 3
6.4　_DMC_01_get_node_table---------- 4
6.5　_DMC_01_reset_card ---------- 5
6.6　_DMC_01_check_nodeno ---------- 6
6.7　_DMC_01_get_master_connect_status---------- 7
6.8　_DMC_01_get_mailbox_Error ---------- 8
6.9　_DMC_01_get_mailbox_cnt---------- 9
6.10　_DMC_01_get_dsp_cnt---------- 10
6.11　_DMC_01_set_dio_output ---------- 11
6.12　_DMC_01_get_dio_output ---------- 12
6.13　_DMC_01_get_dio_input ---------- 13
6.14　_DMC_01_get_cycle_time ---------- 14
6.15　_DMC_01_initial_bus2 ---------- 15
6.16　_DMC_01_motion_cnt ---------- 16
6.17　_DMC_01_pdo_Rx_error---------- 17
6.18　_DMC_01_pdo_Tx_error ---------- 18
6.19　_DMC_01_get_motion_error_code ---------- 19
6.20　_DMC_01_get_motion_error_cnt ---------- 20
6.21　_DMC_01_get_DLL_path ---------- 21
6.22　_DMC_01_get_DLL_version ---------- 22
6.23　_DMC_01_get_DLL_path_Single ---------- 23
6.24　_DMC_01_get_DLL_version_Single ---------- 24

第七章　伺服驅動器參數讀取/寫入 API ---------- 1
7.1　_DMC_01_read_servo_parameter ---------- 2
7.2　_DMC_01_write_servo_parameter---------- 3

第八章　使用 SDO 協定 API ---- 1
8.1 CANopen SDO 協定格式 ---- 1
8.2 _DMC_01_check_canopen_lock ---- 4
8.3 _DMC_01_get_canopen_ret ---- 5
8.4 _DMC_01_set_pdo_mode ---- 6
8.5 _DMC_01_send_message ---- 7
8.6 _DMC_01_send_message2 ---- 9
8.7 _DMC_01_send_message3 ---- 10
8.8 _DMC_01_send_message4 ---- 11
8.9 _DMC_01_read_message ---- 12
8.10 _DMC_01_read_message2 ---- 13
8.11 _DMC_01_get_message ---- 15
8.12 _DMC_01_reset_sdo_choke ---- 16
8.13 _DMC_01_get_sdo_retry_history ---- 17

第九章　點對點運動控制使用封包協定 API ---- 1
9.1 _DMC_01_set_sdo_driver_speed_profile ---- 1
9.2 _DMC_01_start_sdo_driver_r_move ---- 2
9.3 _DMC_01_start_sdo_driver_a_move ---- 3
9.4 _DMC_01_start_sdo_driver_new_position_move ---- 4

第十章　原點復歸運動控制使用封包協定 API ---- 1
10.1 _DMC_01_set_home_config ---- 1
10.2 _DMC_01_set_home_move ---- 6
10.3 _DMC_01_set_home_config2 ---- 7
10.4 _DMC_01_set_home_config_offset2 ---- 8
10.5 _DMC_01_set_home_move2 ---- 9
10.6 _DMC_01_escape_home_move ---- 10

第十一章　速度運動控制使用封包協定 API ---- 1
11.1 _DMC_01_set_velocity_mode ---- 1
11.2 _DMC_01_set_velocity ---- 2
11.3 _DMC_01_set_velocity_stop ---- 3
11.4 _DMC_01_get_rpm ---- 4
11.5 _DMC_01_set_velocity_torque_limit ---- 5

第十二章　轉矩運動控制使用封包協定 API ---- 1
12.1 _DMC_01_set_torque_mode ---- 1
12.2 _DMC_01_set_torque ---- 2
12.3 _DMC_01_set_torque_stop ---- 3
12.4 _DMC_01_set_torque_velocity_limit ---- 4

第十三章 使用 PDO 協定 API ---- 1
13.1 _DMC_01_ipo_set_svon ---- 1
13.2 _DMC_01_get_buffer_length ---- 2
13.3 _DMC_01_command_buf_clear ---- 3
13.4 _DMC_01_buf_dwell ---- 4
13.5 _DMC_01_set_group ---- 5
13.6 _DMC_01_set_dmc_mode ---- 6

第十四章 停止運動控制 API ---- 1
14.1 _DMC_01_emg_stop ---- 2
14.2 _DMC_01_sd_stop ---- 3
14.3 _DMC_01_sd_abort ---- 4
14.4 _DMC_01_set_sd_mode ---- 5
14.5 _DMC_01_set_sld_mode ---- 7
14.6 _DMC_01_set_sld_stop ---- 8
14.7 _DMC_01_set_sd_time ---- 9
14.8 _DMC_01_set_scurve_rate ---- 10

第十五章 運動狀態 API ---- 1
15.1 _DMC_01_motion_done ---- 2
15.2 _DMC_01_motion_status ---- 3
15.3 _DMC_01_set_01_ph_inposition ---- 5
15.4 _DMC_01_get_01_ph_inposition ---- 6

第十六章 運動計數值 API ---- 1
16.1 _DMC_01_get_command ---- 1
16.2 _DMC_01_set_command ---- 2
16.3 _DMC_01_get_position ---- 3
16.4 _DMC_01_set_position ---- 4
16.5 _DMC_01_get_target_pos ---- 5
16.6 _DMC_01_get_torque ---- 6
16.7 _DMC_01_get_current_speed ---- 7
16.8 _DMC_01_get_current_speed_rpm ---- 8

第十七章 軟體極限 API ---- 1
17.1 _DMC_01_set_soft_limit ---- 2
17.2 _DMC_01_enable_soft_limit ---- 3
17.3 _DMC_01_disable_soft_limit ---- 4
17.4 _DMC_01_get_soft_limit_status ---- 5

第十八章　單軸運動控制 API ---- 1
18.1 _DMC_01_start_tr_move ---- 1
18.2 _DMC_01_start_sr_move ---- 3
18.3 _DMC_01_start_ta_move ---- 4
18.4 _DMC_01_start_sa_move ---- 5
18.5 _DMC_01_p_change ---- 6
18.6 _DMC_01_v_change ---- 7
18.7 _DMC_01_start_tr_move_2seg ---- 8
18.8 _DMC_01_start_sr_move_2seg ---- 10
18.9 _DMC_01_start_ta_move_2seg ---- 11
18.10 _DMC_01_start_sa_move_2seg ---- 13
18.11 _DMC_01_start_tr_move_2seg2 ---- 14
18.12 _DMC_01_start_sr_move_2seg2 ---- 16
18.13 _DMC_01_start_ta_move_2seg2 ---- 17
18.14 _DMC_01_start_sa_move_2seg2 ---- 18
18.15 _DMC_01_feedrate_overwrite ---- 19
18.16 _DMC_01_start_v3_move ---- 22
18.17 _DMC_01_multi_table_V3_move ---- 24

第十九章　二軸線性補間運動控制 API ---- 1
19.1 _DMC_01_start_tr_move_xy ---- 2
19.2 _DMC_01_start_sr_move_xy ---- 4
19.3 _DMC_01_start_ta_move_xy ---- 5
19.4 _DMC_01_start_sa_move_xy ---- 6
19.5 _DMC_01_start_v3_move_xy ---- 7

第二十章　二軸圓弧補間運動控制 API ---- 1
20.1 _DMC_01_start_tr_arc_xy ---- 3
20.2 _DMC_01_start_sr_arc_xy ---- 5
20.3 _DMC_01_start_ta_arc_xy ---- 6
20.4 _DMC_01_start_sa_arc_xy ---- 7
20.5 _DMC_01_start_tr_arc2_xy ---- 8
20.6 _DMC_01_start_sr_arc2_xy ---- 10
20.7 _DMC_01_start_ta_arc2_xy ---- 11
20.8 _DMC_01_start_sa_arc2_xy ---- 12
20.9 _DMC_01_start_tr_arc3_xy ---- 13
20.10 _DMC_01_start_sr_arc3_xy ---- 15
20.11 _DMC_01_start_ta_arc3_xy ---- 17
20.12 _DMC_01_start_sa_arc3_xy ---- 19
20.13 _DMC_01_start_spiral_xy ---- 21
20.14 _DMC_01_start_spiral2_xy ---- 23

20.15 _DMC_01_start_v3_arc_xy ---- 25
20.16 _DMC_01_start_v3_arc2_xy ---- 27
20.17 _DMC_01_start_v3_arc3_xy ---- 29
20.18 _DMC_01_start_v3_spiral_xy ---- 31
20.19 _DMC_01_start_v3_spiral2_xy ---- 33
20.20 _DMC_01_start_v3_ellipse_xy ---- 35

第二十一章 三軸線性補間運動控制 API ---- 1
21.1 _DMC_01_start_tr_move_xyz ---- 1
21.2 _DMC_01_start_sr_move_xyz ---- 2
21.3 _DMC_01_start_ta_move_xyz ---- 3
21.4 _DMC_01_start_sa_move_xyz ---- 4
21.5 _DMC_01_start_v3_move_xyz ---- 5

第二十二章 三軸螺旋補間運動控制 API ---- 1
22.1 _DMC_01_start_tr_heli_xy ---- 1
22.2 _DMC_01_start_sr_heli_xy ---- 3
22.3 _DMC_01_start_ta_heli_xy ---- 5
22.4 _DMC_01_start_sa_heli_xy ---- 7
22.5 _DMC_01_start_v3_heli_xy ---- 9
22.6 _DMC_01_start_v3_sphere_xyz ---- 11

第二十三章 速度運動控制 API ---- 1
23.1 _DMC_01_tv_move ---- 2
23.2 _DMC_01_sv_move ---- 4

第二十四章 同步運動控制 API ---- 1
24.1 _DMC_01_sync_move ---- 2
24.2 _DMC_01_sync_move_config ---- 3

第二十五章 遠端模組控制 API ---- 1
25.1 _DMC_01_get_rm_input_value ---- 1
25.2 _DMC_01_set_rm_input_filter ---- 2
25.3 _DMC_01_set_rm_input_filter_enable ---- 4
25.4 _DMC_01_set_rm_output_value ---- 5
25.5 _DMC_01_set_rm_output_value_error_handle ---- 6
25.6 _DMC_01_get_rm_output_value ---- 7
25.7 _DMC_01_get_rm_output_value_error_handle ---- 8
25.8 _DMC_01_set_rm_output_active ---- 9

第二十六章　電子手輪與 JOG 操作 API---------- 1
26.1 _DMC_01_set_rm_mpg_axes_enable ---------- 2
26.2 _DMC_01_set_rm_mpg_axes_enable2---------- 4
26.3 _DMC_01_set_rm_jog_axes_enable ---------- 7

第二十七章　四組脈波介面專用 API ---------- 1
27.1 _DMC_01_set_rm_04pi_ipulser_mode ---------- 1
27.2 _DMC_01_set_rm_04pi_opulser_mode ---------- 2
27.3 _DMC_01_set_rm_04pi_svon_polarity---------- 3
27.4 _DMC_01_set_rm_04pi_DO2 ---------- 4
27.5 _DMC_01_set_rm_04pi_homing_ratio ---------- 5
27.6 _DMC_01_rm_04PI_get_buffer ---------- 6
27.7 _DMC_01_set_rm_04pi_md2_feedback_mode---------- 7
27.8 _DMC_01_set_rm_04pi_home_config_Tdec ---------- 8
27.9 _DMC_01_set_rm_04pi_Home_counter_Delay_time ---------- 9

第二十八章　四組脈波介面（模式 1）運動控制 API---------- 1
28.1 _DMC_01_rm_04pi_md1_start_move ---------- 1
28.2 _DMC_01_rm_04pi_md1_v_move ---------- 3
28.3 _DMC_01_rm_04pi_md1_start_line2 ---------- 4
28.4 _DMC_01_rm_04pi_md1_start_line3 ---------- 6
28.5 _DMC_01_rm_04pi_md1_start_line4 ---------- 8
28.6 _DMC_01_rm_04pi_md1_start_arc ---------- 10
28.7 _DMC_01_rm_04pi_md1_start_arc2 ---------- 12
28.8 _DMC_01_rm_04pi_md1_start_arc3 ---------- 14
28.9 _DMC_01_rm_04pi_md1_start_heli---------- 16
28.10 _DMC_01_rm_04pi_md1_p_change---------- 18
28.11 _DMC_01_rm_04pi_md1_v_change ---------- 19
28.12 _DMC_01_rm_04pi_md1_set_gear ---------- 20
28.13 _DMC_01_rm_04pi_md1_set_soft_limit---------- 21
28.14 _DMC_01_rm_04pi_md1_get_soft_limit_status---------- 22
28.15 _DMC_01_rm_04pi_md1_set_sld---------- 24
28.16 _DMC_01_rm_04pi_md1_get_mc_error_code ---------- 25
28.17 _DMC_01_set_rm_04pi_ref_counter---------- 26

第二十九章　四組類比輸出遠端擴充模組 API ---------- 1
29.1 _DMC_01_rm_04da_set_output_value ---------- 1
29.2 _DMC_01_rm_04da_get_output_value ---------- 2
29.3 _DMC_01_rm_04da_get_return_code ---------- 3
29.4 _DMC_01_rm_04da_set_output_range ---------- 6
29.5 _DMC_01_rm_04da_set_output_enable ---------- 7

29.6 _DMC_01_rm_04da_set_output_overrange ---- 8
29.7 _DMC_01_rm_04da_set_output_error_clear ---- 9
29.8 _DMC_01_rm_04da_set_output_error_handle ---- 10
29.9 _DMC_01_rm_04da_set_output_offset_value ---- 11
29.10 _DMC_01_rm_04da_get_output_offset_value ---- 13
29.11 _DMC_01_rm_04da_read_data ---- 15

第三十章 四組類比輸入遠端擴充模組 API ---- 1
30.1 _DMC_01_set_04ad_input_range ---- 1
30.2 _DMC_01_get_04ad_input_range ---- 2
30.3 _DMC_01_set_04ad_conversion_time ---- 3
30.4 _DMC_01_get_04ad_conversion_time ---- 4
30.5 _DMC_01_get_04ad_data ---- 5
30.6 _DMC_01_set_04ad_average_mode ---- 6
30.7 _DMC_01_get_04ad_average_mode ---- 7
30.8 _DMC_01_set_04ad_input_enable ---- 8

第三十一章 Slave 資訊 API ---- 1
31.1 _DMC_01_get_devicetype ---- 2
31.2 _DMC_01_get_slave_version ---- 4
31.3 _DMC_01_get_slave_subversion ---- 5

第三十二章 監控參數 API ---- 1
32.1 _DMC_01_set_monitor ---- 2
32.2 _DMC_01_get_monitor ---- 4
32.3 _DMC_01_get_servo_command ---- 6
32.4 _DMC_01_get_servo_DI ---- 7
32.5 _DMC_01_get_servo_DO ---- 8

第三十三章 警報訊息 API ---- 1
33.1 _DMC_01_set_ralm ---- 1
33.2 _DMC_01_get_alm_code ---- 2
33.3 _DMC_01_master_alm_code ---- 3
33.4 _DMC_01_slave_error ---- 4
33.5 _DMC_01_set_ralm_servo_all ---- 5
33.6 _DMC_01_set_limit_auto_ralm ---- 6

第三十四章 多軸運動控制 API ---- 1
34.1 _DMC_01_multi_axes_move ---- 1
34.2 _DMC_01_liner_speed_master ---- 3

34.3 _DMC_01_start_v3_multi_axes ---- 5
34.4 _DMC_01_start_v3_msbrline_move ---- 7

第三十五章 緩衝區操作 API ---- 1
35.1 _DMC_01_set_trigger_buf_function ---- 2

第三十六章 Interrupt API ---- 1
36.1 _DMC_01_set_int_factor ---- 2
36.2 _DMC_01_int_enable ---- 3
36.3 _DMC_01_int_disable ---- 4
36.4 _DMC_01_get_int_count ---- 5
36.5 _DMC_01_get_int_status ---- 6
36.6 _DMC_01_link_interrupt ---- 7

第三十七章 Security API ---- 1
37.1 _DMC_01_read_security ---- 2
37.2 _DMC_01_read_security_status ---- 3
37.3 _DMC_01_write_security ---- 4
37.4 _DMC_01_write_security_status ---- 5
37.5 _DMC_01_check_userpassword ---- 6
37.6 _DMC_01_write_userpassword ---- 7
37.7 _DMC_01_check_verifykey ---- 8
37.8 _DMC_01_write_verifykey ---- 9
37.9 _DMC_01_read_serialno ---- 10
37.10 _misc_slave_check_userpassword ---- 11
37.11 _misc_slave_write_userpassword ---- 12
37.12 _misc_slave_get_serialno ---- 13
37.13 _misc_security ---- 14
37.14 _misc_slave_write_verifykey ---- 15
37.15 _misc_slave_check_verifykey ---- 16
37.16 _misc_slave_user_data_buffer_read ---- 17
37.17 _misc_slave_user_data_buffer_write ---- 18
37.18 _misc_slave_user_data_to_flash ---- 19

第三十八章 極限反向 API ---- 1
38.1 _DMC_01_rm_04pi_set_MEL_polarity ---- 2
38.2 _DMC_01_rm_04pi_get_MEL_polarity ---- 3
38.3 _DMC_01_rm_04pi_set_PEL_polarity ---- 4
38.4 _DMC_01_rm_04pi_get_PEL_polarity ---- 5
38.5 _DMC_01_set_01ph_alm_polarity ---- 6

38.6 _DMC_01_set_01ph_org_polarity ---- 7

第三十九章 Compare API ---- 1
39.1 _DMC_01_set_compare_channel_position ---- 2
39.2 _DMC_01_get_compare_channel_position ---- 3
39.3 _DMC_01_set_compare_ipulser_mode ---- 4
39.4 _DMC_01_set_compare_channel_direction ---- 5
39.5 _DMC_01_set_compare_channel_trigger_time ---- 6
39.6 _DMC_01_set_compare_channel_one_shot ---- 7
39.7 _DMC_01_set_compare_channel_source ---- 8
39.8 _DMC_01_channel0_position_cmp ---- 9
39.9 _DMC_01_channel1_output_enable ---- 10
39.10 _DMC_01_channel1_output_mode ---- 11
39.11 _DMC_01_channel1_get_io_status ---- 13
39.12 _DMC_01_channel1_set_gpio_out ---- 14
39.13 _DMC_01_channel1_position_compare_table ---- 15
39.14 _DMC_01_channel1_position_compare_table_level ---- 16
39.15 _DMC_01_channel1_position_compare_table_cnt ---- 17
39.16 _DMC_01_set_compare_channel_polarity ---- 18
39.17 _DMC_01_channel0_position_cmp_by_gpio ---- 19
39.18 _DMC_01_channel1_position_re_compare_table ---- 20
39.19 _DMC_01_channel1_position_re_compare_table_level ---- 21
39.20 _DMC_01_set_compare_channel_enable ---- 22
39.21 _DMC_01_channel1_get_fifo_status ---- 23
39.22 _DMC_01_channel1_get_fifo_counter ---- 24

第四十章 連續速度 API ---- 1
40.1 _DMC_01_speed_continue ---- 2
40.2 _DMC_01_speed_continue_mode ---- 3
40.3 _DMC_01_speed_continue_combine_ratio ---- 5

第四十一章 特殊運用 API ---- 1
41.1 _DMC_01_enable_electcam ---- 2
41.2 _DMC_01_set_cam_enable ---- 4
41.3 _DMC_01_set_cam_pitch ---- 5
41.4 _DMC_01_set_cam_org ---- 6
41.5 _DMC_01_set_cam_mode ---- 7
41.6 _DMC_01_set_cam_table ---- 8
41.7 _DMC_01_set_cam_table2 ---- 10
41.8 _DMC_01_set_backlash ---- 12
41.9 _DMC_01_set_backlash_info ---- 13

41.10 _DMC_01_set_ecam_repeat ---- 15

第四十二章 其它 API ---- 1
42.1 _misc_app_get_circle_endpoint ---- 3
42.2 _misc_app_get_circle_center_point ---- 4
42.3 _misc_set_record_debuging ---- 5
42.4 _misc_open_record_debuging_file ---- 6
42.5 _DMC_01_enable_dda_mode ---- 7
42.6 _DMC_01_set_dda_data ---- 8
42.7 _DMC_01_get_dda_cnt ---- 9
42.8 _DMC_01_clear_dda_data ---- 10
42.9 _DMC_01_save_device_info ---- 11
42.10 _DMC_01_compare_device_info ---- 12
42.11 _DMC_01_enable_record_mode ---- 13
42.12 _DMC_01_get_record_data ---- 14
42.13 _DMC_01_set_mc_fifo_mode ---- 15
42.14 _DMC_01_set_function_time ---- 16
42.15 _DMC_01_get_fifo_cnt ---- 17
42.16 _DMC_01_set_gear ---- 18
42.17 _DMC_01_get_gear ---- 19
42.18 _DMC_01_set_position_ref_source ---- 20
42.19 _DMC_01_get_position_ref_source ---- 21
42.20 _DMC_01_set_virtual_command ---- 22
42.21 _DMC_01_get_virtual_command ---- 23
42.22 _DMC_01_set_virtual_position ---- 24
42.23 _DMC_01_disable_virtual_pos ---- 25
42.24 _DMC_01_set_stepping_move_rate ---- 26
42.25 _DMC_01_MRAM_read_word ---- 27
42.26 _DMC_01_MRAM_write_word ---- 28
42.27 _DMC_01_MRAM_read_dword ---- 29
42.28 _DMC_01_MRAM_write_dword ---- 30
42.29 _DMC_01_get_watchdog_cnt ---- 31
42.30 _DMC_01_set_watchdog_cnt ---- 32
42.31 _DMC_01_get_watchdog_value ---- 33
42.32 _DMC_01_set_watchdog_value ---- 34

第一章 API 函式庫介紹

PCI-DMC-A01/B01/F01 提供可操作的函式庫與動態連結(DLL). 您可以藉由呼叫這些函式庫來完成您所需完成的各項功能。以下各小節將告訴您如何將這些函式庫導入您的開發環境。

1.1 函式庫的使用

當您將安裝程式安裝完成，您能在「lib」的資料夾中找到 4 個函式庫，該函式庫分別是給 Visual Studio C 與 Borland 開發環境下使用。

表 1.1

PCI_DMC_01.lib	Visual Studio C 函式庫(A01/B01 專用)
BCBPCI_DMC_01.lib	Borland C 函式庫(A01/B01 專用)
PCI_DMC.lib	Visual Studio C 函式庫(所有卡通用)
BCBPCI_DMC.lib	Borland C 函式庫(所有卡通用)

1.2 開始編輯新的專案

1.2.1 使用 VC

1. 在您的專案中加入以下指令:
   # include "..\inc\VC\PCI_DMC.h"
   # include "..\inc\VC\PCI_DMC_Err.h"
2. 在 Visual C 開發環境下選擇 Project / Setting / Link
   在 Object / Library modules 中鍵入"..\lib\PCI_DMC.lib"
3. 設置完成，你可以開始使用 API 來操作 PCI-DMC 軸卡。

1.2.2 使用 Borland C

1. 在您的專案中加入以下指令:

   # include "..\inc\BCB \PCI_DMC.h"

   # include "..\inc\BCB\PCI_DMC_Err.h"
2. 在 Borland C++ Build 的開發環境下選擇 View/ Project Manger

   並在新專案中加入"..\lib\BCBPCI_DMC.lib"此函式庫在您的新專案中。
3. 設置完成，你可以開始使用 API 來操作 PCI-DMC 軸卡。

1.2.3 使用 VB

在安裝目錄"..\inc\VB"中，您可將"PCI_DMC.bas"與 "PCI_DMC_Err.bas"兩個檔案加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

1.2.4 使用 Delphi

在安裝目錄"..\ inc\Delphi" 中，您可將"PCI_DMC.pas" 加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

1.2.5 使用 VB.Net

在安裝目錄"..\inc\VB.Net" 中，您可將"PCI_DMC.vb"與 "PCI_DMC_ERR.vb"兩個檔案加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

1.2.6 使用 C#

在安裝目錄"..\ inc\C#" 中，您可將"PCI_DMC.cs"與 "PCI_DMC_ERR.cs"兩個檔案加入您創建的專案中，您就可以立即使用 API 來操作 PCI-DMC 軸卡了。

第二章　命令傳回值與訊息描述

2.1　錯誤傳回值一覽

當您在使用 PCI-DMC 的 API 函式庫時，API 函式多半會回傳一個錯誤代碼，如表 2.1。

若該 API 函式傳回的值為 0 時，這表示此 API 函式是成功地被執行；反之，若此 API 函式回傳其它的錯誤代碼，這表示可能在操作程序或者是硬體連結方面產生問題。請依該錯誤代碼的描述進行問題排解。

表 2.1

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_A/B01 錯誤代碼表		
0	ERR_NoError	API 成功運作
3	ERR_CardNoError	卡號錯誤請確認卡片上的 DIP Switch 調整的號碼
5	ERR_bootmodeErr	無法啟動 DSP 數位程序
6	ERR_downloadcode	DSP 記憶體程序讀/寫錯誤
7	ERR_downloadinit	DSP 記憶體資料讀/寫錯誤
8	ERR_PCI_boot_first	“_DMC_01_pci_initial” API 函式必須先運作
9	ERR_FeedRate_updata	FeedRate overwrite 的值相同
11	ERR_AxisNoError	軸數目設置錯誤(過大)
12	ERR_IPO_First	需要為 IPO 模式
13	ERR_Target_reach	Mode 1 運行時,需 Target 到達
14	ERR_Servo_on_first	需要先 servo on
15	ERR_MPG_Mode	在手輪模式下無法清除位置
16	ERR_PDO_TG	使用 PDO 模式下指令給模組,無法回傳收到
17	ERR_ConfigFileOpenError	創建存放除錯資訊的檔案錯誤
18	ERR_Ctrl_value	使用控制指令錯誤
19	ERR_Security_Fifo	使用 Security Fpga write Error
20	ERR_Security_Fifo_busy	使用 Security Fpga busy
21	ERR_SpeedLimitError	設置的速度大於最大速度
22	ERR_Security_Page	使用 Security page 需要小於 16
23	ERR_Slave_Security_op	使用 Security slave_operate 指令無效
24	ERR_channel_no	channel no 錯誤
25	ERR_start_ring_first	“_DMC_01_start_ring” API 函式必須先運作

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
26	ERR_NodeIDError	NodeID 不存在
27	ERR_MailBoxErr	DSP 忙碌，指令無法下達
28	ERR_SdoData	SDO 資料有送出但無回應
29	ERR_IOCTL	作業系統無法處理此 IRP
30	ERR_SdoSvonFirst	使用 SDO 操作軸控時,需先 Servo On
31	ERR_SlotIDError	Slave 模組(GA or RM)無此 Slot 號碼
32	ERR_PDO_First	使用 PDO 協定操作，必須先啟動 PDO 模式
33	ERR_Protocal_build	Protocal,電氣尚未建立
34	ERR_Maching_TimeOut	模組配對 time Out
35	ERR_Maching_NG	模組配對 NG
36	ERR_Group_Num	設定群組最大值為 6 組
37	ERR_Master_Alarm	故障發生(通訊不良,Driver Alm)
38	ERR_Alarm_reset	Reset Alarm Fail
40	ERR_Master_Security_Wr	使用 Security Master Write 指令無效
41	ERR_Master_Security_Rd	使用 Security Master Read 指令無效
42	ERR_Master_Security_Pw	需要先輸入正確的 password
50	ERR_NonSupport_CardVer	Master Card 的版本錯誤,請連繫代理商,購買正確 Master Card
51	ERR_Compare_Source	Ver Type : B Compare Source 選擇錯誤
52	ERR_Compare_Direction	Compare 的方向錯誤　dir 需為 1 或 0 1:ccw,0:cw
60	ERR_GetDLLPath	Dll 路徑錯誤
61	ERR_GetDLLVersion	擷取 Dll 版本資料發生錯誤
70	ERR_04PISTOP_Timeout	對 04PI 下達 Stop 指令逾時未成功
71	ERR_ServoSTOP_Timeout	對伺服下達 Stop 指令逾時未成功
72	ERR_04PISTOP_status	04PI Stop MC_done not to Error
73	ERR_04PIHoming_err	04PI 於回 Home 模式狀態錯誤
74	ERR_04PISdo_trans	04PI 下達 SDO 指令，但資料回覆錯誤

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
112	ERR_RangeError	設定軸的號碼錯誤
114	ERR_MotionBusy	Motion 的命令重疊
116	ERR_SpeedError	設置最大速度為 0
117	ERR_AccTimeError	加/減速時大於 1000 秒
124	ERR_PitchZero	操作螺旋位移，參數「pitch」的值為 0
127	ERR_BufferFull	Motion 命令緩衝區已滿
128	ERR_PathError	Motion 命令錯誤
130	ERR_NoSupportMode	不支持速度變化
132	ERR_FeedHold_support	Feedhold Stop 啟動，無法接收新的命令
133	ERR_SDStop_On	執行減速停止命令，無法接收新的命令
134	ERR_VelChange_supper	無法執行速度變化功能(Feddhold,同動與減速)
135	ERR_Command_set	無法連續執行 FeedHold 的功能
136	ERR_sdo_message_choke	Sdo 指令回傳有誤，請檢查網路線接線是否接妥
137	ERR_VelChange_buff_feedhold	Feedhold 功能須先啟動，無法速度變化
138	ERR_VelChange_sync_move	正在等待同動命令，無法速度變化
139	ERR_VelChange_SD_On	正在等待減速命令，無法速度變化
140	ERR_P_Change_Mode	單軸點對點模式，加速段,速度等於 0,非單軸點對點模式
141	ERR_BufferLength	當模式在 _Path_p_change,_Path_velocity_change_onfly, _Path_Start_Move_2seg 時 Buffer Length 需要為 0
142	ERR_2segMove_Dist	距離需要同向
143	ERR_CenterMatch	經過反向運算圓心要一致
144	ERR_EndMatch	經過反向運算圓心要一致
145	ERR_AngleCalcu	經過計算角度錯誤
146	ERR_RedCalcu	經過半徑錯誤
147	ERR_GearSetting	Gear 的分子或分母為 0
148	ERR_CamTable	Table Setting First Arrary Point Error 設定 table 不能到負值 table[-1]沒有這種設定
149	ERR_AxesNum	多軸設定值必須要兩軸以上
150	ERR_SpiralPos	最終位置會到達螺旋圓心
151	ERR_SpeedMode_Slave	在速度連續時使用的 Slave 軸,無法執行 Motion 指令
152	ERR_SpeedMode_SlaveSet	設定軸的虛擬必須在軸的前半部
154	ERR_Backlash_step	accstep+contstep+decstep 需小於 100
155	ERR_Backlash_status	設定時 motion done 及 buffer length 必須都為 0

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
201	Compare_Cards_Not_Equal	比對結果ㄧ軸卡資訊(卡號、數量)不符
202	Compare_Nodes_Not_Equal	比對結果ㄧ站號資訊(數量)不符
203	Compare_Node_ID_Not_ Equal	比對結果ㄧ站號資訊(站號)不符
204	Compare_Node_Device_ Type_Not_Equal	比對結果ㄧ模組資訊(模組第二分類)不符
205	Compare_Node_Identity_ Object_Not_Equal	比對結果ㄧ模組資訊(模組第一分類)不符
206	Compare_File_Path_NULL	檔案路徑錯誤
207	Compare_File_Open_Fail	檔案開啟失敗(請確定路徑輸入正確)
208	Compare_File_Not_Exist	檔案不存在

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_F01 錯誤代碼表		
0	ERR_NoError	API 成功運作
301	ERR_NoCardFound	無此卡號或尚未 Initial
302	ERR_CardInitial	卡片 Initial 失敗
303	ERR_MemoryAccess_Failed	記憶體讀寫失敗
304	ERR_MemoryOutOfRange	記憶體使用超過 Range
305	ERR_UartTxIsBusy	Uart Tx Is Busy
306	ERR_UartRxError	Uart Rx 讀取錯誤
307	ERR_UartRxIsNotReady	Uart Rx 尚未準備完成
308	ERR_NotSupportFunc	不支援此 Function
309	ERR_NoNodeFound	Slave 站號設置錯誤
310	ERR_APIInputError	API 參數輸入錯誤(超出限定值)
311	ERR_SDOFailed	SDO 傳送失敗
312	ERR_SDOBusy	SDO 忙碌中或 SDO 同時有兩個指令被寫入
313	ERR_APITypeErr	此模組不支援此 API
314	ERR_ScaleFailed	AD 校正失敗
315	ERR_F_BufferFull	MailBox_Buffer 已滿
316	ERR_ConnectErr	通訊連線異常
317	ERR_MBWordChFailed	MailBox 同時有兩個指令寫入
318	ERR_MailBoxFailed	MailBox 傳送失敗
319	ERR_CantResetCard	無法 ResetCard
320	ERR_PDOFailed	PDO 未回應

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
321	ERR_MBCmding	MailBox 正在處理指令
322	ERR_SVOFF	此動作需 Servo On 方可執行
323	ERR_DriverError	此動作因 DriverErr 無法執行，請先執行 Rest Alrm
324	ERR_ConnReset_Failed	初始化重置失敗
325	ERR_F01SlotIDError	此裝置不支援 Slot 或輸入的 Slot 編號超出範圍
326	ERR_UartData_NoMatch	Download Code 讀取 Uart 的資料時，回傳無法符合正確值
327	ERR_SVON	此動作需先將伺服 Servo Off 方可執行
328	ERR_Mpg_Already_On	此軸已經致能手輪、Jog 或 DDA 功能，其餘功能暫時無法使用
329	ERR_MpgNumber_Over	手輪或 Jog 致能數量已達上限(最大 3 組)
330	ERR_Mpg_Data_Failed	手輪或 Jog 資料傳遞失敗
331	ERR_DDA_Buffer_Full	DDA Buffer 已滿
332	ERR_F_Slave_Security_op	Slave Secutiry 寫入失敗
333	ERR_F_Security_Page	Page 超過設置
334	ERR_F_GetDLLPath	找不到 DLL 路徑
335	ERR_F_GetDLLVersion	找不到 DLL 版本訊息
336	F_Compare_File_Open_Fail	檔案開啟失敗(請確定路徑輸入正確)
337	F_Compare_File_Not_Exist	檔案不存在
338	F_Compare_Cards_Not_Equal	比對結果─軸卡資訊(卡號、數量)不符
339	F_Compare_File_Path_NULL	檔案路徑錯誤
340	F_Compare_Node_ID_Not_Equal	比對結果─站號資訊(站號)不符
341	F_Compare_Node_Device_Type_Not_Equal	比對結果─模組資訊(模組第二分類)不符
342	F_Compare_Node_Identity_Object_Not_Equal	比對結果─模組資訊(模組第一分類)不符
343	F_Compare_Nodes_Not_Equal	比對結果─站號資訊(數量)不符
344	ERR_SecurityNoRet	Security 傳送結果未回傳
345	ERR_SDORetTimeOut	SDO 回傳時間過長
346	ERR_Uart_Connect_Fail	Uart 通訊失敗
399	ERR_UseGetError	系統型錯誤, 需使用 _DMC_01_get_card_error_code 讀取錯誤碼

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_F01 sub code of Error Code=399		
0	ERR_RangeError	此卡號所對應之軸卡沒有發生錯誤
1	ERR_MotionBusy	DMC 通訊連結無法生成
2	ERR_SpeedError	SDO 傳送失敗
3	ERR_AccTimeError	無法重置通訊

錯誤傳回代碼 (十進制)	錯誤碼標示名稱	錯誤問題描述
PCI_DMC_A01/B01/F01 General Error Code		
901	ERR_SameCardNumber	卡號重複(所有軸卡卡號不能相同)
902	ERR_CardType	此軸卡不支援此 API
903	ERR_CantFindDLL	此 API 與其對應的 DLL 連結失敗(沒有 DLL 或是 DLL 函式庫中沒有此函式)
904	ERR_SerchErrorCode	此 API 的錯誤需使用 _DMC_01_get_cardtype_errorcode 來取得

2.2　錯誤代碼回傳範例

下述的範例為一個傳回的函式，您可參考其作用，並對於你的控制需求，創建新的函式來滿足您的需求。

■　範例

```
Void error(unsigned short rc)        // 傳回錯誤代碼的函數，rc 為代入的傳回值
{
        Switch(rc)
        {
          Case 3:
                  printf(“Card No. Error, Please check Card No. again.”);
                   break;
          default:
                  break;
        }
```

第三章　操作原理

3.1　介面卡初始化

3.1.1　函式列表

表 3.1

函式名稱
_DMC_01_open
_DMC_01_get_CardNo_seq
_DMC_01_reset_card
_DMC_01_close
_DMC_01_pci_initial
_DMC_01_initial_bus
_DMC_01_start_ring
_DMC_01_get_device_table
_DMC_01_get_node_table

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.1.2 流程圖

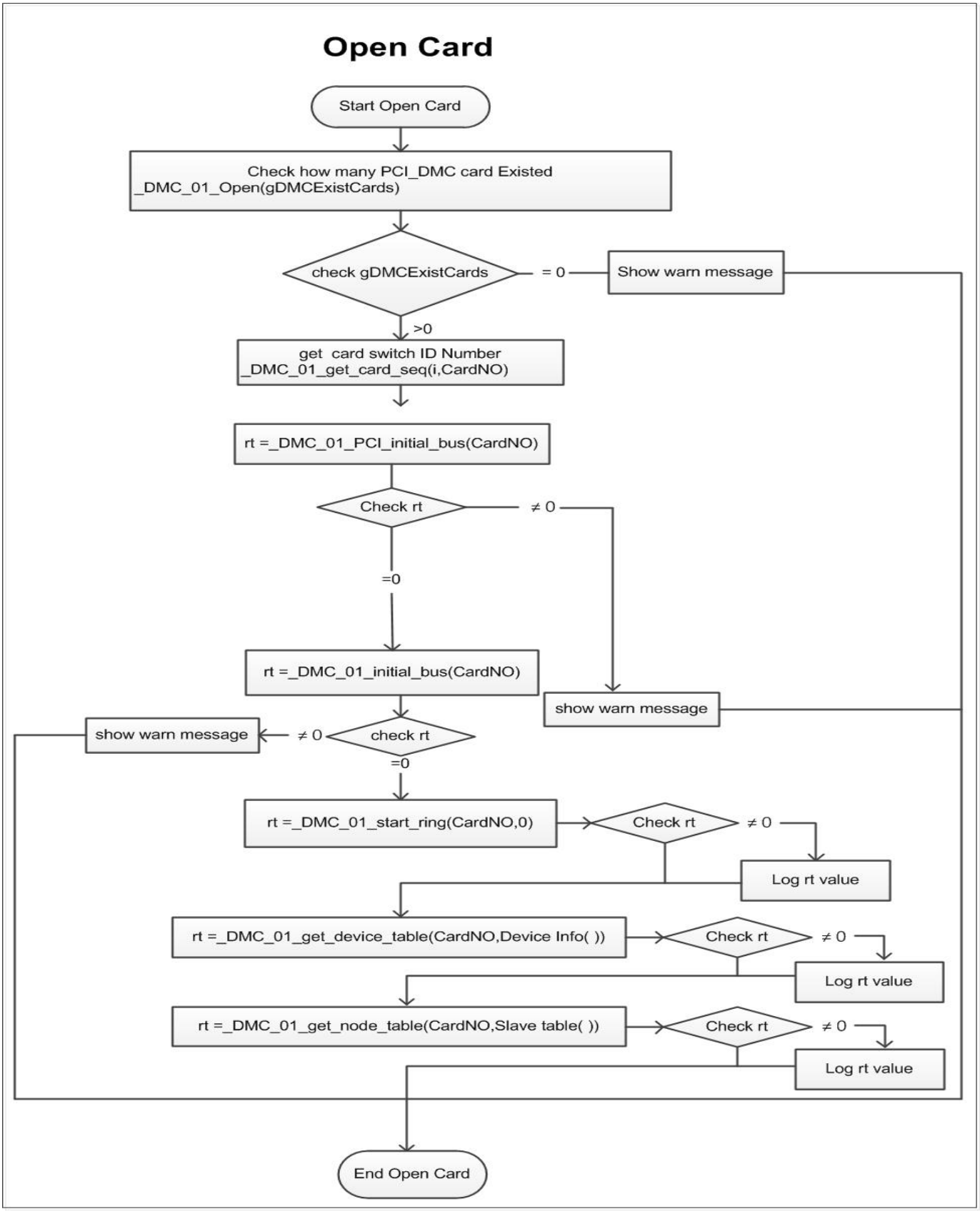


圖 3.010

3.1.3　應用範例

程式外觀

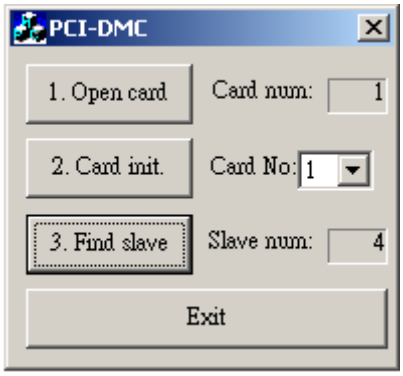


圖 3.011

1) 開啟介面卡



圖 3.012

按壓圖 3.012 中的「Open card」鍵開始執行以下程序：

```
/* gDMCExistCards 變數會被填入 PC 上 PCI-DMC-01 的數量*/
rt = _DMC_01_open(&gDMCExistCards);
```

2) 初始化介面卡



圖 3.013

按壓圖 3.013 中的「Card init」鍵開始執行以下程序：

```
for(i=0; i<gDMCExistCards; i++)
{
/*  取得 PC 上第 i 張介面卡號，卡號即 Switch 指撥開關對應的數值*/
  rt = _DMC_01_get_CardNo_seq(i, &CardNo);
  gpDMCCardNoList[i] = CardNo;

/*進行介面卡初始化  */
   rt = _DMC_01_pci_initial(gpDMCCardNoList[i]);    // 初始化介面卡
   if(rt != 0) AfxMessageBox("Can't boot PCI_DMC Master Card!");
  rt = _DMC_01_initial_bus(gpDMCCardNoList[i]); //  通訊協定初始化
  gbpDSPBoot[gpDMCCardNoList[i]] = true;
}
```

// PCI_DMC_F01 在進行 _DMC_01_initial_bus 時，若 dll 的版本與 IC 內部的版本不同，會進行 DownLoad Code 的動作，時間大約 5 分鐘，DownLoadCode 的過程中要避免關閉 PC 電源等動作發生，若有使用軸卡 Security 功能，還要避免 DownLoadCode
的過程中，更換軸卡或調整卡號的動作(若 DownLoadCode 的過程中，PC 當機或斷電，請原卡原卡號再做一次_DMC_01_initial_bus)。

3) 建立通訊



圖 3.014

按壓圖 3.014 中的「Find slave」鍵開始執行以下程序：

```
rt = _DMC_01_start_ring(gDMCCardNo, 0); //  開始進行通訊
rt = _DMC_01_get_device_table(gDMCCardNo, &gpDeviceInfo[gDMCCardNo]);
rt = _DMC_01_get_node_table(gDMCCardNo, &gpSlaveTable[0][gDMCCardNo]);
```

上述程序執行完成後，顯示找尋到的 Slave 裝置於「Slave num」欄位。

NOTE

_DMC_01_get_device_table → 取得使用 PDO 協定的 Slot ID。
例如：gpDeviceInfo[gDMCCardNo]為一資料型別為 “WORD”的變數，若其值為 7，以二進制表示為” 0000 0000 0000 0111 “，即表示存在的 Slot ID 為”1”, “2”, “3” 的 Slave 裝置。
_DMC_01_get_node_table → 取得使用 SDO 定的 Node ID。
例如：gpSlaveTable[0][gDMCCardNo] 為一資料型別為 “DWORD”的變數，若其值為 7，以二進制表示為” 0000 0000 0000 0000 0000 0000 0000 0111 “，，即表示存在的 Node ID 為”1”, “2”, “3” 的 Slave 裝置。

※您可以使用下列的演算法來找出 SDO 的站號(Node ID)

```
lMask = 0x1;
for(i=0; i<32; i++)
{
/*  第 i 個 bit 值為 1 時,條件式成立.*/
if((gpSlaveTable[0][gDMCCardNo]>>i) & lMask) {
/*  取得的 i 值+1 即為站號(Node ID)，對應驅動器參數「P3-00」  */
     gpNodeID[gNodeNum] = (unsigned short)(i+1);
     gNodeNum++;
   }
}
```

4) 離開程序



圖 3.015

按壓圖 3.015 中的「Exit」鍵開始執行以下程序：

```
for(i=0; i<gDMCExistCards; i++) {
        rt = _DMC_01_reset_card(gpDMCCardNoList[i]);     // 將介面卡重置
}
_DMC_01_close();   // 終止 PCI-DMC-01 運行
```

3.2 讀取/寫入驅動器參數

3.2.1 函式列表

表 3.2

函式名稱
_DMC_01_read_servo_parameter
_DMC_01_write_servo_parameter

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.2.2 應用範例

程式外觀

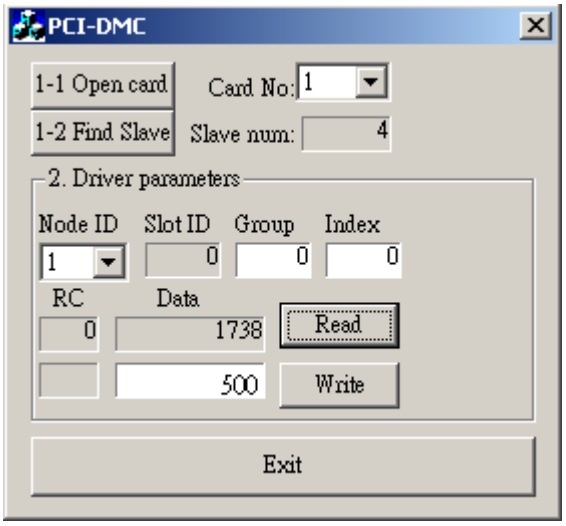


圖 3.020

1) 介面卡開啟與初始化

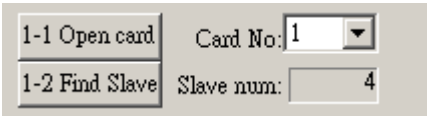


圖 3.021

按壓圖 3.021 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.021 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化詳細說明，請參閱 3.1.3 節「開啟介面卡」到「建立通訊」的函式運用。

2) 填入欲變更的驅動器 (包含驅動器站號、參數群組號,與索引值)



圖 3.022

以例子來說，您可填入以下數值如圖 3.022.

第 1 個欄位“**Node ID**” ：若值填入 1，表示將操作站號為 1 的驅動器。

第 2 個欄位“**Slot ID**”：此欄位無法變更，它會顯示目前 Slave 裝置(驅動器的 Slot ID 值為 0)。

第 3 個欄位”**Group**” ：指該裝置(通常為驅動器)的群組編號，關於該編號功能詳細的說明，請參閱”ASDA-A2 使用手冊。如圖 3.022 群組值設置為“0”，表示將設置”P0-xx”的驅動器參數值(xx 的值請見下方 index 的說明)。

第 4 個欄位”**Index**”：如上述，此數值需與 Group 的值搭配使用。如圖 3.022，index 的值為 0，是故，此圖範例將會對 Node ID 為 1 的驅動器的”P0-00”參數（驅動器版本）進行讀取/寫入。

3) 讀取驅動器參數

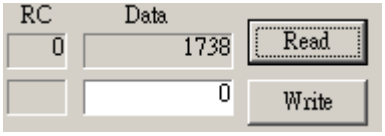


圖 3.023

按壓圖 3.023 中的「Read」鍵開始執行以下程序：

```
rt = _DMC_01_read_servo_parameter(gDMCCardNo, NodeID, SlotID, group, idx, &data);
// 將傳回 data 值，該值即為此驅動器的驅動器參數目前設置的值。
// rt 的值將顯示在「RC」欄位，data 的值將顯示在「Data」欄位。
```

4) 寫入驅動器參數

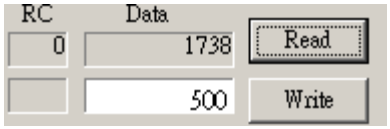


圖 3.024

如圖 3.024，若您欲在驅動器中寫入參數值，您必須在 edit box 的欄位中填入欲寫入的值，接著按壓「Write」鍵開始執行以下程序：

```
rt = _DMC_01_write_servo_parameter(gDMCCardNo, NodeID, SlotID, group, idx, data);
// 該值會寫入您設置的驅動器群組參數中，詳細設置請參閱上述說明。
```

5) 離開程序

按壓圖 3.020 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.3　CANopen 協定的操作

3.3.1　函式列表

表 3.3

函式名稱
_DMC_01_send_message
_DMC_01_read_message
_DMC_01_get_message

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.3.2　應用範例

程式外觀

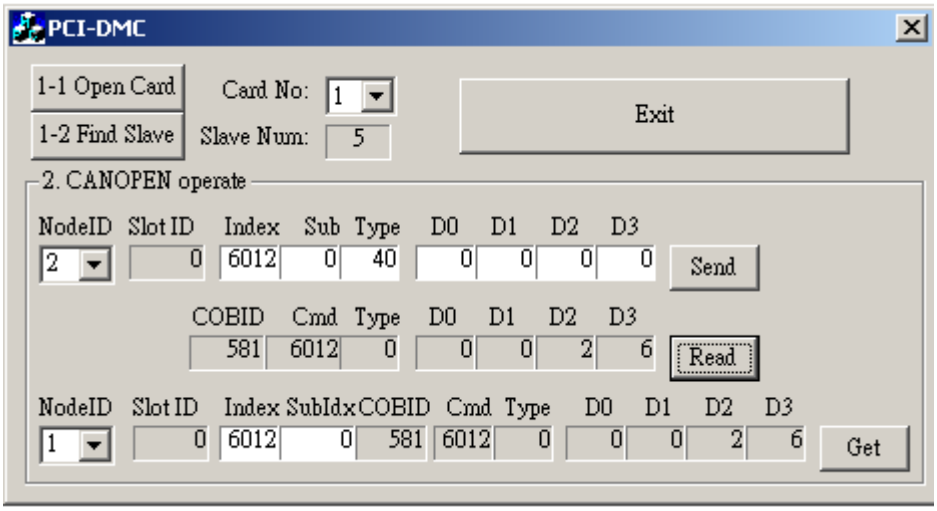


圖 3.030

1) 介面卡開啟與初始化
按壓圖 3.030 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.030 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 送出 SDO 協定的命令

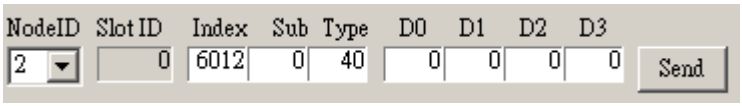


圖 3.031

您必須填入“NodeID”的值，該值會參照到您欲設置的 slave 裝置。關於“Index” 、“Sub”與“Type”設定的值，請參考 CANopen 手冊(DS 402)。“D0”到”D3”的值，這些值是用來鍵入您欲設定的 SDO 命令資料(有效資料：Byte). 當您完成上述資料的填入動作，按壓圖 3.031 中的「Send」鍵即開始執行以下 API 函式：

```
rt = _DMC_01_send_message(gDMCCardNo, NodeID, SlotID, Index, SubIdx, DataType, Value0, Value1, Value2, Value3);
```

3) 讀取 SDO 協定的命令

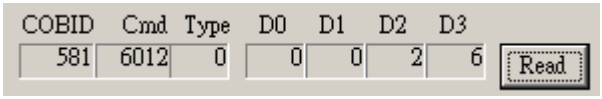


圖 3.032

按壓圖 3.032 中的「Read」鍵開始執行以下程序：

```
/* 取得您送出最近一筆 SDO 命令所傳回的資料 */
rt = _DMC_01_read_message(gDMCCardNo, & COBID, &Cmd , &DataType, &Value0, &Value1, &Value2, &Value3);
```

您可以建立一些變數來存放這些 SDO 命令回傳的資料，關於此傳回資料的詳細說明，請參閱第 8.7 節敘述。

4) 取得 SDO 協定的命令



圖 3.033

您需填入對應的 Node ID、Object Dictionary(OD) index 與 sub-index，以取得您欲知道 CANopen 介面協定的相關資訊。當您完成資料的填入，按壓「Get」鍵執行：

```
Rt = _DMC_01_get_message(gDMCCardNo,NodeID,SlotID,Index,SubIndex, &COBID, &Cmd , &DataType, &Value0, &Value1, &Value2, &Value3);
```

您可以建立一些變數來存放這些 SDO 命令回傳的資料，關於此傳回資料的詳細說明，請參閱第 8.9 節敘述。

5) 離開程序

按壓圖 3.030 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.4　原點復歸操作運動控制

3.4.1　概述

大部分的應用程式利用增量式編碼器(incremental encoder)來取得 position feedback。在運行任何正確地運動控制前，進行原點復歸的操作是必要的。當電源被開啟後，機台上的 position 的狀況將出現 3 種可能， 其一，position 停在一般原復歸設置的位置上等待下一個命令；其二，position 停在 ORG 感測器的位置；其三，position 停留在 ORG 與 Limit Switch(PEL 與 MEL)之間的位置。請參考下方圖 3.040。

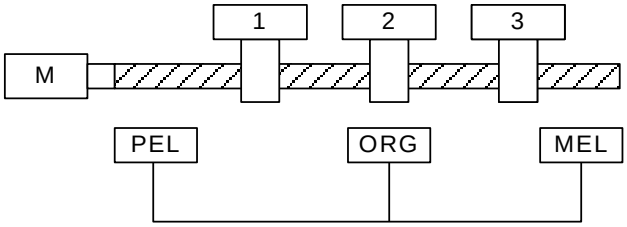


圖 3.040

PCI-DMC 針對上述條件有不同函式的支持。在一般 position 原點復歸的情況下，PCI-DMC 提供多達 35 種原點復歸模式的支持(包含預留的部分)。使用者僅需使用軟件進行相關設置，該硬件即可完成使用者指定之原點復歸的操作。當完成原點復歸的動作時，對應之命令(command)與反饋位置(feed back position) 將會清除為零值，但目標位置(target position)並不會被清除為零。下圖示意原點復歸執行條件：

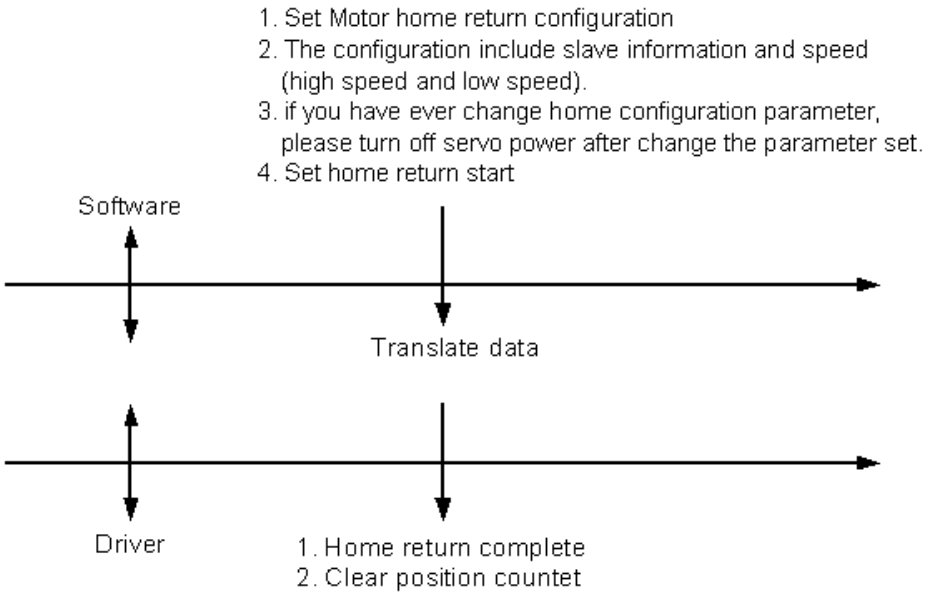


圖 3.041

3.4.2 函式列表

表 3.4

函式名稱
_DMC_01_set_home_config
_DMC_01_set_home_move
_DMC_01_escape_home_move

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.4.3　流程圖

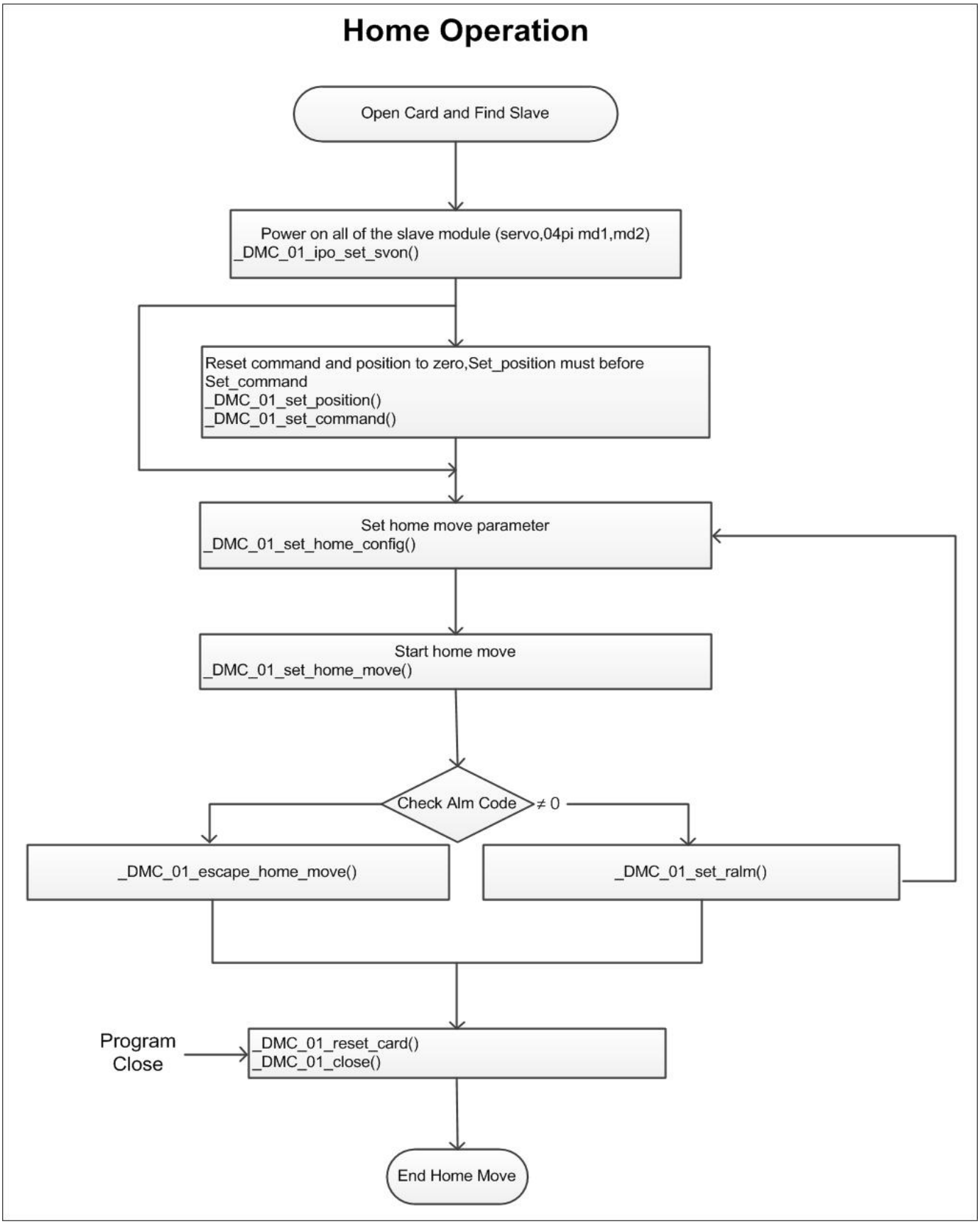


圖 3.042

3.4.4 應用範例

程式外觀

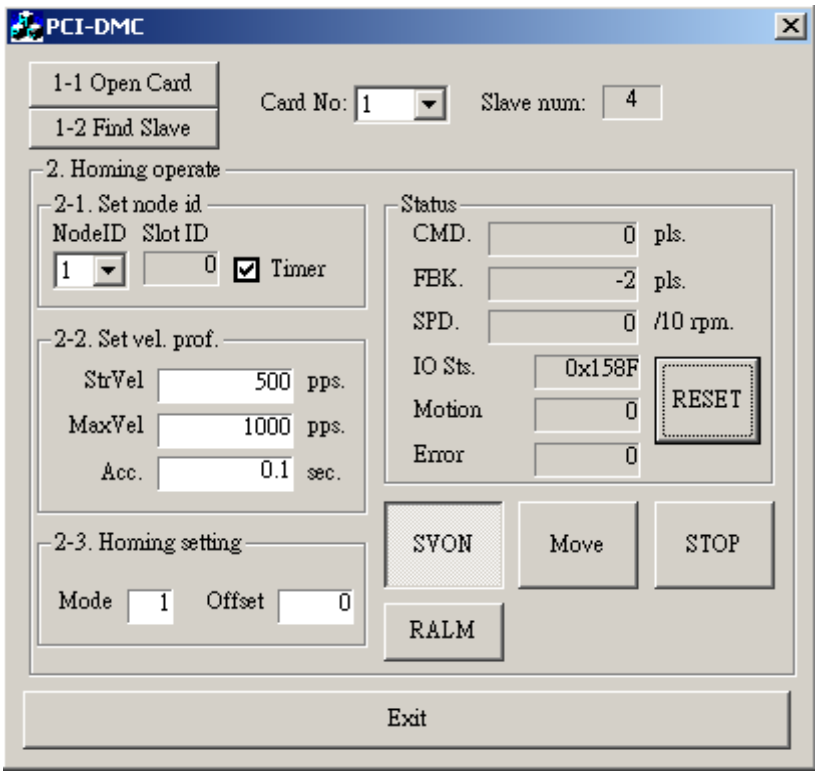


圖 3.043

1) 介面卡開啟與初始化

按壓圖 3.043 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.043 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 填入運動控制的引數內容值

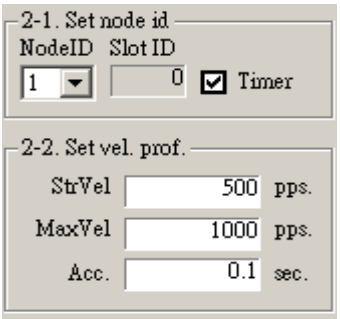


圖 3.044

NodeID 項目：API 函式的引數變數 “NodeID”。

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

StrVel.項目：二段速，每秒送出的脈衝值。API 函式的引數變數 “StrVel”。

MaxVel.項目：一段速，每秒送出的脈衝值。API 函式的引數變數 “MaxVel”。

Acc.項目：達到最大速度所花費時間。API 函式的引數變數 “acc”。

3) 設置原點復歸配置參數(原點復歸模式與偏移量)

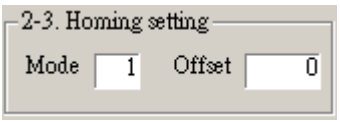


圖 3.045

Mode 項目：原點復歸模式 1 至 35。API 函式的引數變數 “home_mode”。
Offset 項目：原點復歸偏移量。API 函式的引數變數 “home_offset”。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

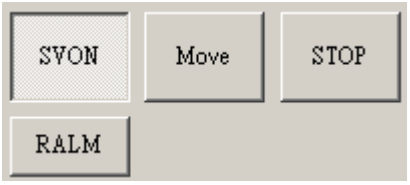


圖 3.046

按壓圖 3.046 中的「SVON」鍵開始執行以下程序：

```
rt =  _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 原點復歸操作

如圖 3.046。按壓「Move」鍵開始執行以下程序：

```
/* 設置原點復歸模式：1~35、偏移量與速度參數 */
rt = _DMC_01_set_home_config(gDMCCardNo, NodeID, SlotID, home_mode,
home_offset, StrVel, MaxVel, acc);
/* 啟動原點復歸運動 */
rt = _DMC_01_set_home_move(gDMCCardNo, NodeID, SlotID);
```

6) 停止原點復歸運動

當您欲停止原點復歸運動操作，您必須按壓圖 3.046 中的「STOP」鍵執行以下程序：

```
/* 中斷原點復歸運動 */
rt = _DMC_01_escape_home_move(gDMCCardNo, NodeID, SlotID);
```

7) 離開程序

按壓圖 3.043 中的「Exit」鍵開始執行程序的離開與關閉。
欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.5 轉矩運動控制

3.5.1 函式列表

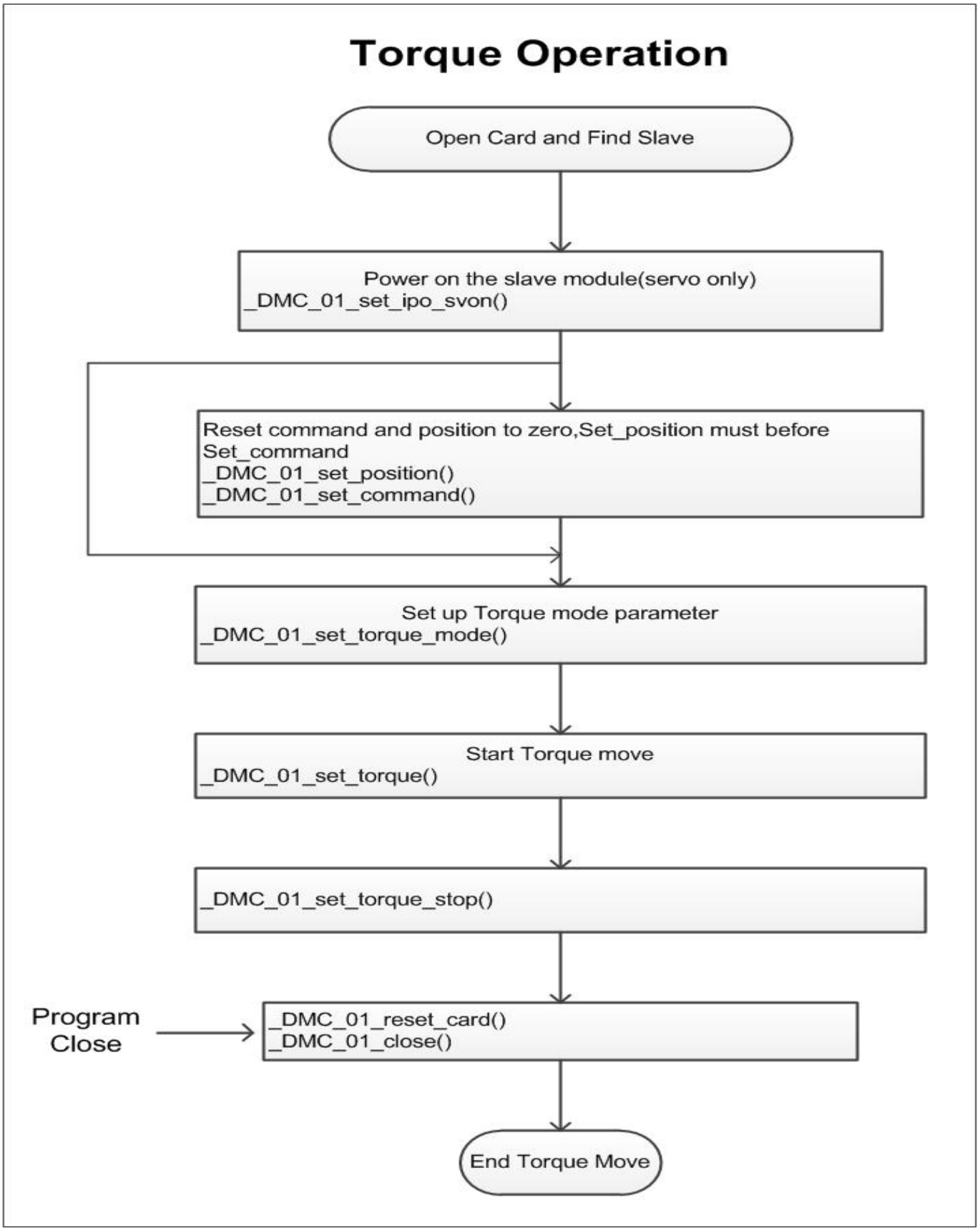
表 3.5

函式名稱
_DMC_01_set_torque_mode
_DMC_01_set_torque
_DMC_01_set_torque_stop
_DMC_01_get_torque

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.5.2　流程圖


Torque Operation
Open Card and Find Slave
Power on the slave module(servo only)
_DMC_01_set_ipo_svon()
Reset command and position to zero,Set_position must before Set_command
_DMC_01_set_position()
_DMC_01_set_command()
Set up Torque mode parameter
_DMC_01_set_torque_mode()
Start Torque move
_DMC_01_set_torque()
_DMC_01_set_torque_stop()
Program Close
_DMC_01_reset_card()
_DMC_01_close()
End Torque Move

3.050

3.5.3 應用範例

程式外觀

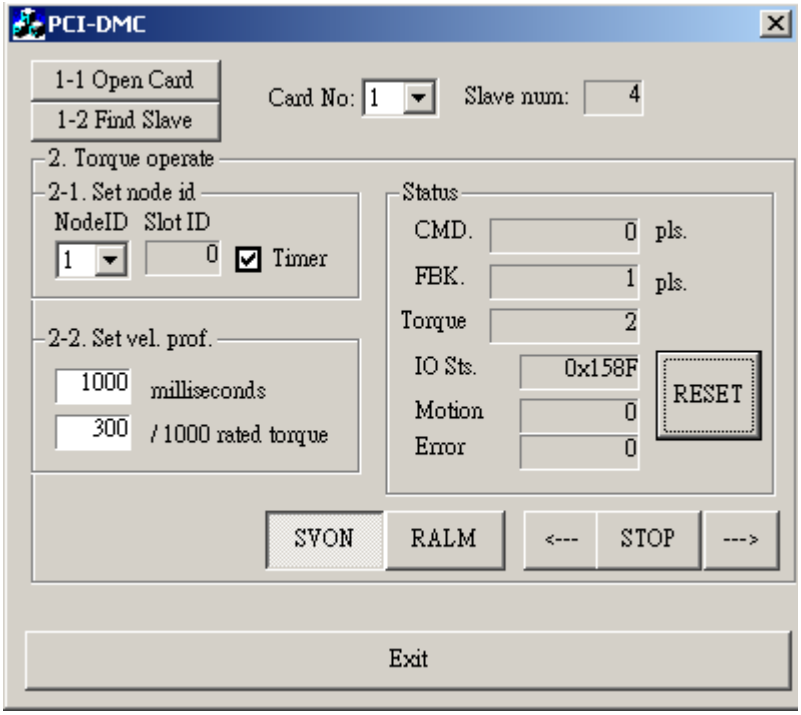


圖 3.051

1) 介面卡開啟與初始化
按壓圖 3.051 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.051 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態



圖 3.052

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函式的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入 slope 與 ratio 值

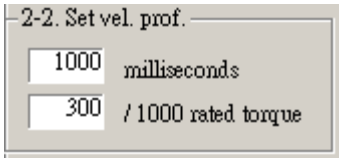


圖 3.053

Slop 項目：由 0 至 100%額定扭力所需時間。(單位：毫秒)
Ratio 項目：額定扭力值千分比。舉例來說，若此值為 300 時表示 30% 額定扭力。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)

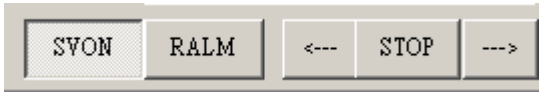


圖 3.054

按壓圖 3.054 中的「SVON」鍵開始執行以下程序：

```
rt =   _DMC_01_ipo_set_svon (gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 –  伺服動力 OFF；1 –  伺服動力 ON
```

5) 轉矩運動控制

按壓圖 3.054 中的「→」或「←」鍵執行以下程序：

```
/*  配置轉矩參數(slope 值) */
rt = _DMC_01_set_torque_mode(gDMCCardNo, NodeID, SlotID, slope);
/*  啟動轉矩運動  */
rt = _DMC_01_set_torque(gDMCCardNo, NodeID, SlotID, ratio);
//  若 ratio 值大於 0，表示馬達將正轉運動；值小於 0 則是反轉運動。
```

Press the「STOP」button to excute torque stop or not.

```
/*  馬達轉矩運動停止與否，當 stop 值為 1 時，表示轉矩運動停止。  */
rt = _DMC_01_set_torque_stop(gDMCCardNo, NodeID, SlotID, stop);
```

6) 重置運動計數值及監控顯示

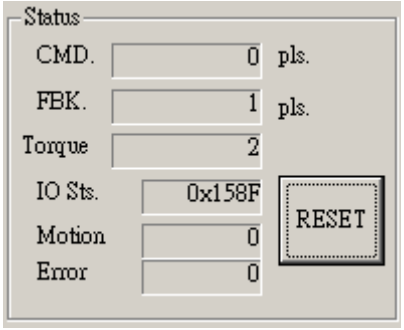


圖 3.055

按壓圖 3.055 的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

7) Alm_Reset

當圖 3.055 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.054 中的「RALM」鍵開始執行 Alm_Code 重置動作。

8) 離開程序

按壓圖 3.051 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.6　速度運動控制

3.6.1　函式列表

表 3.6

函式名稱
_DMC_01_set_velocity_mode
_DMC_01_set_velocity
_DMC_01_set_velocity_stop
_DMC_01_get_rpm

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.6.2 流程圖

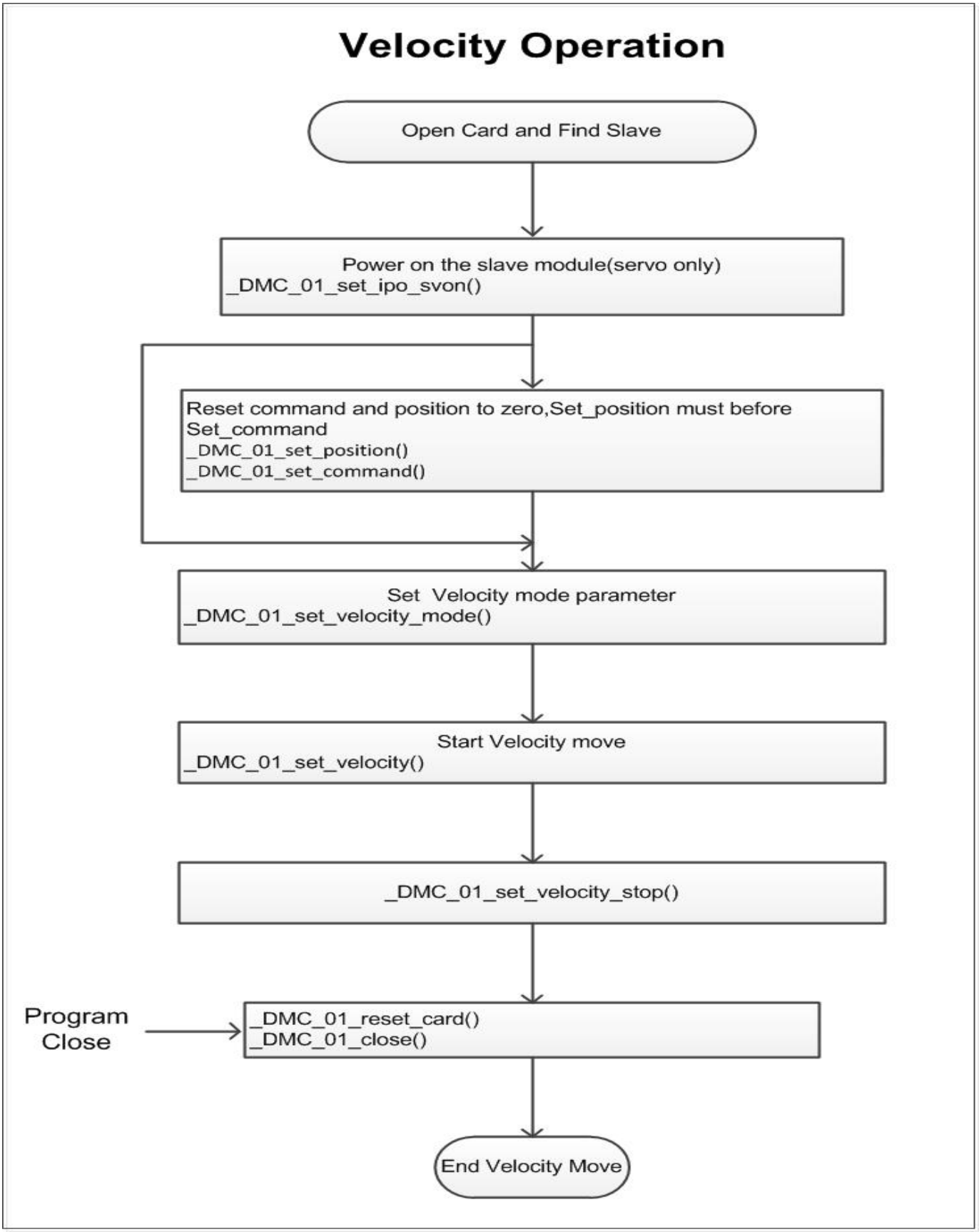


圖 3.060

3.6.3　應用範例

程式外觀

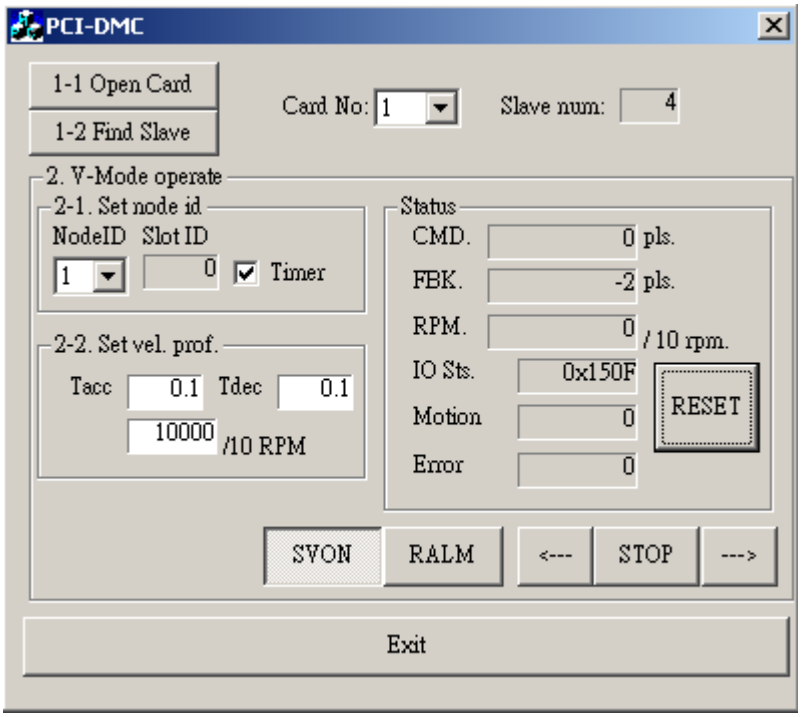


圖 3.061

1) 介面卡開啟與初始化

按壓圖 3.061 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.061 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態



圖 3.062

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函式的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入加減速時間的值和每分鐘旋轉速度(RPM)值

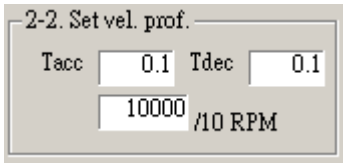


圖 3.063

Tacc 項目：API 函式的引數變數 “Tacc”.

Tdec 項目: API 函式的引數變數 “Tdec”.

RPM 項目: API 函式的引數變數 “rpm”.

※實際 RPM 值為變數 rpm 的 0.1 倍。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.064

按壓圖 3.064 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 速度運動控制

按壓圖 3.064 中的「→」或「←」鍵執行以下程序：

```
/* 配置速度模式參數(加速時間與減速時間的值) */
rt = _DMC_01_set_velocity_mode(gDMCCardNo, NodeID, SlotID, Tacc, Tdec);
/* 啟動速度模式運動 */
rt = _DMC_01_set_velocity(gDMCCardNo, NodeID, SlotID, rpm);
// 若 rpm 的值大於 0 時，驅動器馬達正轉；反之，值小於 0 為反轉。
```

Press the「STOP」button to excute velocity stop or not.

```
/* 設置是否停止速度運動控制。若 stop 的值為 1，速度運動停止 */
rt = _DMC_01_set_velocity_stop(gDMCCardNo, NodeID, SlotID, stop);
```

6) 重置運動計數值及監控顯示

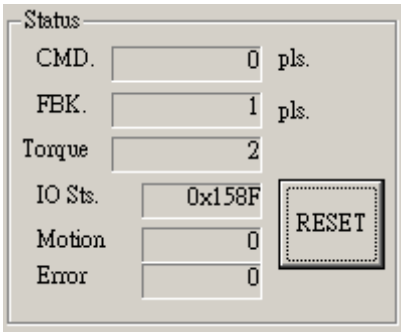


圖 3.065

按壓圖 3.065 的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

7) Alm_Reset

當圖 3.065 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.064 中的「RALM」鍵開始執行 Alm_Code 重置動作。

8) 離開程序

按壓圖 3.061 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.7 單軸運動控制

3.7.1 函式列表

表 3.7

函式名稱
_DMC_01_start_sa_move
_DMC_01_start_ta_move
_DMC_01_start_sr_move
_DMC_01_start_tr_move
_DMC_01_p_change
_DMC_01_v_change
_DMC_01_feedrate_overwrite
_DMC_01_start_sa_move_2seg
_DMC_01_start_ta_move_2seg
_DMC_01_start_sr_move_2seg
_DMC_01_start_tr_move_2seg
_DMC_01_start_sa_move_2seg2
_DMC_01_start_ta_move_2seg2
_DMC_01_start_sr_move_2seg2
_DMC_01_start_tr_move_2seg2
_DMC_01_tv_move
_DMC_01_sv_move
_DMC_01_start_v3_move
_DMC_01_set_position
_DMC_01_set_command
_DMC_01_get_command
_DMC_01_get_position
_DMC_01_get_current_speed
_DMC_01_motion_status
_DMC_01_motion_done
_DMC_01_get_alm_code
_DMC_01_emg_stop

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.7.2　應用範例

程式外觀

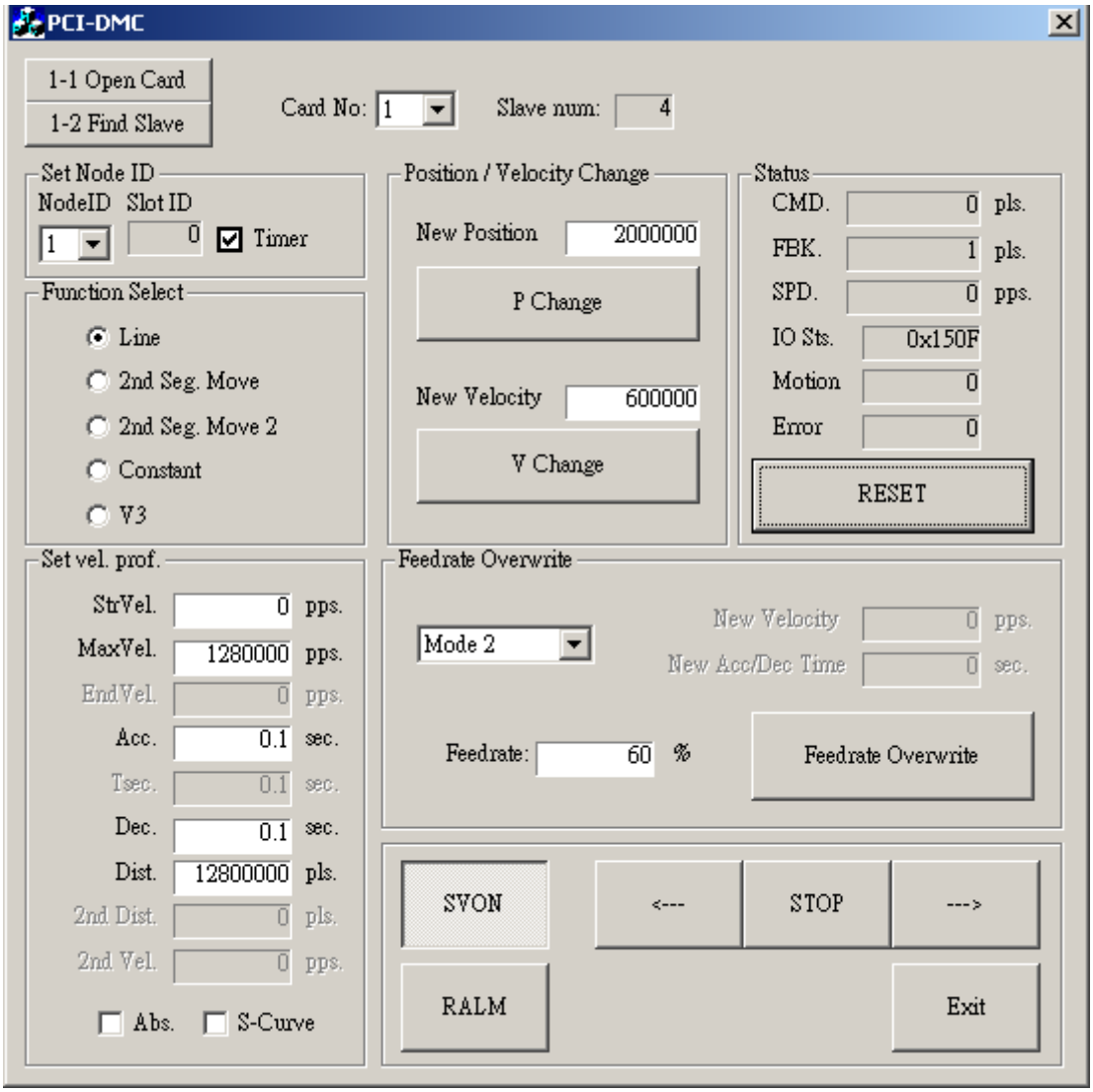


圖 3.070

1) 介面卡開啟與初始化

按壓圖 3.070 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.070 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態



圖 3.071

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 功能選擇

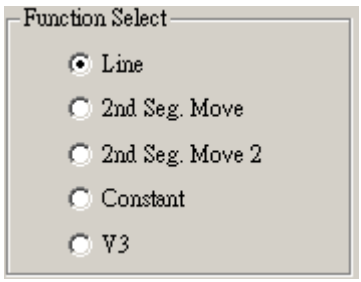


圖 3.072

Line 項目：點對點運動。
2nd Seg. Move 項目：二段式運動模式一。
2nd Seg. Move2 項目：二段式運動模式二。
Constant 項目：直線運動模式。
V3 項目：V3 運動模式。

4) 選擇 Line 項目時運動控制的引數內容值

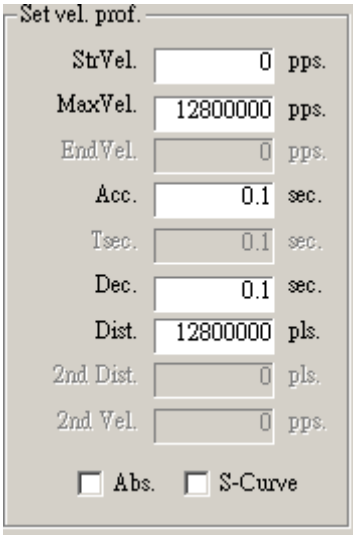


圖 3.073

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

5) 選擇 Line 項目時設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.074

按壓圖 3.074 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 選擇 Line 項目時開始進行速度運動控制
按壓圖 3.074 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位移
```

7) 選擇 Line 項目時開始進行位置變換或速度變換運動控制

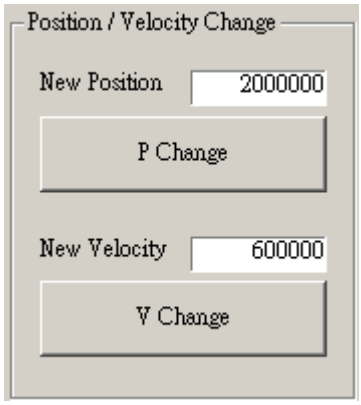


圖 3.075

當按壓圖 3.074 中的「→」或「←」鍵執行 Line 運動控制時，若您欲將運動中的位置置換為新的位置，您必須按壓圖 3.075 中的「P change」執行以下程序：

```
rt = _DMC_01_p_change (CardNo, NodeID, SlotID, NewPos);
// 以新的位置值置換目前的位置
```

若您欲將運動中的速度置換為新的速度，您必須按壓圖 3.075 中的「V change」執行以下程序：

```
rt = _DMC_01_v_change (CardNo, NodeID, SlotID, NewSpeed, sec);
// 以新的速度值置換目前的速度
```

8) 選擇 Line 項目時開始進行 FeedrateOverwrite 運動控制

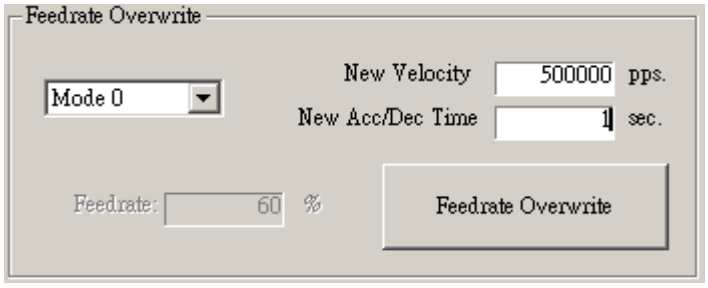


圖 3.076

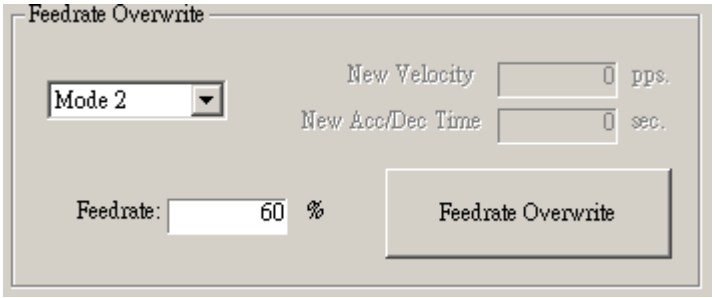


圖 3.077

當按壓圖 3.074 中的「→」或「←」鍵執行 Line 運動控制時，若您欲進行 Mode0、1 的 FeedrateOverwrite，必須按壓圖 3.076 中的「Feedrate Overwrite」執行以下程序：

```
rt = _DMC_01_feedrate_overwrite (CardNo, NodeID, SlotID ,Mode,Feedrate,
NewAccDecT); // Mode=0，1
```

若您欲進行 Mode2 的 FeedrateOverwrite，您必須按壓圖 3.077 中的「Feedrate Overwrite」執行以下程序：

```
rt = _DMC_01_feedrate_overwrite (CardNo, NodeID, SlotID ,Mode,Feedrate,
NewAccDecT); // Mode=2
```

9) 選擇 2nd Seg. Move 或 2nd Seg. Move 2 項目時，畫面如圖 3.078 所示：

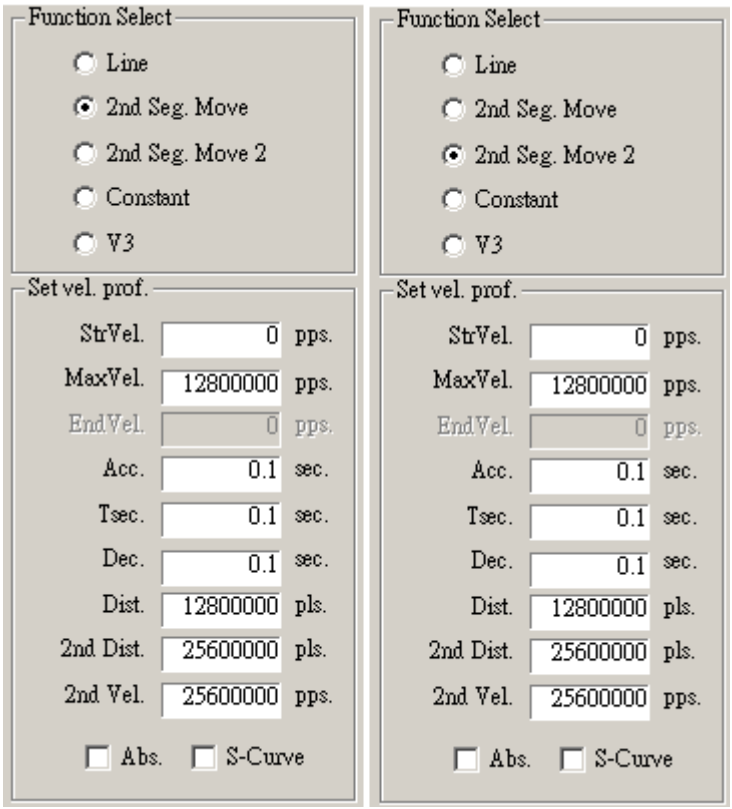


圖 3.078

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：第一段行程中設置的最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Tsec 項目：第一段行程切換至第二段行程的加/減速時間。API 函數引數變數 “Tsec”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：指定的第一段絕對座標行程。API 函數的引數變數 “Dist”。
2ndDist.項目：指定的第二段絕對座標行程。API 函數的引數變數 “Dist2”。
2ndVel 項目：第二段行程中設置的最大速度。API 函數的引數變數 “MaxVel2”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

當按壓圖 3.074 中的「→」或「←」鍵執行 2nd Seg. Move 或 2nd Seg. Move 2 時，將執行以下程序：

```
rt = _DMC_01_start_sa_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
     MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的絕對座標進行二段速型式一運動位移
rt = _DMC_01_start_ta_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
     MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 T-curve 速度剖面下的絕對座標進行二段速型式一運動位移
rt = _DMC_01_start_sr_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的相對座標進行二段速型式一運動位移
rt = _DMC_01_start_tr_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
     MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 T-curve 速度剖面下的相對座標進行二段速型式一運動位移

rt = _DMC_01_start_sa_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
     MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的絕對座標進行二段速型式二運動位移
rt = _DMC_01_start_ta_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
     MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 T-curve 速度剖面下的絕對座標進行二段速型式二運動位移
rt = _DMC_01_start_sr_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 S-curve 速度剖面下的相對座標進行二段速型式二運動位移
rt = _DMC_01_start_tr_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2,StrVel,
     MaxVel, MaxVel2, Tacc, Tsec, Tdec);
//參照 T-curve 速度剖面下的相對座標進行二段速型式二運動位移
```

10) 選擇 Constant 項目時，畫面如圖 3.079 所示：

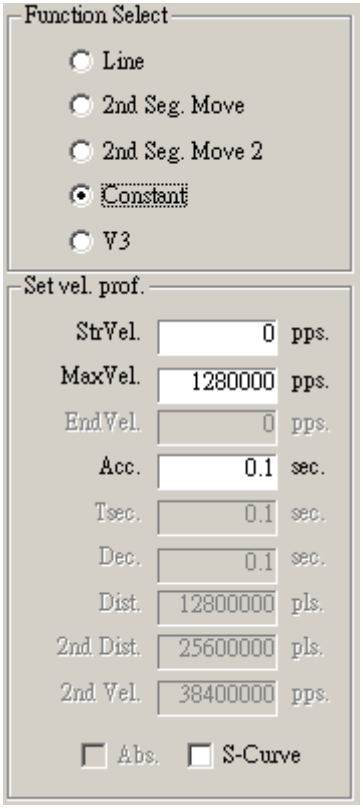


圖 3.079

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：行程中設置的最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

當按壓圖 3.074 中的「→」或「←」鍵執行 Constant 時，將執行以下程序：

```
rt = _DMC_01_tv_move (CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
//S-curve 速度曲線
rt = _DMC_01_sv_move (CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
//T-curve 速度曲線
```

11) 選擇 V3 項目時，畫面如圖 3.07A 所示：

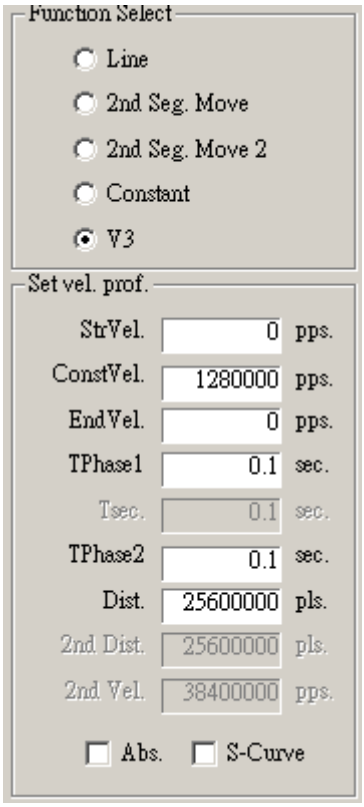


圖 3.07A

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：行程中的常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費時間。API 函數的引數變數 “TPhase2”。
Dist.項目：指定的運動行程。API 函數的引數變數 “Dist”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

當按壓圖 3.074 中的「→」或「←」鍵執行 V3 運動時，將執行以下程序：

```
rt = _DMC_01_start_v3_move (CardNo, NodeID, SlotID, Dist, StrVel, ConstVel,
      EndVel, TPhase1, TPhase2, m_curve, m_r_a);
//增加 EndVel 的單軸運動位移
```

12) 重置運動位移計數值(Command 與 Feedback)及監控顯示

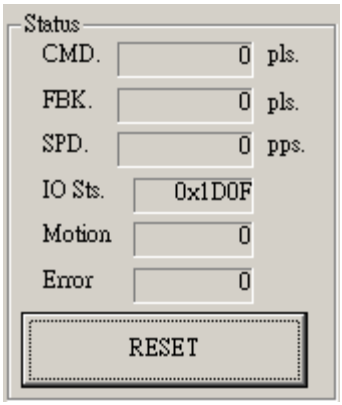


圖 3.07B

按壓圖 3.07B 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

13) 停止運動及 Alm_Code 重置



圖 3.07C

不論正在執行哪一項運動，當按壓圖 3.07C 中的「STOP」鍵開始執行緊急停止：

rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.07B 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.07C 中的「RALM」鍵開始執行 Alm_Code 重置動作。

14) 離開程序

按壓圖 3.07C 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.8　雙軸運動控制

3.8.1　函式列表

表 3.8

函式名稱
_DMC_01_start_sa_move_xy
_DMC_01_start_ta_move_xy
_DMC_01_start_sr_move_xy
_DMC_01_start_tr_move_xy
_DMC_01_start_v3_move_xy
_DMC_01_start_sa_arc_xy
_DMC_01_start_ta_arc_xy
_DMC_01_start_sr_arc_xy
_DMC_01_start_tr_arc_xy
_DMC_01_start_v3_arc_xy
_DMC_01_start_sa_arc2_xy
_DMC_01_start_ta_arc2_xy
_DMC_01_start_sr_arc2_xy
_DMC_01_start_tr_arc2_xy
_DMC_01_start_v3_arc2_xy
_DMC_01_start_sa_arc3_xy
_DMC_01_start_ta_arc3_xy
_DMC_01_start_sr_arc3_xy
_DMC_01_start_tr_arc3_xy
_DMC_01_start_v3_arc3_xy
_DMC_01_start_spiral_xy
_DMC_01_start_v3_spiral_xy
_DMC_01_start_spiral2_xy
_DMC_01_start_v3_spiral2_xy

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.8.2 應用範例

程式外觀

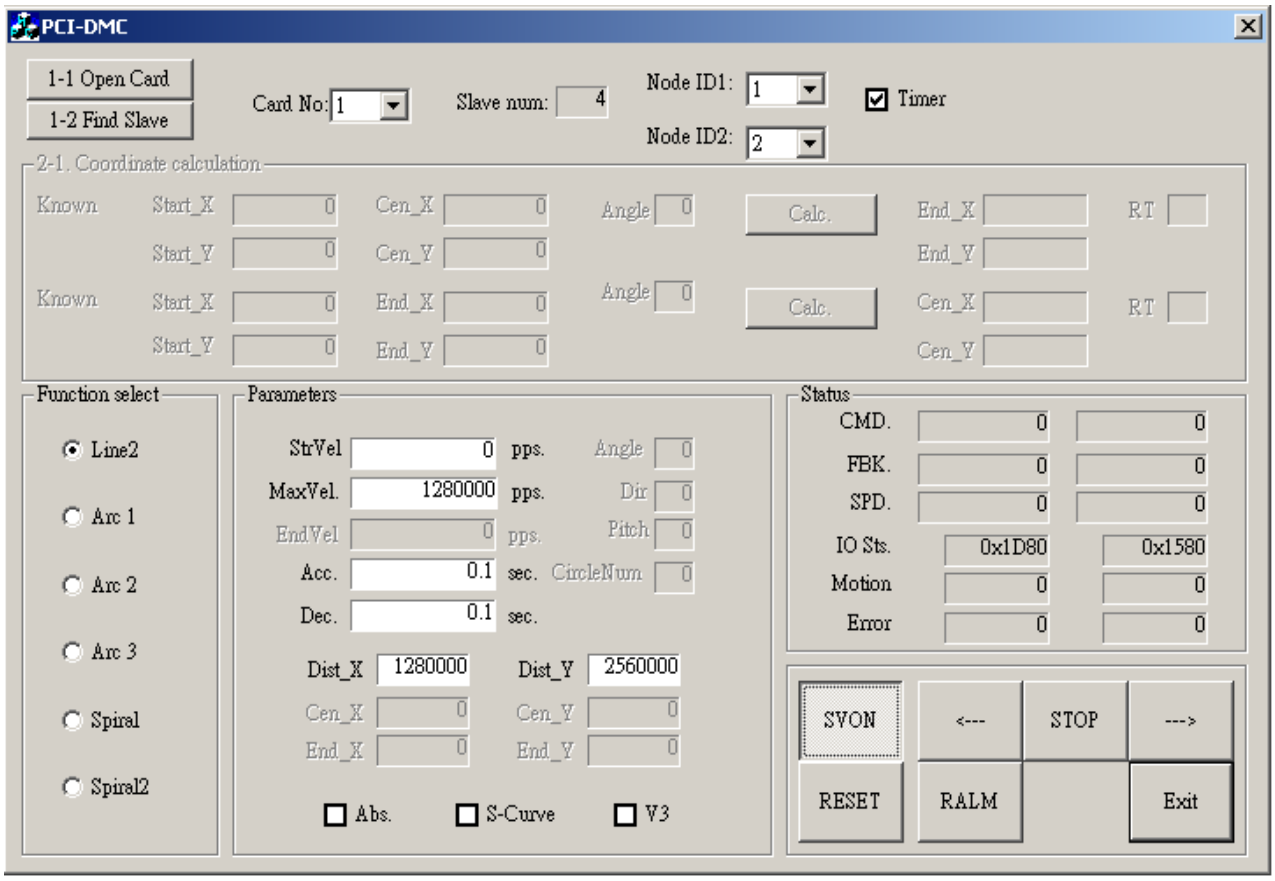


圖 3.080

1) 介面卡開啟與初始化

按壓圖 3.080 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.080 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

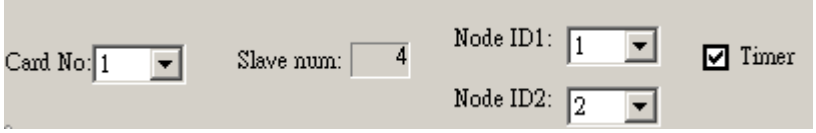


圖 3.081

選取 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 功能選擇

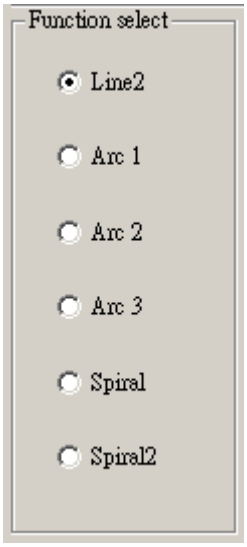


圖 3.082

Line2 項目：線性補間運動控制。
Arc1 項目 ：圓弧補間運動控制型式一。
Arc2 項目 ：圓弧補間運動控制型式二。
Arc3 項目 ：圓弧補間運動控制型式三。
Spiral 項目 ：螺旋補間運動控制型式一。
Spiral2 項目 ：螺旋補間運動控制型式二。

4) 選擇 Line2 運動控制的引數內容值如圖 3.083 所示:

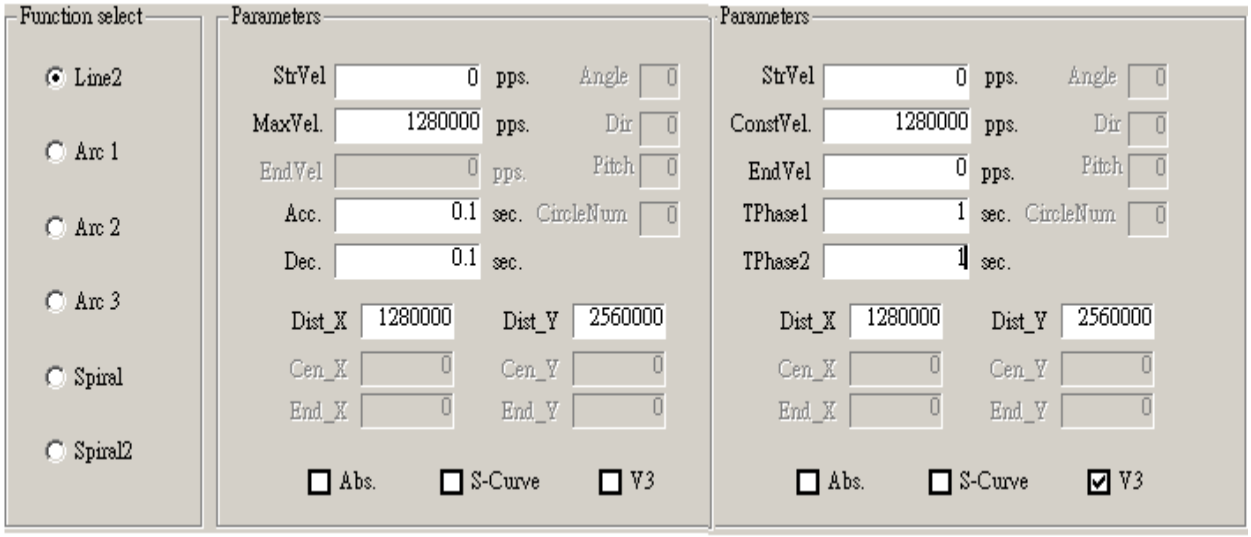


圖 3.083

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist_X 項目: X 軸的運動行程參數。API 函數的引數變數 “DisX”。
Dist_Y 項目: Y 軸的運動行程參數。API 函數的引數變數 “DisY”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.083 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

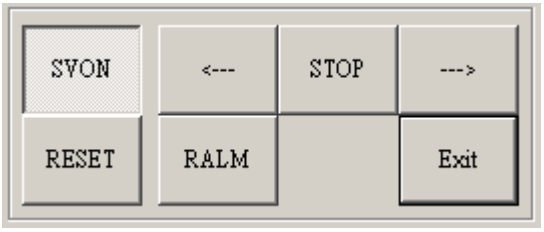


圖 3.084

按壓圖 3.084 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 當按壓圖 3.084 中的「→」或「←」鍵執行 Line2(未勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_sa_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
    DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸線性補間運動
rt = _DMC_01_start_ta_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
    DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸線性補間運動
rt = _DMC_01_start_sr_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
    DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸線性補間運動
rt = _DMC_01_start_tr_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
    DistY, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸線性補間運動
```

7) 當按壓圖 3.084 中的「→」或「←」鍵執行 Line2(勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_move_xy(gDMCCardNo, NodeIDArray, SlotID, DistX,
    DistY, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸線性補間運動
```

8) 選擇功能 Arc1、Arc2 及 Arc3 將會出現圖 3.085「Arc 計算功能」，填入已知條件並選擇使用的計算方式，將可得出使用該選擇 Arc 的參數設定資料。

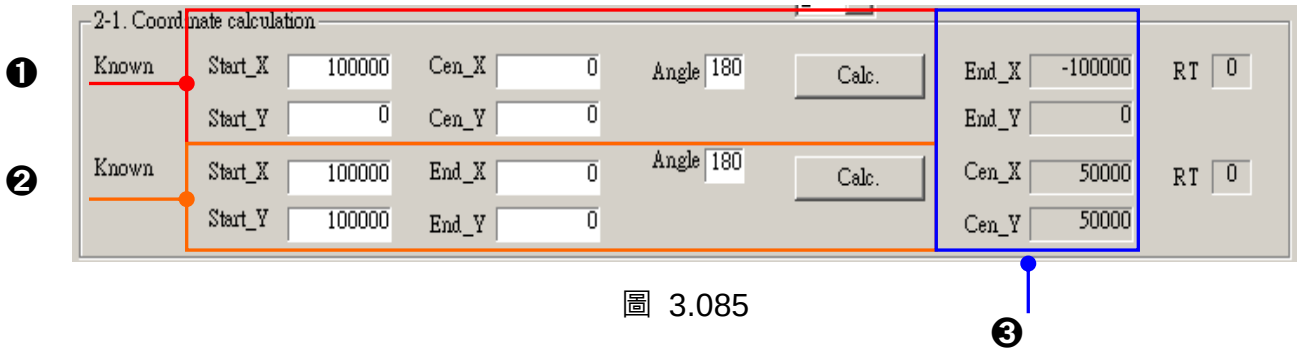


圖 3.085

1. 若您已知起始座標 XY 值、中心座標 XY 值和對應的角度值，然後即可藉由計算式取得終點座標 XY 值。故當您按壓「Calc.」鍵開始執行以下程序：

```
/* 取得 X 軸與 Y 軸的終點座標值 */
rt = _misc_app_get_circle_endpoint(Start_X, Start_Y , Center_X, Center_Y, Angle, &End_X, &End_Y);
```

2. 若您已知起始座標 XY 值、終點座標 XY 值和對應的角度值，然後即可藉由計算式取得中心座標 XY 值。故當您按壓「Calc.」鍵開始執行以下程序：

```
/* 取得 X 軸與 Y 軸的中心座標值 */
rt = _misc_app_get_circle_center_point(Start_X, Start_Y , End_X, End_Y, Angle, &Center_X, &Center_Y);
```

3. 當您完成執行程序 1 或程序 2，計算式計算出來的結果將顯示在區塊❸。

9) 選擇 Arc1 運動控制的引數內容值如圖 3.086 所示:

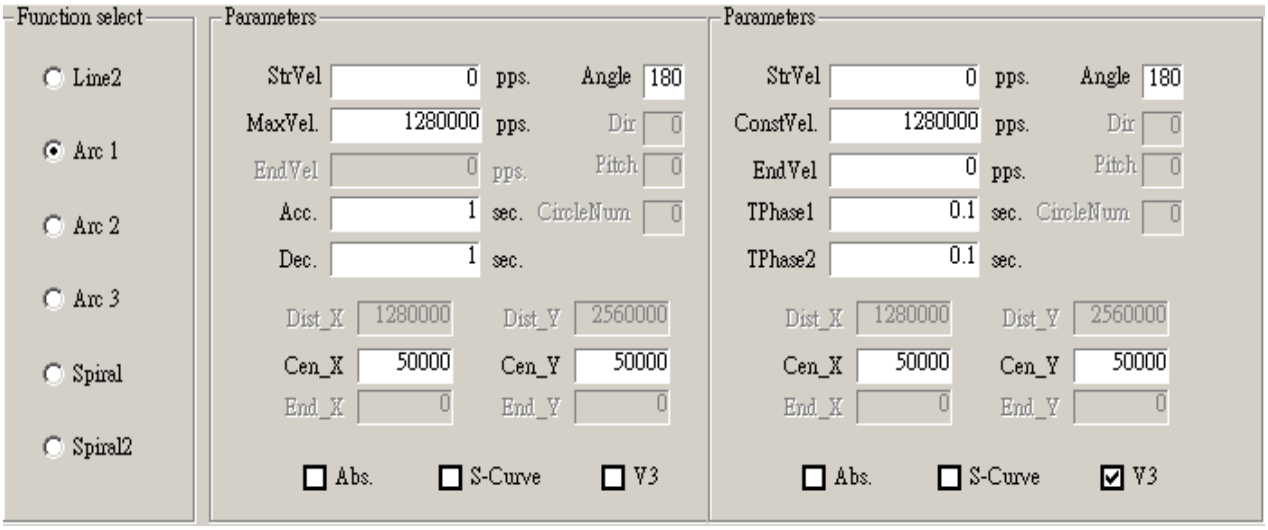


圖 3.086

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應的圓心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應的圓心位置 Y。API 函數的引數變數 “Center_Y”。
Angle 項目: 圓弧的角度設置。API 函數的引數變數 “Angle”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.086 右部的圖示選項。
StrVel 項目　：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目　：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

10) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc1(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式一)
rt = _DMC_01_start_ta_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式一)
rt = _DMC_01_start_sr_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式一)
rt = _DMC_01_start_tr_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
    Center_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式一)
```

11) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc1 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_arc_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸圓弧補間運動(型式一)
```

12) 選擇 Arc2 運動控制的引數內容值如圖 3.087 所示:

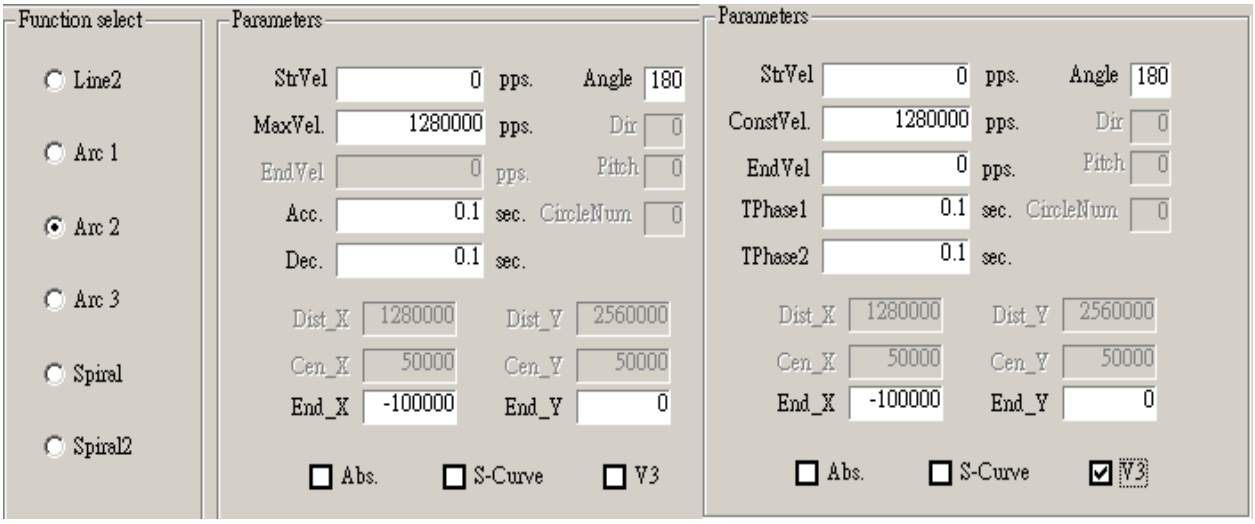


圖 3.087

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
End_X 項目: 指定軸對應的終點位置 X。API 函數的引數變數 “End_X”。
End_Y 項目: 指定軸對應的終點位置 Y。API 函數的引數變數 “End_Y”。
Angle 項目: 圓弧的角度設置。API 函數的引數變數 “Angle”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.087 右部的圖示選項。
StrVel 項目　：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目　：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

13) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc2(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End_X,
    End_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式二)
rt = _DMC_01_start_ta_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
    End _Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式二)
rt = _DMC_01_start_sr_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
    End _Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式二)
rt = _DMC_01_start_tr_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
    End _Y, Angle, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式二)
```

14) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc2 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_arc2_xy(gDMCCardNo, NodeIDArray, SlotID, End _X,
End _Y, Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸圓弧補間運動(型式二)
```

15) 選擇 Arc3 運動控制的引數內容值如圖 3.088 所示:

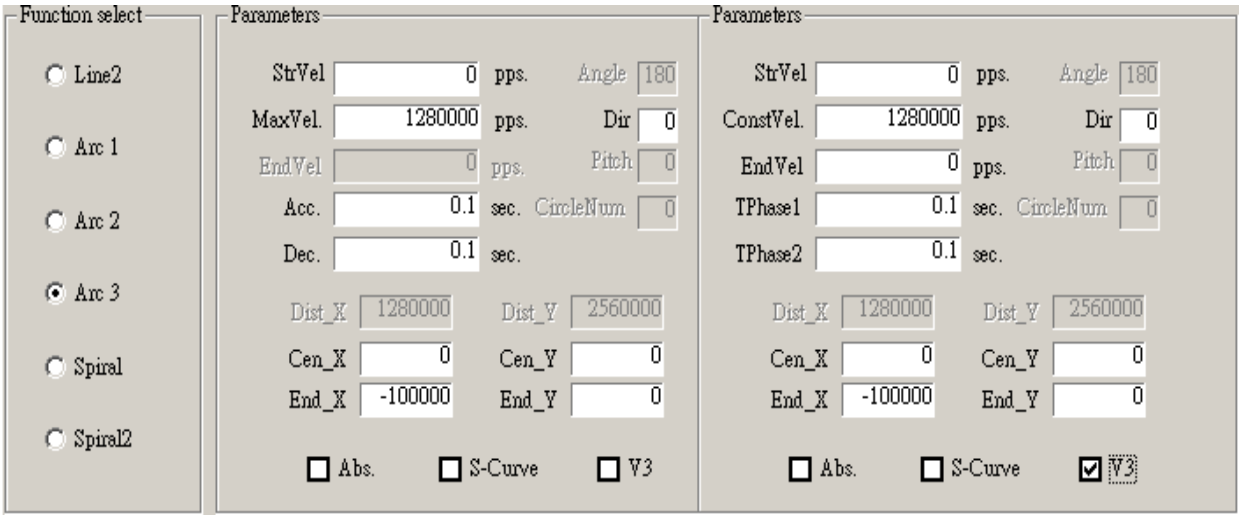


圖 3.088

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應的圓心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應的圓心位置 Y。API 函數的引數變數 “Center_Y”。
End_X 項目: 指定軸對應的終點位置 X。API 函數的引數變數 “End_X”。
End_Y 項目: 指定軸對應的終點位置 Y。API 函數的引數變數 “End_Y”。
Dir 項目: 指定的方向(0:順時針方向 1:逆時針方向)。API 函數的引數變數 “Dir”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.088 右部的圖示選項。
StrVel 項目　：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目　：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

16) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc3(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_arc3_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式三)
rt = _DMC_01_start_ta_arc3_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(型式三)
rt = _DMC_01_start_sr_arc3_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式三)
rt = _DMC_01_start_tr_arc3_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(型式三)
```

17) 當按壓圖 3.084 中的「→」或「←」鍵執行 Arc3 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_arc3_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, End_X, End_Y, Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2,
m_curve, m_r_a);
// 增加 EndVel 的兩軸圓弧補間運動(型式三)
```

18) 選擇 Spiral 運動控制的引數內容值如圖 3.089 所示:

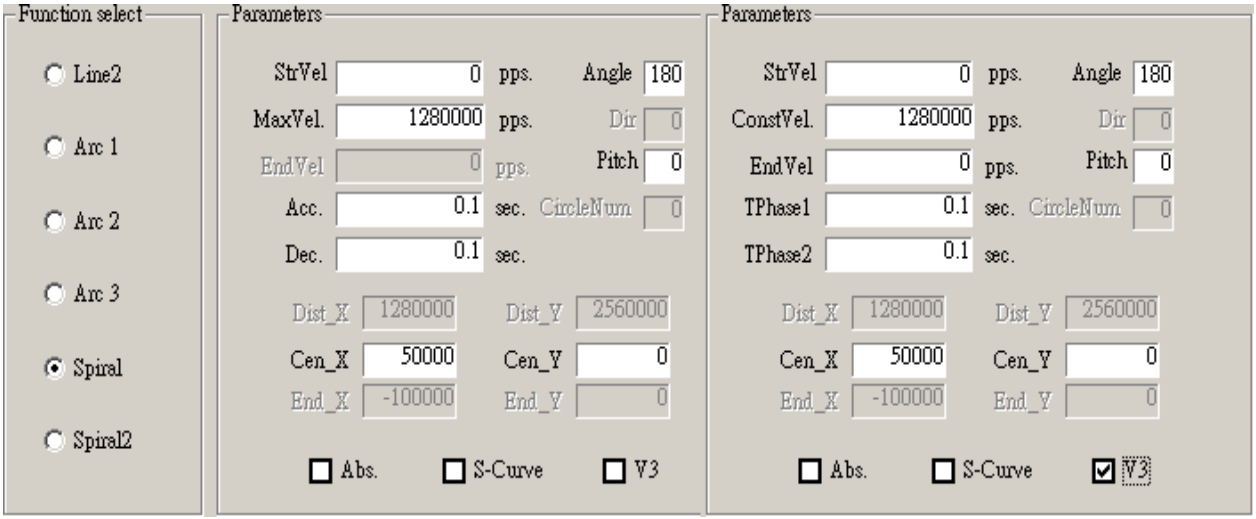


圖 3.089

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應中心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應中心位置 Y。API 函數的引數變數 “Center_Y”。
Angle 項目: 螺旋運動之總角度。API 函數的引數變數 “spiral_angle”。
Pitch 項目: 螺旋之間相對性位置距離。API 函數的引數變數 “spiral_interval”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.089 右部的圖示選項。
StrVel 項目　：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目　：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

19) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_spiral_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Spiral_interval, Spiral_angle, StrVel, MaxVel, Tacc, Tdec, m_curve,
M_r_a);//進行兩軸螺旋運動(型式一)
```

20) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_spiral _xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Spiral_interval, Spiral_angle, StrVel, ConstVel, EndVel, TPhase1,
TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸螺旋運動(型式一)
```

21) 選擇 Spiral2 運動控制的引數內容值如圖 3.08A 所示:

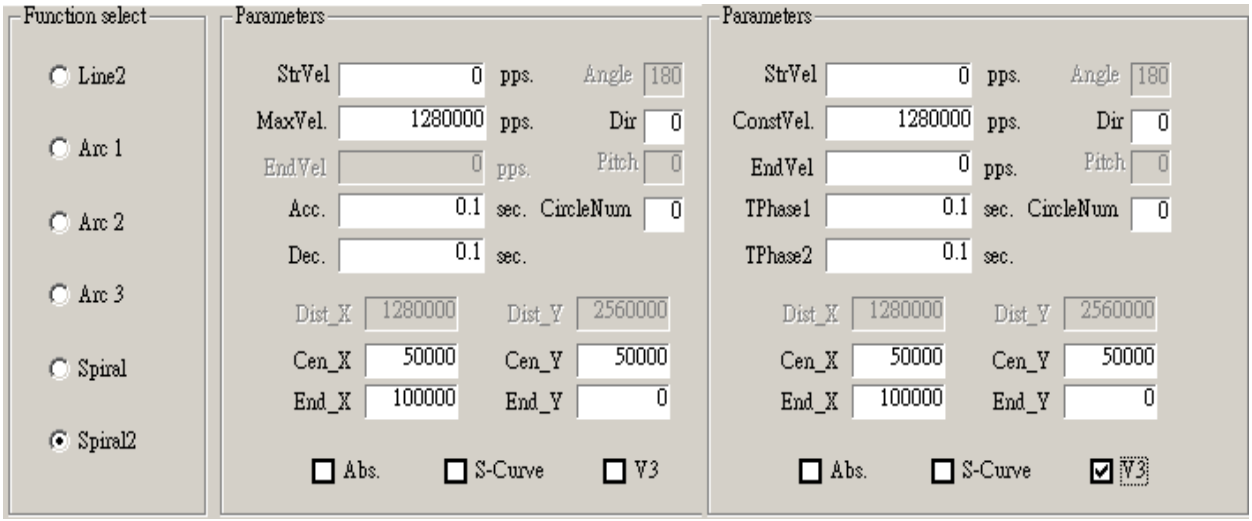


圖 3.08A

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Cent_X 項目: 指定軸對應中心位置 X。API 函數的引數變數 “Center_X”。
Cent_Y 項目: 指定軸對應中心位置 Y。API 函數的引數變數 “Center_Y”。
End_X 項目: 指定軸對應的終點位置 X。API 函數的引數變數 “End_X”。
End_Y 項目: 指定軸對應的終點位置 Y。API 函數的引數變數 “End_Y”。
Dir 項目: 指定的方向(0:順時針方向 1:逆時針方向)。API 函數的引數變數 “dir”。
CircleNum 項目: 螺旋運動的圈數。API 函數的引數變數 “circlenum”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.08A 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。

TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數“TPhase2”。

22) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral2(未勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_spiral2_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, End_X, End_Y, dir, circelnum, StrVel, MaxVel, Tacc, Tdec, m_curve,
m_r_a);//進行兩軸螺旋運動(型式二)
```

23) 當按壓圖 3.084 中的「→」或「←」鍵執行 spiral2(勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_spiral2 _xy(gDMCCardNo, NodeIDArray, SlotID,
Center_X, Center_Y, End_X, End_Y, dir, circelnum, StrVel, ConstVel, EndVel,
TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的兩軸螺旋運動(型式二)
```

24) 監控顯示

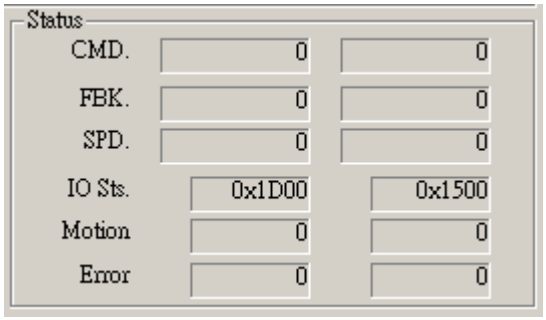


圖 3.08B

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

25) 重置運動計數值、停止運動及 Alm_Reset

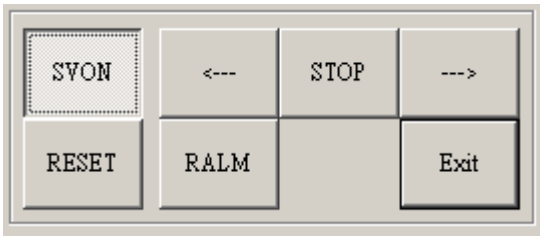


圖 3.08C

按壓圖 3.08C 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.08C 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.08B 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.08C 中的「RALM」鍵開始執行 Alm_Code 重置動作。

26) 離開程序

按壓圖 3.08C 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.9　三軸運動控制

3.9.1　函式列表

表 3.9

函式名稱
_DMC_01_start_sa_move_xyz
_DMC_01_start_ta_move_xyz
_DMC_01_start_sr_move_xyz
_DMC_01_start_tr_move_xyz
_DMC_01_start_v3_move_xyz
_DMC_01_start_sa_heli_xy
_DMC_01_start_ta_heli_xy
_DMC_01_start_sr_heli_xy
_DMC_01_start_tr_heli_xy
_DMC_01_start_v3_heli_xy

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.9.2 應用範例

程式外觀

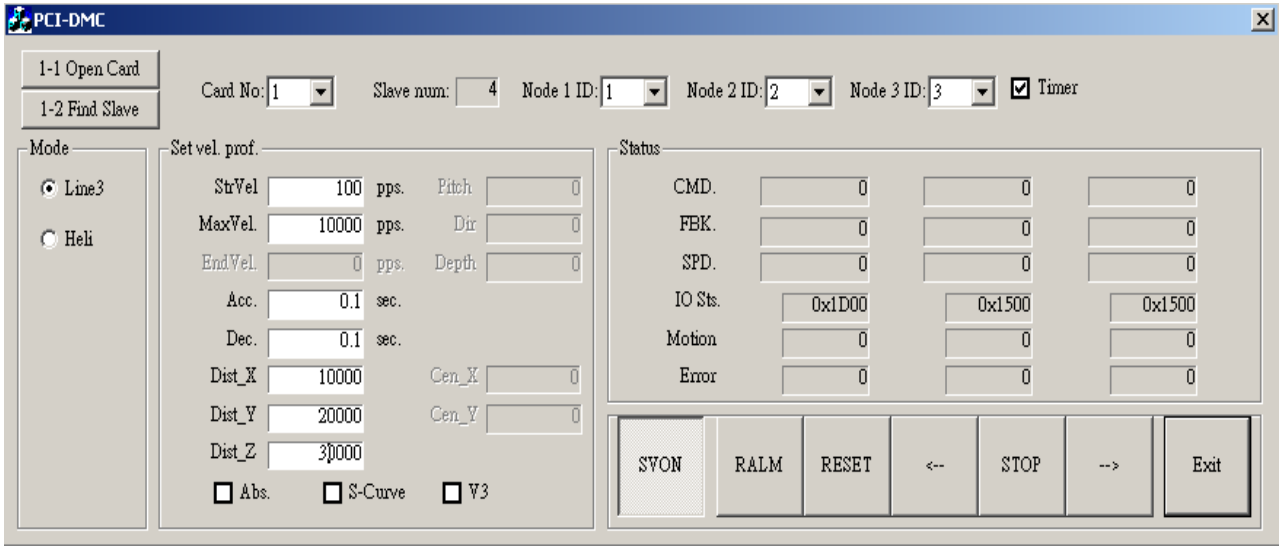


圖 3.090

1) 介面卡開啟與初始化

按壓圖 3.090 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.090 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態



圖 3.091

勾選「Timer」核選方塊啟用顯示運動狀態。
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 功能選擇



圖 3.092

Line3 項目：線性補間運動控制。

Heli 項目 ：螺旋補間運動控制。

4) 選擇 Line3 運動控制的引數內容值如圖 3.093 所示:

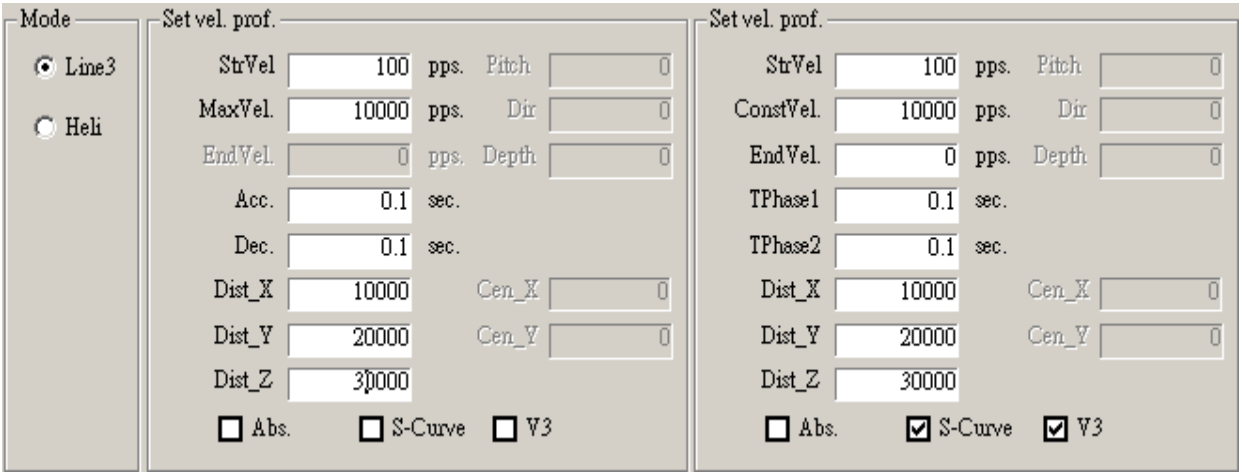


圖 3.093

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist_X 項目: 對應 X 軸站號運動的路徑參數。API 函數的引數變數 “DisX”。
Dist_Y 項目: 對應 Y 軸站號運動的路徑參數。API 函數的引數變數 “DisY”。
Dist_Z 項目: 對應 Z 軸站號運動的路徑參數。API 函數的引數變數 “DisZ”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.093 右部的圖示選項。
StrVel 項目　　：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目　　：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.094

按壓圖 3.094 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 當按壓圖 3.094 中的「→」或「←」鍵執行 Line3(未勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_sa_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行三軸線性補間運動
rt = _DMC_01_start_ta_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行三軸線性補間運動
rt = _DMC_01_start_sr_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行三軸線性補間運動
rt = _DMC_01_start_tr_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行三軸線性補間運動
```

7) 當按壓圖 3.094 中的「→」或「←」鍵執行 Line3 (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_move_xyz(gDMCCardNo, NodeIDArray, SlotID, DisX,
DisY, DisZ, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
// 增加 EndVel 的三軸線性補間運動
```

8) 選擇 Heli 運動控制的引數內容值如圖 3.095 所示:

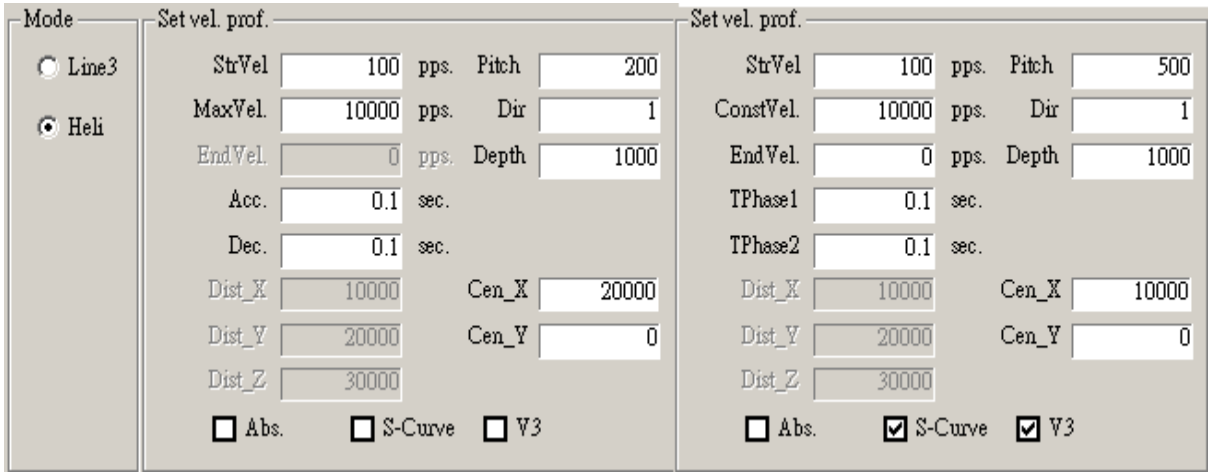


圖 3.095

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Pitch 項目: 兩螺旋之間的高度。API 函數的引數變數 “Pitch”。
Dir 項目: 螺旋圓弧運動的方向 (0:順時針方向 1:逆時針方向)。
API 函數的引數變數 “Dir”。
Depth 項目: 指定軸對應之位置深度(整個 Z 方向高度)API 函數的引數變數 “Depth”。
Cen_X 項目: 指定軸對應的圓心位置 X。API 函數的引數變數 “Center_X”。
Cen_Y 項目: 指定軸對應的圓心位置 Y。API 函數的引數變數 “Center_Y”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
示意圖請參閱圖 3.096

若勾選 V3，則會出現圖 3.095 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

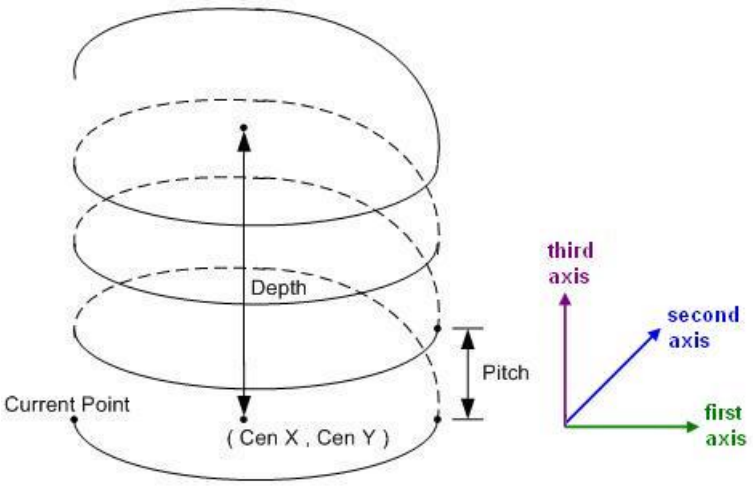


圖 3.096

9) 當按壓圖 3.094 中的「→」或「←」鍵執行 Heli(未勾選 V3)功能時，
將執行以下程序：

```
rt = _DMC_01_start_sa_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的絕對座標進行三軸螺旋補間運動
rt = _DMC_01_start_ta_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的絕對座標進行三軸螺旋補間運動
rt = _DMC_01_start_sr_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 S-curve 速度剖面下的相對座標進行三軸螺旋補間運動
rt = _DMC_01_start_tr_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
// 參照 T-curve 速度剖面下的相對座標進行三軸螺旋補間運動
```

10) 當按壓圖 3.094 中的「→」或「←」鍵執行 Heli (勾選 V3)功能時，將執行以下程序：

```
rt = _DMC_01_start_v3_heli_xy(gDMCCardNo, NodeIDArray, SlotID, Center_X,
Center_Y, Depth, Pitch, Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2,
m_curve, m_r_a);
// 增加 EndVel 的三軸螺旋補間運動
```

11) 監控顯示

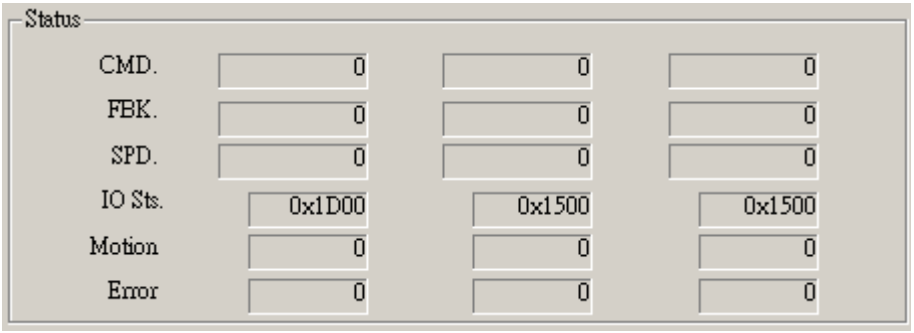


圖 3.097

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

12) 重置運動計數值、停止運動及 Alm_Reset

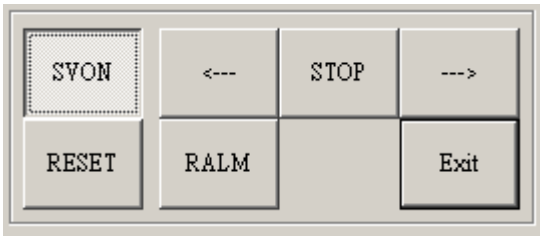


圖 3.098

按壓圖 3.098 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.098 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.097 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.098 中的「RALM」鍵開始執行 Alm_Code 重置動作。

13) 離開程序

按壓圖 3.098 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.10 多軸運動控制

3.10.1 函式列表

表 3.10

函式名稱
_DMC_01_multi_axes_move
_DMC_01_start_v3_multi_axes
_DMC_01_liner_speed_master

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.10.2　應用範例

程式外觀

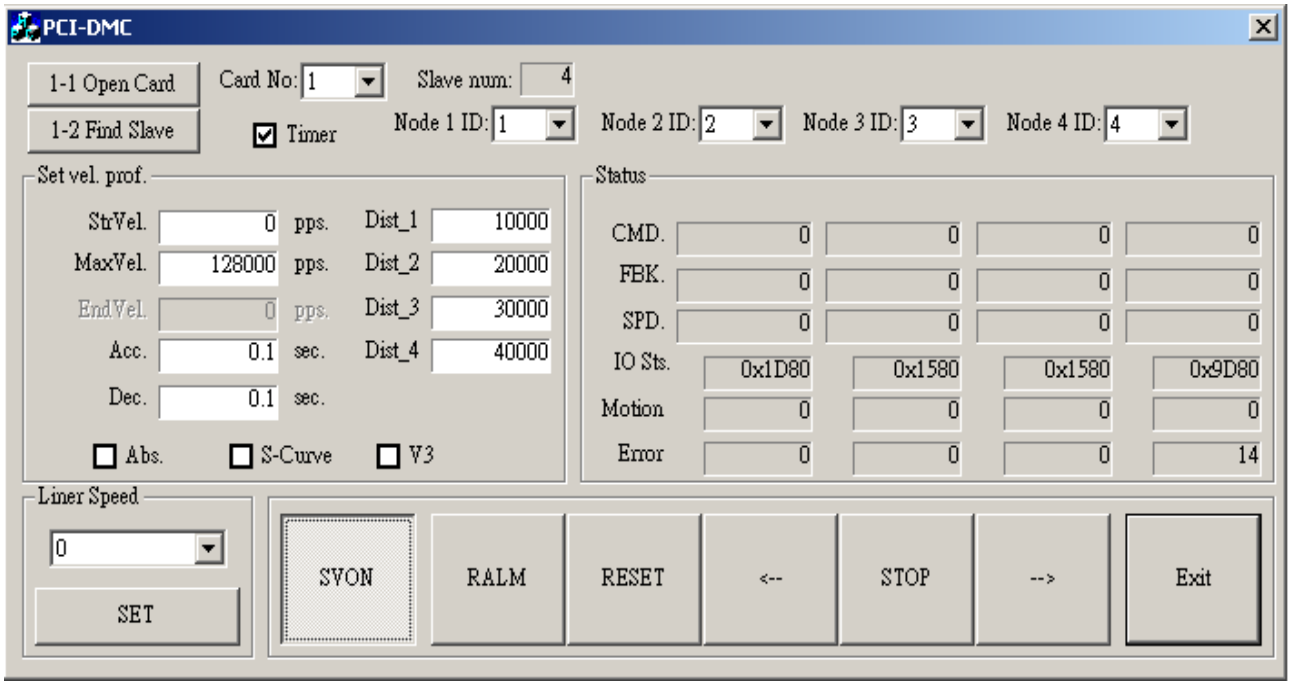


圖 3.100

1) 介面卡開啟與初始化

按壓圖 3.100 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.100 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

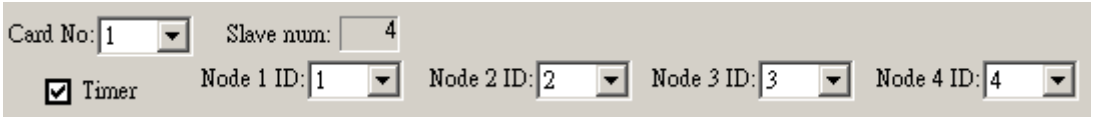


圖 3.101

勾選「Timer」核選方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

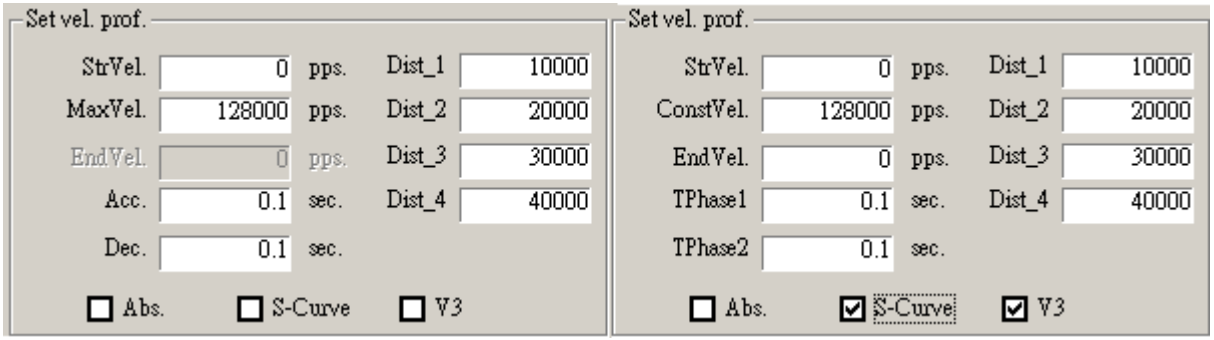


圖 3.102

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

若勾選 V3，則會出現圖 3.102 右部的圖示選項。
StrVel 項目 ：初始速度。API 函數的引數變數 “StrVel”。
ConstVel 項目：常態速度。API 函數的引數變數 “ConstVel”。
EndVel 項目 ：運動結束速度。API 函數的引數變數 “EndVel”。
TPhase1 項目：StrVel 到 ConstVel 所花費時間。API 函數的引數變數 “TPhase1”。
TPhase2 項目：ConstVel 到 EndVel 所花費的時間。API 函數的引數變數 “TPhase2”。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.103

按壓圖 3.103 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 開始進行多軸(未勾選 V3)運動控制

按壓圖 3.103 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_multi_axes_move(gDMCCardNo, AxisNum, NodeIDArry, SlotIDArry,
DistArry, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
// 設置一軸(含)以上的運動控制。
```

6) 開始進行多軸(勾選 V3)運動控制

按壓圖 3.103 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_v3_multi_axes(gDMCCardNo, AxisNum, NodeIDArry,
SlotIDArry, DistArry, StrVel, ConstVel, EndVel, Tacc, Tdec, m_curve, m_r_a);
// 設置增加 EndVel 一軸(含)以上的運動控制。
```

7) 設定 Liner Speed Master 功能



圖 3.104

Mode 項目：模式設定(0：向量速度，1：距離最遠軸的最大分量速度)。
API 函數的引數“Mode”。

按壓圖 3.104 中的「SET」鍵開始執行以下程序：

```
rt = _DMC_01_liner_speed_master(gDMCCardNo, NodeID, SlotID , Mode);
// 0：向量速度  1：距離最遠軸的最大分量速度
```

8) 監控顯示

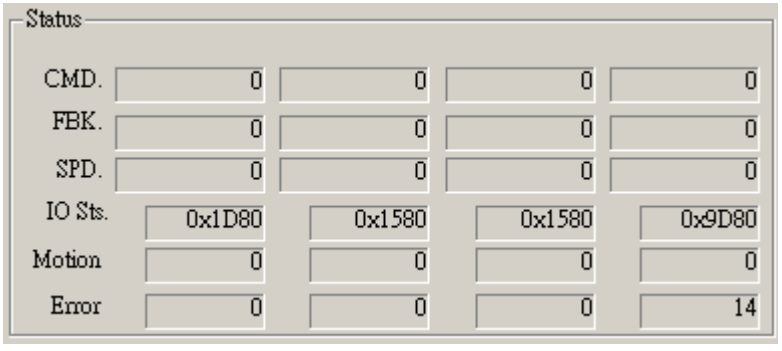


圖 3.105

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

9) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.103 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.103 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.105 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.103 中的「RALM」鍵開始執行 Alm_Code 重置動作。

10) 離開程序

按壓圖 3.103 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.11　同步運動控制

3.11.1　函式列表

表 3.11

函式名稱
_DMC_01_sync_move
_DMC_01_sync_move_config

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.11.2 流程圖

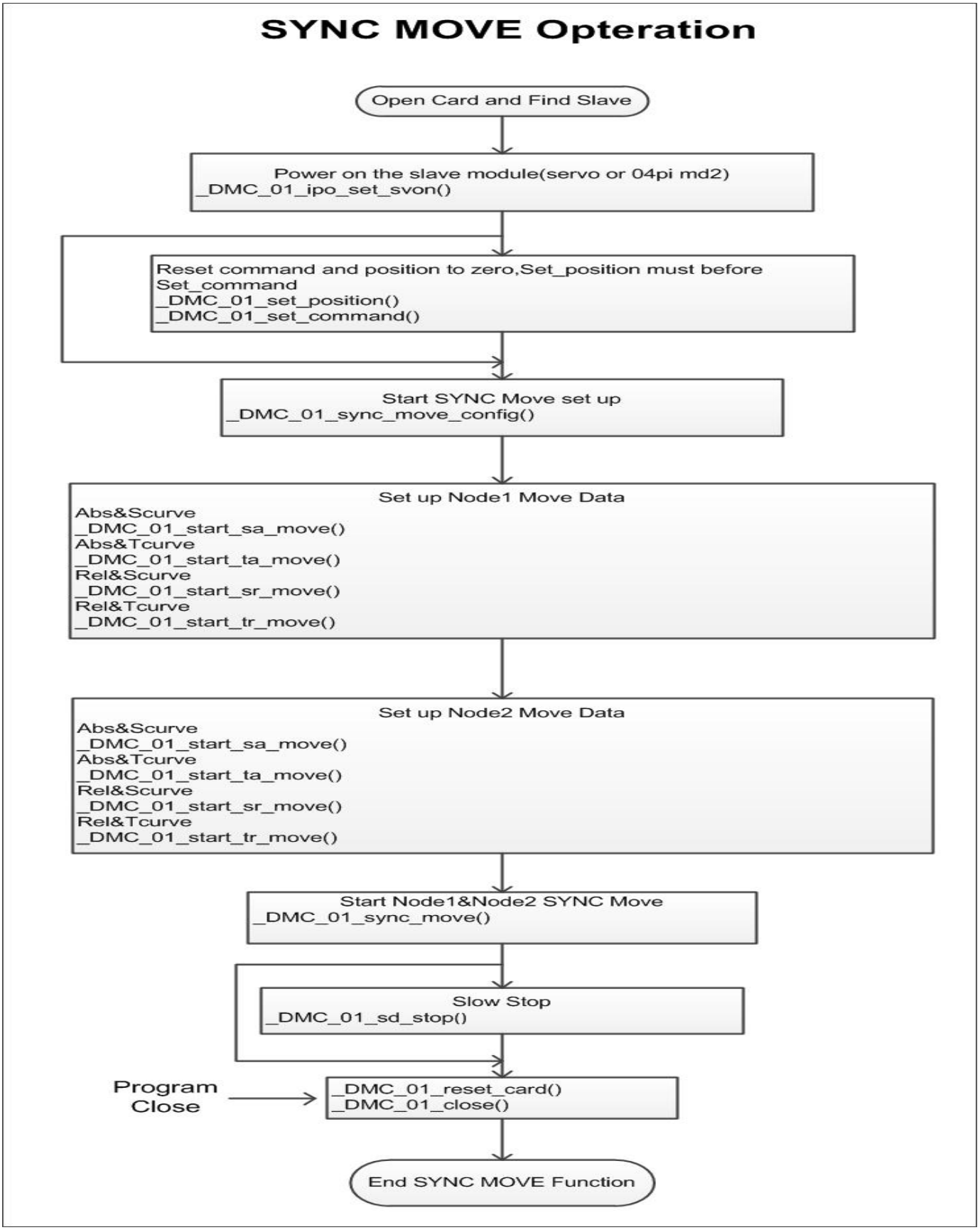


圖 3.110

3.11.3　應用範例

程式外觀

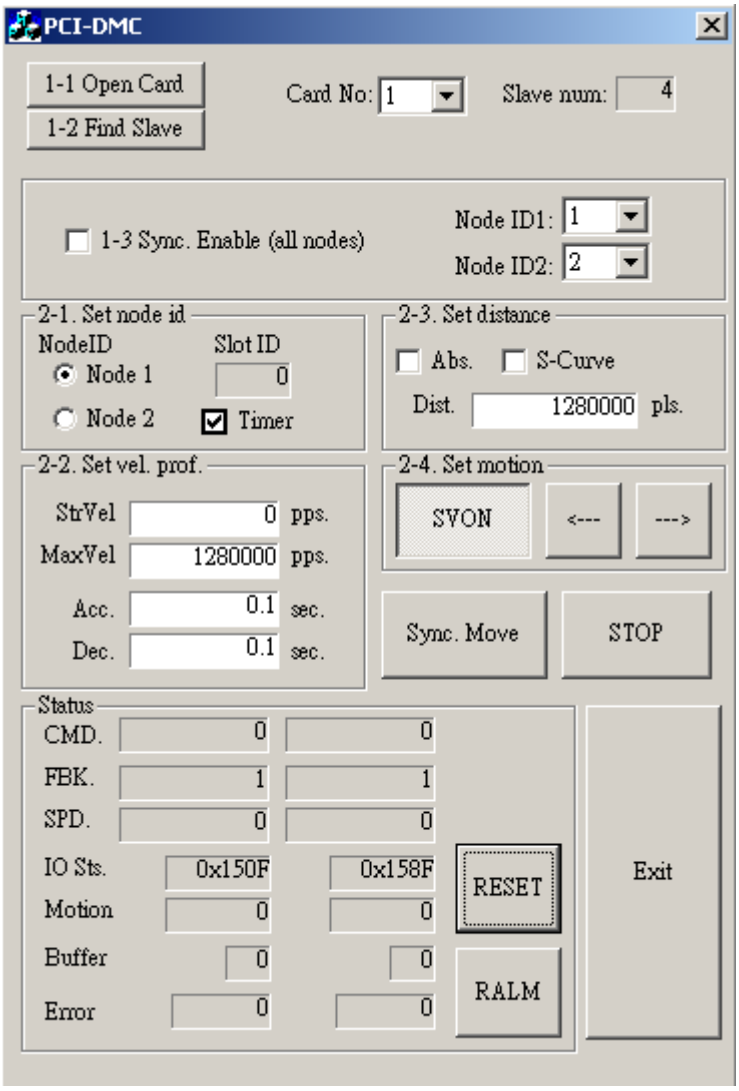


圖 3.111

1) 介面卡開啟與初始化

按壓圖 3. 111 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3. 111 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

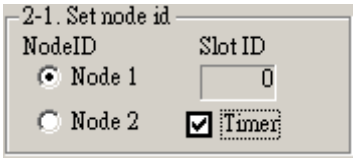


圖 3.112

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

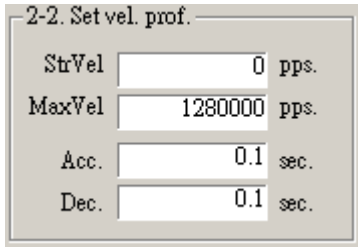


圖 3.113

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

4) 選擇運動模式與設置運動行程

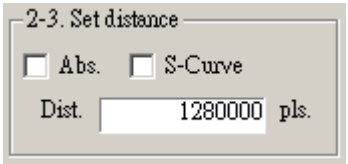


圖 3.114

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

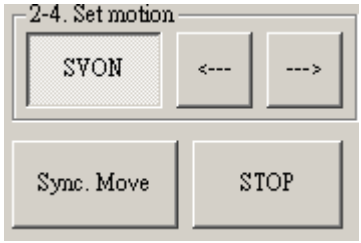


圖 3.115

按壓圖 3. 115 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON.
```

6) 選擇是否啟動同步運動控制



圖 3.116

Sync. Enable 核取方塊：啟動馬達運動同步與否。

勾選圖 3. 116 中的 **Sync. Enable**，您可以執行以下程序啟用同步運動：

```
rt = _DMC_01_sync_move_config(gDMCCardNo, gpNodeID[i], SlotID, enable);
// 若 enable 引數值為 1，啟動同步運動
```

7) 按壓圖 3. 115 中的「→」或「←」鍵開始執行點對點單軸運動命令。由於您僅設置一軸作運動，此時 motion 仍尚未動作，您必須回到 2) 設置驅動器的 Node ID 的步驟開始設置其它軸的運動命令，待設置完成後，按壓圖 3. 115 中「Sync. Move」鍵開始執行同動命令。

此同動命令如下列函數動作執行：

```
rt = _DMC_01_sync_move(gDMCCardNo);
// 當同步運動完成，同動的設置即會回復到關閉的狀態。如您欲再次使用同動命令，則必須重新啟動設置同動命令。
```

8) 重置運動位移計數值、Alm_Reset 及監控顯示

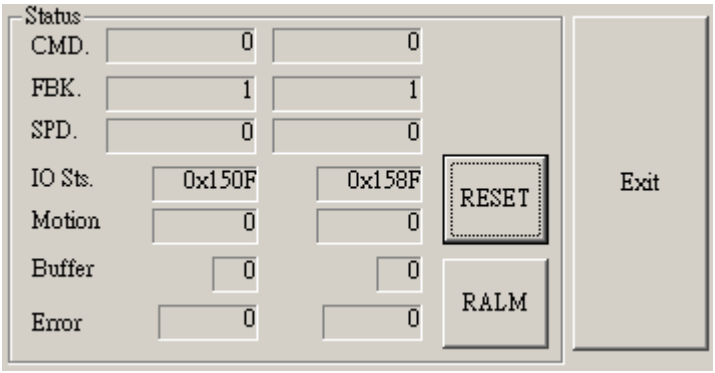


圖 3.117

按壓圖 3.117 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

當圖 3.117 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.117 中的「RALM」鍵開始執行 Alm_Code 重置動作。

9) 停止運動

不論正在執行哪一項運動，當按壓圖 3.115 中的「STOP」鍵開始執行緊急停止：

rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

10) 離開程序

按壓圖 3.117 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.12 Speed Continue

3.12.1 函式列表

表 3.12

函式名稱
_DMC_01_speed_continue
_DMC_01_speed_continue_mode
_DMC_01_speed_continue_ratio
_DMC_01_start_sa_move
_DMC_01_start_ta_move
_DMC_01_start_sr_move
_DMC_01_start_tr_move

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

3.12.2　應用範例

程式外觀

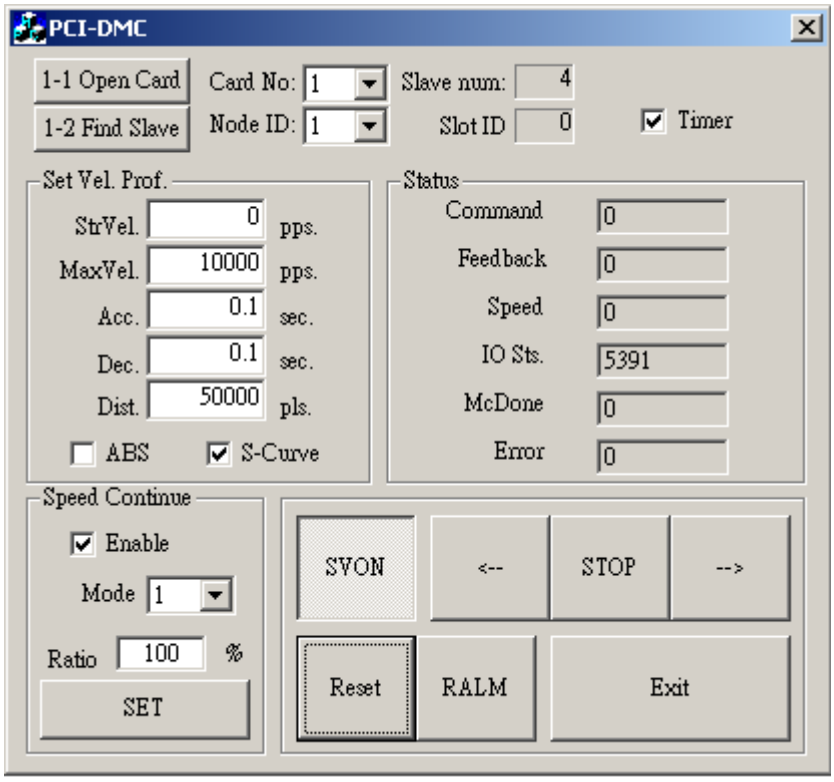


圖 3.120

1) 介面卡開啟與初始化
按壓圖 3. 120 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3. 120 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

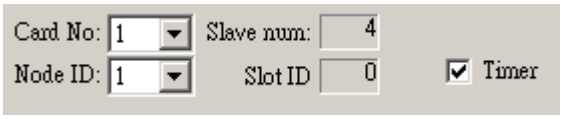


圖 3.121

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

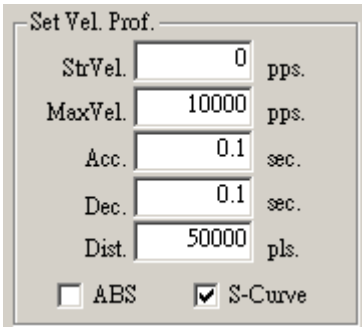


圖 3.122

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：運動行程。API 函數的引數變數 “Distance”。
ABS 核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定 Speed Continue 相關參數

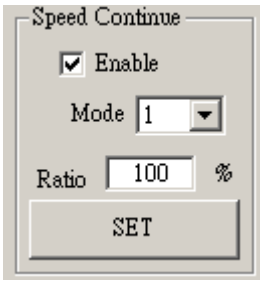


圖 3.123

Enable 核取方塊：若欲啟動 Speed Continue 功能，必須勾選它，勾選執行以下程序：

```
rt = _DMC_01_speed_continue(gDMCCardNo, NodeID, SlotID , enable);
 //0：Disable 1：Enable
```

Mode 項目：設定連續速度模式。API 函數的引數變數 “Mode”。
Ratio 項目：設定連續速度合成百分比。API 函數的引數變數 “Ratio”。
按壓圖 3.123 中的「SET」鍵開始執行以下程序：

```
rt = _DMC_01_speed_continue_mode(gDMCCardNo, NodeID, SlotID , mode);
 //0：等加速模式 1：加、減速模式 2：最大速度模式
rt = _DMC_01_speed_continue_ratio(gDMCCardNo, NodeID, SlotID , ratio);
 //0~100
```

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.124

按壓圖 3.124 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON.
```

6) 按壓圖 3.124 中的「→」或「←」鍵開始執行連續運動並執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位移
```

7) 監控顯示

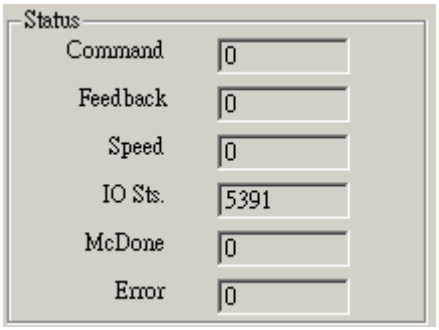


圖 3.125

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.124 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.124 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.125 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.124 中的「RALM」鍵開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.124 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.13 螺旋補間 Helix 應用 Normal_Follow

3.13.1 函式列表

表 3.13

函式名稱
_DMC_01_start_tr_heli_xy
_DMC_01_start_v3_msbrline_move
_DMC_01_start_v3_multi_axes

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.13.2 應用範例

程式外觀

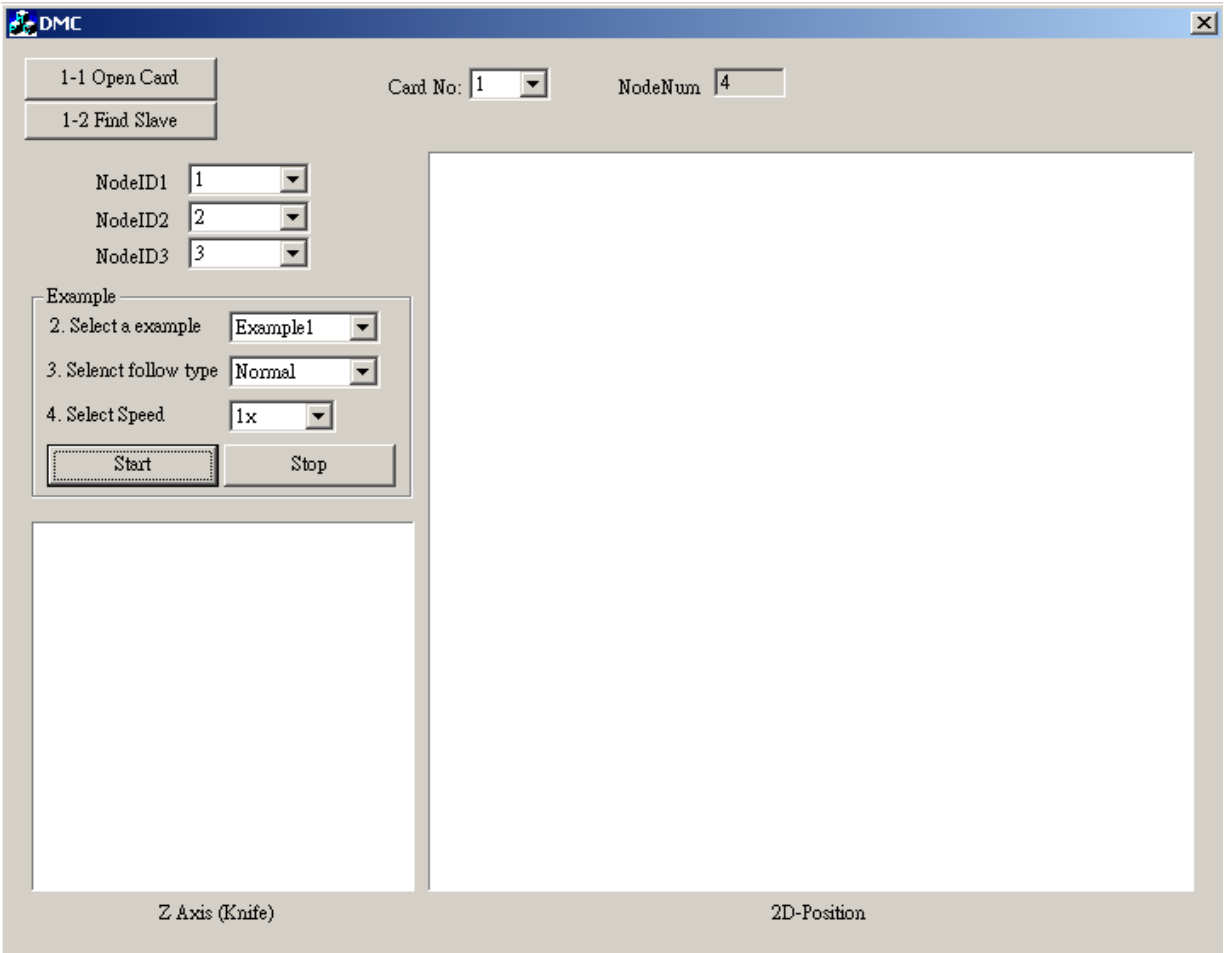


圖 3.130

1) 介面卡開啟與初始化

按壓圖 3.130 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.130 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 卡號、站數顯示:

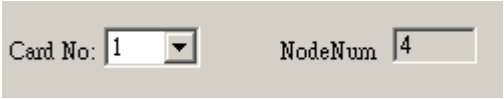


圖 3.131

Card num 項目：顯示卡片張數。

Slave num 項目：顯示站數。

3) Example 參數設定及操作:

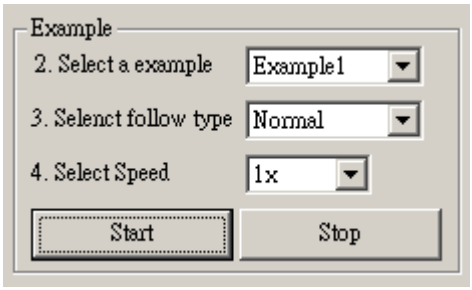


圖 3.132

Select a example 項目：有 Example1 、Example2 及 MSBRline 可供選擇。

Select follow type 項目：設有 Normal 及 Tangent 可供選擇。

Speed 項目：選擇預定速度倍率。

Start 項目：當點選"Start"鍵將執行以下設定。

_DMC_01_start_tr_heli_xy(CardNo,NodeIDArry,SlotIDArry,CenterX,CenterY, Depth,Pitch, Dir, StartVel, MaxVel, Tacc, Tdec);

參數：

CardNo：卡號。

NodeIDArry：3 軸站號的陣列。

SlotIDArry：3 軸號的陣列。

CenterX：X,Y 軸圓弧動作時的圓心(X 座標)。

CenterY：X,Y 軸圓弧動作時的圓心(Y 座標)。

Depth：Z 軸移動的距離(等於圓弧的角度除以 360 再乘上 Pitch)。

Pitch：Z 軸旋轉一圈的距離淨量(恆為正數)。

Dir：圓弧動作的方向，0 為順時針，1 為逆時針。

備註：若是 Z 軸走 Pitch 的距離為順時針旋轉一圈，則如果 Dir = 0，Depth 也為正數，Dir = 1 則 Depth 為負數；若 Z 軸走 pitch 的距離為逆時針旋轉一圈，則反之。

當選擇 Example1 及 Normal 時，執行結果如圖 3.133:

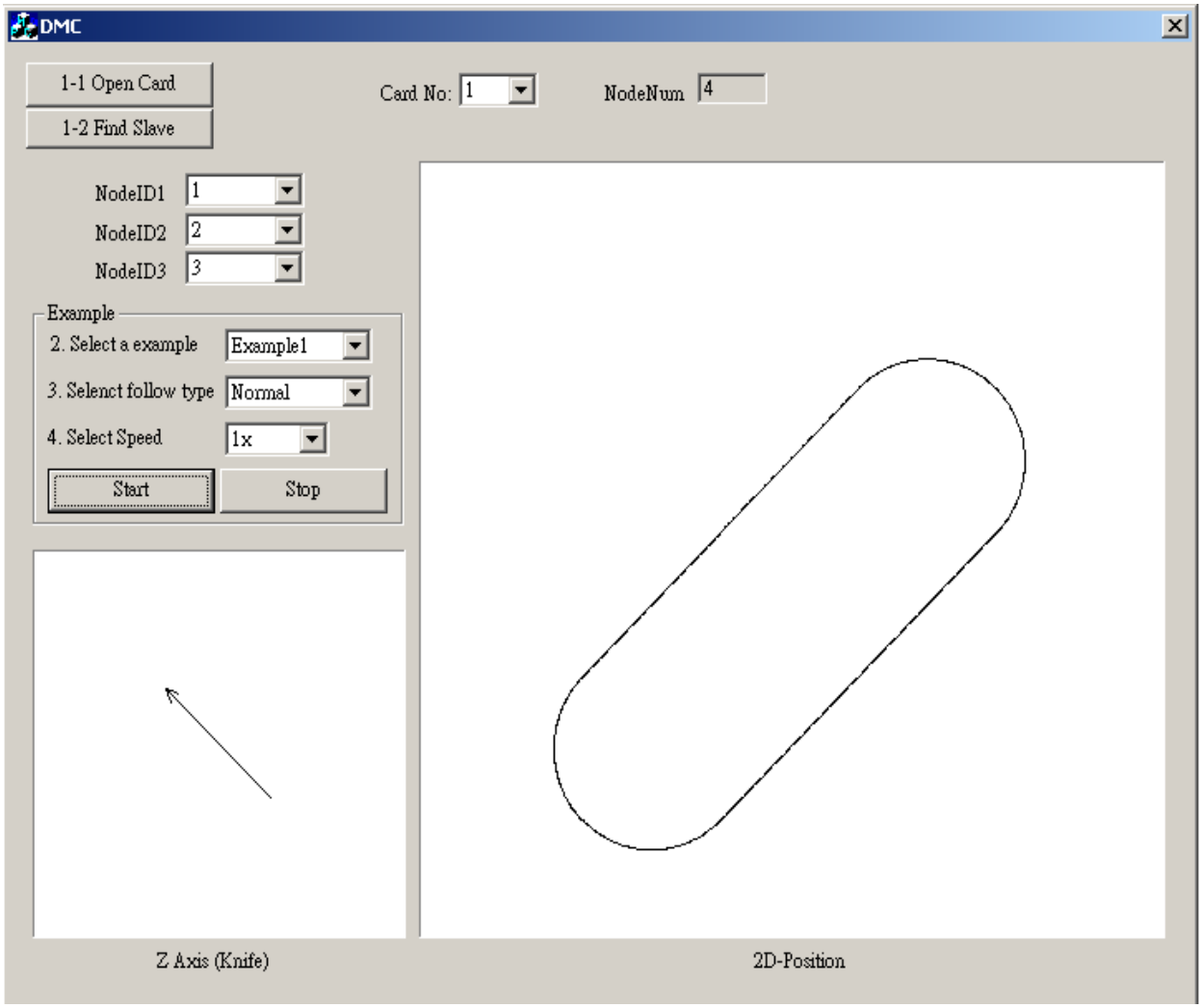


圖 3.133

當選擇 Example2 及 Tangent 時，執行結果如圖 3.134:

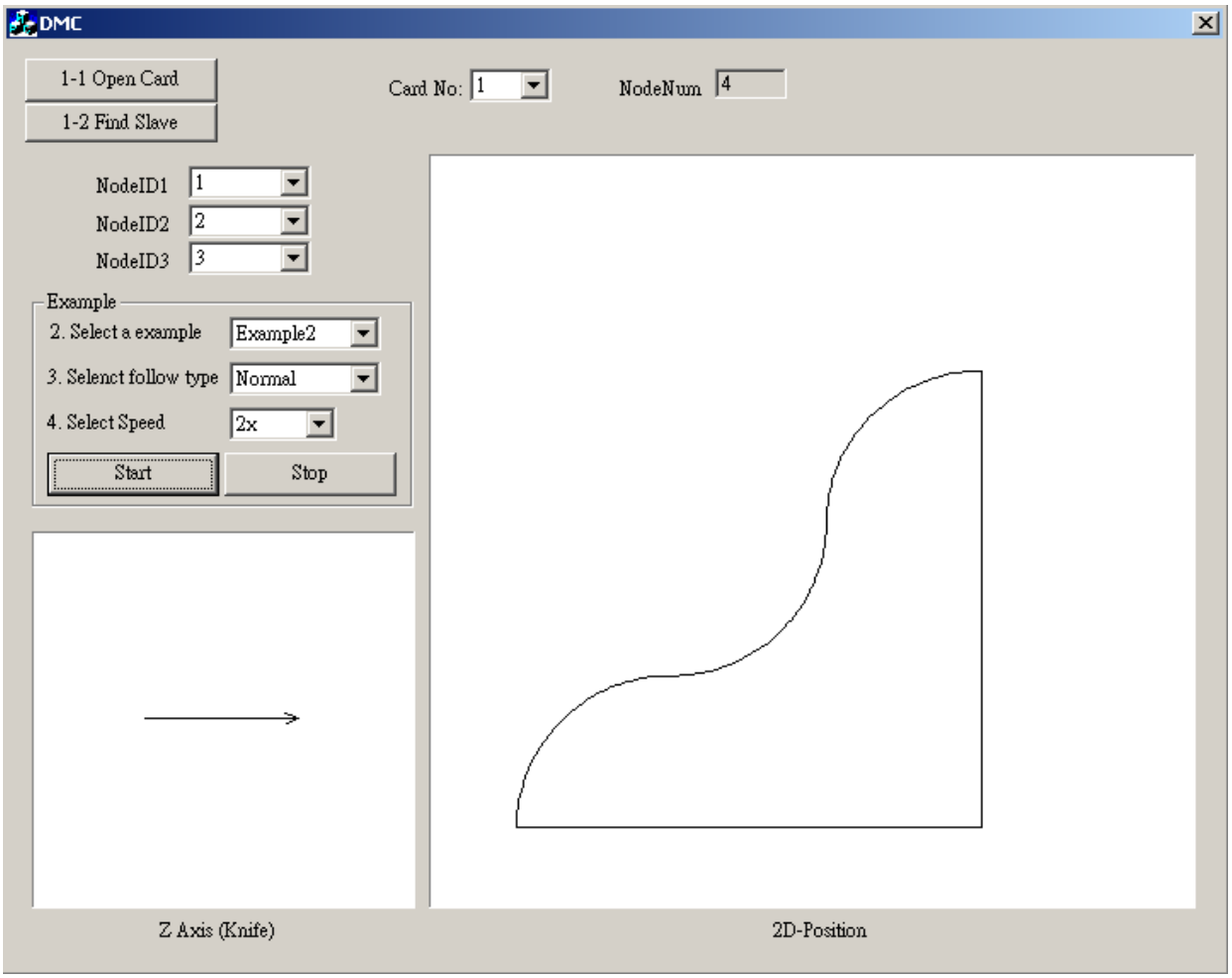


圖 3.134

當選擇 MSBRline 及 Normal 時，執行結果如圖 3.135:

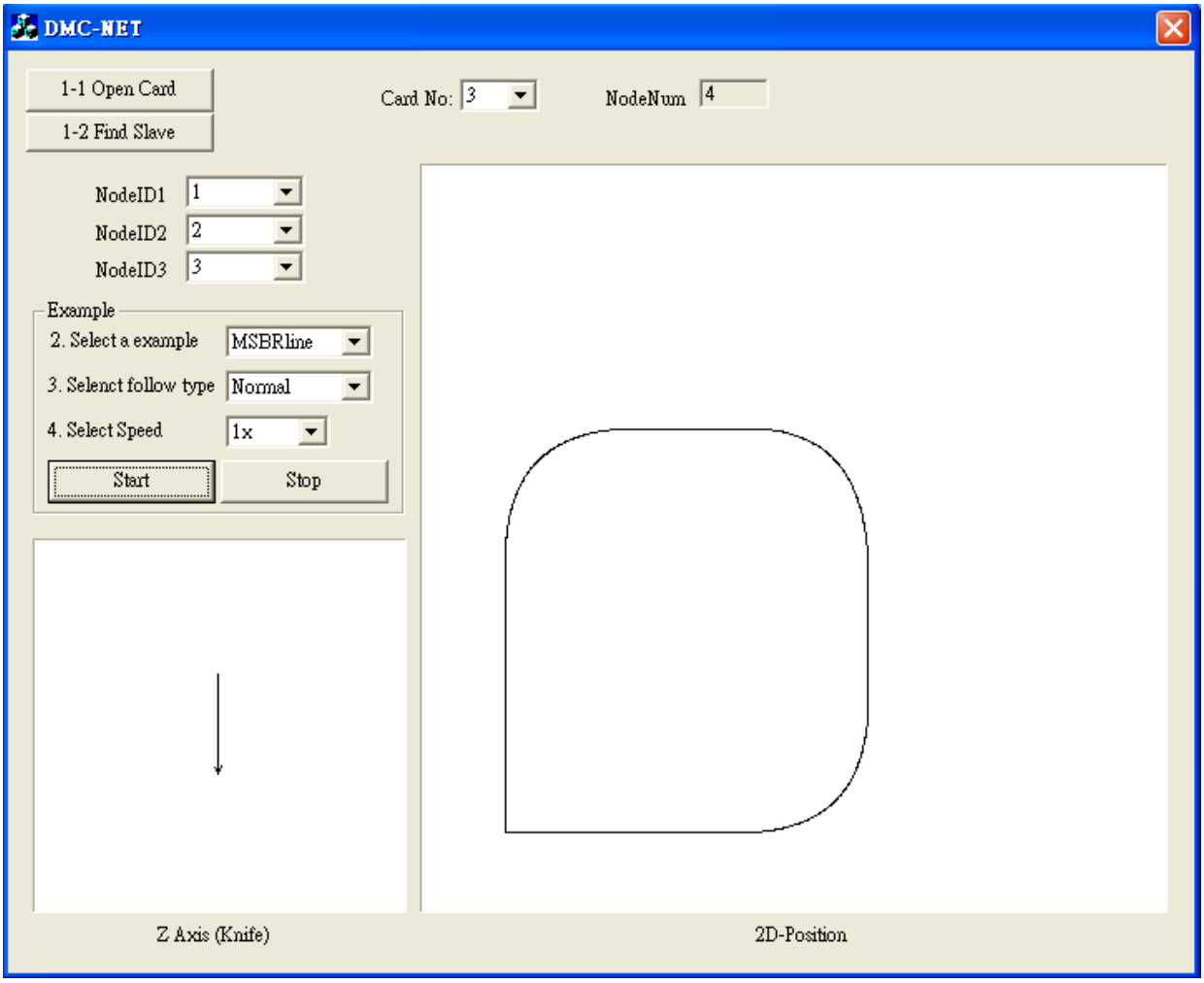


圖 3.135

例子 1：此例是透過 X 軸與 Y 軸，使得工具繞圓角及直線移動，並且讓 Z 軸的工具始終與圖形外緣垂直(Normal)或切線(Tangent)如圖 3.136。
(以下解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

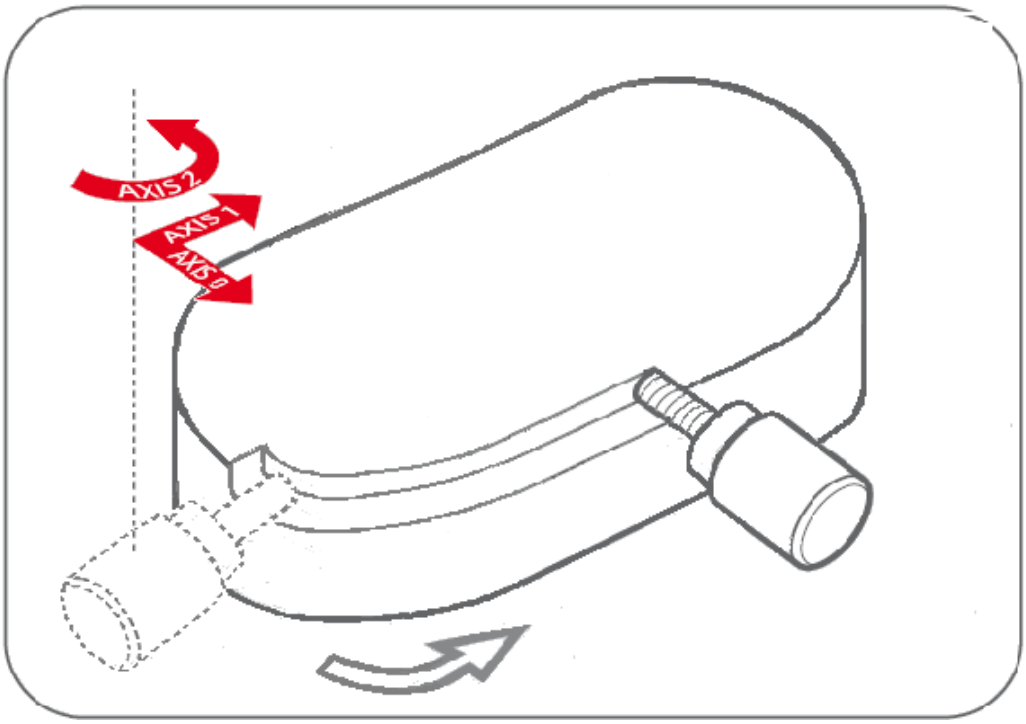


圖 3.136

```
/* MotionStep1.為將三軸平移至加工起始點 */

_DMC_01_start_sa_move(CardNo, NodeID3[0], SlotID3[0], 0, 0, MaxVel, 0.1, 0.1);
_DMC_01_start_sa_move(CardNo, NodeID3[1], SlotID3[1], 0, 0, MaxVel, 0.1, 0.1);
_DMC_01_start_sa_move(CardNo, NodeID3[2], SlotID3[2], -13500, 0, MaxVel, 0.1, 0.1);

/* MotionStep2.等待 Motion Done，並確定 Step1 完成 */
for(i=0;i<3;i++)
{
      rt[i] = _DMC_01_get_command(CardNo, NodeID3[i], SlotID3[i], &Cmd[i]);
}
if((rt[0] + rt[1] + rt[2] == 0) && (Cmd[0] == 0) && (Cmd[1] == 0) && (Cmd[2] == -13500))
{
      MotionStep = 3;
}

/* MotionStep3. 開始加工 */
_DMC_01_start_tr_move_xy(CardNo, NodeID2, SlotID2, 40000, 40000, MaxVel/2, MaxVel, 0.1, 0.1);
_DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, -10000, 10000, -Pitch*(180.0/360),
                                 Pitch,1, MaxVel/2, MaxVel, 0.1, 0.1);
_DMC_01_start_tr_move_xy(CardNo, NodeID2, SlotID2, -40000, -40000, MaxVel/2, MaxVel,
                                 0.1,0.1);
_DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, 10000, -10000, - Pitch *(180.0/360), Pitch, 1,
                                 MaxVel/2, MaxVel,0.1,0.1);
```

例子 2：此例是透過 X 軸與 Y 軸，使得工具沿圓角及直線移動，並且讓 Z 軸的工具始終與圖形保持垂直(Normal)或切線(Tangent)。如圖 3.137
(下例解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

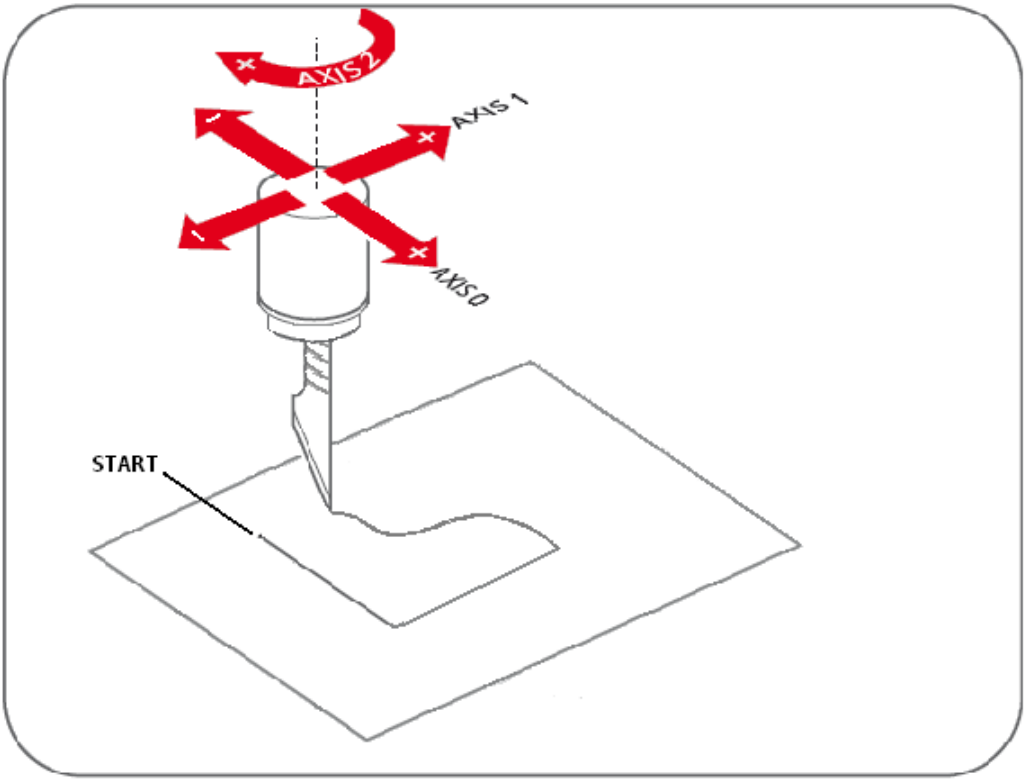


圖 3.137

```
 /* MotionStep1.為將三軸平移至加工起始點 */
_DMC_01_start_sa_move(CardNo, NodeID3[0], SlotID3[0], 0, 0, MaxVel, 0.1, 0.1);
_DMC_01_start_sa_move(CardNo, NodeID3[1], SlotID3[1], 0, 0, MaxVel, 0.1, 0.1);
_DMC_01_start_sa_move(CardNo, NodeID3[2], SlotID3[2], 0, 0, MaxVel, 0.1, 0.1);

/* MotionStep2.等待 Motion Done，並確定 Step1 完成 */
for(i=0;i<3;i++)
{
      rt[i] = _DMC_01_get_command(CardNo, NodeID3[i], SlotID3[i], &Cmd[i]);
}

if((rt[0] + rt[1] + rt[2] == 0) && (Cmd[0] == 0) && (Cmd[1] == 0) && (Cmd[2] == 0))
{
      MotionStep = 3;
}
```

```
/* MotionStep3. 開始加工第一步(直線) */
_DMC_01_start_ta_move_xy(CardNo, NodeID2, SlotID2, 30000, 0, MaxVel/2, MaxVel, 0.1, 0.1);

/* MotionStep4. 等候加工第一步執行結束 */

rt[0] = _DMC_01_get_command(CardNo, NodeID3[0], SlotID3[0], &Cmd[0]);
rt[1] = _DMC_01_get_command(CardNo, NodeID3[1], SlotID3[1], &Cmd[1]);
if ((rt[0] + rt[1] == 0) && (Cmd[0] == 30000) && (Cmd[1] == 0))
{
    MotionStep = 5;
}

/* MotionStep5. 旋轉 Z 軸使工具保持切線(或法線) */
rt[0] = _DMC_01_start_tr_move(CardNo, NodeID3[2], SlotID3[2], -Pitch*(90.0/360.0),
                              MaxVel/2, MaxVel, 0.1, 0.1);

/* MotionStep6. 等候 Z 軸工具定位(動作結束) */
rt[2] = _DMC_01_get_command(CardNo, NodeID3[2], SlotID3[2], &Cmd[2]);
if ((rt[2] == 0) && (Cmd[2] == TempSaveData))
{
    MotionStep = 7;
}

/* MotionStep7. 開始加工第二步(直線) */
_DMC_01_start_ta_move_xy(CardNo, NodeID2, SlotID2, 30000, 30000, MaxVel/2, MaxVel,
                         0.1, 0.1);

/* MotionStep8. 等候加工第二步執行結束 */
rt[0] = _DMC_01_get_command(CardNo, NodeID3[0], SlotID3[0], &Cmd[0]);
rt[1] = _DMC_01_get_command(CardNo, NodeID3[1], SlotID3[1], &Cmd[1]);
if ((rt[0] + rt[1] == 0) && (Cmd[0] == 30000) && (Cmd[1] == 30000))
{
    MotionStep = 9;
}

/* MotionStep9. 旋轉 Z 軸使工具保持切線(或法線) */
_DMC_01_start_tr_move(CardNo, NodeID3[2], SlotID3[2], -Pitch*(90.0/360.0), MaxVel/2,
                      MaxVel, 0.1, 0.1);
```

```
/* MotionStep10.  等候 Z 軸工具定位(動作結束) */
rt[2] = _DMC_01_get_command(CardNo, NodeID3[2], SlotID3[2], &Cmd[2]);
if ((rt[2] == 0) && (Cmd[2] == TempSaveData))
{
      MotionStep = 11;
}

/* MotionStep11.  開始加工(連續圓弧) */
_DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, 0, -10000, -Pitch*(90.0/360.0), Pitch,
                                    1, MaxVel/2, MaxVel, 0.1, 0.1);
_DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, -10000, 0, Pitch *(90.0/360.0), Pitch,
                                    0, MaxVel/2, MaxVel, 0.1, 0.1);
_DMC_01_start_tr_heli_xy(CardNo, NodeID3, SlotID3, 0, -10000, - Pitch *(90.0/360.0),
                                    Pitch, 1, MaxVel/2, MaxVel, 0.1, 0.1)
```

例子 3：此例是透過 X 軸與 Y 軸，使用 Rline 運動指令下達轉折 90˚ 路徑座標。透過調整 Parameter 參數來決定轉折的平滑程度。Parameter 越大，移動路徑中距離轉折點座標的最小距離越遠。並透過將起始速度與上一筆運動的結束速度相同，來達到運動指令銜接時向量速度連續的效果。

(下例解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

```
/* MotionStep20. 開始加工(連續圓弧) */

I32 Table1[7][12]={
{40000,      0,        0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{40000, 20000, -9000, 0, 0, 0, 0, 0, 0 ,0, 0, 0},
{40000, 40000, -9000, 0, 0, 0, 0, 0, 0, 0, 0, 0},
{20000, 40000,-18000, 0, 0, 0, 0, 0, 0, 0, 0,0},
{00000, 40000,-18000, 0, 0, 0, 0, 0, 0, 0, 0,0},
{00000, 20000,-27000, 0, 0, 0, 0, 0, 0, 0, 0,0},
{00000, 00000,-27000, 0, 0, 0, 0, 0, 0, 0, 0,0}
};
rt[0] = _DMC_01_start_v3_msbrline_move(gDMCCardNo, 3, NodeID3, SlotID3, 0, Table1[0], Table1[1], 0,
                                       Parameter, 1, 0, 0, 0,    MaxVel, MaxVel, 1, 0, 2, 1);
rt[0] = _DMC_01_start_v3_msbrline_move(gDMCCardNo, 3,    NodeID3, SlotID3,    0,    Table1[2], Table1[3], 0,
                                       Parameter,    1, 0, 0, MaxVel, MaxVel, MaxVel, 0, 0, 2, 1);
rt[0] = _DMC_01_start_v3_msbrline_move(gDMCCardNo, 3, NodeID3, SlotID3, 0, Table1[4], Table1[5], 0,
                                       Parameter, 1,    0, 0, MaxVel, MaxVel, MaxVel, 0, 0, 2, 1);
rt[0] = _DMC_01_start_v3_multi_axes(gDMCCardNo, 3, NodeID3, SlotID3, Table1[6], MaxVel, MaxVel, 0, 0, 1, 1,
                                       1);


/* MotionStep21. 等待運動指令結束*/

rt[0] = _DMC_01_get_buffer_length(gDMCCardNo, gpNodeID[0], 0, &bufferLength);
    if (bufferLength==0)
    {
        rt[0] = _DMC_01_motion_done(gDMCCardNo, gpNodeID[0], 0, &i);
        if(i==0)
        {
            MotionStep = 0;
        }
    }
```

3.14　電子凸輪

3.14.1　函式列表

表 3.14

函式名稱
_DMC_01_enable_electcam

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

3.14.2 流程圖

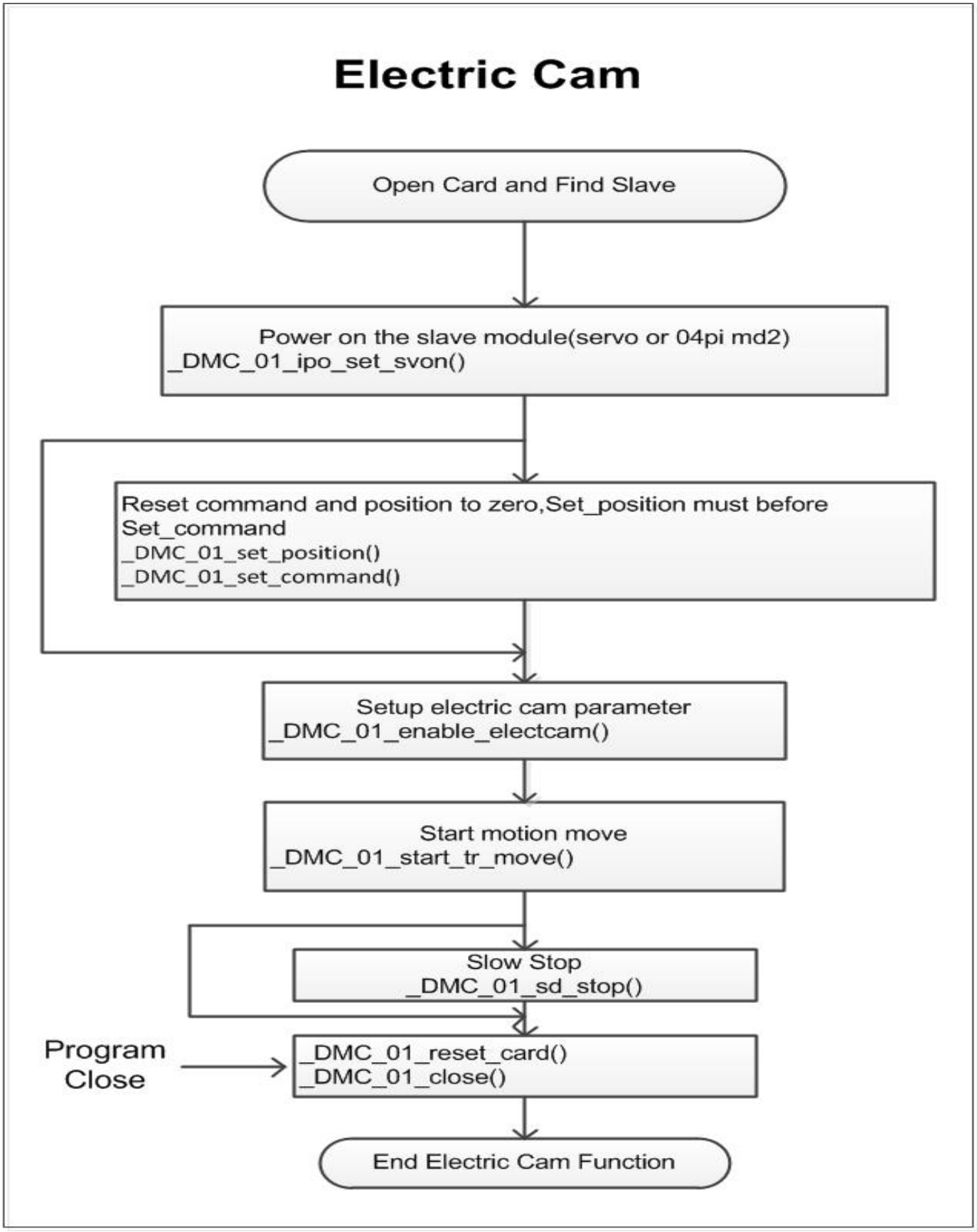


圖 3.140

3.14.3　應用範例

程式外觀

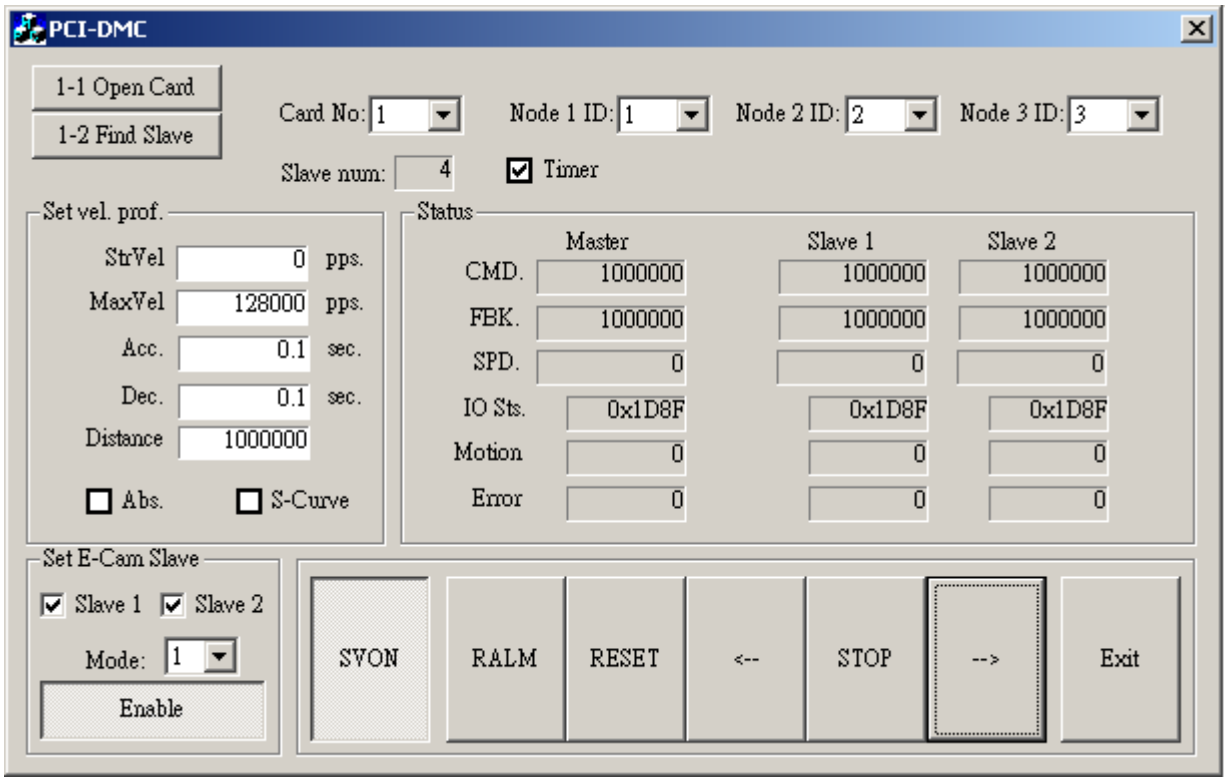


圖 3.141

1) 介面卡開啟與初始化
   按壓圖 3.141 中的「Open card」鍵開始執行介面卡初始化。
   按壓圖 3.141 中的「Find Slave」鍵開始搜尋串接模組。
   關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

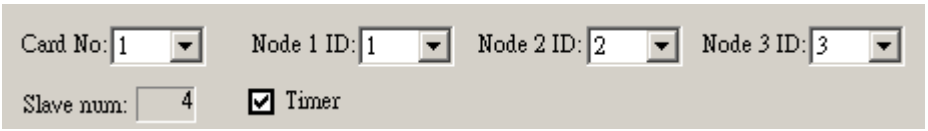


圖 3.142

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

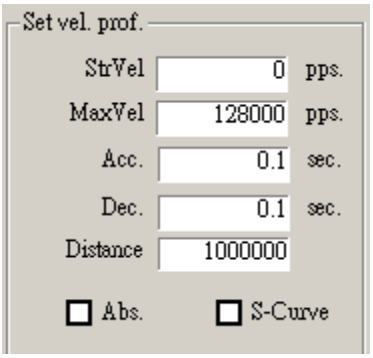


圖 3.143

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Distance 項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定 Electric Cam 相關參數



圖 3.144

Slave1 核取方塊：設定第一跟隨軸資料。API 函數的引數變數 “axisbit”。
Slave2 核取方塊：設定第二跟隨軸資料。API 函數的引數變數 “axisbit”。
Mode 項目：設定跟隨主軸的 Command 或 Position。API 函數的引數變數 “mode”。
按壓圖 3.144 中的「Enable」鍵開始執行以下程序：

```
rt = _DMC_01_enable_electcam(gDMCCardNo, NodeID, SlotID , enable, axisbit,
    mode); //mode 0：跟隨主軸 Command   1：跟隨主軸 Position
```

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.145

按壓圖 3.145 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON.
```

6) 按壓圖 3.145 中的「→」或「←」鍵開始執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位移
```

7) 監控顯示

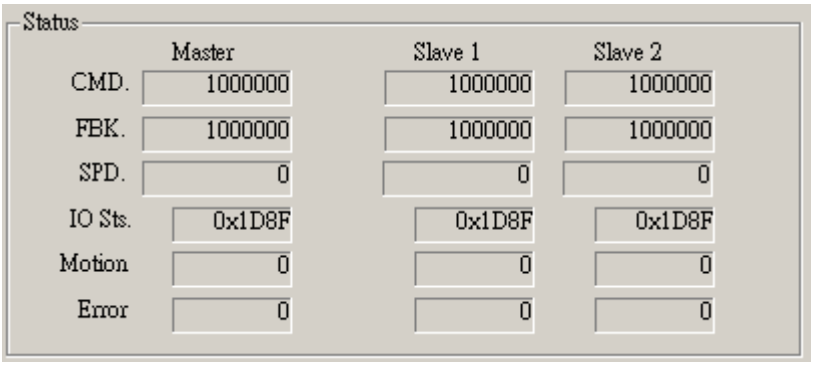


圖 3.146

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.145 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.145 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.146 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.145 中的「RALM」鍵開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.145 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.15 間隙補償

3.15.1 函式列表

表 3.15

函式名稱
_DMC_01_set_cam_enable
_DMC_01_set_cam_mode
_DMC_01_set_cam_pitch
_DMC_01_set_cam_org
_DMC_01_set_cam_table

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

3.15.2　流程圖

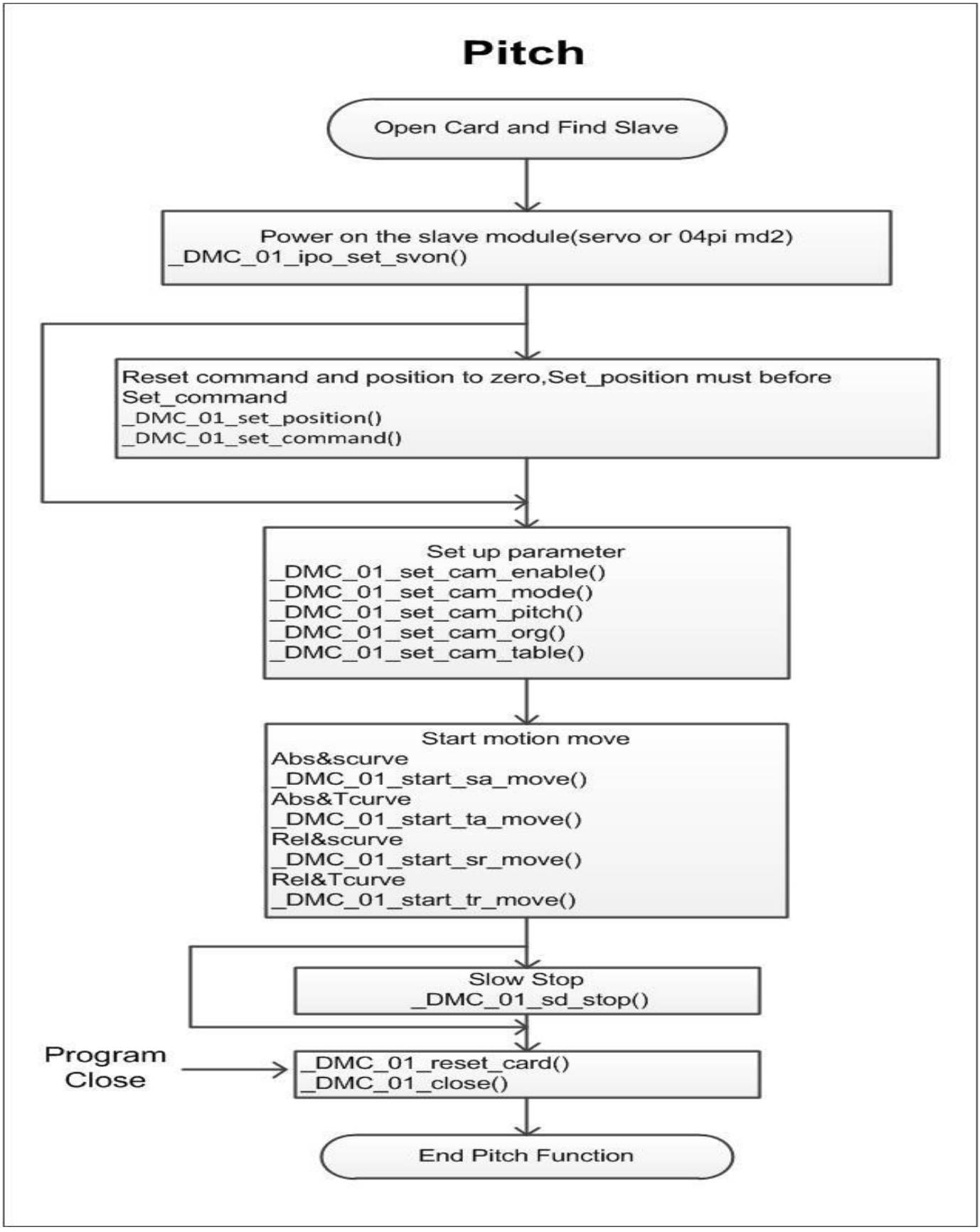


圖 3.150

3.15.3 應用範例

程式外觀

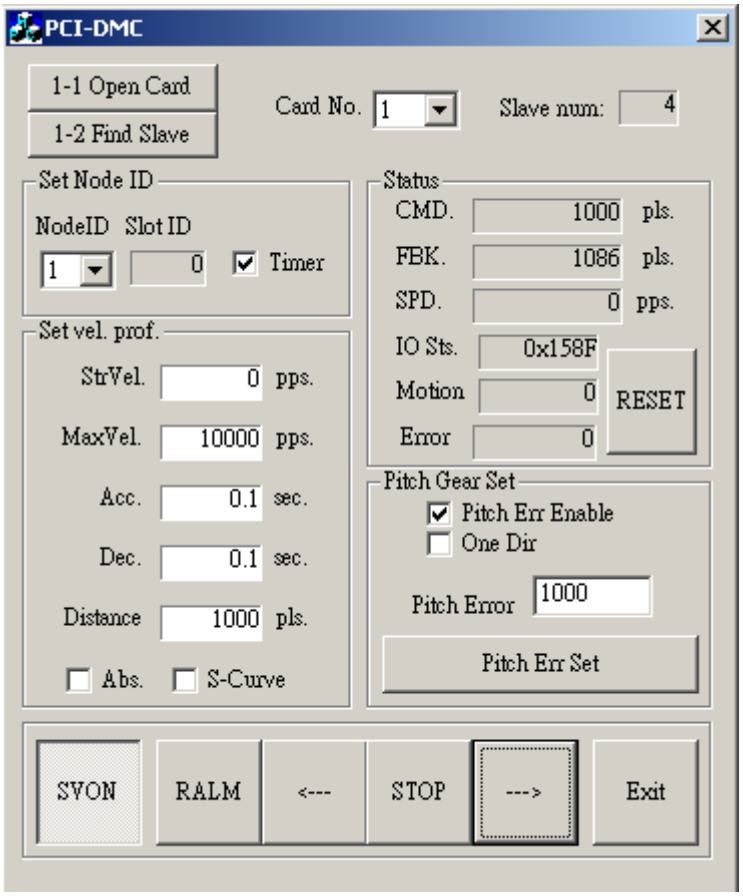


圖 3.151

1) 介面卡開啟與初始化
   按壓圖 3.151 中的「Open card」鍵開始執行介面卡初始化。
   按壓圖 3.151 中的「Find Slave」鍵開始搜尋串接模組。
   關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

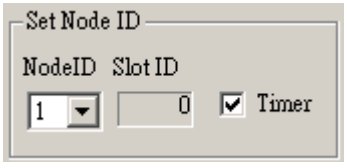


圖 3.152

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

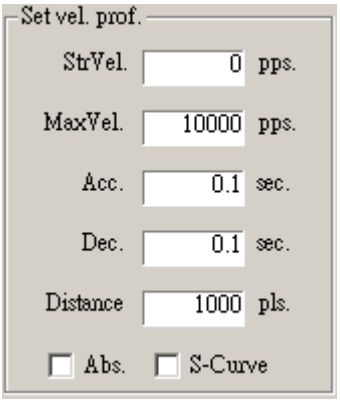


圖 3.153

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Distance 項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.154

按壓圖 3.154 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);

// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 設定動態補償參數



圖 3.155

Pitch Err Enable 項目：啟動/關閉間隙補償功能。API 函數的引數變數 “on_off”。
One Dir 項目：間隙補償模式(單向/雙向)。API 函數的引數變數 “Mode”。
Pitch Err 項目：間隙補償間隔(Pulse)。API 函數的引數變數 “pitch”。
按壓圖 3.155 中的「Pitch Err Set」鍵開始執行以下程序，並且在 C 槽留下記錄**間隙**補償資料表的檔案(PitchErrLog.txt):

```
rt = _DMC_01_set_cam_enable(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 關閉間隙補償；1 –啟動間隙補償
rt = _DMC_01_set_cam_mode(gDMCCardNo, NodeID, SlotID , Mode);
// Mode: 0 – 設定雙向表格，來回用不同表格做間隙補償；
         1 – 設定單向表格，來回都用同一表格做間隙補償。
rt = _DMC_01_set_cam_pitch(gDMCCardNo, NodeID, SlotID , pitch);
// pitch: 設定間隙補償間隔(Pulse)
rt = _DMC_01_set_cam_org(gDMCCardNo, NodeID, SlotID, Dir, OrgPos);
// Dir: 0 – 設定正向間隙補償原點；1 –設定負向間隙補償原點
//OrgPos: 間隙補償原點起始位置
rt = _DMC_01_set_cam_table(gDMCCardNo, NodeID, SlotID ,Dir, table);
// Dir: 0 – 設定正向間隙補償；1 –設定負向間隙補償
// table: 間隙補償參數資料
```

6) 開始進行運動控制

按壓圖 3.154 中的「→」或「←」鍵執行以下程序並根據設定進行間隙補償：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位
```

7) 重置運動計數值及監控顯示

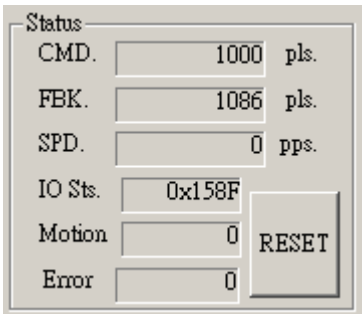


圖 3.156

按壓圖 3.156 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
```

```
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 停止運動及 Alm_Reset

不論正在執行哪一項運動，當按壓圖 3.154 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.156 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.154 中的「RALM」鍵開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.154 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.16　背隙補償

3.16.1　函式列表

表 3.16

函式名稱
_DMC_01_set_backlash_info
_DMC_01_set_backlash

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

3.16.2 流程圖

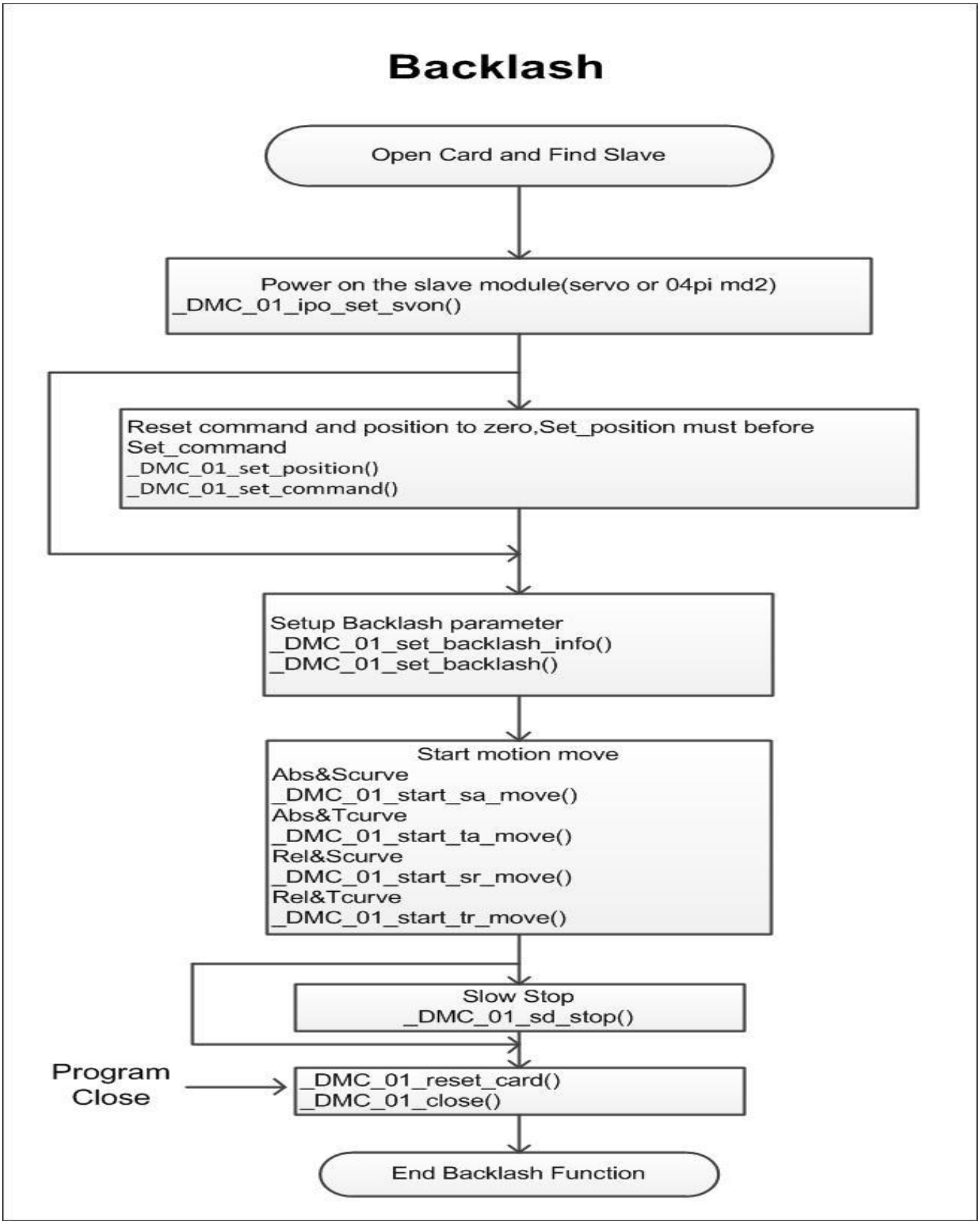


圖 3.160

3.16.3　應用範例

程式外觀

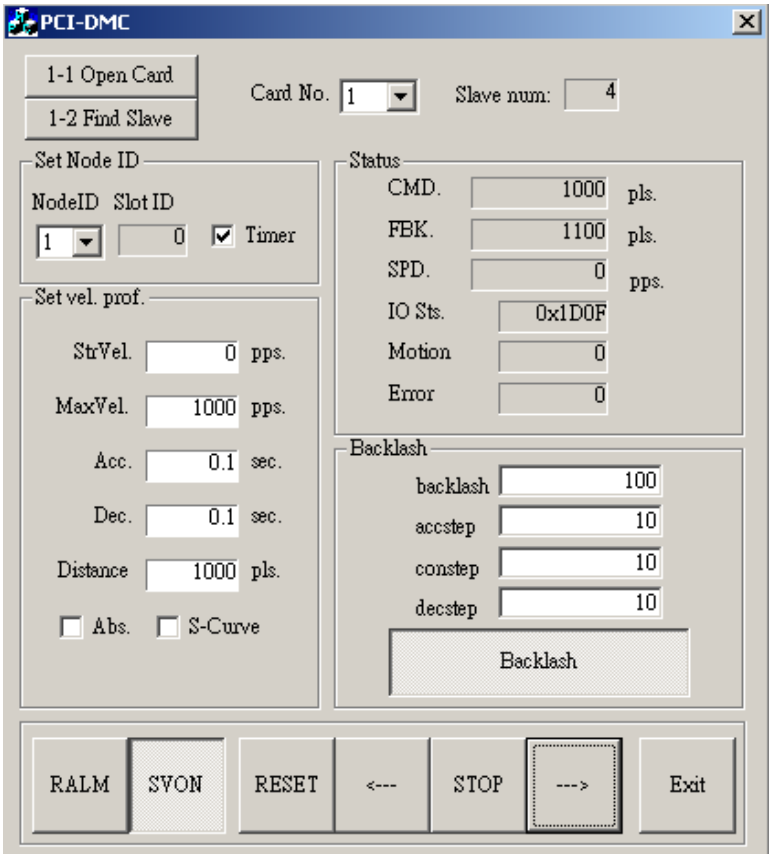


圖 3.161

1) 介面卡開啟與初始化

按壓圖 3.161 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.161 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

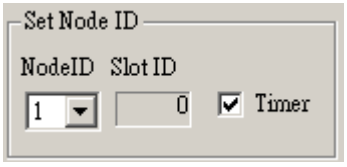


圖 3.162

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

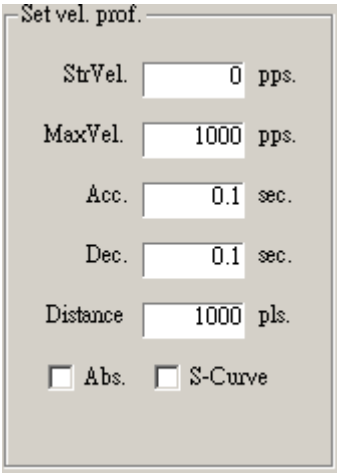


圖 3.163

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Distance 項目：運動行程。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

4) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.164

按壓圖 3.164 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

5) 設定背隙補償參數

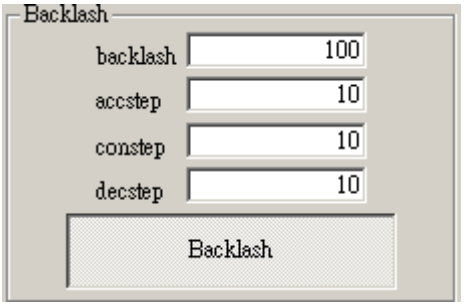


圖 3.165

按壓圖 3.165 中的「BackLash」鍵開始執行以下程序：

```
rt = _DMC_01_set_bachlash_info(gDMCCardNo, NodeID, SlotID , BackLash,
accstep, constep, decstep);
// BackLash:  單位距離所要背隙補償之 Pulse 量
rt = _DMC_01_set_backlash(gDMCCardNo, NodeID, SlotID , enable);
// enable: 0 –  關閉背隙補償功能；1 –啟動背隙補償功能
```

6) 開始進行運動控制，按壓圖 3.164 中的「→」或「←」鍵執行以下程序並依據設定進行背隙補償：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的絕對座標進行運動位移

rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel,
MaxVel, acc, dec);
// 參照 T-curve 速度剖面下的相對座標進行運動位
```

7) 監控顯示

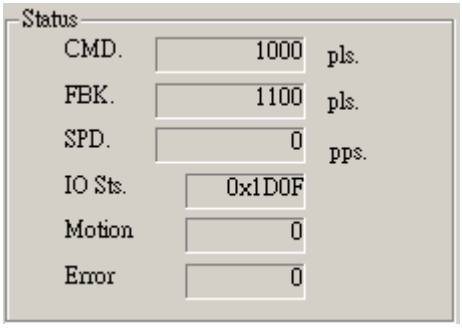


圖 3.166

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

8) 重置運動計數值、停止運動及 Alm_Reset

按壓圖 3.164 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.164 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.166 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.164 中的「RALM」鍵開始執行 Alm_Code 重置動作。

9) 離開程序

按壓圖 3.164 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.17 軟體極限操作

3.17.1 函式列表

表 3.17

函式名稱
_DMC_01_start_sr_move
_DMC_01_start_tr_move
_DMC_01_start_sa_move
_DMC_01_start_ta_move
_DMC_01_set_soft_limit
_DMC_01_enable_soft_limit
_DMC_01_disable_soft_limit
_DMC_01_get_soft_limit_status

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.17.2　流程圖

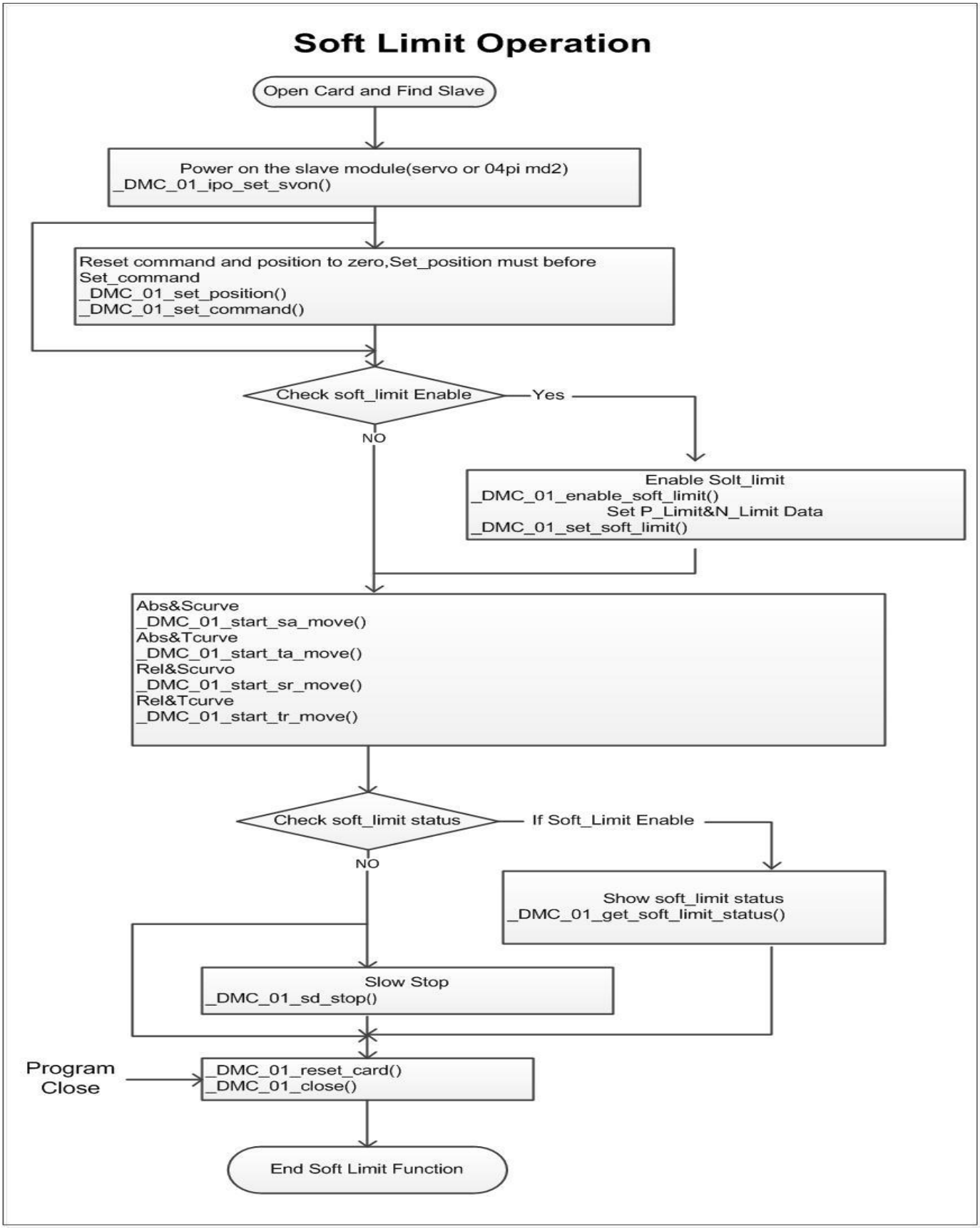


圖 3.170

3.17.3 應用範例

程式外觀

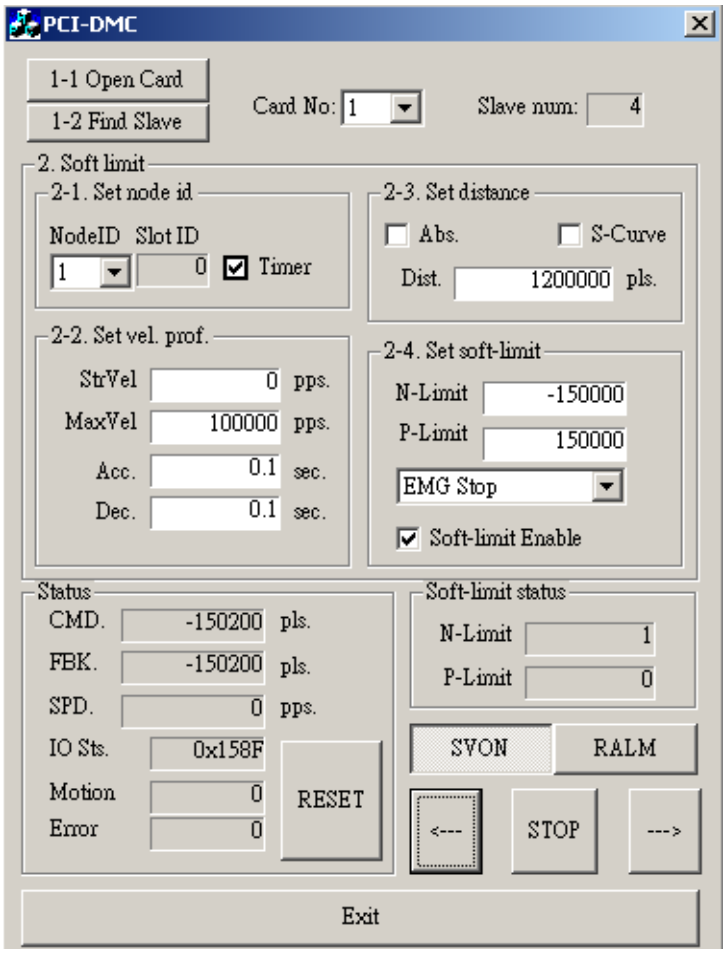


圖 3.171

1) 介面卡開啟與初始化

按壓圖 3.171 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.171 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

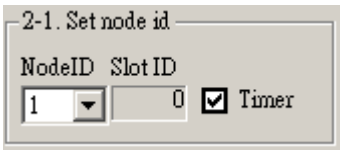


圖 3.172

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

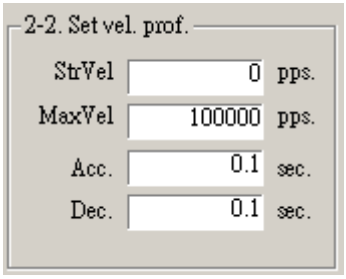


圖 3.173

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。

MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。

Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。

Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

4) 選擇運動模式與設置運動行程

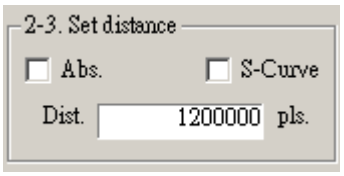


圖 3.174

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。

Dist.項目：運動行程。API 函數的引數變數 “Distance”。

5) 設定正/負極限值、停止運動模式與是否啟用軟體極限

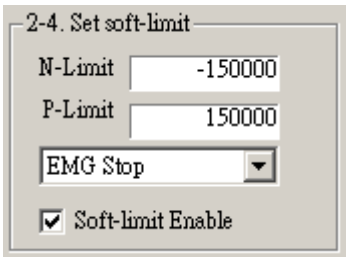


圖 3.175

N-Limit 項目：負極限值。API 函數的引數變數 “NLimit”。
P-Limit 項目：正極限值。API 函數的引數變數 “PLimit”。
Stop mode 下拉選單：停止的方式。API 函數的引數變數 “action”。
Soft-limit 核取方塊：啟動/關閉軟體極限。

6) 若您欲啟用軟體極限功能，您必須將圖 3.175 中的「Soft-limit Enable」核取方塊勾選，
並執行以下程序：

```
/* 啟動設定軟體極限 */
rt = _DMC_01_enable_soft_limit(gDMCCardNo, NodeID, SlotID, action);
// action 引數表示碰觸到極限時，使用的停止方式。值為 1 時，表示立即停止；值為
   2 時，表示使用減速停止
/* 正極限與負極限的設定值 */
rt = _DMC_01_set_soft_limit(gDMCCardNo, NodeID, SlotID, PLimit, NLimit);
// PLimit 引數表示正極限的設定值；NLimit 則為負極限的設定值
```

7) 若您欲關閉軟體極限功能時，您必須使用以下程序關閉軟體極限：

```
rt = _DMC_01_disable_soft_limit(gDMCCardNo, NodeID, SlotID);
```

8) 設定伺服馬達動力 ON/OFF(servo on/servo off)

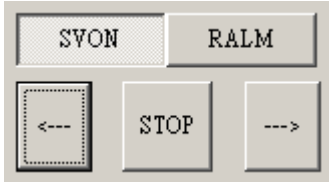


圖 3.176

按壓圖 3.176 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

9) 開始進行運動控制(此處以點對點運動為例)

按壓圖 3.176 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的相對座標進行運動位移
```

10) 若您欲觀察進行點對點運動時，碰觸正/負極限的狀態，您能執行以下程序進行觀察：

```
/* 觀察碰觸正/負極限所回應的狀態 */
rt = _DMC_01_get_soft_limit_status(gDMCCardNo, NodeID, SlotID, &PLimit_sts, &NLimit_sts);
```

如圖 3.177 中的所示，左圖表示此運動期間尚未碰觸到正極限與負極限；中間的圖表示此運動期間有碰觸到正極限；而右圖表示此運動期間有碰觸到負極限。

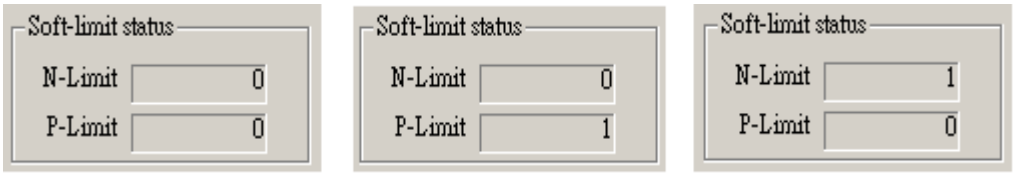


圖 3.177

11) 重置運動計數值及監控顯示

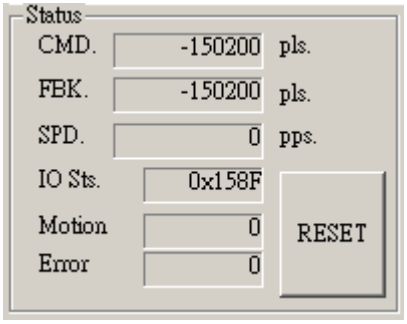


圖 3.178

按壓圖 3.178 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

12) 停止運動及 Alm_Reset

不論正在執行哪一項運動，當按壓圖 3.176 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.178 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.176 中的「RALM」鍵開始執行 Alm_Code 重置動作。

13) 離開程序

按壓圖 3.171 中「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.18　遠端擴充 IO 模組 Input 應用(RM32MN、RM64MN)操作

3.18.1　函式列表

表 3.18

函式名稱
_DMC_01_set_rm_input_filter
_DMC_01_set_rm_input_filter_enable
_DMC_01_get_rm_input_value

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.18.2 應用範例

程式外觀

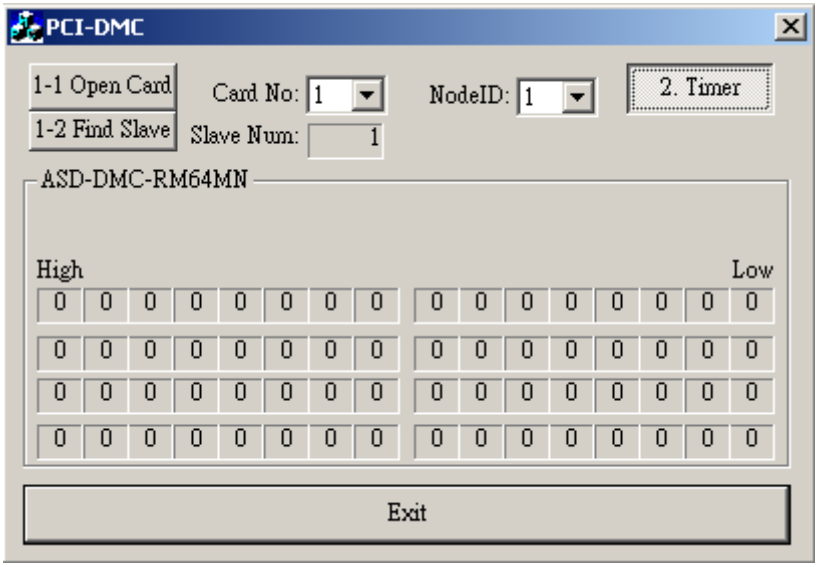


圖 3.180

1) 介面卡開啟與初始化
按壓圖 3. 180 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3. 180 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化介面卡的詳細說明，請參閱第 3.1.2 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

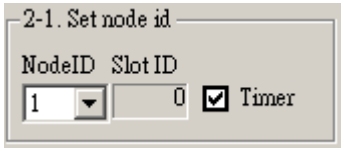


圖 3.181

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) ASD-DMC-RM32MN/ RM64MN 模組初始化參數設定。您必須執行以下程序：
rt = _DMC_01_set_rm_input_filter(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter);
// port 變數表示可設置 remote 模組上的 2 組 Port，當其值為 0，將會操作模組上的 Port 0；值若為 1，表示將操作在 Port 1。 filter 變數是設定軟體濾波器的等級，當您

將值設置為 0 時，軟體濾波時間為 1ms；值為 1 時，濾波時間為 2ms，依此類推。

```
rt = _DMC_01_set_rm_input_filter_enable(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter_enable);
// filter_enable 變數的值範圍為 0~0xFFFF，祺是用來控制 Port 上 0 到 15 的軟體濾波遮罩。
```

4) 數位輸入(DI)

當您欲從數位輸入端取得您在數位輸出模組所送出的資料，您需要使用 ASD-DMC-RM32MN/RM64MN 模組，並執行以下程序：

```
/*  取得數位輸入 Port 0 的值  RM32MN/RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 0, &input_value[0]);
// Input_value[0]變數將會回覆從 Port 0 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 1 的值  RM32MN/RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 1, &input_value[1]);
// Input_value[1]  變數將會回覆從 Port 1 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 2 的值  RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 2, &input_value[1]);
// Input_value[1]  變數將會回覆從 Port 2 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 3 的值  RM64MN */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 3, &input_value[1]);
// Input_value[1]  變數將會回覆從 Port 3 的 bit 0 到  bit 15 所取得的資料
```

如圖 3.182 表示 ASD-DMC-RM64MN 模組上 Port 0、1、2 無取得任何資料；而 Port 3 的 bit 0 為 ON，其餘的 bit 為 OFF。

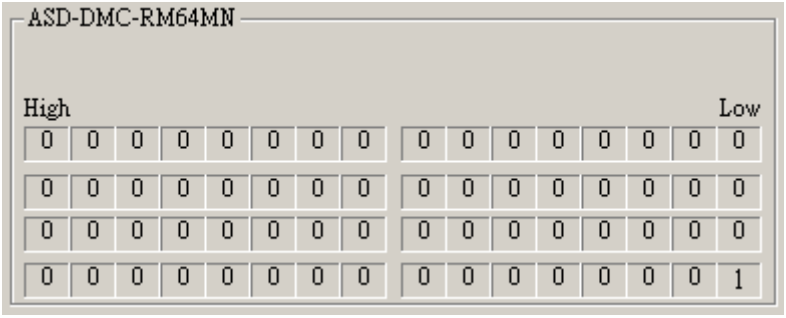


圖 3.182

5) 離開程序

按壓圖 3.180 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.19　遠端擴充 IO 模組 Output 應用(RM32NT、RM64NT)操作

3.19.1　函式列表

表 3.19

函式名稱
_DMC_01_set_rm_output_active
_DMC_01_set_rm_output_value
_DMC_01_set_rm_output_value_error_handle

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.19.2 應用範例

程式外觀

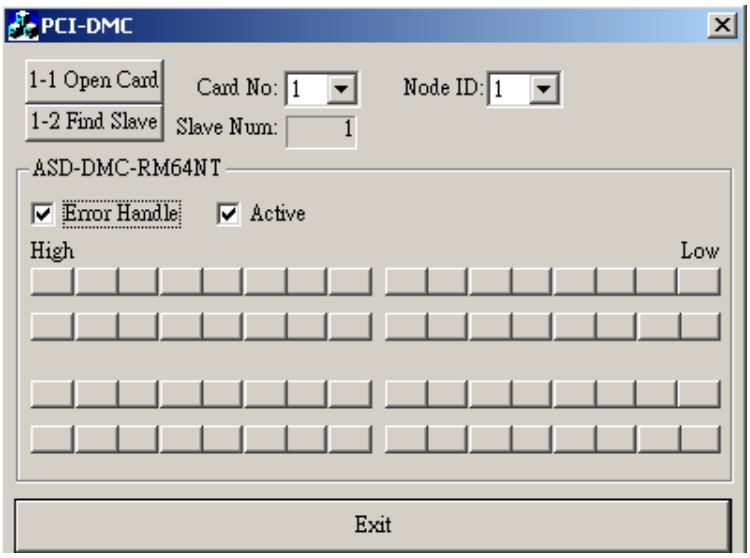


圖 3.190

1) 介面卡開啟與初始化

按壓圖 3.190 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.190 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 數位輸出(DO)操作

當您欲操作數位輸出時，首先勾選圖 3.190 中的 Active 選項並執行下列程序：

```
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，上述設置的輸出值才會從輸出埠輸出。
```

再來使用 ASD-DMC-RM32NT/RM64NT 模組，並執行以下程序：

```
/* 設定數位輸出 Port 0 的值 RM32NT/RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 0,
output_value[0]);
// Output_value[0]變數將存放 Port 0 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 1 的值 RM32NT/RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 1,
output_value[1]);
// Output_value[1]變數將存放 Port 1 的 bit 0 到 bit 15 欲輸出的值
```

```
/*  設定數位輸出 Port 2 的值  RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 2,
output_value[1]);
// Output_value[1]變數將存放 Port 2 的 bit 0 到 bit 15 欲輸出的值

/*  設定數位輸出 Port 3 的值  RM64NT*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 3,
output_value[1]);
// Output_value[1]變數將存放 Port 3 的 bit 0 到 bit 15 欲輸出的值
```

如圖 3.191 表示 ASD-DMC-RM64NT 模組上 Port 0、1、2、3 無輸出任何資料。

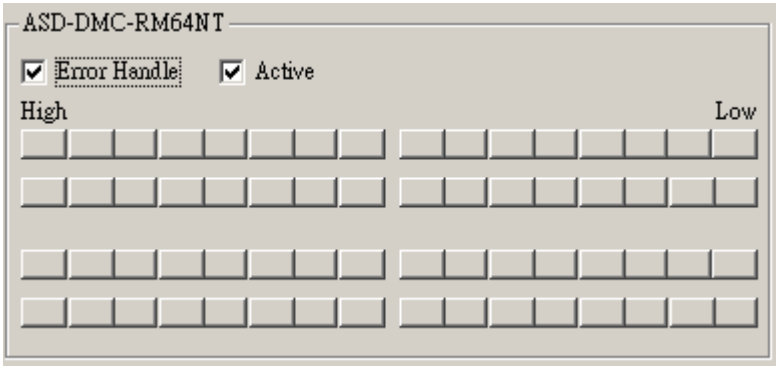


圖 3.191

3) 保留輸出埠的輸出狀態（值）

當您欲將當前的輸出，在模組電源尚未關閉前持續保留，必須將圖 3.191 中的 Error Handle 選項勾選並執行下列程序：

```
rt = _DMC_01_set_rm_output_value_error_handle(gDMCCardNo,
gpNodeID[gNodeNum], 0, port, gErrorHandle);
// port 變數表示可設置 remote 模組上的 2 組 Port，當其值為 0，將會操作模組上的
Port 0；值若為 1，表示將操作在 Port 1。而 gErrorHandle 變數的值是設置當錯誤發
生時，系統是否保留該錯誤輸出的值。當其值為 1 時，系統將保留該輸出值，直到模
組的電源關閉或移除；若其值為 0，則錯誤輸出值將被重置為 0。
```

4) 離開程序

按壓圖 3.190 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.20 遠端擴充 IO 模組 In-Out 應用(RM32PT、GA+GE16MN/NT)操作

3.20.1 函式列表

表 3.20

函式名稱
_DMC_01_set_rm_input_filter
_DMC_01_set_rm_input_filter_enable
_DMC_01_set_rm_output_value_error_handle
_DMC_01_get_slave_version
_DMC_01_get_devicetype
_DMC_01_set_rm_output_value
_DMC_01_get_rm_input_value

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.20.2　應用範例

程式外觀

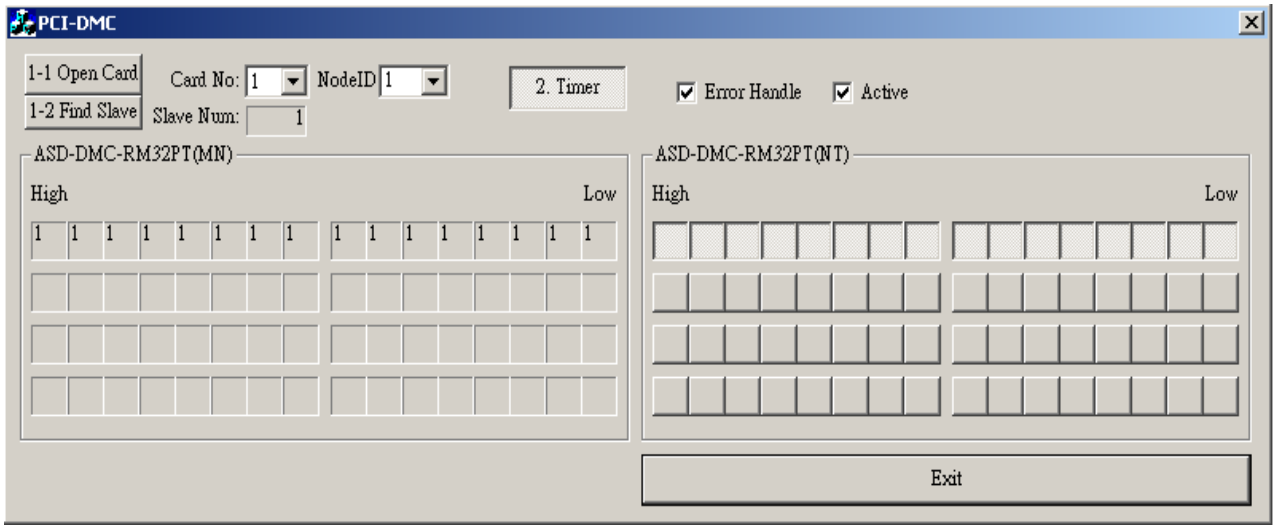


圖 3. 200

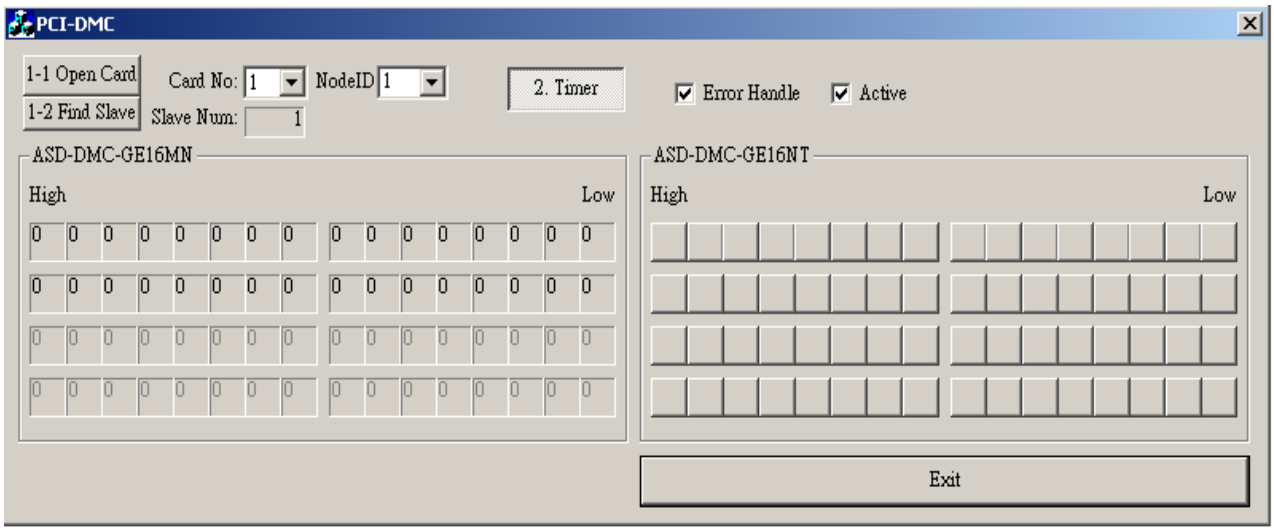


圖 3.201

1) 介面卡開啟與初始化

按壓圖 3.200 或圖 3.201 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.200 或圖 3.201 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) ASD-DMC-RM32PT/GA+GE16MN/NT 模組初始化參數設定。您可以執行以下程序：

```
rt = _DMC_01_set_rm_input_filter(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter);
// port 變數表示可設置 remote 模組上的 1~4 組 Port，當其值為 0，將會操作模組上的 Port 0；值若為 1，表示將操作在 Port 1，以此類推。 filter 變數是設定軟體濾波器的等級，當您將值設置為 0 時，軟體濾波時間為 1ms；值為 1 時，濾波時間為 2ms，依此類推。

rt = _DMC_01_set_rm_input_filter_enable(gDMCCardNo, gpNodeID[gNodeNum], 0, port, filter_enable);
// filter_enable 變數的值範圍為 0~0xFFFF，祺是用來控制 Port 上 0 到 15 的軟體濾波遮罩。
```

3) 數位輸入(DI)與數位輸出(DO)操作

當您欲操作數位輸出時，首先勾選圖 3.200 或圖 3.201 中的 Active 選項並執行下列程序：

```
rt = _DMC_01_set_rm_output_active(gDMCCardNo, NodeID, SlotID, active);
//此功能必須啟動後，上述設置的輸出值才會從輸出埠輸出。
```

再來使用 ASD-DMC-RM32PT/GA+GE16NT 模組，並執行以下程序：

```
/* 設定數位輸出 Port 0 的值 RM32PT/GA+GE16NT(最多 4 組) */
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 0, output_value[0]);
// Output_value[0]變數將存放 Port 0 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 1 的值 RM32PT/ GA+GE16NT(最多 4 組)*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 1, output_value[1]);
// Output_value[1]變數將存放 Port 1 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 2 的值 GA+GE16NT(最多 4 組)*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 2, output_value[1]);
// Output_value[1]變數將存放 Port 2 的 bit 0 到 bit 15 欲輸出的值

/* 設定數位輸出 Port 3 的值 GA+GE16NT(最多 4 組)*/
rt = _DMC_01_set_rm_output_value(gDMCCardNo, NodeID, SlotID, 3, output_value[1]);
// Output_value[1]變數將存放 Port 3 的 bit 0 到 bit 15 欲輸出的值
```

當您欲從數位輸入端取得您在數位輸出模組所送出的資料，您需要使用 ASD-DMC-RM32PT/GA+GE16MN 模組，並執行以下程序：

```
/*  取得數位輸入 Port 0 的值  RM32PT/GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 0,
&input_value[0]);
// Input_value[0]變數將會回覆從 Port 0 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 1 的值  RM32PT/ GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 1,
&input_value[1]);
// Input_value[1]  變數將會回覆從 Port 1 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 2 的值  GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 2,
&input_value[1]);
// Input_value[1]  變數將會回覆從 Port 2 的 bit 0 到  bit 15 所取得的資料

/*  取得數位輸入 Port 3 的值  GA+GE16MN(最多 4 組) */
rt = _DMC_01_get_rm_input_value(gDMCCardNo, NodeID, SlotID, 3,
&input_value[1]);
// Input_value[1]  變數將會回覆從 Port 3 的 bit 0 到  bit 15 所取得的資料
```

4) 保留輸出埠的輸出狀態（值）

當您欲將當前的輸出，在模組電源尚未關閉前持續保留，必須將圖 3.200 或圖 3.201 中的 Error Handle 選項勾選並執行下列程序：

```
rt = _DMC_01_set_rm_output_value_error_handle(gDMCCardNo,
gpNodeID[gNodeNum], 0, port, gErrorHandle);
// port 變數表示可設置 remote 模組上的 1~4 組 Port，當其值為 0，將會操作模組上的 Port 0；值若為 1，表示將操作在 Port 1，以此類推。而 gErrorHandle 變數的值是設置當錯誤發生時，系統是否保留該錯誤輸出的值。當其值為 1 時，系統將保留該輸出值，直到模組的電源關閉或移除；若其值為 0，則錯誤輸出值將被重置為 0。
```

5) 離開程序

按壓圖 3.200 或圖 3.201 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.21 遠端擴充 IO 模組應用-手輪操作

3.21.1 函式列表

表 3.21

函式名稱
_DMC_01_get_rm_input_value
_DMC_01_set_rm_mpg_axes_enable2

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.21.2　流程圖

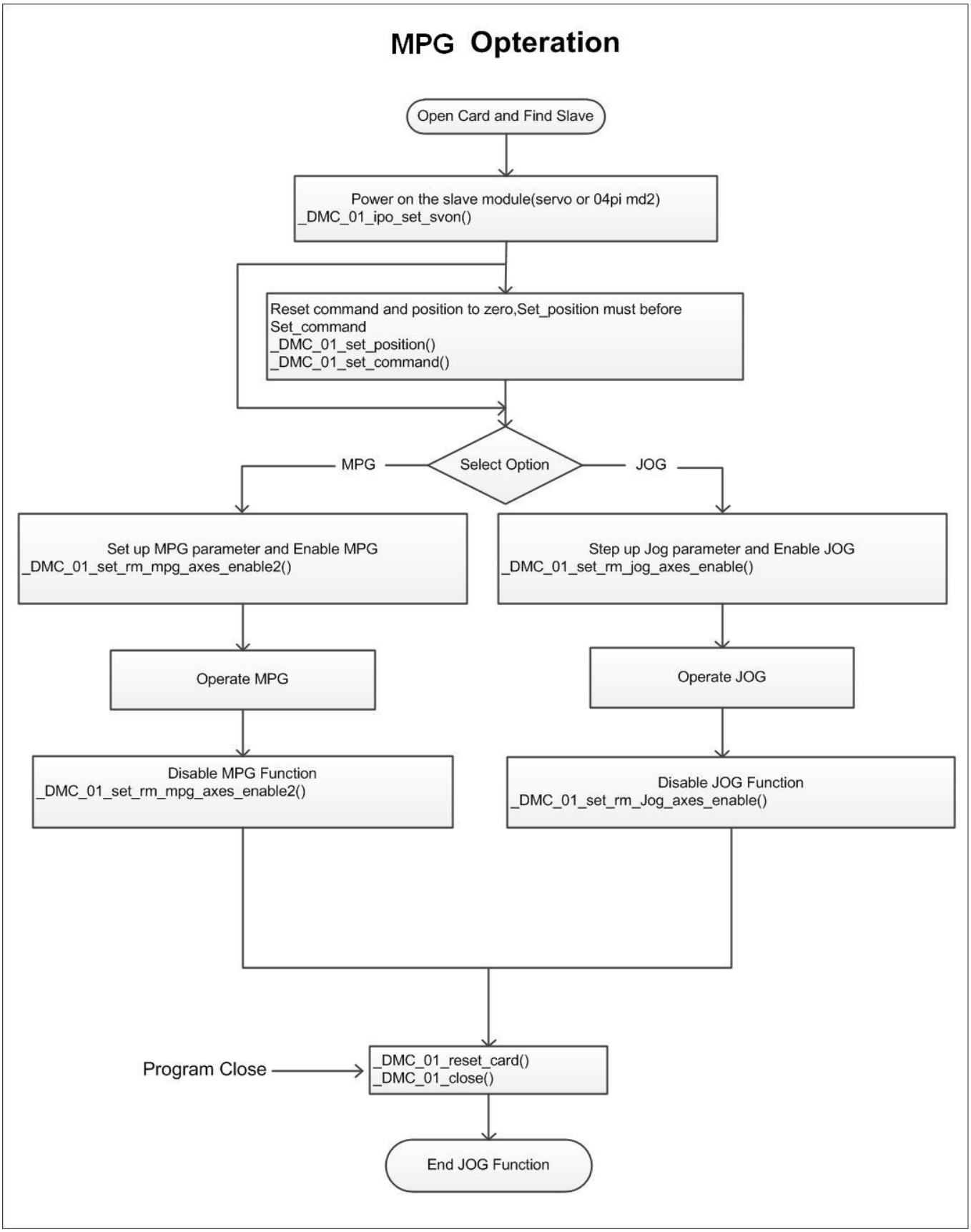


圖 3.210

3.21.3 應用範例

程式外觀

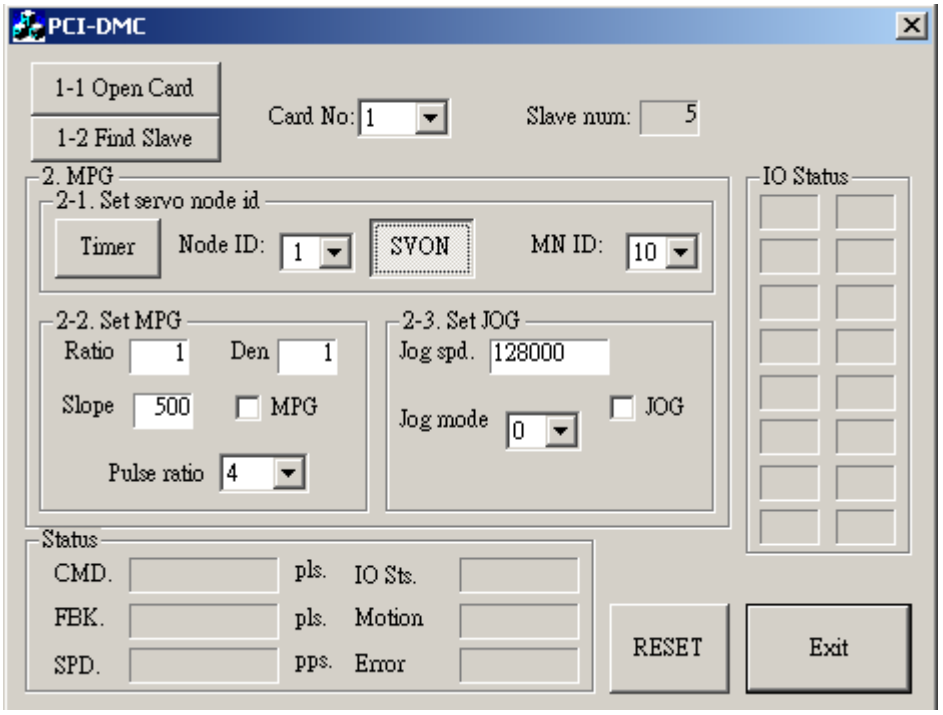


圖 3.211

1) 介面卡開啟與初始化
按壓圖 3. 211 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3. 211 中的「Find Slave」鍵開始搜尋串接模組。
關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID、伺服馬達動力 ON/OFF 及啟用顯示運動狀態



圖 3.212

填入 Node ID 並勾選“Timer”核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。
MN ID 項目：手輪站號設定。API 函數的引數變數 “MNNodeID”。

按壓圖 3.212 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

3) 填入手輪控制的參數內容值

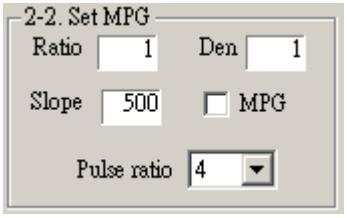


圖 3.213

Ratio 項目：手輪每轉馬達圈數比率。API 函數的引數變數 “ratio”。
Den　項目：手輪每轉馬達圈數分母。API 函數的引數變數 denominator”。
Slope 項目：手輪速度斜率。API 函數的引數變數 “slop”。
Pulse ratio 項目：手輪每格輸出脈波數比率值。API 函數的引數變數 “pulser_ratio”。
MPG 核取方塊：要啟動手輪功能須勾選它(Ratio、Den、Slope 及 Pulse Ratio 的參數修改，都必須要在 MPG 勾選前完成，否則不能完成參數變動)。

4) 若您欲執行「手輪運動控制」時，您必須勾選圖 3.213 中的「MPG」並執行以下程序：

```
rt = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port, &Value);
// 取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
rt = _DMC_01_set_rm_mpg_axes_enable2(CardNo, MasterNodeID, MasterSlotID,
NodeID, SlotID, enable, pulser_ratio, ratio, slop, denominator);
// 手輪運動控制。ratio 為每轉馬達圈數比率分子；denominator 為每轉馬達圈數比分母；藉由這兩個參數值的設定，可任意調整手輪輸出值。
```

此時，即可操作手輪。

5) 停止手輪運動控制
若您欲停止手輪的操作，請將圖 3.213 中的「MPG」核取方塊勾選取消即可停止手輪運動控制。

6) 重置運動計數值

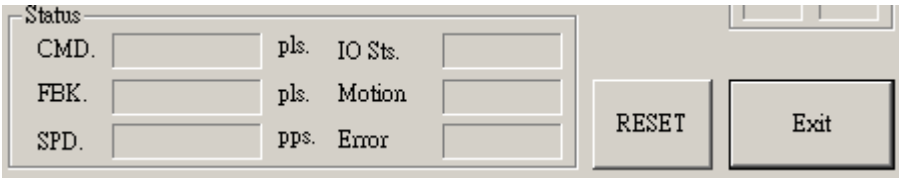


圖 3.214

若您欲將圖 3.214 中的 Command 及 Position 清除，必須將圖 3.213 中的 MPG 核取方塊勾選取消後，按壓圖 3.214 中的「RESET」鍵，執行以下程序：

```
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//將 Command 清為零
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//將 Position 清為零
```

7) 手輪 Jog 功能參數設定



圖 3.215

Jog mode 項目：JOG 模式選擇。API 函數的引數變數 “jog_mode”。
Jog spd 項目：JOG 速度設定。API 函數的引數變數 “jog_speed”。
JOG 核取方塊：要啟動手輪 JOG 功能須勾選它(Jog mode 及 Jog speed 的參數修改，都必須要在 JOG 勾選前完成，否則不能完成參數變動)。

8) 若您欲執行「手輪 JOG 運動控制」時，您必須勾選圖 3.215 中的「JOG」並執行以下程序：

```
rt = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port, &Value);
// 取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
rt = _DMC_01_set_rm_jog_axes_enable(CardNo, MasterNodeID, MasterSlotID,
NodeID, SlotID, enable, jog_mode,jog_speed, sec);
```

此時，即可操作手輪 JOG 功能。

9) 停止手輪運動 JOG 控制

若您欲停止手輪的操作，請將圖 3.215 中的「JOG」核取方塊勾選取消即可停止手輪運動 JOG 控制。

10) 重置運動計數值

若您欲將圖 3.214 中的 Command 及 Position 清除，必須將圖 3.215 中的 JOG 核取方塊勾選取消後，按壓圖 3.214 中的「RESET」鍵，執行以下程序：

```
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//將 Command 清為零
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//將 Position 清為零
```

11) 離開程序

按壓圖 3.214 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.22 遠端擴充脈波介面模組應用-RM04PI Mode 1 操作

3.22.1 函式列表

表 3.22

函式名稱
_DMC_01_get_buffer_length
_DMC_01_set_monitor
_DMC_01_get_monitor
_DMC_01_rm_04pi_md1_get_soft_limit_status
_DMC_01_set_rm_04pi_ipulser_mode
_DMC_01_set_rm_04pi_opulser_mode
_DMC_01_rm_04pi_md1_set_soft_limit
_DMC_01_set_rm_04pi_svon_polarity
_DMC_01_rm_04pi_md1_start_move
_DMC_01_rm_04pi_md1_v_move
_DMC_01_rm_04pi_set_MEL_polarity
_DMC_01_rm_04pi_set_PEL_polarity
_DMC_01_set_home_config
_DMC_01_set_home_move
_DMC_01_rm_04pi_md1_p_change
_DMC_01_rm_04pi_md1_v_change
_DMC_01_set_rm_04pi_DO2

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.22.2　流程圖

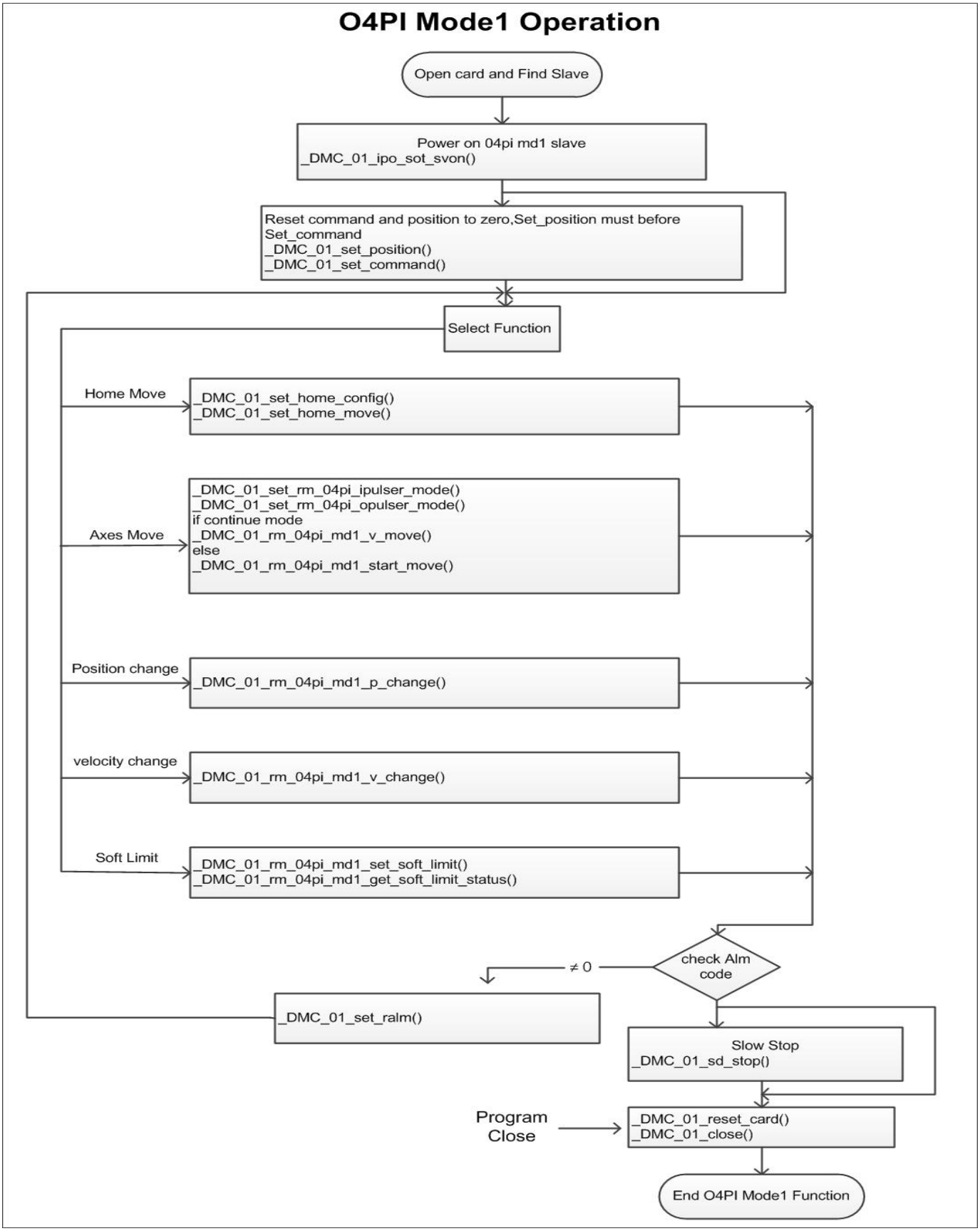


圖 3.220

3.22.3 應用範例

程式外觀

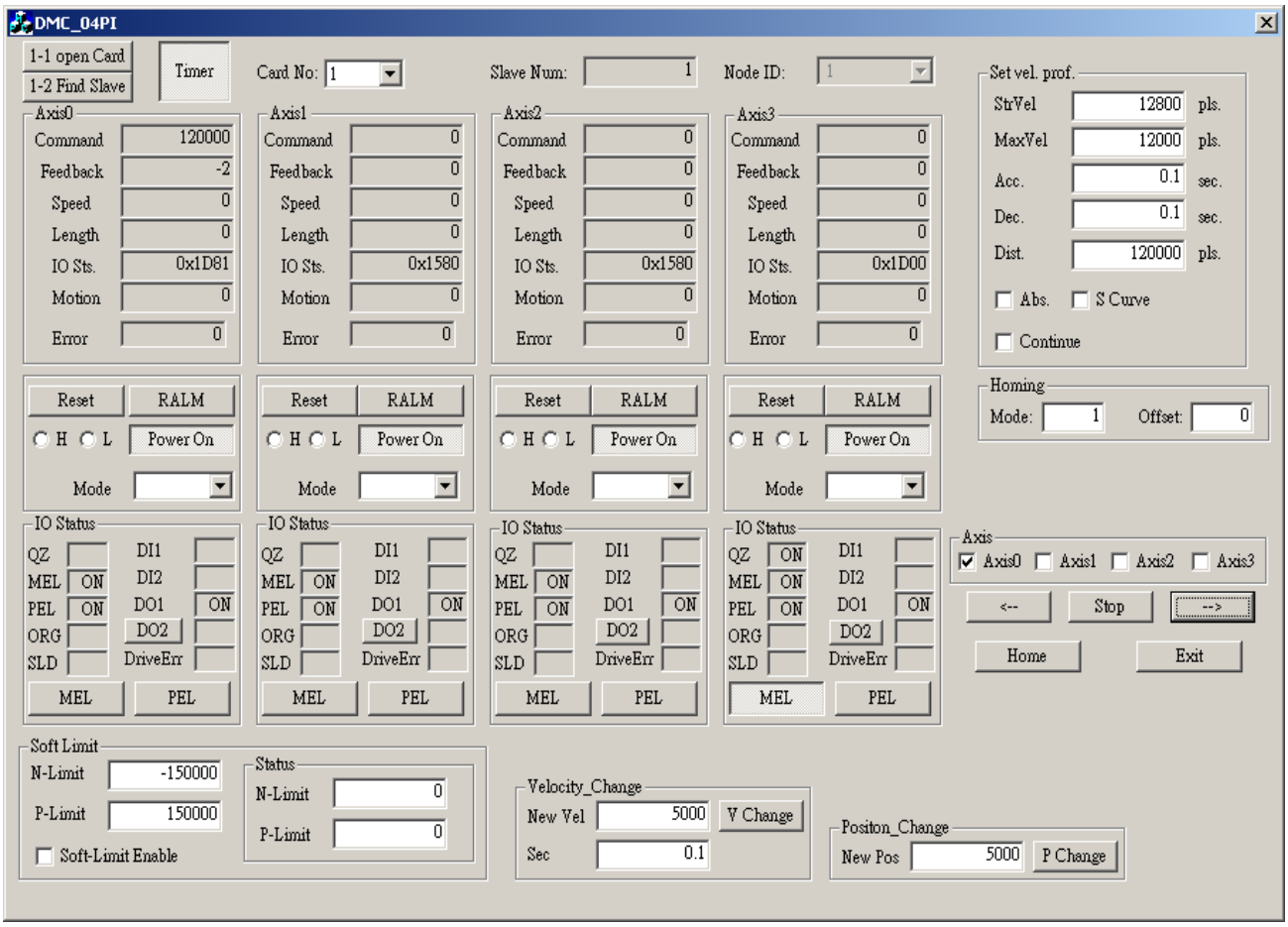


圖 3.221

1) 介面卡開啟與初始化
   按壓圖 3. 221 中的「Open card」鍵開始執行介面卡初始化。
   按壓圖 3. 221 中的「Find Slave」鍵開始搜尋串接模組。
   關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

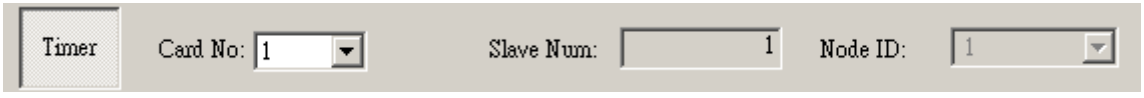


圖 3.222

填入 Node ID 並勾選「Timer」核取方塊啟用顯示運動狀態
NodeID 項目：API 函數的引數變數 “NodeID”.
Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

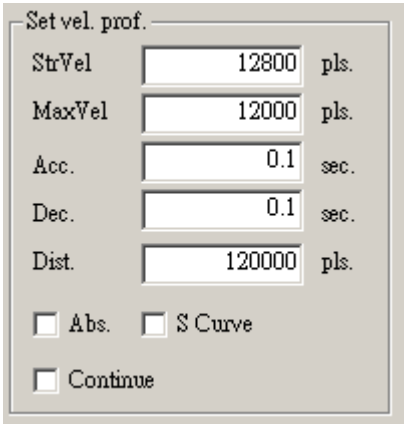


圖 3.223

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist 項目：設定運動距離。API 函數的引數變數 “Distance”。
Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
Continue 核取方塊：若您欲使用 Continue 運動模式，必須勾選它。

4) 選擇運動模式與設置運動行程

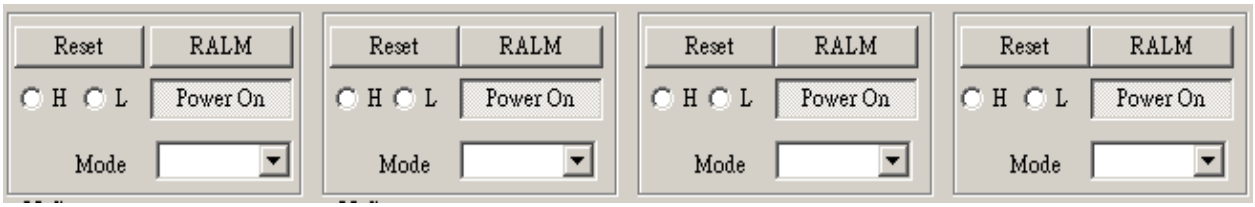


圖 3.224

Power On 核取方塊：按 Power On 啟動該軸電源。
H/L 核取方塊：選擇運行電位。
Mode 項目：輸出相位分 ABphase 及 CW/CCW 兩種。
RALM 項目：可以清除運行過程中產生得異常代碼。
Reset 項目：可以清除 Command 及 feedback 等相關顯示資料。

5) 狀態顯示及正、負極性反相功能

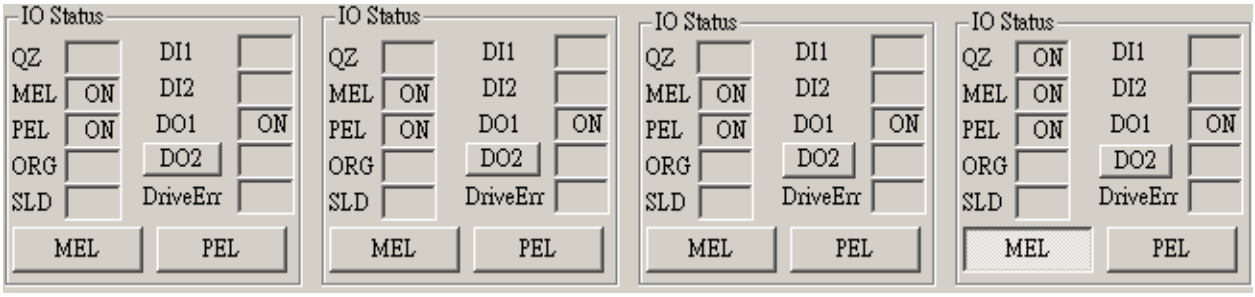


圖 3.225

MEL 核取方塊：設定將負極限反向。
PEL 核取方塊：設定將正極限反向。
按壓圖 3.225 中的「MEL」鍵開始執行以下程序：
rt = _DMC_01_rm_04pi_set_MEL_polarity(gDMCCardNo,NodeID,SlotID, Inverse);
按壓圖 3.225 中的「PEL」鍵開始執行以下程序：
rt = _DMC_01_rm_04pi_set_PEL_polarity(gDMCCardNo,NodeID,SlotID, Inverse);

6) 設定回 Home 運動模式的類別及位移植:

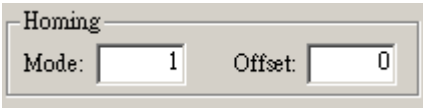


圖 3.226

Mode 項目：選擇回 Home 運動模式的類別。API 函數的引數變數 “home_mode”。
Offset 項目：設定回 Home 運動的位移植。API 函數的引數變數 “home_offset”。

7) 設定正/負極限值、是否啟用軟體極限與顯示是否碰觸軟體極限:

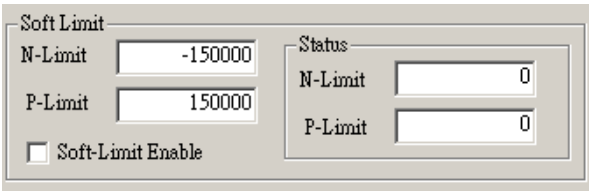


圖 3.227

P-Limit 項目：正極限值。API 函數的引數變數 “PLimit”。
N-Limit 項目：負極限值。API 函數的引數變數 “NLimit”。
Soft-limit Enable 核取方塊：啟動/關閉軟體極限。
P- Limit 項目：碰觸軟體正極限狀態顯示。
N- Limit 項目：碰觸軟體負極限狀態顯示。

8) 設定欲置換的新位置值及置換啟動:

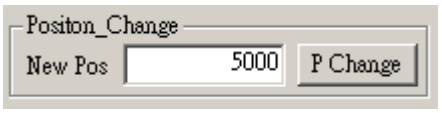


圖 3.228

New Pos 項目：新位置值。API 函數的引數變數 “NewPos”。
P Change 項目：置換啟動。

9) 設定欲置換的新速度值、加、減速時間及置換啟動:

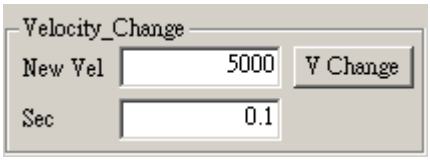


圖 3.229

New Vel 項目：設定新速度值。API 函數的引數變數 “NewSpeed”。
sec 項目：設定加、減速時間。API 函數的引數變數 “sec”。
V Change 項目：置換啟動。

10) 設定伺服馬達動力 ON/OFF(servo on/servo off)

按壓圖 3.224 中的「Power On」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

11) 選定軸號，開始進行運動控制(此處以點對點運動為例)



圖 3.22A

按壓圖 3.22A 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_rm_04pi_md1_start_move (CardNo, NodeID, SlotID, Dist, StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);   //進行運動位移
```

12) 啟用軟體極限功能

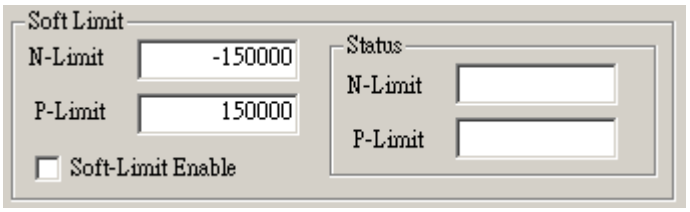


圖 3.22B

若您欲使用軟體極限功能時，必須先設定圖 3.22B 中的「P-Limit」與「N-Limit」的值，並勾選「Soft-limit Enable」核取方塊，執行以下程序：

```
/*  啟動設定軟體極限  */
rt = _DMC_01_rm_04pi_md1_set_soft_limit(CardNo, NodeID, SlotID, PLimit, NLimit, Enable);   // PLimit 引數表示正極限的設定值；NLimit 則為負極限的設定值
/*  軟體極限狀態顯示  */
rt = _DMC_01_rm_04pi_md1_get_soft_limit_status(CardNo, NodeID, SlotID, &NLimit_status, &PLimit_status);
```

※關於軟體極限範例的詳細敘述，請參閱「3.13 軟體極限」一節。

13) Countine 運動控制

若您欲進行 Countine 運動時，請勾選圖 3.223 中的「Countine」核取方塊，再按壓圖 3.22A 中的「→」或「←」鍵執行以下程序：

```
/* Countine 運動  */
rt = _DMC_01_rm_04pi_md1_v_move(CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Tdec, dir, m_curve);
```

14) 原點復歸運動控制

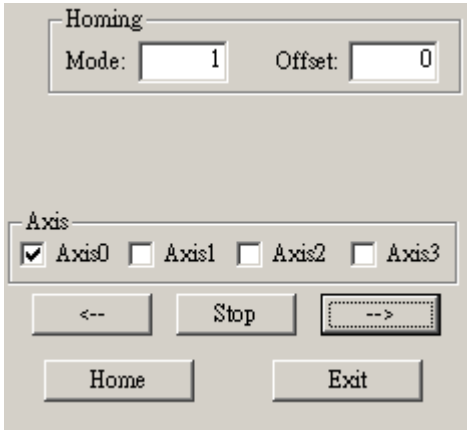


圖 3.22C

若您欲進行回 Home 運動時，請在圖 3.22C 中的 Homing 填入「Mode」及「Offset」並按壓「Home」鍵執行以下程序：

```
/* 設置原點復歸模式：1~35、偏移量與速度參數 */
rt = _DMC_01_set_home_config(gDMCCardNo, NodeID, SlotID, home_mode, home_offset, StrVel, MaxVel, acc);
/* 啟動原點復歸運動 */
rt = _DMC_01_set_home_move(gDMCCardNo, NodeID, SlotID);
```

15) 位置置換功能

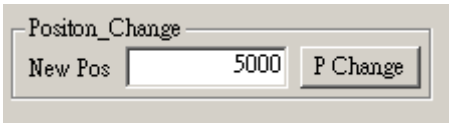


圖 3.22D

將設定的新位置值替換目前的位置，您必須按壓圖 3.22D 中的「P change」鍵執行以下程序:

```
/* P Change */
rt = _DMC_01_rm_04pi_md1_p_change(CardNo, NodeID, SlotID, NewPos);
```

16) 速度置換功能

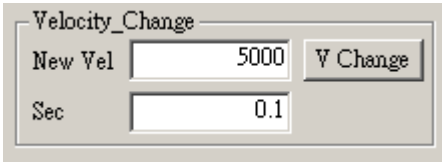


圖 3.22E

將設定的新速度值替換目前的速度，您必須按壓圖 3.22E 中的「V change」鍵執行以下程序:

```
/* P Change */
rt = _DMC_01_rm_04pi_md1_v_change(CardNo, NodeID, SlotID, NewsPeed,sec);
```

17) 停止運動及 Alm_Reset

不論正在執行哪一項運動，當按壓圖 3.22C 中的「Stop」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.221 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.224 中的「RALM」鍵開始執行 Alm_Code 重置動作。

18) 離開程序

按壓圖 3.22C 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.23　遠端擴充脈波介面模組應用-RM04PI Mode 2 操作

3.23.1　函式列表

表 3.23

函式名稱
_DMC_01_start_sa_move
_DMC_01_start_ta_mov
_DMC_01_start_sr_move
_DMC_01_start_tr_move
_DMC_01_set_rm_04pi_ipulser_mode
_DMC_01_set_rm_04pi_opulser_mode
_DMC_01_set_rm_04pi_svon_polarity
_DMC_01_set_monitor
_DMC_01_get_monitor
_DMC_01_send_message

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.23.2 流程圖

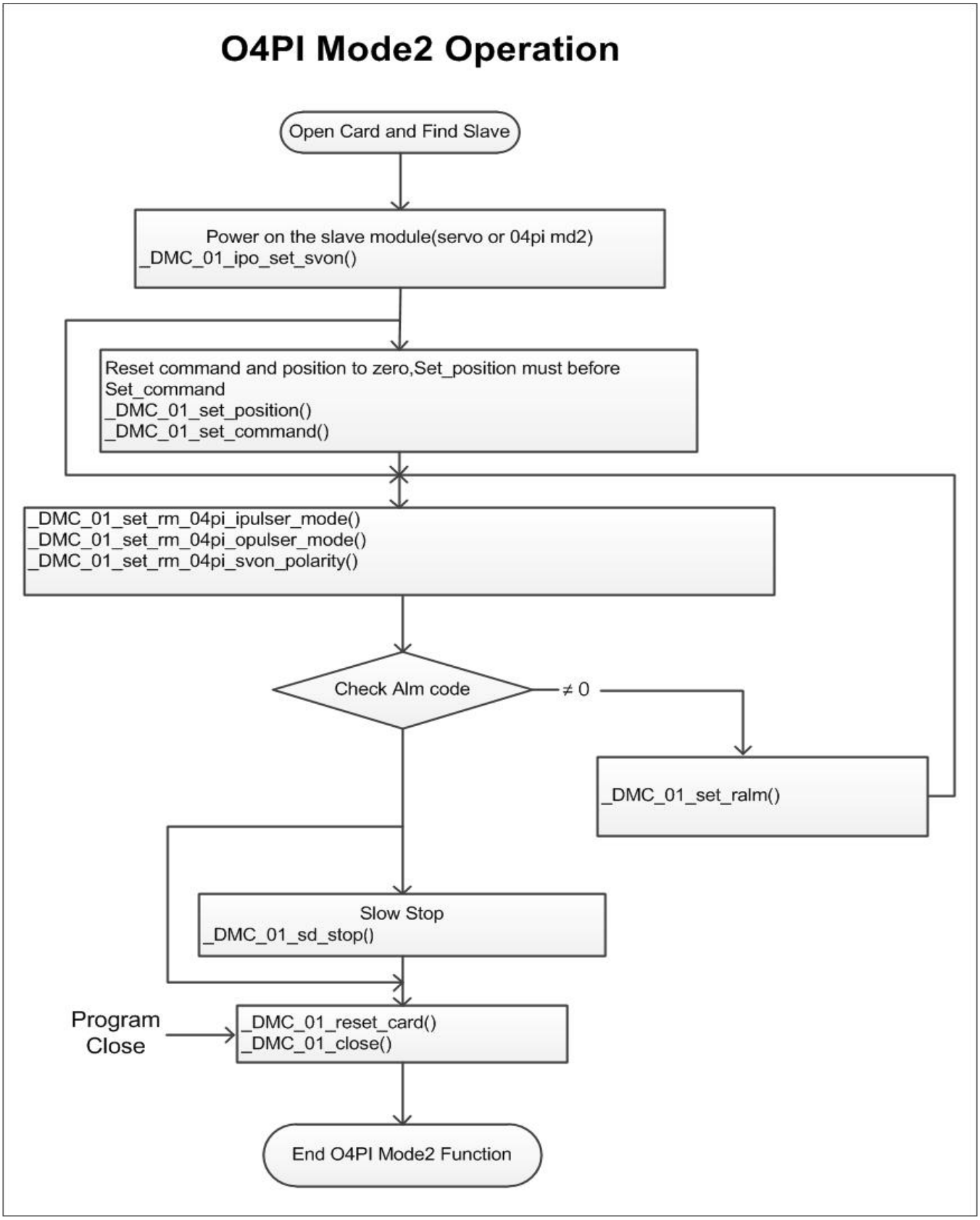


圖 3.230

3.23.3　應用範例

程式外觀

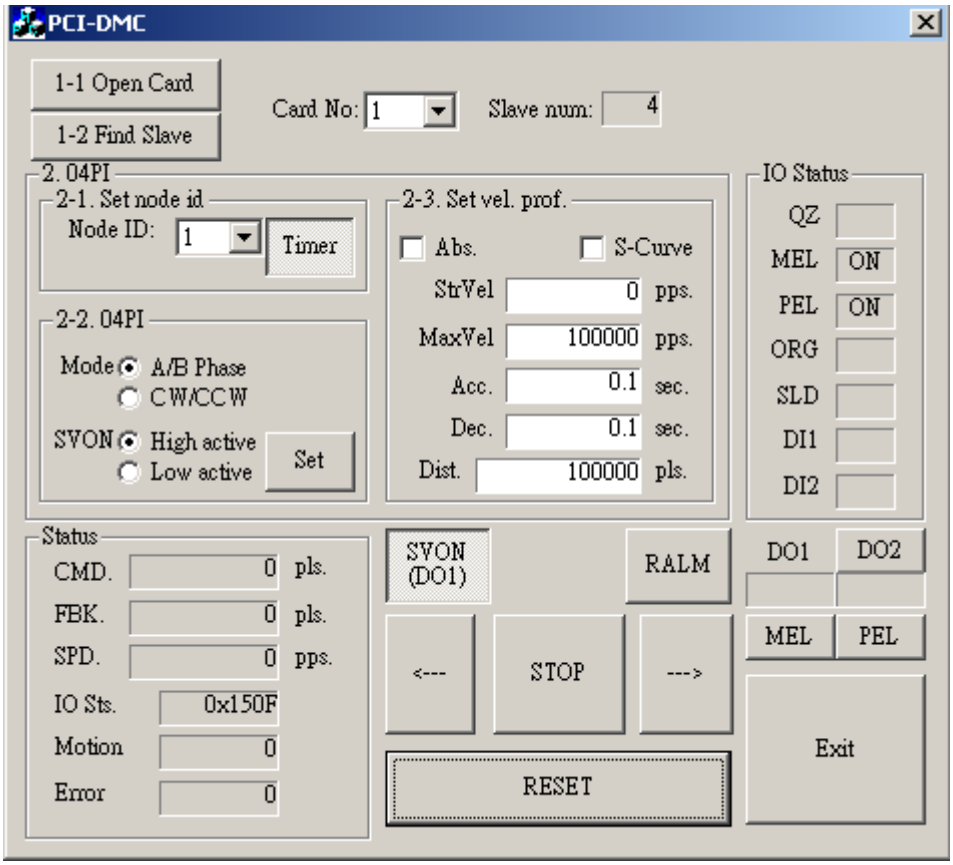


圖 3.231

1) 介面卡開啟與初始化

按壓圖 3. 231 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3. 231 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

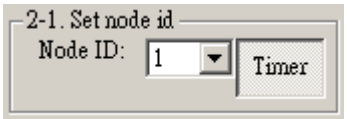


圖 3.232

填入 Node ID 並勾選「Timer」核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 選擇控制的引數內容值

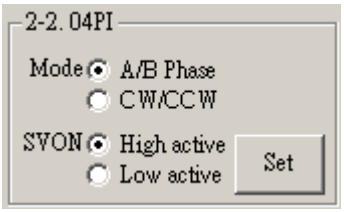


圖 3.233

AB phase 項目： 輸入/輸出模式為 AB phase 模式。
CW/CCW 項目： 輸入/輸出模式為 CW/CCW 模式。
High active 項目： 高準位時觸發。
Low active 項目： 低準位時觸發。

4) 填入運動控制的引數內容值

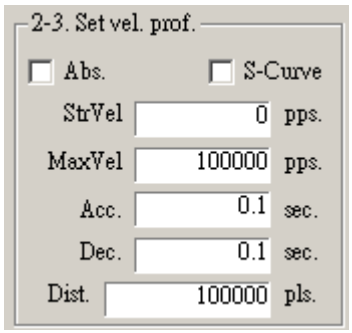


圖 3.234

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。
S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它。
StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。
MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。
Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。
Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。
Dist.項目：設定運動距離。API 函數的引數變數 “Distance”。

5) 設定伺服馬達動力 ON/OFF(servo on/servo off)

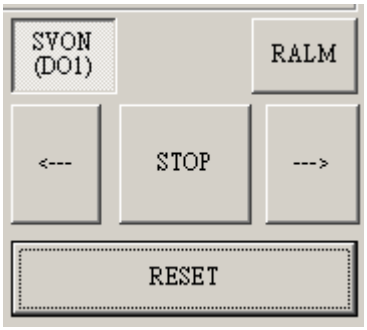


圖 3.235

按壓圖 3.235 中的「SVON(DO1)」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

6) 狀態顯示及正、負極性反相功能



圖 3.236

MEL 核取方塊：設定將負極限反向。

PEL 核取方塊：設定將正極限反向。

按壓圖 3.236 中的「MEL」鍵開始執行以下程序：

```
rt = _DMC_01_rm_04pi_set_MEL_polarity(gDMCCardNo,NodeID,SlotID, Inverse);
```

按壓圖 3.236 中的「PEL」鍵開始執行以下程序：

```
rt = _DMC_01_rm_04pi_set_PEL_polarity(gDMCCardNo,NodeID,SlotID, Inverse);
```

7) 若設定了圖 3.233 中的 AB Phase、CW/CCW 及 High active、Low active 後按壓「Set」鍵開始執行以下程序：

```
rt =_DMC_01_set_rm_04pi_ipulser_mode(CardNo, NodeID, SlotID, mode);
//設定脈波介面模組輸入相位模式
rt = _DMC_01_set_rm_04pi_opulser_mode(CardNo, NodeID, SlotID, mode);
//設定脈波介面模組輸出相位模式
rt = _DMC_01_set_rm_04pi_svon_polarity(CardNo, NodeID, SlotID, polarity);
//設定 POWER ON（SVON）的準位
```

8) 開始進行運動控制(此處以點對點運動為例)

按壓圖 3.235 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的相對座標進行運動位移
```

9) 監控顯示

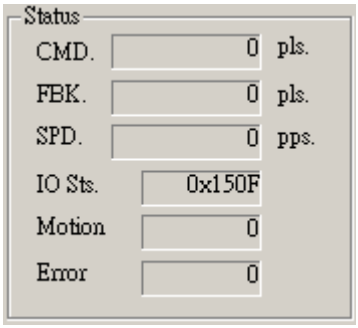


圖 3.237

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

10) 數值重置、停止運動及 Alm_Reset

按壓圖 3.235 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.235 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.237 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.235 中的「RALM」鍵開始執行 Alm_Code 重置動作。

11) 離開程序

按壓圖 3.231 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.24 中斷操作 Interrupt

3.24.1 函式列表

表 3.24

函式名稱
_DMC_01_int_enable
_DMC_01_int_disable
_DMC_01_set_int_factor
_DMC_01_get_int_count

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

3.24.2　流程圖

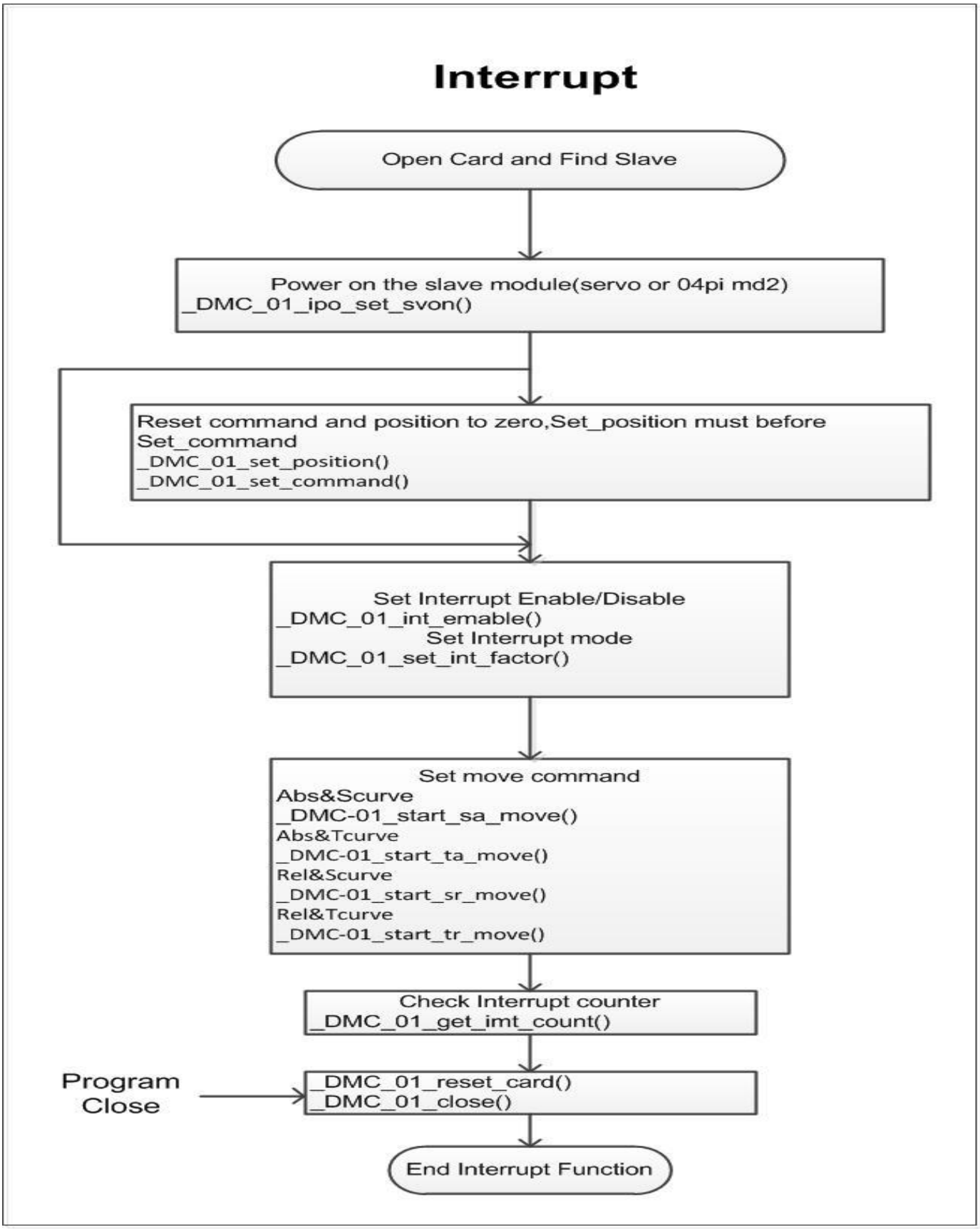


圖 3.240

3.24.3 應用範例

程式外觀

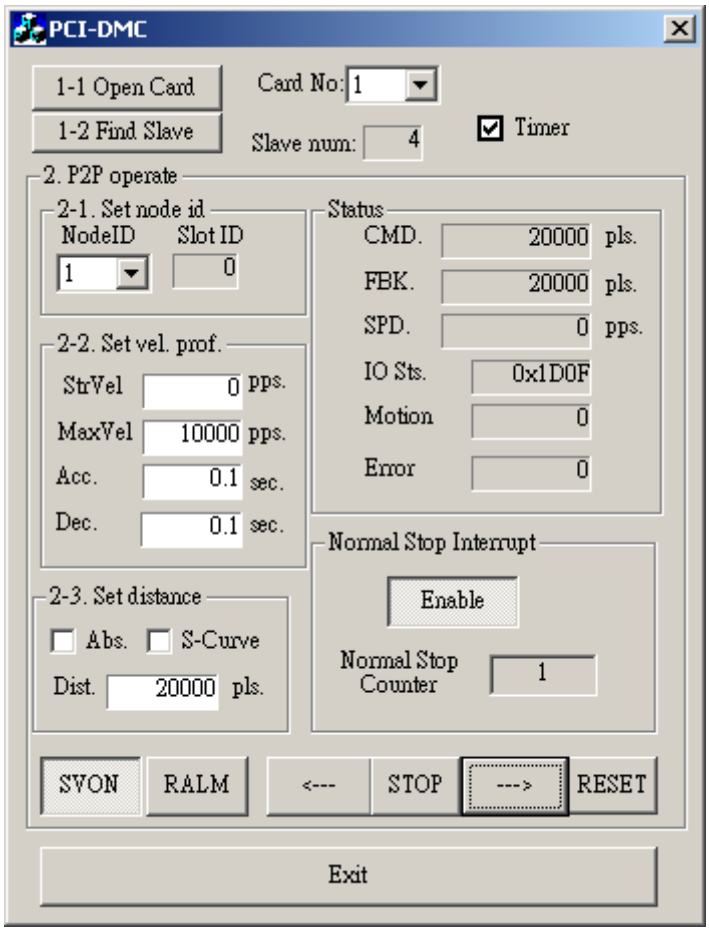


圖 3.241

1) 介面卡開啟與初始化

按壓圖 3.241 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.241 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 設置驅動器的 Node ID 並啟用顯示運動狀態

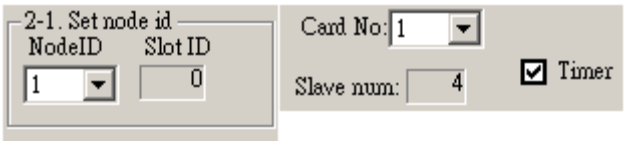


圖 3.242

勾選 NodeID 項目並勾選“Timer”核取方塊啟用顯示運動狀態

NodeID 項目：API 函數的引數變數 “NodeID”.

Timer 核取方塊：勾選它將會顯示運動的狀態，反之，則不顯示。

3) 填入運動控制的引數內容值

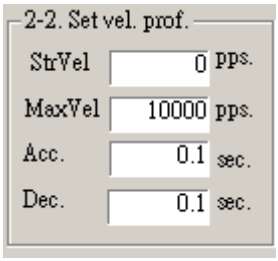


圖 3.243

StrVel 項目：初始速度。API 函數的引數變數 “StrVel”。

MaxVel 項目：最大速度。API 函數的引數變數 “MaxVel”。

Acc 項目：達到最大速度所花費時間。API 函數的引數變數 “acc”。

Dec 項目：從最大速度減至 0 值所花費的時間。API 函數的引數變數 “dec”。

4) 選擇運動模式與設置運動行程

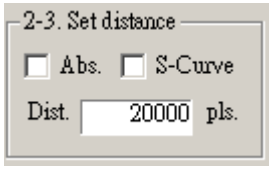


圖 3.244

Abs.核取方塊：若您欲參照絕對座標的方式進行運動位移，必須勾選它。

S-Curve 核取方塊：若您欲使用 S-curve 速度曲線，必須勾選它，Default 為 T-curve。

Dist.項目：運動行程。API 函數的引數變數 “Distance”。

5) 設定 Normal Stop Interrupt 是否啟用

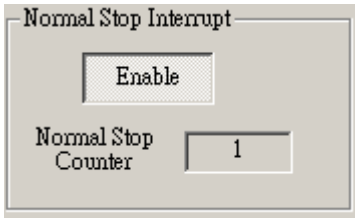


圖 3.245

Enable 項目：啟動/關閉 Normal Stop Interrupt。
Normal Stop Counter 項目：啟動 Normal Stop Interrupt 後累計觸發數。

6) 若您欲啟用 Normal Stop Interrupt 功能，您必須將圖 3.245 中的 “Enable”鍵按下，並執行以下程序：

```
/* 啟動 Interrupt 功能*/
rt = _DMC_01_int_enable(gDMCCardNo, NodeID);
/* 關閉 Interrupt 功能*/
rt = _DMC_01_int_disable(gDMCCardNo, NodeID);
/* 設定 Interrupt 功能為 Normal Stop Interrupt*/
rt = _DMC_01_set_int_factor(gDMCCardNo, NodeID,1);
/* 1: Normal Stop：任一運動結束後會觸發(Mode2)  */
/* 2: Next Buffer：執行 Buffer 動作時會觸發(Mode2)  */
/* 3: Acceleration End：在加速結束時觸發(Mode2)  */
/* 4: Deceleration Start：在減速開始時觸發(Mode2)  */
/* 5: Sdo Finish：（此功能尚未開放）  */
/* 6: DMC Cycle Start：進入 DMC Cycle 時觸發  */
/* 7: RM04PI-FIFO：進入 04PI FIFO 時觸發(Mode1)  */
/* 8: User define：（此功能尚未開放）  */
```

7) 設定伺服馬達動力 ON/OFF(servo on/servo off)



圖 3.246

按壓圖 3.246 中的「SVON」鍵開始執行以下程序：

```
rt = _DMC_01_ipo_set_svon(gDMCCardNo, NodeID, SlotID , ON_OFF);
// ON_OFF: 0 – 伺服動力 OFF；1 – 伺服動力 ON
```

8) 開始進行運動控制(此處以點對點運動為例)

按壓圖 3.246 中的「→」或「←」鍵執行以下程序：

```
rt = _DMC_01_start_sa_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_ta_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的絕對座標進行運動位移
rt = _DMC_01_start_sr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 S-curve 速度剖面下的相對座標進行運動位移
rt = _DMC_01_start_tr_move(gDMCCardNo, NodeID, SlotID, Distance, StrVel, MaxVel, acc, dec); // 參照 T-curve 速度剖面下的相對座標進行運動位移
```

9) 設定 Interrupt 功能為 Normal Stop Interrupt：
當進行運動的距離完成時或於中途按下 Stop 鍵，用下列 API 去讀取 Int_Count 值會較之前增加 1。如上圖 3.245 Normal Stop Counter 所示。

```
rt = _DMC_01_get_int_counter(gDMCCardNo, NodeID, &Int_Count);
```

10) 監控顯示

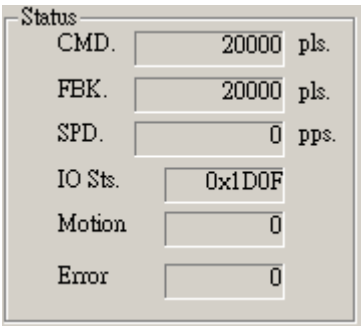


圖 3.247

如圖 3.247 中所示，將使用下列程序取得監控資訊:

運動計數值：

```
rt = _DMC_01_get_command(gDMCCardNo, NodeID, SlotID, &cmd);
// 取得命令計數值
rt = _DMC_01_get_position(gDMCCardNo, NodeID, SlotID, &pos);
// 取得命令反饋計數值
```

運動狀態：Motion status:

```
rt = _DMC_01_get_current_speed(gDMCCardNo, NodeID, SlotID, &speed);
// 取得運動運行的速度值
rt = _DMC_01_motion_status(gDMCCardNo, NodeID, SlotID, &MC_status);
// 取得目前的狀態
rt = _DMC_01_motion_done(gDMCCardNo, NodeID, SlotID, &MC_done);
// 取得目前馬達動作的狀態
rt = _DMC_01_get_alm_code(gDMCCardNo, NodeID, SlotID, &err);
// 取得目前 Alm_Code 的狀態
```

11) 數值重置、停止運動及 Alm_Reset

按壓圖 3.246 中的「RESET」鍵開始執行重置命令：

```
rt = _DMC_01_set_position(gDMCCardNo, NodeID, SlotID, 0);
//先執行清除 feedback
rt = _DMC_01_set_command(gDMCCardNo, NodeID, SlotID, 0);
//再執行清除 command
```

不論正在執行哪一項運動，當按壓圖 3.246 中的「STOP」鍵開始執行緊急停止：

```
rt = _DMC_01_emg_stop (gDMCCardNo, NodeID, SlotID);
```

此例中使用緊急停止的方式停止運動位移，此方式是將減速時間設為 0，快速地停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

當圖 3.247 中的 Error 欄位發生錯誤訊號時，可以按壓圖 3.246 中的「RALM」鍵開始執行 Alm_Code 重置動作。

12) 離開程序

按壓圖 3.246 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.25　主控介面卡安全機制操作 Master Security

3.25.1　函式列表

表 3.25

函式名稱
_DMC_01_check_userpassword
_DMC_01_write_userpassword
_DMC_01_read_serialno
_DMC_01_write_verifykey
_DMC_01_check_verifykey
_DMC_01_read_security
_DMC_01_read_security_status
_DMC_01_write_security
_DMC_01_write_security_status
_misc_security

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.25.2 流程圖

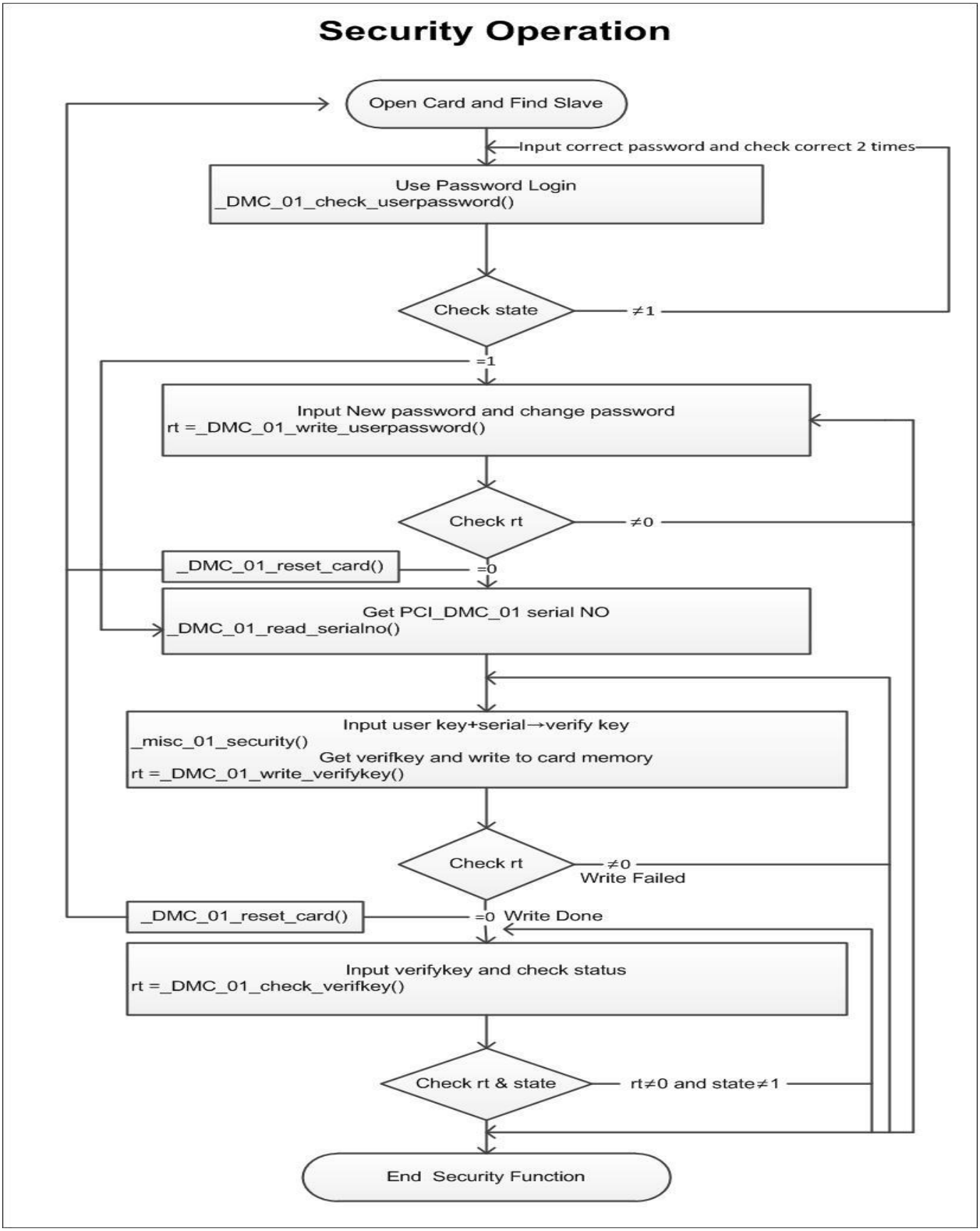


圖 3.250

3.25.3　應用範例

程式外觀

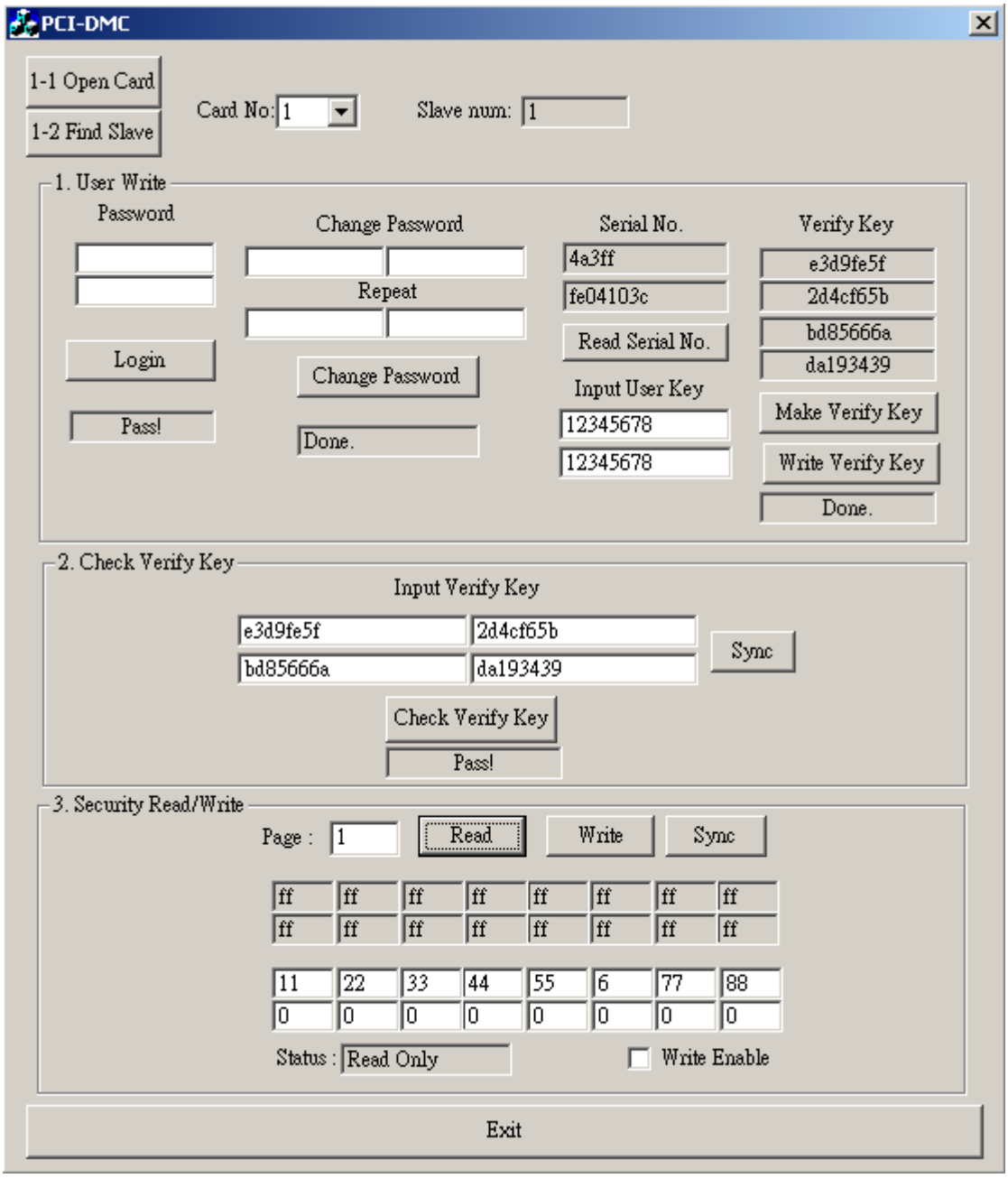


圖 3.251

1) 介面卡開啟與初始化

按壓圖 3.251 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.251 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 進行軸卡登入動作。(SerialNo、Verifykey、Memory Read/Write 均需要 Check Password Pass 才能顯示運作)。



圖 3.252

Password 項目：輸入兩組 1~8 位 0~F 的 16 進制值，軸卡預設密碼 (Password1:FFFFFFFF Password2:FFFFFFFF)。

Login 項目：進行密碼確認動作。

狀態列項目：如果密碼正確，圖 3.252 下方狀態列出現「Pass!」，如果密碼錯誤，狀態列出現「Error!」。

3) 變更軸卡密碼:

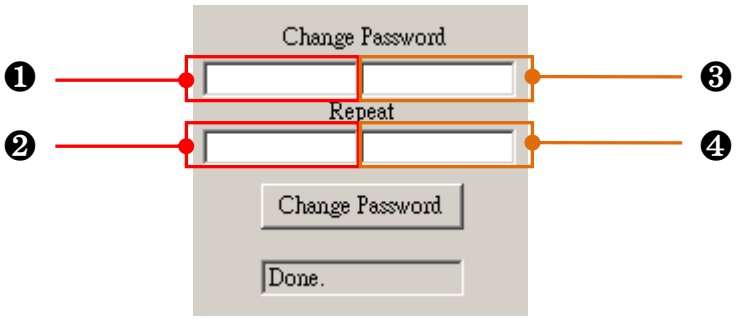


圖 3.253

Change Password 項目：輸入兩組 1~8 位 0~F 的 16 進制值(❶及❸)。

Repeat 項目：重複輸入兩組 1~8 位 0~F 的 16 進制值(❷及❹) (與 Change Password 輸入值相同)。

Change 項目：按下「Change Password」鍵則進行密碼變更。

狀態列項目：如果 Change Password 與 Repeat 輸入的密碼是相同的，則按下圖 3.253 「Change Password」鍵後，變更密碼成功，狀態列出現「Done.」，如果 Change Password 與 Repeat 輸入的密碼是不同的，狀態列會出現「Please rewrite it.」，請重新進行密碼變更動作。

4) 建立 Verifykey 動作

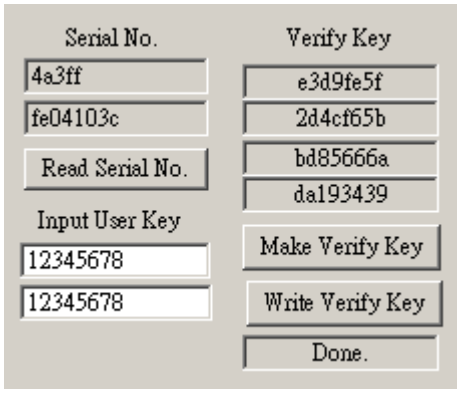


圖 3.254

SerialNo.顯示：軸卡出廠的製造序號，顯示於此。
Read Serial No 項目：進行讀取軸卡製造序號的動作。
Input User Key 項目：輸入客戶端自訂的兩組 1~8 位 0~F 的 16 進制的 Key 值。
Verify key 顯示：由系統產生的 verifykey 值顯示於此。
Make Verify Key 項目：將 SerialNo 及客戶輸入的 Userkey，經加密程序，產生成四組 VerifyKey。
Write Verify Key 項目：將產生的 Verifykey 值寫入軸卡中。
狀態列項目：如果 Verifykey 寫入成功，狀態列出現「Done.」，寫入失敗，出現「Error!」。

5) Verify key 是否正確

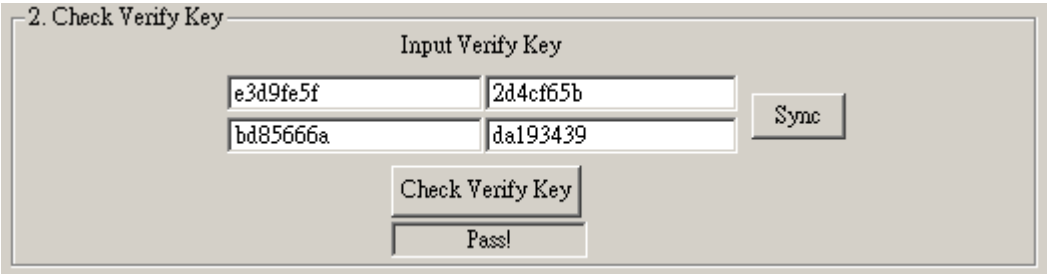


圖 3.255

Input Verify key 項目：輸入要檢查軸卡的 Verify key 值。
Sync 項目：可將圖 3.254 程序產生的 Verify key 值複製於 Input Verify key 欄位。
Check Verify key 項目：按下圖 3.255 中的「Check Verify Key」，若 Key 值正確，狀態列顯示「Pass.」Key 值錯誤，狀態列顯示「Error!」。

6) 記憶體讀/寫功能(PCI_DMC_F01 無此功能)

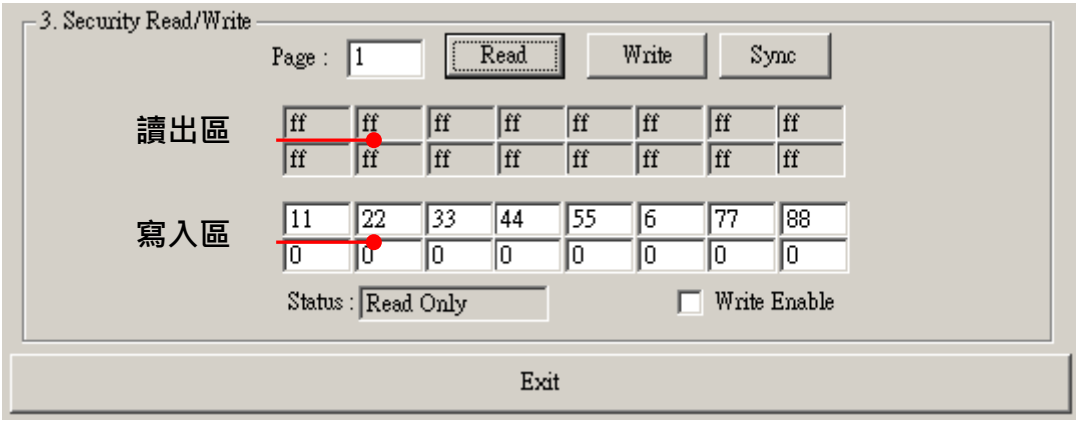


圖 3.256

Page 項目：輸入要讀取或寫入的 Memory Page(0~9)值。
Read 項目：執行讀取指定 Memory Page 動作，並顯示於「讀取 」位置。
Write Enable 項目：在執行寫入動作前，需將 Write Enable 選項打勾，這時於 Status 狀態列會出現「Write/Read」。
Sync 項目：按 Sync 鍵可以將「讀取」位置的值複製到「寫入」位置。
Write 項目：執行將「寫入」位置的值寫入 Memory 中。
Status 項目：Default 值為「Read Only」，Write Enable 選項打勾，顯示「Write/Read」。

7) 若按下圖 3.252 的「login」鍵則執行下列程序：

```
/* 檢查 Password 功能*/
rt = _DMC_01_check_userpassword(gDMCCardNo, Password(0),state);
```

若按下圖 3.253 的「Change Password」鍵則執行下列程序：

```
/* 進行變更密碼動作 */
rt = _DMC_01_write_userpassword(gDMCCardNo, chpassword(0));
```

若按下圖 3.254 的「Read Serial No.」鍵則執行下列程序：

```
/* 讀取 Serial No 動作*/
rt = _DMC_01_read_serialno(gDMCCardNo, serial(0));
```

若按下圖 3.254 的「Make Verify Key」鍵則執行下列程序：

```
/* 進行加密製作 Verify Key 動作*/
rt = _misc_security(userkey(0),userkey(1),SerialNo(0),SerialNo(1),verifykey(0) ,
                    verifykey(1) ,verifykey(2) ,verifykey(3));
```

若按下圖 3.254 的「Write Verify Key」鍵則執行下列程序：

```
/* 將 Verify Key 寫入 Memory 中*/
rt =_DMC_01_write_verifykey(gDMCCardNo,verifykey(0));
```

若按下圖 3.255 的「Check Verify Key」鍵則執行下列程序：

```
/* 檢查 Verify Key 值是否正確*/
rt =_DMC_01_check_verifykey(gDMCCardNo,verifykey(0),state);
```

若按下圖 3.256 的「Read」鍵則執行下列程序：

```
/* 讀取指定 Page 的資料顯示於「讀取」位置*/
rt =_DMC_01_read_security(gDMCCardNo,page,ary(0));
```

若勾選圖 3.256 的「WriteEnable」則執行下列程序：

```
/* 將「讀/寫」狀態改變*/
rt =_DMC_01_write_security_status(gDMCCardNo,on_off);
```

若按下圖 3.256 的「Write」鍵則執行下列程序：

```
/* 將「寫入」位置的資料寫入指定 Page 的 Memory 中*/
rt =_DMC_01_write_security(gDMCCardNo,page,ary(0));
```

/* 執行完這個 API 後，Dll 會將讀/寫狀態自動改為 Read Only，若需要再寫入其他資料，須重新設定「讀/寫」狀態(Write_security_status)*/

8) 離開程序

按壓圖 3.256 的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.26 遠端擴充類比輸入模組應用-RM04AD

3.26.1 函式列表

表 3.26

函式名稱
_DMC_01_set_04ad_input_range
_DMC_01_get_04ad_input_range
_DMC_01_set_04ad_conversion_time
_DMC_01_get_04ad_conversion_time
_DMC_01_get_04da_data
_DMC_01_set_04ad_average_mode
_DMC_01_get_04ad_average_mode
_DMC_01_set_04ad_input_enable
_DMC_01_set_04ad_input_range

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.26.2　應用範例

程式外觀

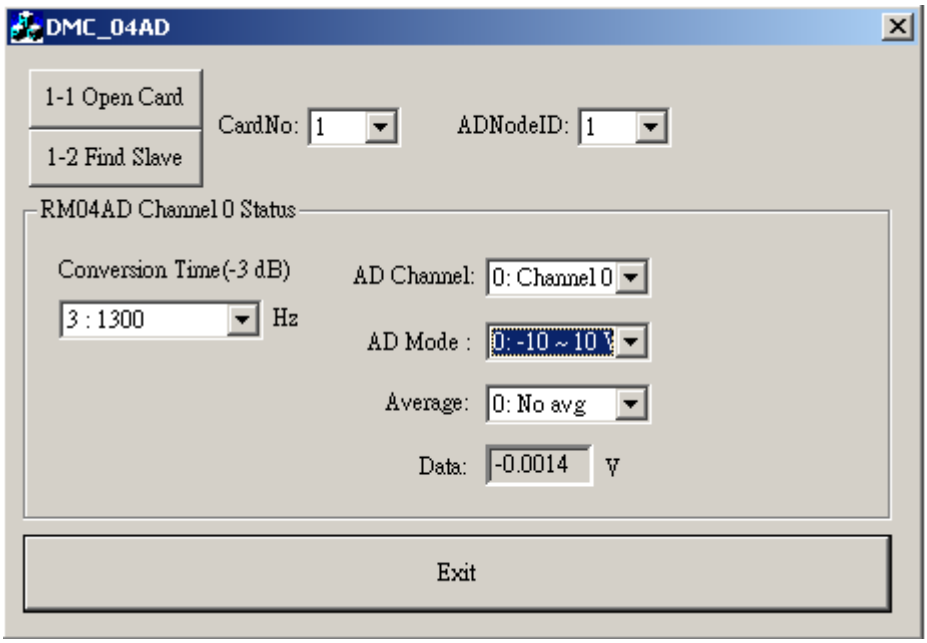


圖 3.260

1) 介面卡開啟與初始化
   按壓圖 3.260 中的「Open card」鍵開始執行介面卡初始化。
   按壓圖 3.260 中的「Find Slave」鍵開始搜尋串接模組。
   關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) AD Conversion Time 模式選擇:



圖 3.261

Conversion Time 項目：選擇 Conversion Time 的模式。

3) 選擇 AD Channel / 顯示模式 / Average 等相關資料:

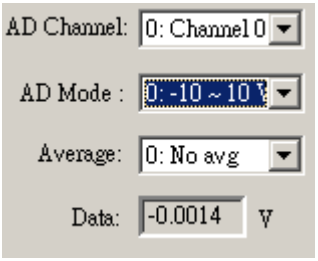


圖 3.262

Channel 項目：選擇 AD 致能輸入之 Channel 位置。
AD Mode 項目：選擇 AD 顯示範圍模式。
Average 項目：選擇波形顯示計算頻率。
Data 項目：輸入 AD 之電壓或電流值顯示位置。

4) 若選擇圖 3.261 的 Conversion Time 選項，則執行下列程序：

```
/* 設定 Conversion Time 模式*/
rt =_DMC_01_set_04ad_conversion_time(gDMCCardNo,ADNodeID,SlotID,mode);
```

若選擇圖 3.262 的 Mode 選項，則執行下列程序：

```
/* 設定 AD 輸入範圍模式*/
rt =_DMC_01_set_04ad_input_range(gDMCCardNo,ADNodeID,SlotID,ChannelAD,
      mode);
```

若選擇圖 3.262 的 Average 選項，則執行下列程序：

```
/* 設定 AD 波形輸出計算頻率*/
rt =_DMC_01_set_04ad_average_mode(gDMCCardNo,ADNodeID,SlotID,
      ChannelAD,mode);
```

圖 3.262 的 Data 顯示，則執行下列程序：

```
/* 顯示 AD 輸入電壓或電流值 */
rt =_DMC_01_get_04ad_data(gDMCCardNo,ADNodeID,SlotID,ChannelAD,data);
```

5) 離開程序

按壓圖 3.260 中的「Exit」鍵開始執行程序的離開與關閉。
欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.27　遠端擴充類比輸出模組應用-RM04DA

3.27.1　函式列表

表 3.27

函式名稱
_DMC_01_rm_04da_set_output_value
_DMC_01_rm_04da_get_output_value
_DMC_01_rm_04da_get_return_code
_DMC_01_rm_04da_read_data
_DMC_01_rm_04da_set_output_range
_DMC_01_rm_04da_set_output_enable
_DMC_01_rm_04da_set_output_overrange
_DMC_01_rm_04da_set_output_error_clear
_DMC_01_rm_04da_set_output_error_handle
_DMC_01_rm_04da_set_output_offset_value
_DMC_01_rm_04da_get_output_offset_value

■　**屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.27.2 應用範例

程式外觀

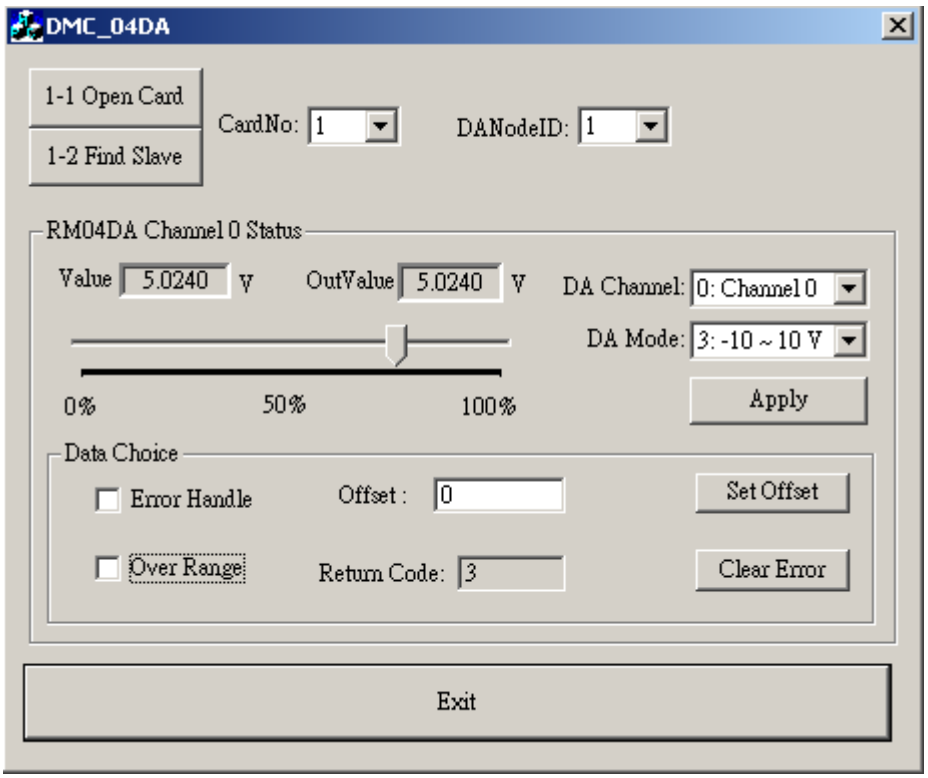


圖 3.270

1) 介面卡開啟與初始化

按壓圖 3.270 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.270 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) DA Channel 選擇及顯示模式選擇:

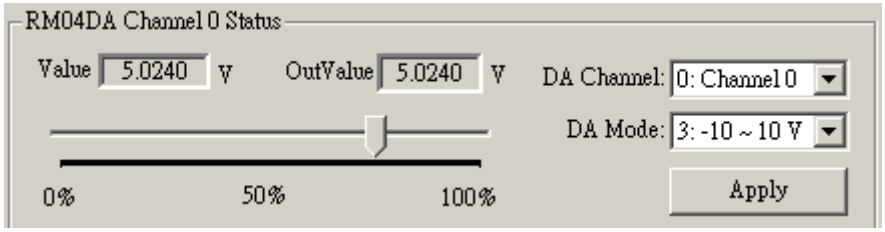


圖 3.271

DA Channel 項目：輸入欲使用之 DA Channel 編號(0~3)。
DA Mode 項目：選擇 DA 顯示範圍模式，API 函數的引數變數 “mode”。
Value 項目: 根據選擇範圍模式及拉把位置顯示預估電壓或電流值。
OutValue 項目: 顯示實際輸出的電壓或電流值。
Apply 項目：按壓 Apply 鍵，會將 Value 的預估電流或電壓值轉為實際輸出。

3) DA Data Choice:

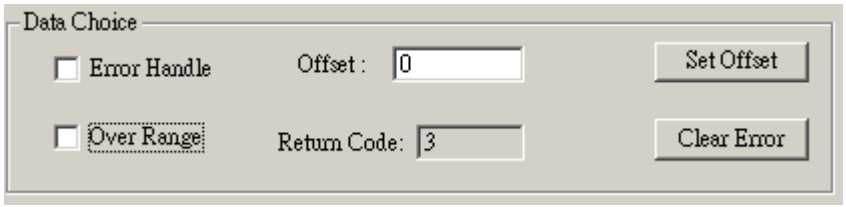


圖 3.272

Error Handle 項目：勾選本項目，可在重新執行本程式時保留上次離開時的狀態。
Over Range 項目：勾選本項目，可使電壓或電流輸出值增加 10%”。
Set Offset 項目：設定 DA 的偏差量。
ClearError 項目：將 Rtn Code 所顯示之 Error 狀態清除。

4) 若選擇圖 3.271 的「DA Mode」選項，則執行下列程序：

```
/* 先將 Offset 值歸 0 */
rt = _DMC_01_rm_04da_set_output_offset_value(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,0);
/* 再將 Output Disable */
rt = _DMC_01_rm_04da_set_output_enable(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,0);
/* 設定 DA 選擇模式 */
rt = _DMC_01_rm_04da_set_output_range(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,DAmode);
```

若選擇圖 3.271 的「Apply」選項，則執行下列程序：

```
/*  設定 DA 輸出值  */
rt = _DMC_01_rm_04da_set_output_value(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,AppValue);
```

若勾選圖 3.272 的 Error Handle 選項，則執行下列程序：

```
/* Enable Error Handle 功能*/
rt =_DMC_01_rm_04da_set_output_error_handle(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,1);
```

若勾選圖 3.272 的 Over Range 選項，則執行下列程序：

```
/* Enable Over Range 功能*/
rt =_DMC_01_rm_04da_set_output_overrange(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,1);
```

若點選圖 3.272 的「Set Offset」選項，則執行下列程序：

```
/*  將輸入之 Offset 值設定進去 DA */
rt =_DMC_01_rm_04da_set_output_offset_value(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,offset);
```

若點選圖 3.272 的「Clear Error」選項，則執行下列程序：

```
/*  將 Error  狀態清除  */
rt =_DMC_01_rm_04da_set_output_error_clear(gDMCCardNo,DANodeID,SlotID,
    ChannelDA,1);
```

5) 離開程序

按壓圖 3.270 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作“_DMC_01_reset_card”與_DMC_01_close”，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.28　位置比較 Compare

3.28.1　函式列表

表 3.28

函式名稱
_DMC_01_set_compare_channel_position
_DMC_01_get_compare_channel_position
_DMC_01_set_compare_ipulser_mode
_DMC_01_set_compare_channel_direction
_DMC_01_set_compare_channel_enable
_DMC_01_set_compare_channel_trigger_time
_DMC_01_set_compare_channel_one_shot
_DMC_01_set_compare_channel_source
_DMC_01_channel0_position_cmp
_DMC_01_channel1_output_enable
_DMC_01_channel1_output_mode
_DMC_01_channel1_get_io_status
_DMC_01_channel1_get_gpio_out
_DMC_01_channel1_position_compare_table
_DMC_01_channel1_position_compare_table_level
_DMC_01_channel1_position_compare_table_cnt
_DMC_01_set_compare_channel_polarity

■　**屬性**

PCI_DMC_B01 軸卡	PCI_DMC_F01 軸卡
●	

3.28.2 流程圖

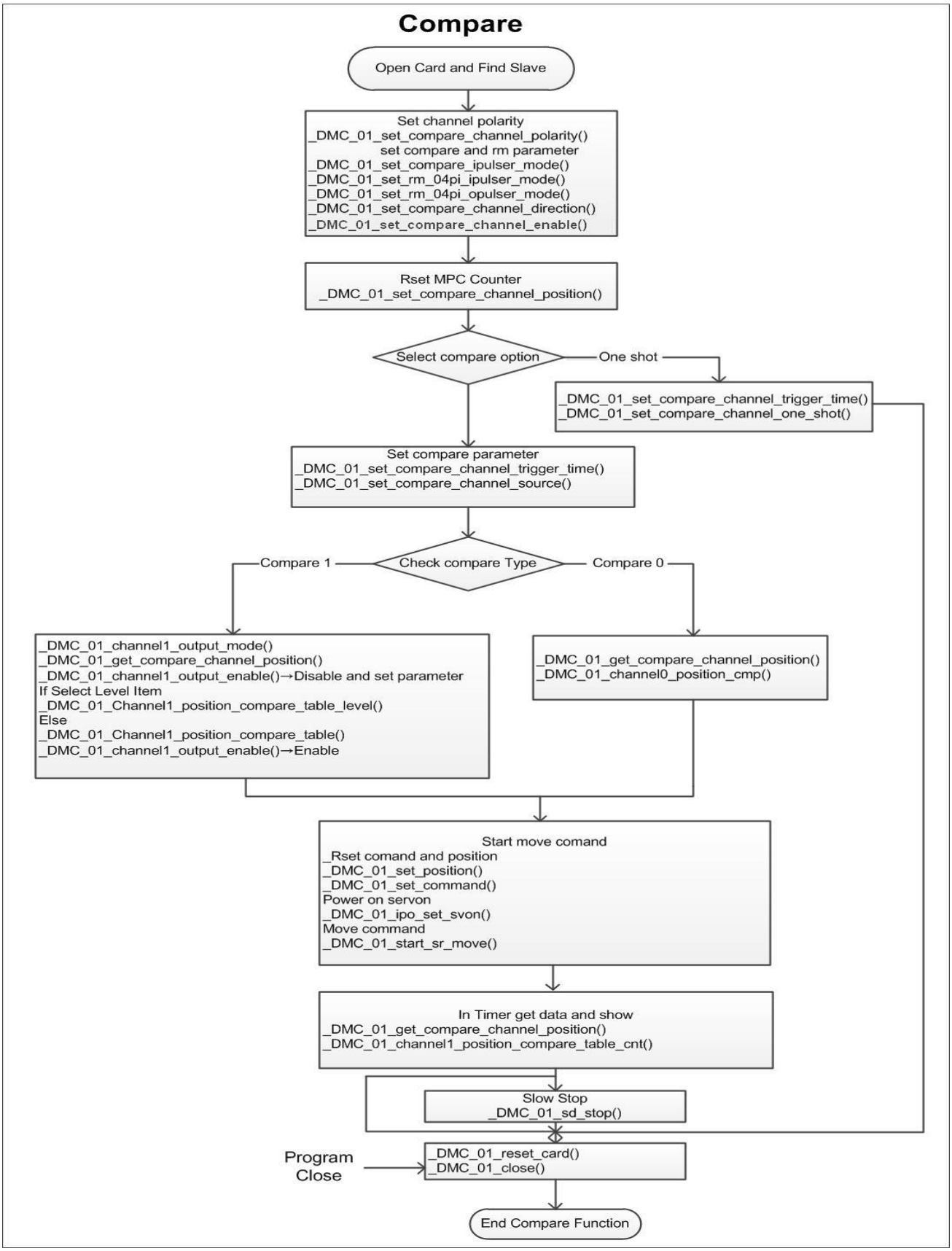


圖 3.280

3.28.3　應用範例

程式外觀

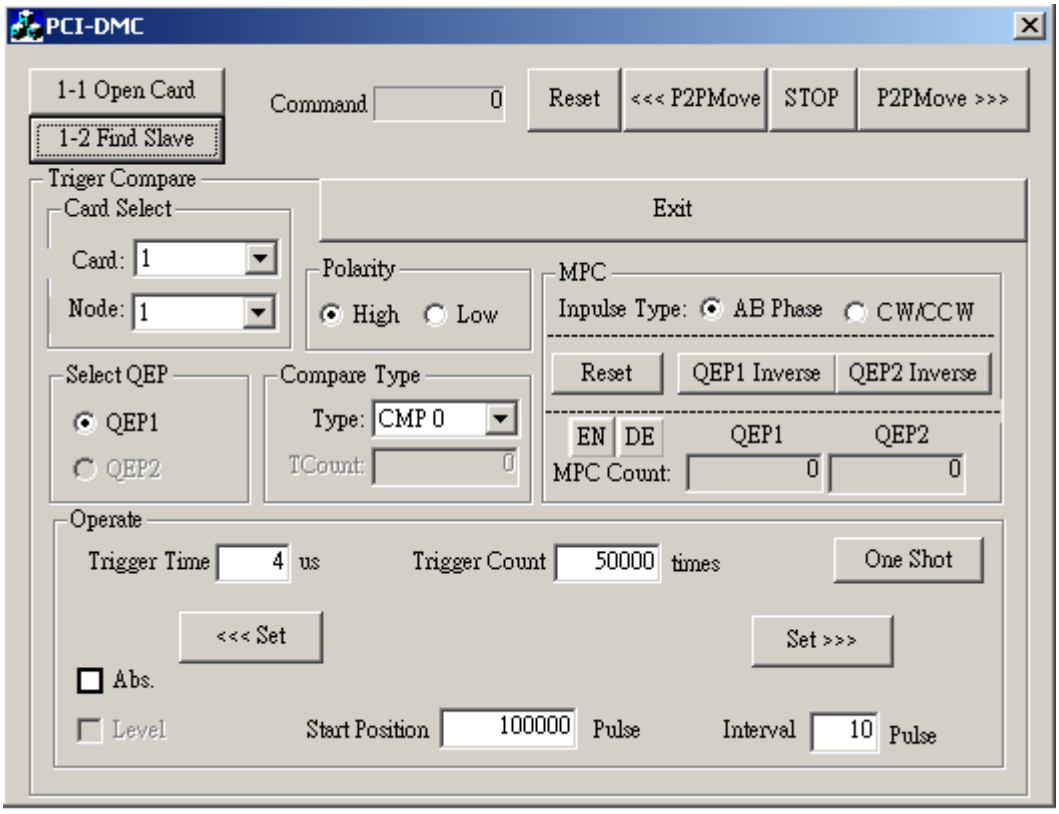


圖 3.281

1) 介面卡開啟與初始化

※首先確認您安裝的為 PCI-DMC-B01 介面卡。

按壓圖 3.281 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.281 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) 卡號、站號及 QEP 選擇:

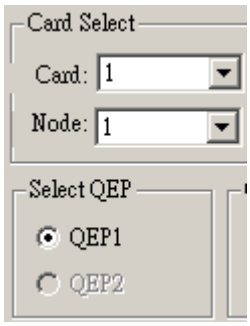


圖 3.282

Card 項目：輸入欲使用之 PCI_DMC_B01 卡片編號。
Node 項目：選擇使用站號，應與 QEP1 與 QEP2 選擇一致。
QEP1 項目: 例如:選擇使用 Channel0，站號應選擇 Node1。
QEP2 項目: 例如:選擇使用 Channel1，站號應選擇 Node2。

3) Compare Type Select 及 Polarity 設定:

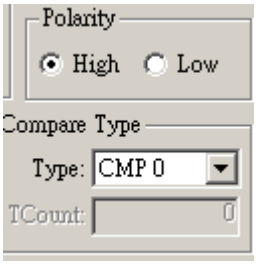


圖 3.283

High 項目：勾選本項目，執行以下設定: **0: High**
rt = _DMC_01_set_compare_channel_polarity(CpCardNo,0);
Low 項目：勾選本項目，執行以下設定: **1: Low**
rt = _DMC_01_set_compare_channel_polarity(CpCardNo,1);
Type 項目：選擇 Comapre1 或是 Compare2 功能。
rt = _DMC_01_set_compare_channel_source(CpCardNo,Compare_Type,CpQEP);
//Compare_type:0=COMP1,1=COMP2
TCount 項目：Trigger 實際執行次數。

4) MPC 參數設定:

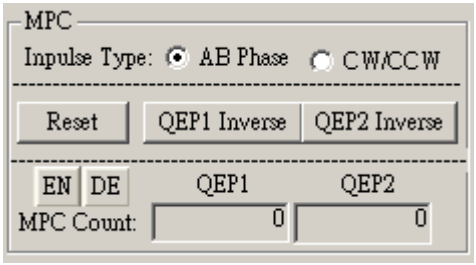


圖 3.284

EN/DE 項目：開啟/關閉 Compare 功能，執行以下設定。

rt = _DMC_01_set_compare_channel_enable(CpCardNo,channel,1); //1：開啟

Inpulse Type 項目：選擇 AB Phase 或是 CW/CCW 的模式。執行以下設定:

rt = _DMC_01_set_compare_ipulser_mode(CpCardNo,mode);

// 0: AB Phase　1: CW/CCW

Reset 項目：按下此鍵，將 QEP1 及 QEP2 的 MPC Count 清除,執行以下設定。

rt = _DMC_01_set_compare_channel_position(CpCardNo,channel,0);

QEP1 Inverse 項目：反向，按下此鍵，執行以下設定:

rt = _DMC_01_set_compare_channel_direction(CpCardNo,0,dir); // dir:0 or 1

QEP2 Inverse 項目：反向，按下此鍵，執行以下設定:

rt = _DMC_01_set_compare_channel_direction(CpCardNo,1,dir); // dir:0 or 1

5) Operate 相關資料設定:

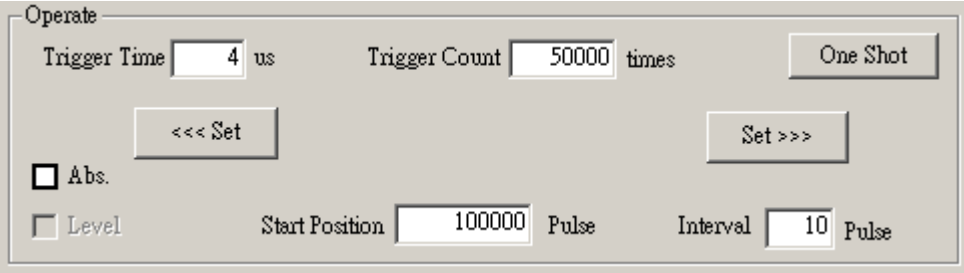


圖 3.285

Trigger Time 項目：輸入一個 Trigger 致能時間。
Trigger count 項目：輸入 Trigger 總致能次數。
Start Position 項目：輸入 Trigger 致能起始位置。
Interval 項目：輸入 Trigger 致能頻率，例如:輸入 10 表示每 10 個 Pulse 致能一次。
Abs 核取方塊：若您欲參照絕對座標的方式進行 Trigger 致能，必須勾選它。
Level 核取方塊：勾選它，執行以下設定:

```
rt = _DMC_01_channel1_output_mode(CpCardNo,mode);
// mode: 0 表示 Normal 型態， 1 表示自訂型態(詳見 39 章 Compare API 說明)
```

One Shot 項目：按下此鍵，只打一個 Trigger，執行以下設定:

```
rt = _DMC_01_set_compare_channel_trigger_time(CpCardNo,compare_channel,
      time_us); // time_us=Trigger time
rt = _DMC_01_set_compare_channel_one_shot(CpCardNo,compare_channel);
```

Set 項目：可選擇>>>及<<<這兩個方向，按下此鍵，執行以下設定:

```
rt = _DMC_01_get_compare_channel_position(CpCardNo,compare_channel,
      position);
rt = _DMC_01_set_compare_channel_trigger_time(CpCardNo,compare_channel,
      time_us); // time_us = Trigger time
If CompareType=CMP1     //Compare1
  rt = _DMC_01_channel0_position_cmp(CpCardNo,start,dir,interval);
  // dir->0:CMP1，1:CMP2
else  //Compare2
 先將 output disable
 rt = _DMC_01_channel1_output_enable(CpCardNo,0); // 0:off 1:on
 若 Level 核取方塊是勾選的，則執行下列設定:
 rt = _DMC_01_channel1_position_compare_table(CpCardNo,pos_table,tab_size);
 若未勾選，則執行下列設定:
 rt = _DMC_01_channel1_position_compare_table_level(CpCardNo,pos_table,
     level_table,tab_size);
```

最後將 output enable

```
rt = _DMC_01_channel1_output_enable(CpCardNo,1); // 0:off 1:on
```

6) Command 顯示及測試操作區:

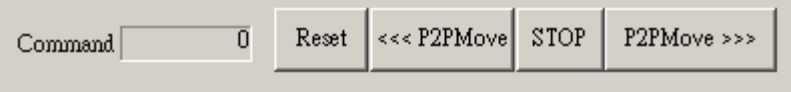


圖 3.286

Reset 項目：按下此鍵會進行 Motion 歸零動作。
P2PMove 項目：按下此鍵會進行 Motion 正向或負向動作。
STOP 項目：按下此鍵會將 Motion 動作停止。
Command 項目：顯示 Motion 動作目前的位置。

7) 離開程序

按壓圖 3.281 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.29 Axis Group

3.29.1 函式列表

表 3.29

函式名稱
_DMC_01_set_group

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	

3.29.2　流程圖

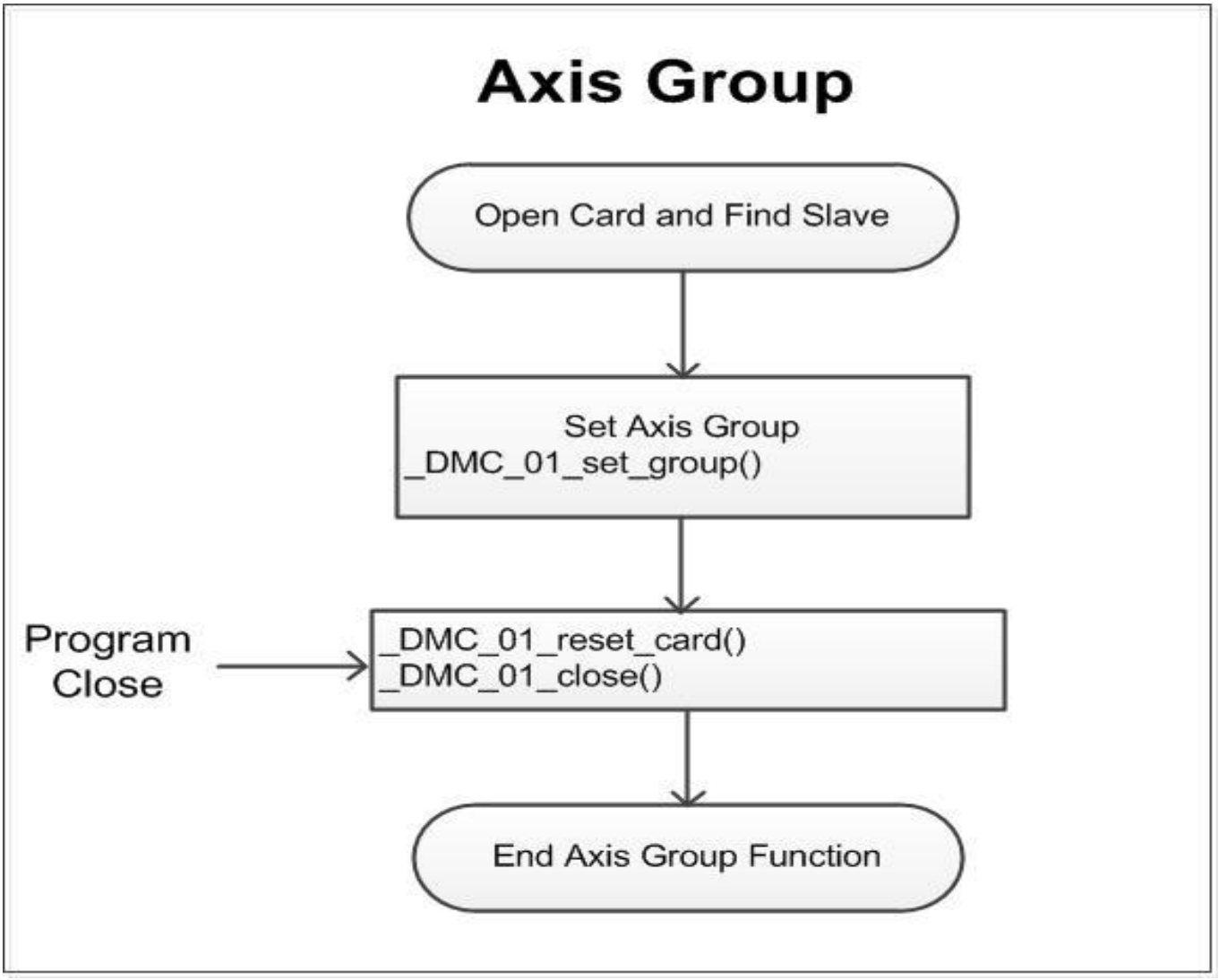


圖 3.290

3.29.3 應用範例

程式外觀

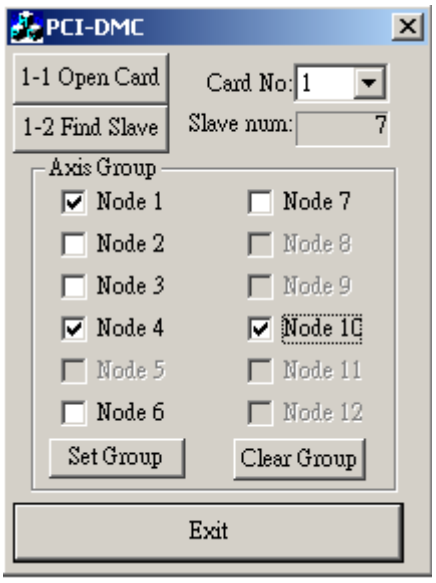


圖 3.291

1) 介面卡開啟與初始化
按壓圖 3.291 中的「Open card」鍵開始執行介面卡初始化。
按壓圖 3.291 中的「Find Slave」鍵開始搜尋串接模組。
關於此按鍵的相關敘述，可參考第 3.1.3 節「介面卡初始化與切換操作模式」說明。

2) 卡號、站數顯示:

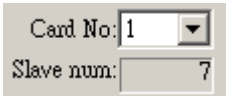


圖 3.292

Card num 項目：顯示卡片張數。
Slave num 項目：顯示站數。

3) Group 設定及解除:

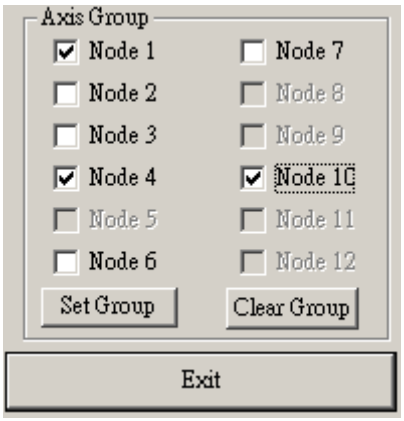


圖 3.293

Nodexx 項目：顯示站號詳細資料。

Set Group 項目：1.一張卡最多可以設定六個群組。

2.假設設定 Node1，Node4，Node10 為一個群組，表示當連線是在 Node4 發生異常，那麼 Node1，4，10 會一起停止，其他 Node6，7 則不受影響，可維持正常運作。

3.當選好要設為同一群組的 Node 目標時，點選"Set Group"將執行以下設定。

rt = _DMC_01__DMC_01_set_group (gDMCCardNo, NodeID, SlotID, NodeNum, 1); //1:表示將所選擇的 Node 設定為同一群組。

Clear All Group 項目：當點選"Clear All Group"將執行以下設定。

rt = _DMC_01_set_group(gDMCCardNo, (U16*)1, 0, 1, 0);

//0:表示將本卡所有的群組設定都清除

4) 離開程序

按壓圖 3.293 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

3.30 DDA 應用 DDA Axes Move

3.13.1 函式列表

表 3.30

函式名稱
_DMC_01_enable_dda_mode
_DMC_01_set_dda_data

■ 屬性

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.30.2 應用範例

程式外觀

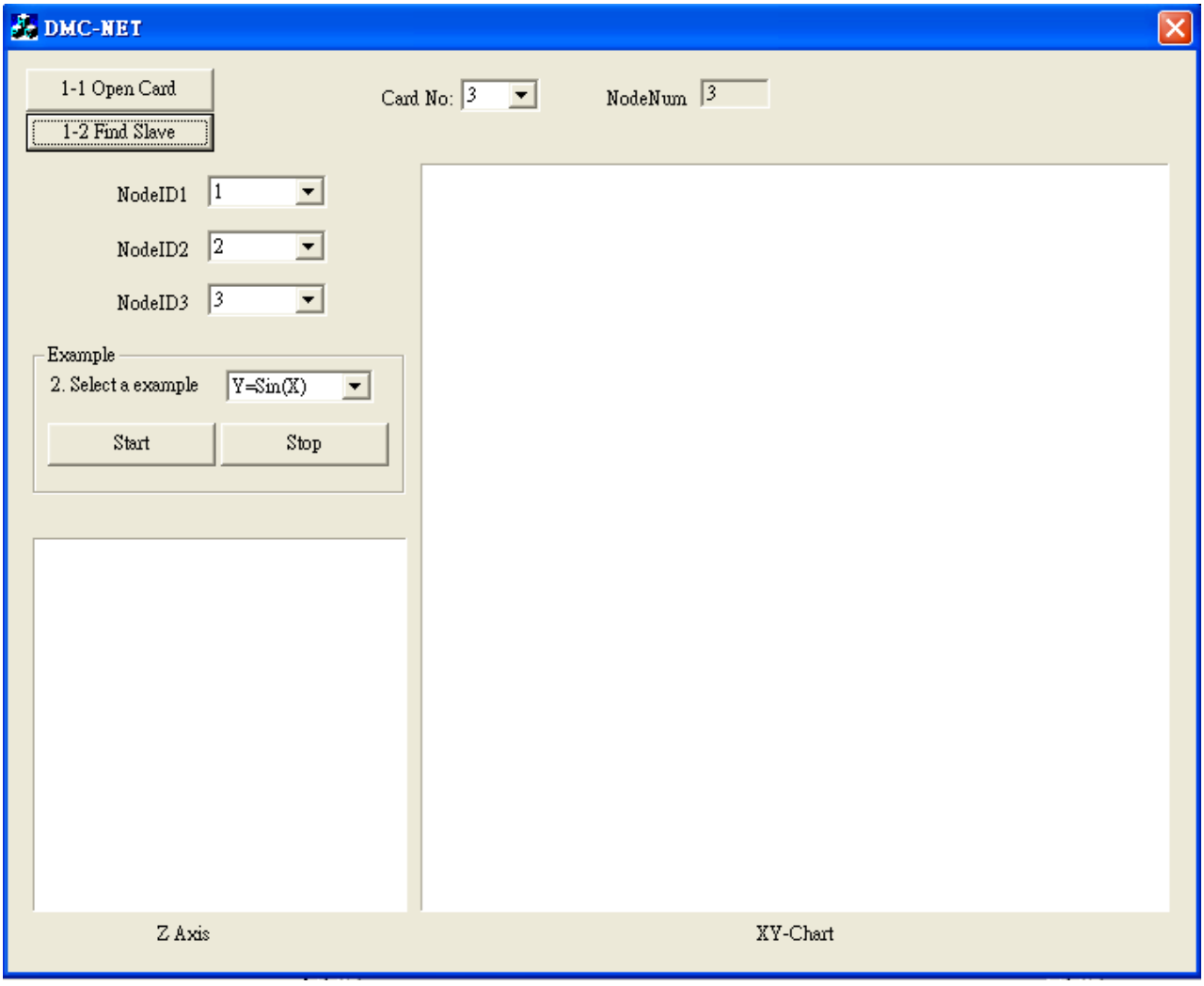


圖 3.300

6) 介面卡開啟與初始化

按壓圖 3.130 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3.130 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

7) 卡號、站數顯示:

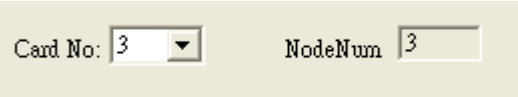


圖 3.301

Card num 項目：顯示卡片張數。

Slave num 項目：顯示站數。

8) Example 參數設定及操作:

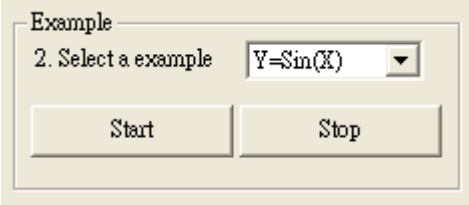


圖 3.302

Select a example 項目：有 Y=Sin(X)可供選擇。

Start 項目：當點選”Start”鍵將執行以下設定。

_DMC_01_enable_dda_mode(CardNo, enable)
_DMC_01_set_dda_data(CardNo, * abs_pos)

參數：

CardNo：卡號。

enable ：bit0~12 代表軸 1~12，為 1 時表示該軸啟用 DDA 功能。

abs_pos：1X12 陣列，代表軸 1~12 的絕對位置。

此例是以每 1ms 為時間單位計算出 X 軸需行走 t*pi/720(pps)的距離，Y 軸行走 Sin(X)的距離。並且讓 Z 軸的始終保持切線方向，如圖 3.303。

(以下解說使用 C 語言，並且使用狀態機型式來執行，詳情請參考 Sample)

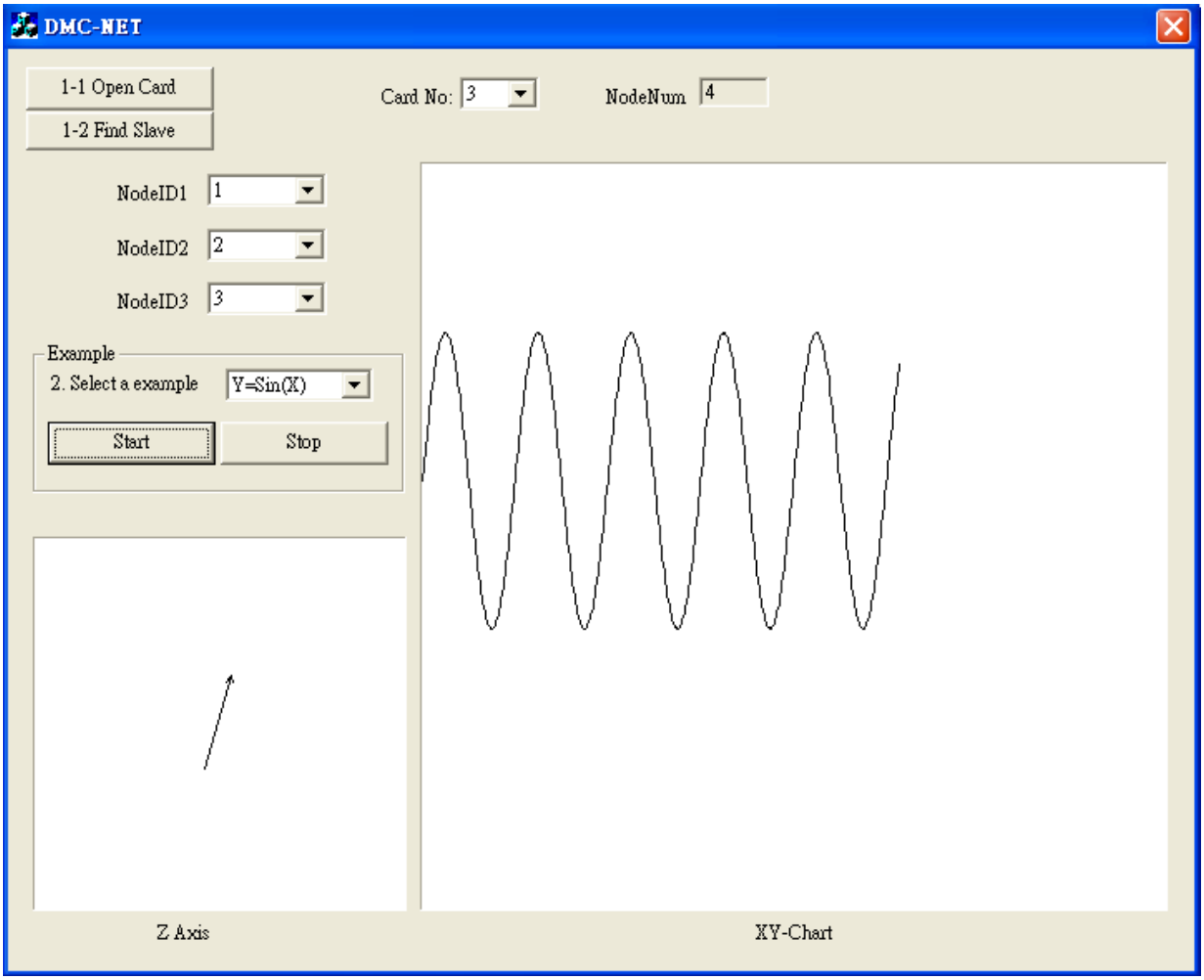


圖 3.303

```
/* MotionStep1.預先填入 100ms 的座標資料到 DDA Buffer 內  */
rc= _DMC_01_get_dda_cnt(gDMCCardNo, &i);
if (i < 100)
{
	for (i=0;i<100;i++)
	{
		X=(i*PI/720);
		Y=32000*sin(X);
		Z=9000*sin(X);
		DDA_Table[i][gpNodeID[2]-1]=Z;
		DDA_Table[i][gpNodeID[1]-1]=Y;
		DDA_Table[i][gpNodeID[0]-1]=1280000*X/360;
	}
	X_Counter = i;
	rc= rc+_DMC_01_enable_dda_mode(gDMCCardNo, DDA_Enable);
}
if(rc == 0)
{	MotionStep = 2;	}

/* MotionStep2.若 DDA Buffer 內資料低於 100 筆，產生的後 100ms 的座標資料到 DDA Buffer 內*/
rc= _DMC_01_get_dda_cnt(gDMCCardNo, &i);
if (i < 100)
{
	for (i=0;i<100;i++)
	{
		X=((X_Counter+i)*PI/720);
		Y=32000*sin(X);
		Z=9000*sin(X);
		DDA_Table[i][gpNodeID[2]-1]=Z;
		DDA_Table[i][gpNodeID[1]-1]=Y;
		DDA_Table[i][gpNodeID[0]-1]=1280000*X/360;
		rc= _DMC_01_set_dda_data(gDMCCardNo, DDA_Table[i]);
	}
X_Counter=X_Counter+i;
}
/*填入 7.5 秒的座標資料後結束*/
if (X_Counter > 7500)
{	MotionStep =0 ;	}
```

3.99 歷程記錄器

3.99.1 函式列表

表 3.99

函式名稱
_misc_set_record_debuging
_misc_open_record_debuging_file

■ **屬性**

PCI_DMC_A01/B01 軸卡	PCI_DMC_F01 軸卡
●	●

3.99.2　流程圖

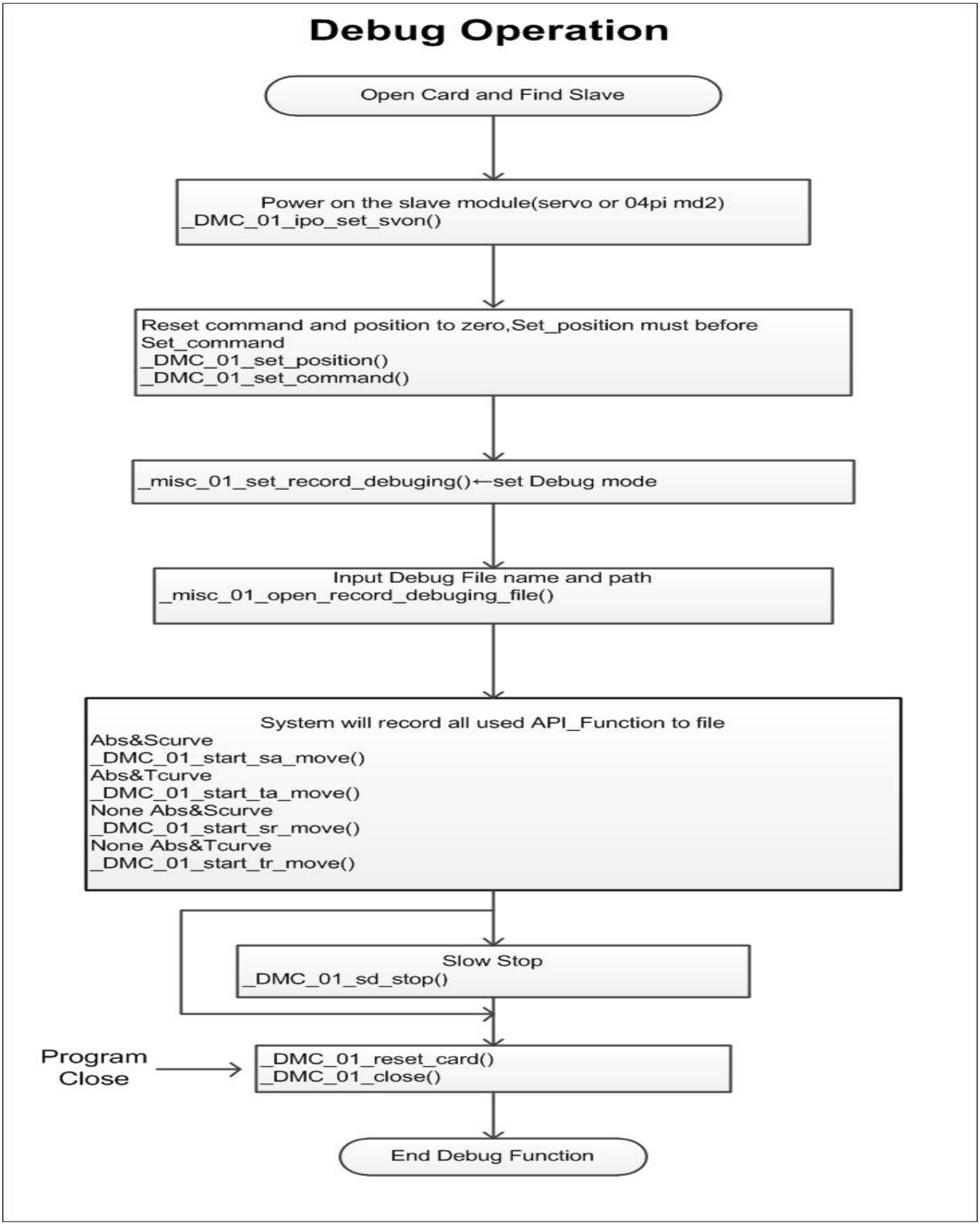


圖 3.990

3.99.3 應用範例

程式外觀

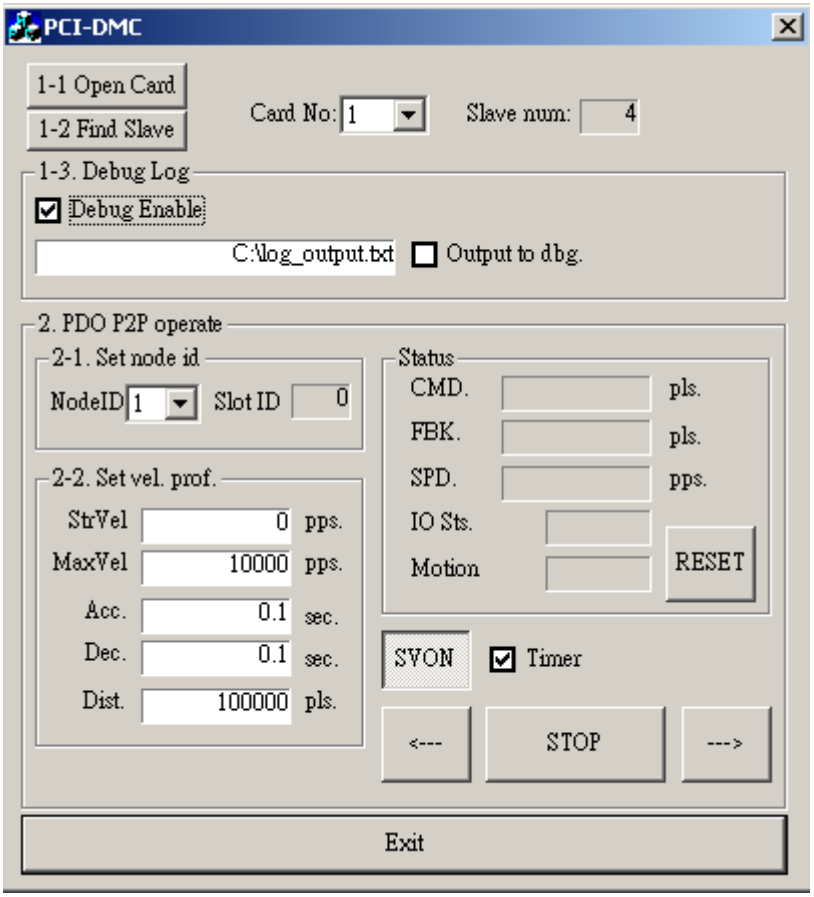


圖 3.991

1) 介面卡開啟與初始化

按壓圖 3. 991 中的「Open card」鍵開始執行介面卡初始化。

按壓圖 3. 991 中的「Find Slave」鍵開始搜尋串接模組。

關於初始化介面卡的詳細說明，請參閱第 3.1.3 節「開啟介面卡」和「初始化介面卡」。

2) Debug 的功能啟動與否

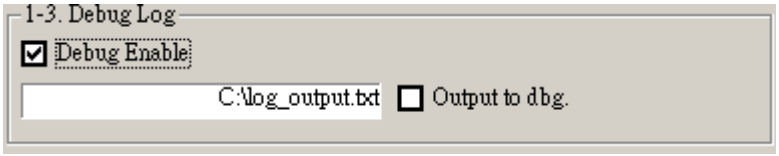


圖 3.992

欲啟動 debug 功能，必須勾選圖 3.992 中的 Debug Enable 選項並執行以下程序：

```
rt = _misc_set_record_debuging(enable);
// enable 引數值為 1 時，表示 debug 功能開啟
```

3) 填入記錄表存放的路徑，並選擇該記錄的輸出方式

當您欲觀察運動控制產生的歷程記錄，您必須輸入圖 3.992 中的檔案存放路徑及檔名並執行以下程序：

```
rt = _misc_open_record_debuging_file(gDMCCardNo ,file_name, open);
// “file_name”為一個用來存放 debug 產生的記錄輸出的檔案路徑的字元陣列變數。而另一個“open”變數則是用來決定該 debug 記錄是否輸出至您所設置檔案路徑的文件裡。即當”open”的值為 1 時，debug 記錄將會輸出存放至您設定的檔案路徑文件中；反之，若其值為 0，則該記錄僅會顯示在 visual studio 6 集成開發環境下的 debug 窗口。
```

4) 開始操作任一在 PDO 模式下運行的運動控制

5) 歷程記錄的輸出

圖 3.993 表示運動控制所使用的函數與值變化的歷程顯示在 visual studio 6 集成開發環境的 debug 窗口。另外，圖 3.994 表示該函數與值變化的歷程記錄將輸出到"log_output.txt"的文件中。

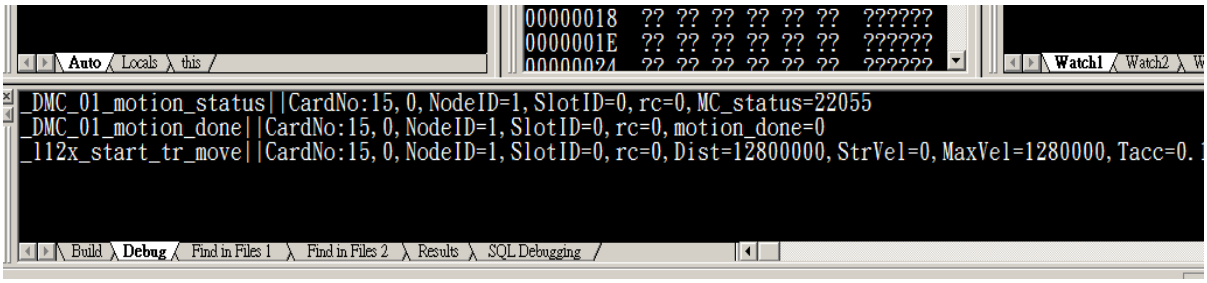


圖 3.993

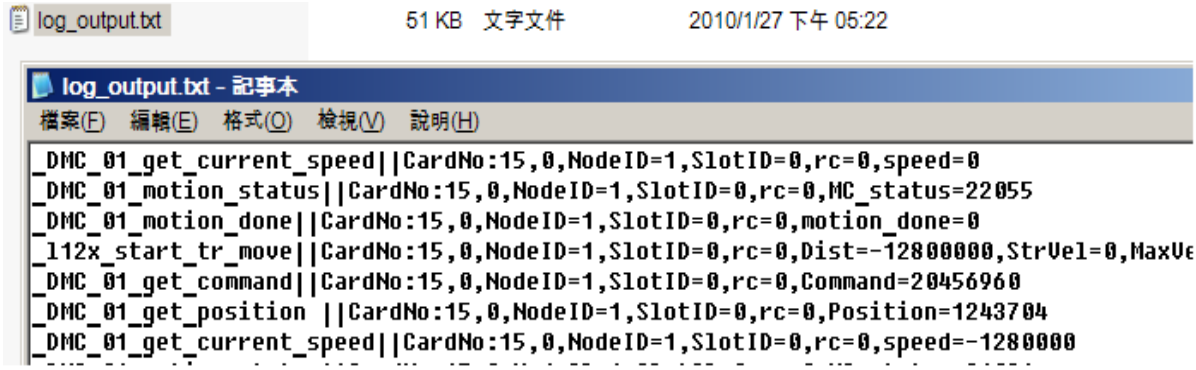


圖 3.994

6) 停止運動

按壓圖 3.991 中的「STOP」鍵開始執行一般停止：

```
rt = _DMC_F01_sd_stop(gDMCCardNo, NodeID, SlotID, 0, dec);
```

此例中使用一般停止的方式停止運動位移，此方式是設定減速時間設，於設定減速時間內停止運動。關於停止運動的詳細內容，請參閱後敘相關「停止運動控制 API」的章節說明。

7) 離開程序

按壓圖 3.991 中的「Exit」鍵開始執行程序的離開與關閉。

欲跳脫函式必須操作"_DMC_01_reset_card"與_DMC_01_close"，關於此兩個 API 函式應用，請參閱 3.1.3 節「離開程序」的函式操作。

第四章 控制 API

4.1 資料型別與範疇

在安裝目錄“inc\VC\”資料夾中，存放一“TYPE_DEF.H”檔案，此檔案定義了一般的資料型別。

如下表 4.1，其標示該檔案中定義的型別、名稱、表示的意義、與值的範疇。

表 4.1

名稱	描述	值的範疇
U8	8-bit ASCII 字符	0 到 255
I16	16-bit 有號整數	-32768 到 32767
U16	16-bit 無號整數	0 到 65535
I32	32-bit 有號長整數	-2147483648 到 2147483647
U32	32-bit 無號長整數	0 到 4294967295
F32	32-bit 單精度浮點數	-3.402823E38 到 3.402823E38
F64	64-bit 雙精度浮點數	-1.797683134862315E308 到 1.797683134862315E309
Boolean	布林值	TRUE, FALSE

4.2　函數描述

表 4.2

註	硬體初始化 API	
	_DMC_01_open	當應用程式啟動時，初始化系統資源
	_DMC_01_close	釋放所有系統資源
	_DMC_01_get_CardNo_seq	取得主機上所有 PCI-DMC-A01 介面卡號
	_DMC_01_pci_initial	初始此 PCI 介面卡
	_DMC_01_get_card_version	取得軸卡版號
	介面卡操作 API	
	_DMC_01_initial_bus	初始外部匯流排 (External Bus)
	_DMC_01_start_ring	起始 ring 的通訊
	_DMC_01_get_device_table	取得裝置列表
	_DMC_01_get_node_table	取得站號列表
	_DMC_01_reset_card	重置該介面卡
	_DMC_01_check_nodeno	查驗該站號是否已存在
	_DMC_01_get_master_connect_status	取得 Master Card 與擴充模組目前的連線狀態
	_DMC_01_get_mailbox_Error	取得 MailBox 報錯的次數
	_DMC_01_get_mailbox_cnt	取得 MailBox 計數值
	_DMC_01_get_dsp_cnt	取得中斷計數值
	_DMC_01_set_dio_output	設置 GPIO 輸出腳位狀態
	_DMC_01_get_dio_output	取得 GPIO 輸出腳位狀態
	_DMC_01_get_dio_input	取得 GPIO 輸入腳位狀態
	_DMC_01_get_cycle_time	取得當前查找裝置的循環時間
	_DMC_01_initial_bus2	初始外部所有匯流排 (External Bus)
	_DMC_01_motion_cnt	取得 MailBox 及 DSP 計數值
	_DMC_01_pdo_Rx_error	取得 PDO 收封包錯誤記錄
	_DMC_01_pdo_Tx_error	取得 PDO 送封包錯誤記錄
	_DMC_01_get_motion_error_code	取得 Motion 動作的錯誤回傳值
	_DMC_01_get_motion_error_cnt	取得 Motion 動作的錯誤次數
	_DMC_01_get_DLL_path	得到 PCI_DMC.dll 詳細路徑資料
	_DMC_01_get_DLL_version	得到 PCI_DMC.dll 版本資料。
	_DMC_01_get_DLL_path_Single	得到 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細路徑資料
	_DMC_01_get_DLL_version_Single	得到 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 版本資料。
	伺服驅動器參數讀取/寫入 API	

	_DMC_01_read_servo_parameter	讀取伺服驅動器參數資料
	_DMC_01_write_servo_parameter	寫入伺服驅動器參數資料
	使用 SDO 協定 API	
	_DMC_01_check_canopen_lock	SDO 模式下，檢查是否可以再執行下一筆命令
	_DMC_01_get_canopen_ret	取得 CANOPEN 傳回的資訊(SDO 相關數據)
	_DMC_01_set_pdo_mode	設置使用的 CANopen 協定(PDO 或 SDO)
■	_DMC_01_send_message	發送 SDO 命令信息至資料緩衝區，會等待資料送達伺服端且讀回資料相符才離開
■	_DMC_01_send_message2	發送 SDO 命令信息至資料緩衝區，只確認資料送達伺服端就離開
■	_DMC_01_send_message3	發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝區
■	_DMC_01_send_message4	發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝區
	_DMC_01_read_message	讀取最後一筆 SDO 命令信息至資料緩衝區
	_DMC_01_read_message2	讀取最後一筆 SDO 命令信息至資料緩衝區，並傳回已完成的次數值
■	_DMC_01_get_message	取得 SDO 命令信息至資料緩衝區
■	_DMC_01_reset_sdo_choke	當 SDO 命令因故阻塞時，重置 SDO
	_DMC_01_get_sdo_retry_history	取得 SDO 重新發送的次數
	點對點運動控制使用封包協定 API	
▲	_DMC_01_set_sdo_driver_speed_profile	使用封包協定的速度參數配置
▲	_DMC_01_start_sdo_driver_r_move	起始一個相對性的運動位移
▲	_DMC_01_start_sdo_driver_a_move	起始一個絕對性的運動位移
▲	_DMC_01_start_sdo_driver_new_position_move	變換新的 position 值進行運動位移
	原點復歸運動控制使用封包協定 API	
	_DMC_01_set_home_config	設置原點復歸各項配置
	_DMC_01_set_home_move	設置開始運行原點復歸運動
	_DMC_01_set_home_config2	設置原點復歸各項配置
	_DMC_01_set_home_config2_offset2	重新設置原點復歸 offset 值
	_DMC_01_set_home_move2	設置開始運行原點復歸運動
■	_DMC_01_escape_home_move	停止原點復歸運動
	速度運動控制使用封包協定 API	
■	_DMC_01_set_velocity_mode	速度運動控制參數配置
■	_DMC_01_set_velocity	設置開始運行速度運動
■	_DMC_01_set_velocity_stop	停止速度運動控制
■	_DMC_01_set_velocity_torque_limit	在 velocity 模式時設定最大 torque 限制
	轉矩運動控制使用封包協定 API	

■	_DMC_01_set_torque_mode	轉矩運動控制參數配置
■	_DMC_01_set_torque	設置開始運行轉矩運動
■	_DMC_01_set_torque_stop	停止轉矩運動
■	_DMC_01_set_torque_velocity_limit	在 torque 模式時設定最大 velocity 限制
	使用 PDO 協定 API	
	_DMC_01_ipo_set_svon	在 PDO 協定下 Servo ON/OFF 設置
	_DMC_01_get_buffer_length	取得仍未實行的運動命令
	_DMC_01_command_buf_clear	清除間隔時間(buffer dwell 計數值)
▲	_DMC_01_buf_dwell	兩個運動命令之間的間隔時間
▲	_DMC_01_set_group	設定關連性群組
	_DMC_01_set_dmc_mode	將模式切成 DMC_NET 模式
	停止運動控制 API	
■	_DMC_01_emg_stop	Buffer 內所有運動指令將立即停止
■	_DMC_01_sd_stop	Buffer 內所有運動指令將依減速時間減速停止
■	_DMC_01_sd_abort	當前正在執行的運動指令將依減速時間停止
	_DMC_01_set_sd_mode	Sd_stop 的模式設定
	_DMC_01_set_sld_mode	設定 SLD 訊號觸發，SLD 運動模式。
	_DMC_01_set_sld_stop	啟動/關閉 SLD 模式設定
	_DMC_01_set_sd_time	設定減速時間。
	運動狀態 API	
	_DMC_01_motion_done	傳回 Master Card 目前實行的運動階段
	_DMC_01_motion_status	傳回 Master Card 目前的運動狀態
	_DMC_01_set_01_ph_inposition	設定 01PH INP 到位訊號的區間範圍
	_DMC_01_get_01_ph_inposition	取得目前 01PH INP 到位訊號設定的區間範圍
	運動計數值 API	
	_DMC_01_get_command	取得 command 計數值
	_DMC_01_set_command	設置新的 command 計數值
	_DMC_01_get_position	取得目前 position 計數值
■	_DMC_01_set_position	設置新的 position 計數值
	_DMC_01_get_target_pos	取得目的位置的 position 值
	_DMC_01_get_torque	取得傳回目前的轉矩計數值
	_DMC_01_get_current_speed	取得目前的運動分量速度
	_DMC_01_get_current_speed_rpm	取得目前每分鐘的轉速乘上 10 倍的值
	軟體極限 API	
	_DMC_01_set_soft_limit	設置軟體正/負極限的參考值
	_DMC_01_enable_soft_limit	設置啟用極限開啟與否和碰觸極限後的停止方式
	_DMC_01_disable_soft_limit	關閉軟體極限
	_DMC_01_get_soft_limit_status	取得運動期間軟體正/負極限的狀態

	單軸運動控制 API	
	_DMC_01_start_tr_move	參照 T-curve 速度剖面下的相對座標進行運動位移
	_DMC_01_start_sr_move	參照 S-curve 速度剖面下的相對座標進行運動位移
	_DMC_01_start_ta_move	參照 T-curve 速度剖面下的絕對座標進行運動位移
	_DMC_01_start_sa_move	參照 S-curve 速度剖面下的絕對座標進行運動位移
	_DMC_01_p_change	以新的位置值替換目前的位置設置
	_DMC_01_v_change	以新的速度值設置替換目前的運動速度
	_DMC_01_start_tr_move_2seg	參照 T-curve 速度剖面下的相對座標進行二段速度運動位移
	_DMC_01_start_sr_move_2seg	參照 S-curve 速度剖面下的相對座標進行二段速度運動位移
	_DMC_01_start_ta_move_2seg	參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移
	_DMC_01_start_sa_move_2seg	參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移
	_DMC_01_start_tr_move_2seg2	參照 T-curve 速度剖面下的相對座標進行二段速度運動位移
	_DMC_01_start_sr_move_2seg2	參照 S-curve 速度剖面下的相對座標進行二段速度運動位移
	_DMC_01_start_ta_move_2seg2	參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移
	_DMC_01_start_sa_move_2seg2	參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移
	_DMC_01_feedrate_overwrite	改變運動向量速度或向量速度比率
	_DMC_01_start_v3_move	增加 EndVel 的單軸運動位移
	二軸線性補間運動控制 API	
	_DMC_01_start_tr_move_xy	參照 T-curve 速度剖面相對座標進行兩軸線性補間運動
	_DMC_01_start_sr_move_xy	參照 S-curve 速度剖面相對座標進行兩軸線性補間運動
	_DMC_01_start_ta_move_xy	參照 T-curve 速度剖面絕對座標進行兩軸線性補間運動
	_DMC_01_start_sa_move_xy	參照 S-curve 速度剖面絕對座標進行兩軸線性補間運動
	_DMC_01_start_v3_move_xy	增加 EndVel 的兩軸線性補間運動
	二軸圓弧補間運動控制 API	
	_DMC_01_start_tr_arc_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
	_DMC_01_start_sr_arc_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
	_DMC_01_start_ta_arc_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
	_DMC_01_start_sa_arc_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧

		補間運動(已知條件：中心座標、角度)
	_DMC_01_start_tr_arc2_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_sr_arc2_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_ta_arc2_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_sa_arc2_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_tr_arc3_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_sr_arc3_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_ta_arc3_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_sa_arc3_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_spiral_xy	兩軸螺旋運動(已知條件：X、Y 軸中心座標)
	_DMC_01_start_spiral2_xy	兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)
	_DMC_01_start_v3_arc_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、角度)
	_DMC_01_start_v3_arc2_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：終點座標、角度)
	_DMC_01_start_v3_arc3_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、終點座標)
	_DMC_01_start_v3_spiral_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標)
	_DMC_01_start_v3_spiral2_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)
	_DMC_01_start_v3_ellipse_xy	增加 EndVel 的兩軸橢圓運動(已知條件：X、Y 軸中心座標)
	三軸線性補間運動控制 API	
	_DMC_01_start_tr_move_xyz	參照 T-curve 速度剖面相對座標進行三軸線性補間運動
	_DMC_01_start_sr_move_xyz	參照 S-curve 速度剖面相對座標進行三軸線性補間運動
	_DMC_01_start_ta_move_xyz	參照 T-curve 速度剖面絕對座標進行三軸線性補間運動
	_DMC_01_start_sa_move_xyz	參照 S-curve 速度剖面絕對座標進行三軸線性補間運動
	_DMC_01_start_v3_move_xyz	增加 EndVel 的三軸線性補間運動
	三軸螺旋補間運動控制 API	
	_DMC_01_start_tr_heli_xy	參照 T-curve 速度剖面相對座標進行三軸螺旋補間運動
	_DMC_01_start_sr_heli_xy	參照 S-curve 速度剖面相對座標進行三軸螺旋

		補間運動
	_DMC_01_start_ta_heli_xy	參照 T-curve 速度剖面絕對座標進行三軸螺旋補間運動
	_DMC_01_start_sa_heli_xy	參照 S-curve 速度剖面絕對座標進行三軸螺旋補間運動
	_DMC_01_start_v3_heli_xy	增加 EndVel 的三軸螺旋補間運動
	_DMC_01_start_v3_sphere_xyz	三點成圓
	速度運動控制 API	
	_DMC_01_tv_move	參照 T-curve 速度剖面下的速度運動控制
	_DMC_01_sv_move	參照 S-curve 速度剖面下的速度運動控制
	同步運動控制 API	
	_DMC_01_sync_move	設置開始運行同步運動
	_DMC_01_sync_move_config	設置同步運動功能是否啟動
	遠端擴充模組控制 API	
	_DMC_01_get_rm_input_value	取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
	_DMC_01_set_rm_input_filter	設定遠端擴充模組輸入埠的軟體濾波等級
	_DMC_01_set_rm_input_filter_enable	設定遠端擴充模組輸入埠 bit 0 到 bit 15 軟體遮罩開啟與否
	_DMC_01_set_rm_output_value	設定遠端擴充模組輸出埠 bit 0 到 bit 15 的值
	_DMC_01_set_rm_output_value_error_handle	設定遠端擴充模組的發生錯誤時獲取的輸出值
	_DMC_01_get_rm_output_value	取得遠端擴充模組的輸出值
	_DMC_01_get_rm_output_value_error_handle	取得遠端擴充模組的輸出值，在發生錯誤時，系統是否將該值保留或重置
	_DMC_01_set_rm_output_active	設定遠端擴充模組輸出是否啟動
	電子手輪與 JOG 操作 API	
▲ ■	_DMC_01_set_rm_mpg_axes_enable	設定手輪運動控制
■	_DMC_01_set_rm_mpg_axes_enable2	手輪運動控制（可設定馬達圈數比例分母值）
■	_DMC_01_set_rm_jog_axes_enable	設定 JOG 運動控制
	四組脈波介面專用 API	
	_DMC_01_set_rm_04pi_ipulser_mode	設定脈波介面模組輸入相位模式
	_DMC_01_set_rm_04pi_opulser_mode	設定脈波介面模組輸出相位模式
	_DMC_01_set_rm_04pi_svon_polarity	設定 POWER ON（SVON）的準位
	_DMC_01_set_rm_04pi_DO2	設定 DO2 埠啟動
	_DMC_01_set_rm_04pi_homing_ratio	設置原點復歸速度增量比
	_DMC_01_rm_04PI_get_buffer	取得仍未執行的運動命令
	四組脈波介面（模式 1）運動控制 API	
	_DMC_01_rm_04pi_md1_start_move	RM04PI 在模式 1 下，進行單軸運動位移
	_DMC_01_rm_04pi_md1_v_move	RM04PI 在模式 1 下，進行速度運動控制
	_DMC_01_rm_04pi_md1_start_line2	RM04PI 在模式 1 下，進行兩軸線性補間運動
	_DMC_01_rm_04pi_md1_start_line3	RM04PI 在模式 1 下，進行三軸線性補間運動
	_DMC_01_rm_04pi_md1_start_line4	RM04PI 在模式 1 下，進行四軸線性補間運動

	_DMC_01_rm_04pi_md1_start_arc	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：中心座標、角度)
	_DMC_01_rm_04pi_md1_start_arc2	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：終點座標、角度)
	_DMC_01_rm_04pi_md1_start_arc3	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：中心座標、終點座標)
	_DMC_01_rm_04pi_md1_start_heli	RM04PI 在模式 1 下，進行三軸螺旋補間運動
	_DMC_01_rm_04pi_md1_p_change	RM04PI 在模式 1 下，將設定的新位置值替換目前的位置
	_DMC_01_rm_04pi_md1_v_change	RM04PI 在模式 1 下，將設定的新速度值替換目前的速度
	_DMC_01_rm_04pi_md1_set_gear	RM04PI 在模式 1 下，設定 Gear 功能啟動與其參數設置
	_DMC_01_rm_04pi_md1_set_soft_limit	RM04PI 在模式 1 下，設定是否啟動軟體極限
	_DMC_01_rm_04pi_md1_get_soft_limit_ status	RM04PI 在模式 1 下，取得目前 4 軸碰撞軟體極限的狀態
	_DMC_01_rm_04pi_md1_set_sld	設定 SLD 埠（DI3）開啟與其相關配置
	_DMC_01_rm_04pi_md1_get_mc_error_ code	當 alarm code 為 299 時，取得該 RM04PI 在模式 1 下，發生運動控制錯誤的訊息
■	_DMC_01_set_rm_04pi_ref_counter	RM04PI 在模式 1 下，選擇重新連線參考的位置
	四組類比輸出遠端擴充模組 API	
	_DMC_01_rm_04da_set_output_value	設定 DA 輸出數值
	_DMC_01_rm_04da_get_output_value	讀取 DA 輸出數值
	_DMC_01_rm_04da_get_return_code	讀取 DA 狀態值
■	_DMC_01_rm_04da_set_output_range	設定 DA 輸出範圍
	_DMC_01_rm_04da_set_output_enable	設定腳位輸出開啟/關閉
■	_DMC_01_rm_04da_set_output_overrange	設定輸出範圍增加 10%
	_DMC_01_rm_04da_set_output_error_clear	清除錯誤狀態
■	_DMC_01_rm_04da_set_output_error_handle	斷線保持 DA 原本設定狀態
■	_DMC_01_rm_04da_set_output_offset_value	設定 DA offset 數值
	_DMC_01_rm_04da_get_output_offset_value	讀取 DA offset 數值
	四組類比輸入遠端擴充模組 API	
■	_DMC_01_set_04ad_input_range	設定 AD 的 Input 範圍
	_DMC_01_get_04ad_input_range	取得目前 AD 的 Input 設定範圍
■	_DMC_01_set_04ad_conversion_time	設定 AD 轉換速度
	_DMC_01_get_04ad_conversion_time	取得目前 AD 所設定的轉換速度
	_DMC_01_get_04ad_data	讀取輸入之電壓數值
■	_DMC_01_set_04ad_average_mode	設定 AD 平均功能之模式

	_DMC_01_get_04ad_average_mode	讀取 AD 平均功能之模式
■	_DMC_01_set_04ad_input_enable	設定 AD Channel 是否致能 Input
	Slave 資訊	
	_DMC_01_get_devicetype	獲取 slave 裝置類型
	_DMC_01_get_slave_version	獲取 slave 裝置韌體版本
	監控參數 API	
	_DMC_01_set_monitor	設定欲監控的參數
	_DMC_01_get_monitor	取得監控的參數獲取的值
	_DMC_01_get_servo_command	取得伺服驅動器發送的命令值
	_DMC_01_get_servo_DI	取得伺服驅動器 DI 訊號的值
	_DMC_01_get_servo_DO	取得伺服驅動器 DO 訊號的值
	警報訊息 API	
	_DMC_01_set_ralm	輸出的伺服驅動器警示訊息重置
	_DMC_01_get_alm_code	取得 Slave 的錯誤資訊
	_DMC_01_master_alm_code	取得 Master Card 連線的錯誤資訊
	_DMC_01_slave_error	取得 slave 通訊中連續錯誤次數
	多軸運動控制 API	
	_DMC_01_multi_axes_move	設置 2 軸以上的運動控制
	_DMC_01_liner_speed_master	多軸直線運動運動速度設定
	_DMC_01_start_v3_multi_axes	增加 EndVel 的多軸(2 軸以上)的運動控制
	緩衝區操作 API	
	_DMC_01_set_trigger_buf_function	使用伺服驅動器 DI3（SLD）觸發 Motion 指令
	Interrupt API	
▲	_DMC_01_set_int_factor	設定中斷模式，共有 8 種模式。
▲	_DMC_01_int_enable	中斷設定致能
▲	_DMC_01_int_disable	中斷設定解除
▲	_DMC_01_get_int_count	中斷累計次數
▲	_DMC_01_get_int_status	查詢目前設定中斷的模式
▲	_DMC_01_Link_interrupt	連結一處理程序，當設定之中斷發生就導入處理。
	Security API	
▲	_DMC_01_read_security	Master Card 讀取指定記憶體區塊中的安全驗證資料
▲	_DMC_01_read_security_status	Master Card 獲取目前的記憶體是處於讀/寫的狀態
▲	_DMC_01_write_security	Master Card 將指定的安全資料寫入指定的記憶體區塊
▲	_DMC_01_write_security_status	Master Card 寫入安全資料前，將記憶體寫入功能致能

	_DMC_01_check_userpassword	Master Card 確認使用者對 Memory 讀/寫的權限
	_DMC_01_write_userpassword	Master Card 變更密碼
	_DMC_01_check_verifykey	Master Card 驗證碼驗證
	_DMC_01_write_verifykey	Master Card 寫入驗證碼
	_DMC_01_read_serialno	Master Card 讀取產品序號
	_misc_slave_check_userpasswaord	Slave(04PI)確認使用者對 Memory 讀/寫的權限
	_misc_slave_write_userpasswaord	Slave(04PI)變更密碼
	_misc_slave_get_serialno	Slave(04PI)讀取產品序號
	_misc_security	使用者 Key+SerialNo 經加密產生驗證碼
	_misc_slave_write_verifykey	Slave(04PI)寫入驗證碼
	_misc_slave_check_verifykey	Slave(04PI)驗證碼驗證
	_misc_slave_user_data_buffer_read	Slave(04PI)讀取指定記憶體區塊中的資料
	_misc_slave_user_data_buffer_write	Slave(04PI)將指定的資料寫入指定的記憶體區塊
	_misc_slave_user_data_to_flash	Slave(04PI)將資料由 Bufeer 寫入 Flash 中
	極限反向 API	
	_DMC_01_rm_04pi_set_MEL_polarity	設定 RM04PI 負極限正向/反向
	_DMC_01_rm_04pi_get_MEL_polarity	取得 RM04PI 負極限目前的狀態
	_DMC_01_rm_04pi_set_PEL_polarity	設定 RM04PI 正極限正向/反向
	_DMC_01_rm_04pi_get_PEL_polarity	取得 RM04PI 正極限目前的狀態
	_DMC_01_set_01ph_alm_polarity	設定 GE01PH ALM 準位
	_DMC_01_set_01ph_org_polarity	設定 RM04PI/GE01PH ORG 準位
	Compare API	
●	_DMC_01_set_compare_channel_position	設定該 Channel 新的 Position 計數值
●	_DMC_01_get_compare_channel_position	讀取該 Channel 目前的 Position 計數值
●	_DMC_01_set_compare_ipulser_mode	設定脈波介面模組輸入相位模式
●	_DMC_01_set_compare_channel_direction	設定該 Channel 脈波方向
●	_DMC_01_set_compare_channel_trigger_time	設定 Trigger 致能持續時間
●	_DMC_01_set_compare_channel_one_shot	設定為執行一次 Trigger 致能狀態
●	_DMC_01_set_compare_channel_source	設定其比較來源
●	_DMC_01_channel0_position_cmp	執行 Compare Type 為 Compare0
●	_DMC_01_channel1_output_enable	設定 Compare1 輸出開啟/關閉
●	_DMC_01_channel1_output_mode	Compare1 設定輸出模式
●	_DMC_01_channel1_get_io_status	讀取 Compare1 狀態
●	_DMC_01_channel1_set_gpio_out	設置 GPIO 輸出腳位狀態

●	_DMC_01_channel1_position_compare_table	Compare1 設定一般 Compare 相關資料
●	_DMC_01_channel1_position_compare_table_level	Compare1 設定自訂 Compare 相關資料
●	_DMC_01_channel1_position_compare_table_cnt	讀取已 Compare 計數次數
●	_DMC_01_set_compare_channel_polarity	設定 Compare 的準位
●	_DMC_01_channel0_position_cmp_by_gpio	設定 Compare 的觸發條件由 GPIO 控制
●	_DMC_01_channel1_position_re_compare_table	用上次的 Compare 條件，重新執行一次 Channel1 的 Compare
●	_DMC_01_channel1_position_re_compare_table_level	用上次的 Compare 條件，重新執行一次 Channel1(Level 模式)的 Compare
●	_DMC_01_set_compare_channel_enable	設定 Compare 功能開啟/關閉
	連續速度 API	
▲	_DMC_01_speed_continue	連續速度功能是否致能
▲	_DMC_01_speed_continue_mode	連續速度模式
▲	_DMC_01_speed_continue_combine_ratio	連續速度合成比率
	特殊運用 API	
▲	_DMC_01_enable_electcam	設定開啟/關閉電子凸輪功能。
▲	_DMC_01_set_cam_enable	開啟/關閉間隙補償功能
▲	_DMC_01_set_cam_pitch	設定間隙補償間隔(Pulse)
▲	_DMC_01_set_cam_org	設定間隙補償開始原點位置
▲	_DMC_01_set_cam_mode	設定間隙補償模式(單向/雙向)
▲	_DMC_01_set_cam_table	建立間隙補償參數表資料
▲	_DMC_01_set_cam_table2	建立間隙補償參數表資料並設定補償數量
▲	_DMC_01_set_backlash	開啟/關閉背隙補償功能
▲	_DMC_01_set_backlash_info	設定背隙補償參數
▲	_DMC_01_set_ecam_repeat	開啟/關閉電子凸輪重複功能
	其它 API	
	_misc_app_get_circle_endpoint	獲取圓弧補間所需之終點座標(X, Y)
	_misc_app_get_circle_center_point	獲取圓弧補間所需之中心座標(X, Y)
	_misc_set_record_debuging	Debug 記錄功能是否啟動
	_misc_open_record_debuging_file	設置輸出 Debug 記錄存放位置
	_DMC_01_enable_dda_mode	啟動 DDA 填表功能
	_DMC_01_set_dda_data	填入 DDA Table 資料
	_DMC_01_get_dda_cnt	取得目前 DDA Table 尚存的資料筆數

	_DMC_01_clear_dda_data	將 DDA Table 中尚存的資料清除。
	_DMC_01_save_device_info	將模組資料存為文字檔。
	_DMC_01_compare_device_info	將目前連線之模組資料與已儲存之資料檔進行比對，並輸出比對結果的紀錄檔。
	_DMC_01_enable_record_mode	開啟/關閉以毫秒為單位擷取 Monitor 所下參數的功能。
	_DMC_01_get_record_data	讀取以毫秒為單位擷取 Monitor 所下參數的功能
▲	_DMC_01_set_mc_fifo_mode	FIFO 參數設定。
▲	_DMC_01_set_function_time	設定每 1ms 最小的通訊時間以確保通訊不會異常。(詳細請參照 API 內說明)
	_DMC_01_set_position_ref_source	設定座標的參考來源模式
	_DMC_01_get_position_ref_source	取得目前座標的參考來源模式
	_DMC_01_set_virtual_command	Set position 的快速模式
	_DMC_01_get_virtual_command	取得實際 command、position 與 virtual_command 的差值
	_DMC_01_set_virtual_position	將實際 Position 值設定為 Virtual Position 的值
	_DMC_01_disable_virtual_pos	選擇重置 Command 與 Position 的連動性
	_DMC_01_Set_Stepping_Move_rate	設定解析放大的比率
◆	_DMC_01_MRAM_read_word	讀取 MH1(MP1)記憶體 MRAM 的值(單位:word)
◆	_DMC_01_MRAM_write_word	寫值到 MH1(MP1)記憶體 MRAM(單位:word)
◆	_DMC_01_MRAM_read_dword	讀取 MH1(MP1)記憶體 MRAM 的值 (單位:double word)
◆	_DMC_01_MRAM_write_dword	寫值到 MH1(MP1)記憶體 MRAM (單位:double word)

NOTE

表 4.2 中，有 ■ 符號標記的 API 表示在使用此 API 時，會比一般 API 耗費較長時間，程式撰寫中不適合無限制的重複下達此 API。

有 ▲ 符號標記的 API 表示 PCI_DMC_F01 軸卡不提供此功能。

有 ● 符號標記的 API 表示僅 PCI_DMC_B01 軸卡提供此功能。

有 ◆ 符號標記的 API 表示僅 MH1(MP1)上主機板提供此功能。

第五章　硬體初始化 API

表 5.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_open	當應用程式啟動時，初始化系統資源
A/B/F	_DMC_01_close	釋放所有系統資源
A/B/F	_DMC_01_get_CardNo_seq	取得主機上所有 PCI-DMC 介面卡號
A/B/F	_DMC_01_pci_initial	初始此 PCI 介面卡
A/B/F	_DMC_01_get_card_version	取得軸卡版號

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

5.1 _DMC_01_open

■ **格式**

I16 PASCAL _DMC_01_open (I16* existcard)

■ **目的**

當應用程式啟動時，初始化系統資源。

■ **參數**

名稱	資料型別	單位	描述
Existcard	I16*	片	取得系統中掛載之該介面卡數量

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 existCards;
I16 status = _DMC_01_open (&existCards);
```

5.2　_DMC_01_close

■　**格式**

Void PASCAL _DMC_01_close ()

■　**目的**

釋放所有系統資源。

■　**參數**

無引用的參數。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
_DMC_01_close();
```

5.3　_DMC_01_get_CardNo_seq

■　**格式**

I16 PASCAL _DMC_01_get_CardNo_seq(U16 CardNo_seq,U16* CardNo)

■　**目的**

取得主機上所有 PCI-DMC-A01 介面卡號。

■　**參數**

名稱	資料型別	單位	描述
CardNo_seq	U16	編號單位	序列數
CardNo	U16*	編號單位	介面卡號 CardNo 為 0~15，藉由 SW1 設置。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

1. 一張 PCI-DMC 安裝於主機時：

```
U16 CardNo_seq = 0;
U16 CardNo;
I16 status = _DMC_01_get_CardNo_seq(CardNo_seq, &CardNo);
```

2. 兩張或多張 PCI-DMC 安裝於主機時：

```
U16 CardNo_seq = 0;
U16 CardNo;
I16 status;
I16 existCards; //此 existCards 的值參閱 _DMC_01_open 的 API，詳述參閱程式手冊第 5.1 節。

for(CardNo_seq=0; CardNo_seq <existCards; CardNo_seq ++)
        status = _DMC_01_get_CardNo_seq(CardNo_seq, &CardNo);
```

NOTE

當僅一張 PCI-DMC 於主機上，且 SW1 設置為 3，此時取得該介面卡號為 3。
當兩張或多張 PCI-DMC 接於主機時，介面卡號將依系統掃描到之第 1 張 PCI-DMC 介面卡上 SW1 的值，並依序取得其它介面卡的卡號。
SW1 的設置，請參閱 PCI-DMC 使用手冊-第 1.5.5 節敘述。

5.4　_DMC_01_pci_initial

■　**格式**

I16 PASCAL _DMC_01_pci_initial (U16 CardNo)

■　**目的**

初始此 PCI 介面卡。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;

I16 status = _DMC_01_pci_initial (CardNo);
```

5.5　_DMC_01_get_card_version

■　**格式**

I16 PASCAL _DMC_01_get_card_version (U16 CardNo, U16 *ver)

■　**目的**

取得軸卡版號。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Ver	U16*	數值單位	軸卡版號資料。 0X6x 是 PCI_DMC_A01 0X7x 是 PCI_DMC_B01 0X8x 是 PCI_DMC_F01

※　**PCI_DMC_A01 軸卡版號 0x64 開始提供 GPIO 為 Input x 8/Output x 4。**
PCI_DMC_B01 軸卡提供 GPIO 均為 Input x 8/Output x 4。
PCI_DMC_F01 軸卡提供 GPIO 均為 Input x 8/Output x 4。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 ver;

I16 status = _DMC_01_get_card_version (CardNo, &ver);
```

第六章　介面卡操作 API

表 6.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_initial_bus	初始外部匯流排(External Bus)
A/B/F	_DMC_01_start_ring	起始 ring 的通訊
A/B/F	_DMC_01_get_device_table	取得裝置列表
A/B/F	_DMC_01_get_node_table	取得站號列表
A/B/F	_DMC_01_reset_card	重置該介面卡
A/B/F	_DMC_01_check_nodeno	查驗該站號是否已存在
A/B/F	_DMC_01_get_master_connect_status	取得 Master Card 與擴充模組目前的連線狀態
A/B/F	_DMC_01_get_mailbox_Error	取得 MailBox 報錯的次數
A/B/F	_DMC_01_get_mailbox_cnt	取得 MailBox 計數值
A/B/F	_DMC_01_get_dsp_cnt	取得中斷計數值
A/B/F	_DMC_01_set_dio_output	設置 GPIO 輸出腳位狀態
A/B/F	_DMC_01_get_dio_output	取得 GPIO 輸出腳位狀態
A/B/F	_DMC_01_get_dio_input	取得 GPIO 輸入腳位狀態
A/B/F	_DMC_01_get_cycle_time	取得當前查找裝置的循環時間
A/B/F	_DMC_01_initial_bus2	初始外部所有匯流排 (External Bus)
A/B/F	_DMC_01_motion_cnt	取得 MailBox 及 DSP 計數值
A/B/F	_DMC_01_pdo_Rx_error	取得 PDO 收封包錯誤記錄
A/B/F	_DMC_01_pdo_Tx_error	取得 PDO 送封包錯誤記錄
A/B/F	_DMC_01_get_motion_error_code	取得 Motion 動作的錯誤回傳值
A/B/F	_DMC_01_get_motion_error_cnt	取得 Motion 動作的錯誤次數
A/B/F	_DMC_01_get_DLL_path	得到 PCI_DMC.dll 詳細路徑資料(合併版)
A/B/F	_DMC_01_get_DLL_version	得到 PCI_DMC.dll 版本資料(合併版)
A/B/F	_DMC_01_get_DLL_path_Single	得到 PCI_DMC.dll 詳細路徑資料(A/B/F01)
A/B/F	_DMC_01_get_DLL_version_Single	得到 PCI_DMC.dll 版本資料(A/B/F01)

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

6.1　_DMC_01_initial_bus

■ **格式**

I16 PASCAL _DMC_01_initial_bus (U16 CardNo)

■ **目的**

初始外部匯流排(External Bus)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;

I16 status = _DMC_01_initial_bus (CardNo);
```

NOTE

PCI_DMC_F01 在進行 _DMC_01_initial_bus 時，若 dll 的版本與 IC 內部的版本不同，會進行 DownLoad Code 的動作，時間大約 5 分鐘，DownLoadCode 的過程中要避免關閉 PC 電源等動作發生，若有使用軸卡 Security 功能，還要避免 DownLoadCode 的過程中，更換軸卡或調整卡號的動作(若 DownLoadCode 的過程中，PC 當機或斷電，請原卡原卡號再做一次 _DMC_01_initial_bus)。

6.2　_DMC_01_start_ring

■　**格式**

I16 PASCAL _DMC_01_start_ring (U16 CardNo, U8 RingNo)

■　**目的**

起始 ring 的通訊。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
RingNo	U8	編號單位	欲運行的 Ring 編號

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U8 RingNo = 0;

I16 status = _DMC_01_start_ring (CardNo, RingNo);
```

PS: 若是 status <> 0 ， 且無對應錯誤碼, 可呼叫 DMC_01_master_alm_code 查詢是否發生通訊錯誤.

6.3 _DMC_01_get_device_table

■ **格式**

I16 PASCAL _DMC_01_get_device_table (U16 CardNo, U16* value)

■ **目的**

取得裝置列表。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
value	U16*	編號單位	裝置編號列表

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 value=0;

I16 status = _DMC_01_get_device_table (CardNo, &value);
```

6.4　_DMC_01_get_node_table

■　**格式**

I16 PASCAL _DMC_01_get_node_table (U16 CardNo, U32* NodeIDTable)

■　**目的**

取得站號列表。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDTable	U32*	編號單位	站號列表

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 NodeIDTable=[4];

I16 status = _DMC_01_get_node_table( CardNo, &NodeIDTable);
```

6.5 _DMC_01_reset_card

■ **格式**

I16 PASCAL _DMC_01_reset_card (U16 CardNo)

■ **目的**

重置該介面卡。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo=0;

I16 status = _DMC_01_reset_card (CardNo);

6.6　_DMC_01_check_nodeno

■　**格式**

I16 PASCAL _DMC_01_check_nodeno(U16 CardNo, U16 NodeID,U16 SlotID,U16* exist)

■　**目的**

查驗該站號是否已存在。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
exist	U16*	選項單位	0：站號不存在 1：站號已存在

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
U16 exist ;

I16 status = _DMC_01_check_nodeno (CardNo, NodeID, SlotID, &exist);
```

6.7　_DMC_01_get_master_connect_status

■　**格式**

I16 PASCAL _DMC_01_get_master_connect_status (U16 CardNo, U16* Protocol)

■　**目的**

取得 Master Card 與擴充模組目前的連線狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Protocol	U16*	選項單位	1：未接 Node1 模組 3：與模組 Node1 連線正常 4：與模組 Node1 斷線

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 Protocol ;

I16 status = _DMC_01_get_master_connect_status (CardNo, &Protocol);
```

6.8　_DMC_01_get_mailbox_Error

■　**格式**

I16 PASCAL _DMC_01_get_mailbox_Error (U16 CardNo, U32* error_cnt)

■　**目的**

取得 MailBox 報錯的次數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
error_cnt	U32*	編號單位	MailBox 報錯的計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 error_cnt ;

I16 status = _DMC_01_get_mailbox_Error (CardNo, &error_cnt);
```

6.9　_DMC_01_get_mailbox_cnt

■　**格式**

I16 PASCAL _DMC_01_get_mailbox_cnt (U16 CardNo, U32* PC_cnt, U32* DSP_cnt)

■　**目的**

取得 MailBox 計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
PC_cnt	U32*	次數	由 PC 端取得的 MailBox 計數值
DSP_cnt	U32*	次數	由 DSP 端取得的 MailBox 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U32 PC_cnt , DSP_cnt ;

I16 status = _DMC_01_get_mailbox_cnt (CardNo, &PC_cnt, &DSP_cnt);
```

6.10　_DMC_01_get_dsp_cnt

■　**格式**

I16 PASCAL _DMC_01_get_dsp_cnt (U16 CardNo, U32* int_cnt, U32* main_cnt)

■　**目的**

取得中斷計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
int_cnt	U32*	次數	當通訊建構時發出的中斷計數值
main_cnt	U32*	次數	當 DSP 建構時發出的中斷計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 int_cnt , main_cnt;

I16 status = _DMC_01_get_dsp_cnt (CardNo, &int_cnt, &main_cnt);
```

6.11 _DMC_01_set_dio_output

■ **格式**

I16 PASCAL _DMC_01_set_dio_output (U16 CardNo, U16 On_Off)

■ **目的**

設置 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
On_Off	U16	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo=0;
U16 On_Off =1;

I16 status = _DMC_01_set_dio_output (CardNo, On_Off);

6.12　_DMC_01_get_dio_output

■　**格式**

I16 PASCAL _DMC_01_get_dio_output (U16 CardNo, U16* On_Off)

■　**目的**

取得 GPIO 輸出腳位狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
On_Off	U16*	選項單位	0：關閉 1：開啟

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 On_Off ;

I16 status = _DMC_01_get_dio_output (CardNo, &On_Off);
```

6.13 _DMC_01_get_dio_input

■ **格式**

I16 PASCAL _DMC_01_get_dio_input (U16 CardNo, U16* On_Off)

■ **目的**

取得 GPIO 輸入腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
On_Off	U16*	選項單位	0：關閉 1：開啟

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 On_Off ;

I16 status = _DMC_01_get_dio_input (CardNo, &On_Off);
```

6.14　_DMC_01_get_cycle_time

■　**格式**

I16 PASCAL _DMC_01_get_cycle_time (U16 CardNo, I32 *time)

■　**目的**

取得當前查找裝置的循環時間，該時間必須小於 1000us。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
time	I32*	數值單位	該 time 的值必須小於 1000

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
I32 time;

/*  取得裝置的循環時間, time 的值必須小於 1000    */
I16 status = _DMC_01_get_cycle_time (CardNo, &time);
```

6.15 _DMC_01_initial_bus2

■ **格式**

I16 PASCAL _DMC_01_initial_bus2 ( )

■ **目的**

初始外部所有匯流排 (External Bus)。

■ **參數**

此函式無任何參數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I16 status = _DMC_01_initial_bus2( );
```

NOTE

PCI_DMC_F01 在進行 _DMC_01_initial_bus2 時，若 dll 的版本與 IC 內部的版本不同，會進行 DownLoad Code 的動作，時間大約 5 分鐘，DownLoadCode 的過程中要避免關閉 PC 電源等動作發生，若有使用軸卡 Security 功能，還要避免 DownLoadCode 的過程中，更換軸卡或調整卡號的動作(若 DownLoadCode 的過程中，PC 當機或斷電，請原卡原卡號再做一次_DMC_01_initial_bus2)。

6.16　_DMC_01_motion_cnt

■　**格式**

I16 PASCAL _DMC_01_motion_cnt (U16 CardNo, U16 NodeID, U16 SlotID,
U16 *pc_mc_cnt, U16 *dsp_mc_cnt)

■　**目的**

取得 MailBox 及 DSP 計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pc_mc_cnt	U16*	數值單位	PC 端入 MailBox Command 總數值。
dsp_mc_cnt	U16*	數值單位	DSP 由 MailBox 取得 Command 總數值。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 pc_mc_cnt;
U16 dsp_mc_cnt;

I16 status = _DMC_01_motion_cnt (CardNo, NodeID, SlotID, &pc_mc_cnt, &pc_mc_cnt);
```

6.17　_DMC_01_pdo_Rx_error

■　**格式**

I16 PASCAL _DMC_01_pdo_Rx_error (U16 CardNo, U16 NodeID,U16 SlotID,
U32 *Rx_error1,U32 *Rx_error2,U16 set_get)

■　**目的**

取得 PDO 封包錯誤記錄。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Rx_error1	U32*	數值單位	PDO　A Channel Rx Error Counters
Rx_error2	U32*	數值單位	PDO　B Channel Rx Error Counters
set_get	U16	選項單位	0：read Rx_error1,Rx_error2 value 1：write Rx_error1,Rx_error2 value

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Rx_error1;
U32 Rx_error2;
U16 ser_get=0;


I16 status = _DMC_01_pdo_Rx_error (CardNo, NodeID, SlotID, &Rx_error1, &Rx_error2,
set_get);
```

6.18　_DMC_01_pdo_Tx_error

■　**格式**

I16 PASCAL _DMC_01_pdo_Tx_error (U16 CardNo, U16 NodeID, U16 SlotID, U32 *Tx_error1, U32 *Tx_error2, U16 set_get)

■　**目的**

取得 PDO 封包錯誤記錄。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tx_error1	U32*	數值單位	PDO　A Channel Tx Error Counters
Tx_error2	U32*	數值單位	PDO　B Channel Tx Error Counters
set_get	U16	選項單位	0：read Tx_error value 1：write Tx_error value

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Tx_error1,Tx_error2;
U16 ser_get=0;

I16 status = _DMC_01_pdo_Tx_error (CardNo, NodeID, SlotID, &Tx_error1, &Tx_error2,
set_get);
```

6.19 _DMC_01_get_motion_error_code

■ **格式**

I16 PASCAL _DMC_01_get_motion_error_code (U16 CardNo, U16 NodeID,U16 SlotID, I16* error_code)

■ **目的**

取得 Motion 動作的錯誤回傳值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
error_code	I16*	數值單位	Motion 動作的錯誤回傳值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 error_code;

I16 status = _DMC_01_get_motion_error_code (CardNo, NodeID, SlotID, & error_code);
```

6.20 _DMC_01_get_motion_error_cnt

■ **格式**

I16 PASCAL _DMC_01_get_motion_error_cnt (U16 CardNo, U16 NodeID, U16 SlotID, I16* error_cnt)

■ **目的**

取得 Motion 動作的錯誤次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
error_cnt	I16*	數值單位	Motion 動作的錯誤次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 error_cnt;

I16 status = _DMC_01_get_motion_error_cnt (CardNo, NodeID, SlotID, & error_cnt);
```

6.21　_DMC_01_get_DLL_path

■　**格式**

I16 PASCAL _DMC_01_get_DLL_path (char *lpFilePath, U32 nSize, U32 *nLength)

■　**目的**

得到 PCI_DMC.dll 詳細路徑資料(合併版)。

■　**參數**

名稱	資料型別	單位	描述
lpFilePath	Char*	字元陣列單位	PCI_DMC.dll 詳細路徑資料字元
nSize	U32	數值單位	路徑資料擷取長度
nLength	U32*	數值單位	傳回路徑資料實際長度

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
Char lpFilePath =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_path ( &lpFilePath, nSize, &nLength);
```

6.22　_DMC_01_get_DLL_version

■　**格式**

I16 PASCAL _DMC_01_get_DLL_version (char *lpBuf, U32 nSize, U32 *nLength)

■　**目的**

得到 PCI_DMC.dll 版本資料(合併版)。

■　**參數**

名稱	資料型別	單位	描述
lpBuf	Char*	字元陣列單位	PCI_DMC.dll 詳細版本資料字元
nSize	U32	數值單位	版本資料擷取長度
nLength	U32*	數值單位	傳回版本資料實際長度

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
Char lpBuf =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_version ( & lpBuf, nSize, &nLength);
```

6.23　_DMC_01_get_DLL_path_Single

■　**格式**

I16 PASCAL _DMC_01_get_DLL_path_Single (U16 CardNo, char *lpFilePath, U32 nSize, U32 *nLength)

■　**目的**

得到 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細路徑資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
lpFilePath	Char*	字元陣列單位	PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細路徑資料字元
nSize	U32	數值單位	路徑資料擷取長度
nLength	U32	數值單位	傳回路徑資料實際長度

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo= 0;
Char lpFilePath =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_path_Single (CardNo , &lpFilePath, nSize, &nLength);
```

6.24　_DMC_01_get_DLL_version_Single

■　**格式**

I16 PASCAL _DMC_01_get_DLL_version_Single (U16 CardNo, char *lpBuf, U32 nSize, U32 *nLength)

■　**目的**

得到 PCI_DMC_01.dll 或 PCI_DMC_F01.dll 版本資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
lpBuf	Char*	字元陣列單位	PCI_DMC_01.dll 或 PCI_DMC_F01.dll 詳細版本資料字元
nSize	U32	數值單位	版本資料擷取長度
nLength	U32	數值單位	傳回版本資料實際長度

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
Char lpBuf =[50];
U32 nSize=50, nLength=0;

I16 status = _DMC_01_get_DLL_version_Single (CardNo , &lpBuf, nSize, &nLength);
```

（此頁有意留為空白）

第七章　伺服驅動器參數讀取/寫入 API

表 7.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_read_servo_parameter	讀取伺服驅動器參數資料
A/B/F	_DMC_01_write_servo_parameter	寫入伺服驅動器參數資料

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

7.1　_DMC_01_read_servo_parameter

■　**格式**

I16 PASCAL _DMC_01_read_servo_parameter (U16 CardNo, U16 NodeID, U16 SlotID, U16 group,U16 idx,U32* data)

■　**目的**

讀取伺服驅動器參數資料(屬於 SDO 指令，執行時間較長，不建議放在常態掃描程式區)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
group	U16	數值	裝置(驅動器)群組編號
idx	U16	數值	驅動器參數群組的索引值
data	U32*	數值	該群組索引所傳回的資料

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
U16 group = 3;
U16 idx = 0;        // 當您設定此例的群組與索引值，表示您欲讀取驅動器參數”P3-00”
U32 data;

I16 status = _DMC_01_read_servo_parameter(CardNo, NodeID, SlotID, group, idx, &data);
```

7.2　_DMC_01_write_servo_parameter

■　**格式**

I16 PASCAL _DMC_01_write_servo_parameter (U16 CardNo, U16 NodeID, U16 SlotID, U16 group,U16 idx,U32 data)

■　**目的**

寫入伺服驅動器參數資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
group	U16	數值	裝置(驅動器)群組編號
idx	U16	數值	驅動器參數群組的索引值
data	U32	數值	欲寫入該群組索引的資料

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
U16 group = 3;
U16 idx = 0;        // 當您設定此例的群組與索引值，表示您將把資料寫入驅動器參數”P3-00”
U32 data = 1;

I16 status = _DMC_01_write_servo_parameter(CardNo, NodeID, SlotID, group, idx, data);
```

（此頁有意留為空白）

第八章　使用 SDO 協定 API

表 8.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_check_canopen_lock	SDO 模式下，檢查是否可以再執行下一筆命令
A/B/F	_DMC_01_get_canopen_ret	取得 CANopen 傳回的資訊(SDO 相關數據)
A/B/F	_DMC_01_set_pdo_mode	設置使用的 CANopen 協定(PDO 或 SDO)
A/B/F	_DMC_01_send_message	發送 SDO 命令信息至資料緩衝區，會等待資料送達伺服端且讀回資料相符才離開
A/B	_DMC_01_send_message2	發送 SDO 命令信息至資料緩衝區，只確認資料送達伺服端就離開
A/B	_DMC_01_send_message3	發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝區
F	_DMC_01_send_message4	發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝區
A/B/F	_DMC_01_read_message	讀取最後一筆 SDO 命令信息至資料緩衝區
A/B/F	_DMC_01_read_message2	讀取最後一筆 SDO 命令信息至資料緩衝區，並傳回已完成的次數值
A/B/F	_DMC_01_get_message	取得 SDO 命令信息至資料緩衝區
A/B/F	_DMC_01_reset_sdo_choke	當 SDO 命令因故阻塞時，重置 SDO
A/B/F	_DMC_01_get_sdo_retry_history	取得 SDO 重新發送的次數

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

說明：本章節是特別針對有需要直接對伺服做存取運用所做的說明，若您無此需求，

可直接跳過本章節。

8.1　CANopen SDO 協定格式

※CANopen 的 SDO 封包格式範列 1 – Index 讀取/寫入成功

當讀取 Index 為「成功」，該封包格式傳回值如下表：

表 8.2

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x40	0x64	0x60	0x00	0x00	0x00	0x00	0x00
回傳	0x43	0x64	0x60	0x00	0x11	0x22	0x33	0x44

表格說明：

(1)　讀取 Feedback PUU(此時 CANopen 的 Index 值填入 0x6064，SubIdx 值填入 0x00)。

(2)　當 Datatype 值填入 0x40，表示讀取。

(3)　隨後回傳的 Datatype 值為 0x43，

若回傳之 Datatype 值為 0x43 表示收到的資料為 32bit [Data1~Data4 均為有效]，

值為 0x4b 表示收到的資料為 16bit [Data1~Data2 有效]，

值為 0x4f 表示收到的資料為　8bit [Data1 有效]。

(4)　Data1~Data4 的欄位為低 Word 優先讀取，而無效的高位 Data 則為 0，

因此上述表格陳列之回傳值表示收到的 Feedback PUU 為 0x44332211。

當寫入 Index 為「成功」，該封包格式傳回值如下表：

表 8.3

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x2b	0x40	0x60	0x00	0x08	0x00	0x00	0x00
回傳	0x60	0x40	0x60	0x00	0x00	0x00	0x00	0x00

表格說明：

(1)　寫入控制碼 ControlWord(此時 CANopen 的 Index 值填入 0x6040，SubIdx 的值填入 0x00)。

(2)　Datatype 填入 0x2b，表示寫入。

(3)　若填入 Datatype 的值為 0x23 表示寫入的資料為 32bit [Data1~Data4 均為有效]，

填入值為 0x2b 表示寫入的資料為 16bit [Data1~Data2 有效]，

填入值為 0x2f 表示寫入的資料為　8bit [Data1 有效]。

(4)　其回傳的 Datatype 值為 0x60，表示寫入成功；此時回傳的 Data1~Data4 無意義。

(5)　Data1~Data4 的欄位為低 Word 優先，而無效的高位 Data 請以 0 遞補。

因此上述陳列之回傳值表示寫入的控制碼 ControlWord 為 0x00000008。

(6)　該 Index 的資料長度請參考台達 ASDA-A2 CANopen Technical Guide。

※CANopen 的 SDO 封包格式範列 2 – Index 讀取/寫入失敗

當讀取 Index 為「失敗」(即查無此索引)，該封包格式傳回值如下表：

表 8.4

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x40	0x59	0x60	0x00	0x00	0x00	0x00	0x00
回傳	0x80	0x59	0x60	0x00	0x00	0x00	0x02	0x06

表格說明：

(1) 讀取一個無效的索引(此時 CANopen 的 Index 值填入 0x6059，SubIdx 的值填入 0x00)。

(2) 當 Datatype 值填入 0x40，表示讀取。

(3) 當驅動器內無此索引，故此讀取失敗，隨後回傳的 Datatype 值為 0x80，表示回傳讀取失敗。

(4) 上表格欄位表示回傳之 Data1~Data4 組成的資料為 0x06020000[無此索引]錯誤碼。

(5) 詳細說明，請參考台達 ASDA-A2 CANopen Technical Guide 中的 SDO Abort Code。

當寫入 Index 為「失敗」(即寫入的數值超出設置範圍)，該封包格式傳回值如下表：

表 8.5

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x2f	0x60	0x60	0x00	0x0F	0x00	0x00	0x00
回傳	0x80	0x60	0x60	0x00	0x30	0x00	0x09	0x06

表格說明：

(1) 寫入操作模式(此時 CANopen 的 Index 值填入 0x6060，SubIdx 的值填入 0x00)。

(2) Datatype 填入 0x2f，表示寫入的資料為 8bit [Data1 有效]。

(3) 因操作模式的範圍為 0x00~0x07，故此命令寫入失敗，隨後回傳的 Datatype 值為 0x80，表示回傳寫入失敗。

(4) 上表格欄位表示回傳之 Data1~Data4 組成的資料為 0x06090030[寫入數值超出範圍]錯誤碼。

(5) 該 index 的資料長度請參考台達 ASDA-A2 CANopen Technical Guide。

其它詳細說明，請參考台達 ASDA-A2 CANopen Technical Guide 中的 SDO Abort Code。.

※CANopen 的 SDO 封包格式範列 3 – 讀取/寫入 ASDA-A2 參數

當欲讀取 ASDA-A2 參數，該封包格式傳回值如下表：

表 8.6

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x40	0x00	0x23	0x00	0x00	0x00	0x00	0x00
回傳	0x4b	0x00	0x23	0x00	0x7F	0x00	0x00	0x00

表格說明：

(1) 可用標準 CANopen SDO 格式讀取 A2 參數。

(2) P3-00[站號] (CANopen 的 Index 值填入 0x2300 SubIdx 的值填入 0x00)。
格式：**P△-○○** [驅動器參數] ←→ **0x2△□□** [Index 值] (□□為○○的 16 進制表示)。
即若 Index 值設置為 0x2300，表示讀取 P3-00 的參數
若 Index 值設置為 0x212D，表示讀取 P1-45 的參數(2D 為 45 的 16 進制表示)

(3) SubIdx 的值均為 0x00。

當欲寫入 ASDA-A2 參數，該封包格式傳回值如下表：

表 8.7

欄位名稱	Datatype	Index low	Index high	Sub index	Data1	Data2	Data3	Data4
命令	0x2b	0x00	0x23	0x00	0x7F	0x00	0x00	0x00
回傳	0x60	0x00	0x23	0x00	0x00	0x00	0x00	0x00

表格說明：

(1) 可用標準 CANopen SDO 格式寫入 A2 參數。

(2) P3-00[站號] (CANopen 的 Index 值填入 0x2300 SubIdx 的值填入 0x00)。
格式：**P△-○○** [驅動器參數] ←→ **0x2△□□** [Index 值] (□□為○○的 16 進制表示)。

(3) SubIdx 的值均為 0x00。

(4) ASDA-A2 參數的資料長度為 16bit 或 32bit

(5) 您可參考 ASDA-A2 的 EDS 檔，尋找對應參數 Index 的 DataType。
若 DataType 為 3，此時資料長度為 16bit，命令寫入的 DataType 應填入值為 0x2b
若 DataType 為 4，此時資料長度為 32bit，命令寫入的 DataType 應填入值為 0x23

8.2　_DMC_01_check_canopen_lock

■　**格式**

I16 PASCAL _DMC_01_check_canopen_lock (U16 CardNo, U16 *lock)

■　**目的**

在 SDO 模式下，檢查是否可以再執行下一筆命令。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
lock	U16*	選項單位	0：未栓鎖(傳送成功)，可執行下一筆命令。 1：已栓鎖，須等候目前命令執行完成。 2：傳送失敗。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 lock;

I16 status = _DMC_01_check_canopen_lock (CardNo, &lock);
```

8.3　_DMC_01_get_canopen_ret

■　**格式**

I16 PASCAL _DMC_01_get_canopen_ret (U16 CardNo, U16* COBID, U8* value)

■　**目的**

取得 CANOPEN 傳回的資訊(SDO 相關數據)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
COBID	U16*	數值	CAN 物件的 ID
value	U8*	數值	物件的數據內文 Value[0]：SDO 信息回覆 數據內文長度為 32-bit，值為 43 數據內文長度為 16-bit，值為 4b 數據內文長度為 8-bit，值為 4f 數據寫入成功，值為 60 報錯，值為 80 Value[1]：Index(低位元組) Value[2]：Index(高位元組) Value[3]：Subindex Value[4]：Data low word(低位元組) Value[5]：Data low word(高位元組) Value[6]：Data high word(低位元組) Value[7]：Data high word(高位元組)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 COBID=0;
U8 value[8]={0};

I16 status = _DMC_01_get_canopen_ret (CardNo, &COBID, value);
```

8.4　_DMC_01_set_pdo_mode

■　**格式**

I16 PASCAL _DMC_01_set_pdo_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 Enable)

■　**目的**

設置使用的 CANopen 協定(PDO 或 SDO)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Enable	U16	選項單位	0: PDO 關閉 PDO，使用 SDO 1: PDO 開啟 PDO

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Enable=1;

I16 status = _DMC_01_set_pdo_mode (CardNo, NodeID, SlotID, Enable);

8.5 _DMC_01_send_message

■ **格式**

I16 PASCAL _DMC_01_send_message (U16 CardNo, U16 NodeID,U16 SlotID ,
U16 Index,U16 SubIdx,U16 DataType,U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■ **目的**

發送 SDO 命令信息至資料緩衝區。(此 API 函式會等待資料送達伺服端且讀回資料相符才離開)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
/*  相關參數說明
    CardNo：Card No；NodeID：站號 ID；SlotID：插槽 ID；  Index：SDO 索引值；  SubIdx：
    SDO 子索引值；  DataType：(讀取命令)讀取數據資料，填入值 0x40；   (寫入命令)寫入
    8-bit 資料，填入值 0x2f；(寫入命令)寫入 16-bit 資料，填入值 0x2b；(寫入命令)寫入 32-bit
    資料，填入值為 0x23；
      Value0：SDO data low word(低位元組),   Value1：SDO data low word(高位元組)；
      Value2：SDO data high word(低位元組),   Value3：SDO data high word(高位元組)；
*/
  U16 CardNo=0;
  U16 NodeID=1;
  U16 SlotID=0;
  U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

  I16 status = _DMC_01_send_message (CardNo, NodeID, SlotID, Index, SubIdx, DataType,
               value0, value1, value2, value3);
```

NOTE

_DMC_01_send_message、_DMC_01_send_message2、_DMC_01_send_message3 與 _DMC_01_send_message4 的比較。

命令名稱	比較說明
_DMC_01_send_message	**於 SDO 訊息送出(TX)，等待目的地 RX 回應後，再發送訊息至目的地將資料讀出，與送出訊息做比對是否正確。 (PCI_DMC_01 與 PCI_DMC_F01 都支援此 API)**
_DMC_01_send_message2	**於 SDO 訊息送出(TX)，等待目的地 RX 回應後，就完成此 API 離開。下達參數與_DMC_01_send_message 完全相同。 (PCI_DMC_01 與 PCI_DMC_F01 都支援此 API)**
_DMC_01_send_message3	**於 SDO 訊息送出(TX)，不等待目的地的 RX 回應馬上離開此 API(一次對 12 軸下達)。 (僅 PCI_DMC_01 支援此 API)**
_DMC_01_send_message4	**於 SDO 訊息送出(TX)，不等待目的地的 RX 回應馬上離開 此 API，下達參數與_DMC_01_send_message3 完全 相同。 (僅 PCI_DMC_F01 支援此 API) 可以使用_DMC_01_check_canopen_lock (CardNo, &lock);這個 API 的回傳值 Lock 來判斷傳送的狀態，0 代表傳送成功，1 代表 SDO 處理中，2 代表傳送失敗。**

8.6　_DMC_01_send_message2

■　**格式**

I16 PASCAL _DMC_01_send_message2 (U16 CardNo, U16 NodeID,U16 SlotID ,
U16 Index,U16 SubIdx,U16 DataType,U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■　**目的**

發送 SDO 命令信息至資料緩衝區。 (此 API 函式只確認資料送達伺服端就離開)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

I16 status = _DMC_01_send_message2 (CardNo, NodeID, SlotID, Index, SubIdx, DataType,
             value0, value1, value2, value3);
```

8.7　_DMC_01_send_message3

■　**格式**

I16 _DMC_01_send_message3 (I16 CardNo, U16 Index,U16 SubIdx,U16 DataType, U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■　**目的**

發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝區(僅限 PCI_DMC_01 支援此 API，且一次對 12 軸下達相同訊息)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■　**範例**

```
U16 CardNo=0 , status = 0, lock = 0;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;
status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0, value1,
          value2, value3);
do{
       status = _DMC_01_check_canopen_lock (CardNo, &lock);
} while(lock);
value0=0x2;
status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0, value1,
          value2, value3);
```

8.8 _DMC_01_send_message4

■ **格式**

I16 PASCAL _DMC_01_send_message4 (U16 CardNo, U16 NodeID,U16 SlotID ,
U16 Index,U16 SubIdx,U16 DataType,U16 Value0,U16 Value1,U16 Value2,U16 Value3)

■ **目的**

發送 SDO 命令信息至資料緩衝區，但發出命令後立即離開資料緩衝。（僅限 PCI_DMC_F01 支援此 API）

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
DataType	U16	數值	物件字典的資料類型
Value0	U16	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

I16 status = _DMC_01_send_message4 (CardNo, NodeID, SlotID, Index, SubIdx, DataType,
             value0, value1, value2, value3);
```

8.9 _DMC_01_read_message

■ **格式**

I16 PASCAL _DMC_01_read_message (I16 CardNo, U16* COBID, U16* Cmd, U16* SubIdx, U16* Value0,U16* Value1,U16* Value2,U16* Value3)

■ **目的**

讀取最後一筆 SDO 命令信息至資料緩衝區。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
COBID	U16*	數值	CAN 物件 ID(0x580 +目前的 Node ID)
Cmd	U16*	數值	物件字典的索引值
SubIdx	U16*	數值	物件字典的子索引值
Value0	U16*	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16*	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16*	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16*	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
/*  相關參數說明
   CardNo：Card No；COBID : CAN 物件 ID(0x580 +目前的 Node ID); NodeID：站號 ID；
   Cmd：SDO 索引值；  SubIdx：SDO 子索引值；
   Value0：SDO data low word(低位元組),   Value1：SDO data low word(高位元組)；
   Value2：SDO data high word(低位元組),   Value3：SDO data high word(高位元組)；
*/
U16 CardNo=0,
U16 COBID, Cmd, SubIdx, Value0, Value1, Value2, Value3;

I16 status = _DMC_01_read_message(CardNo, &COBID, &Cmd, & SubIdx,
             &Value0, &Value1, &Value2, &Value3);
```

8.10　_DMC_01_read_message2

■　**格式**

I16 PASCAL _DMC_01_read_message2 (I16 CardNo, U16 NodeID, U16 *Cmd,
U16 *COBID ,U16 * SubIdx, U16 *Value0,U16 *Value1,U16 *Value2,U16 *Value3,U16 *cnt)

■　**目的**

讀取最後一筆 SDO 命令信息至資料緩衝區，並記錄已完成的次數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
Cmd	U16*	數值	物件字典的索引值
COBID	U16*	數值	CAN 物件 ID(0x580 +目前的 Node ID)
SubIdx	U16*	數值	物件字典的子索引值
Value0	U16*	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16*	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16*	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value2	U16*	數值	信息緩衝區(數據 4) –Data high word(高位元組)
cnt	U16*	數值	SDO 訊息傳送完成次數

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ 範例

```
U16 CardNo=0, Cmd, COBID, Value0, Value1, Value2, Value3, cnt, temp;
U16 Index=0x6060, SubIdx=0, DataType=0x2f, value0=0x1, value1=0, value2=0, value3=0;

I16 status = _DMC_01_read_message2(CardNo, NodeID , &Cmd, &COBID, & SubIdx,
              &Value0, &Value1, &Value2, &Value3, &cnt)
temp = cnt;
status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0, value1,
              value2, value3);
status = _DMC_01_read_message2(CardNo, NodeID , &Cmd, &COBID, & SubIdx, &Value0,
             &Value1, &Value2, &Value3, &cnt)
if(cnt-temp)
{
        status = _DMC_01_send_message3 (CardNo, Index, SubIdx, DataType, value0,
                value1,value2, value3);
}
```

8.11　_DMC_01_get_message

■　**格式**

I16 PASCAL _DMC_01_get_message (I16 CardNo, U16 NodeID, U16 SlotID, U16 Index, U16 SubIdx,U16* Cmd, U16* COBID, U16* DataType, U16* Value0, U16* Value1, U16* Value2, U16* Value3)

■　**目的**

取得 SDO 命令信息至資料緩衝區。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Index	U16	數值	物件字典的索引值
SubIdx	U16	數值	物件字典的子索引值
Cmd	U16*	數值	物件字典的索引值
COBID	U16*	數值	CAN 物件 ID
DataType	U16*	數值	物件字典的資料類型
Value0	U16*	數值	信息緩衝區(數據 1) –Data low word(低位元組)
Value1	U16*	數值	信息緩衝區(數據 2) –Data low word(高位元組)
Value2	U16*	數值	信息緩衝區(數據 3) –Data high word(低位元組)
Value3	U16*	數值	信息緩衝區(數據 4) –Data high word(高位元組)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■　**範例**

U16 CardNo=0, NodeID=1,　SlotID=0, Index=0x6060, SubIdx1=0;
U16 Cmd, COBID, SubIdx2, Value0, Value1, Value2, Value3;

I16 status = _DMC_01_get_message(CardNo, NodeID, SlotID, Index, SubIdx1, &Cmd, &COBID, & SubIdx2, &Value0, &Value1, &Value2, &Value3);

8.12　_DMC_01_reset_sdo_choke

■　**格式**

I16 PASCAL _DMC_01_reset_sdo_choke (U16 CardNo)

■　**目的**

當 SDO 命令因故阻塞時，重置 SDO。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;

I16 status = _DMC_01_reset_sdo_choke (CardNo);
```

8.13　_DMC_01_get_sdo_retry_history

■　**格式**

I16 PASCAL _DMC_01_get_sdo_retry_history (U16 CardNo, U32* cnt)

■　**目的**

取得 SDO 重新發送的次數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
cnt	U32*	次數	傳回 SDO 重新發送信息的次數

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 cnt=0;

I16 status = _DMC_01_get_sdo_retry_history (CardNo , &cnt);
```

第九章　點對點運動控制使用封包協定 API

表 9.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_sdo_driver_speed_profile	使用封包協定的速度參數配置
A/B	_DMC_01_start_sdo_driver_r_move	起始一個相對性的運動位移
A/B	_DMC_01_start_sdo_driver_a_move	起始一個絕對性的運動位移
A/B	_DMC_01_start_sdo_driver_new_ position_move	變換新的 position 值進行運動位移

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

9.1 _DMC_01_set_sdo_driver_speed_profile

■ **格式**

I16 PASCAL _DMC_01_set_sdo_driver_speed_profile (U16 CardNo, U16 NodeID, U16 SlotID, U32 MaxVel, F64 acc, F64 dec)

■ **目的**

使用封包協定的速度參數配置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
MaxVel	U32	脈波數/秒	最大速度參數
acc	F64	秒	指定的加速時間
dec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
U16 MaxVel =1000000;
F64 acc=0.1;
F64 dec=0.1;

I16 status = _DMC_01_set_sdo_driver_speed_profile (CardNo, NodeID, SlotID, MaxVel, acc,
             dec);
```

9.2　_DMC_01_start_sdo_driver_r_move

■　**格式**

I16 PASCAL _DMC_01_start_sdo_driver_r_move (U16 CardNo, U16 NodeID, U16 SlotID, I32 Distance)

■　**目的**

起始一個相對性的運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Distance	I32	脈波數	相對性的運動行程

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 Distance =10000000;

I16 status = _DMC_01_start_sdo_driver_r_move (CardNo, NodeID, SlotID, Distance);
```

9.3　_DMC_01_start_sdo_driver_a_move

■　**格式**

I16 PASCAL _DMC_01_start_sdo_driver_a_move (I16 CardNo, U16 NodeID, U16 SlotID, I32 Position)

■　**目的**

起始一個絕對性的運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Position	I32	脈波數	絕對性的運動位置

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 Position =10000000;

I16 status = _DMC_01_start_sdo_driver_a_move (CardNo, NodeID, SlotID, Position);
```

9.4 _DMC_01_start_sdo_driver_new_position_move

■ **格式**

I16 PASCAL _DMC_01_start_sdo_driver_new_position_move (I16 CardNo, U16 NodeID, U16 SlotID, I32 Position,U16 abs_rel)

■ **目的**

變換新的 position 值進行運動位移。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Position	I32	數值	新的 position 值
abs_rel	U16	選項單位	0：參考絕對座標 1：參考相對座標

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
I32 Position =20000000;
U16 abs_rel =1; //參考相對座標進行運動位移

I16 status = _DMC_01_start_sdo_driver_r_move (CardNo, NodeID, SlotID, Position, abs_rel);
```

（此頁有意留為空白）

第十章　原點復歸運動控制使用封包協定 API

表 10.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_home_config	設置原點復歸各項配置
A/B/F	_DMC_01_set_home_move	設置開始運行原點復歸運動
A/B/F	_DMC_01_set_home_config2	設置原點復歸各項配置
A/B/F	_DMC_01_set_home_config2_offset2	重新設置原點復歸 offset 值
A/B/F	_DMC_01_set_home_move2	設置開始運行原點復歸運動
A/B/F	_DMC_01_escape_home_move	停止原點復歸運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

10.1 _DMC_01_set_home_config

■ **格式**

I16 PASCAL _DMC_01_set_home_config (U16 CardNo,U16 NodeID,U16 SlotID ,
U16 Mode,I32 offset,U16 lowSpeed,U16 highSpeed,F64 acc)

■ **目的**

設置原點復歸各項配置(回 Home 前，請關閉軟體極限設定)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	原點復歸模式：1~35 (參考後頁 note 說明)
offset	I32	脈波數	原點復歸偏移量
lowSpeed	U16	轉速/分	歸回零所使用的速度參數
highSpeed	U16	轉速/分	尋找各極限位置所使用的速度參數
acc	F64	秒	原點復歸參照的加速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

※ lowSpeed 與 highSpeed 參數的單位, 會因連結的 Slave 模組有所區別。
伺服驅動器（ASDA-A2F）➔轉速/分
脈波介面模組（RM04PI, GE01PH）➔**Pulse/Sec**
脈波模組回原點最高速為 65535Pulse/Sec，若要運用更高的速度，需搭配 27.5 節的
DMC_01_set_rm_04pi_homing_ratio(詳見)。
線性馬達➔**um/sec**。

注意:_DMC_01_Set_home_config 與_DMC_01_set_home_move 是一組回 Home 完整的參數設定，回 Home 進行中，不可重複下達_DMC_01_Set_home_config。

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;

/* 使用原點復歸模式*/
 I16 status = _DMC_01_set_home_config (CardNo, NodeID, SlotID, Mode, offset, lowSpeed,
          highSpeed, acc);
```

NOTE

#1. 以負極限與索引脈波信息之原點復歸模式

使用此方法，若負極限開關為無效(為低電位)時，初始方向為向左進行位移運動。此原點位置為當觸碰負極限開關時即往右方之索引脈衝位移。

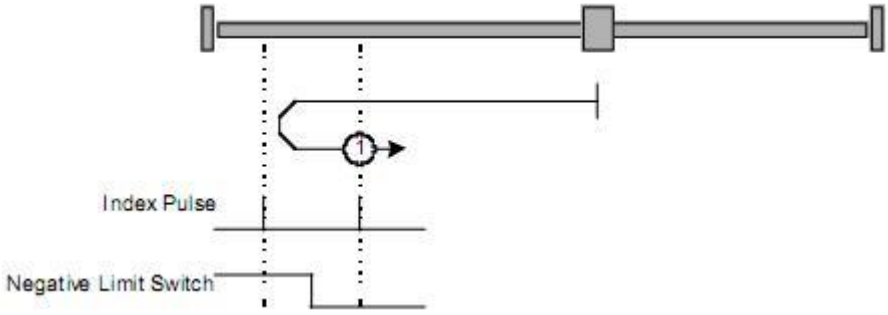


圖 10.1

#2. 以正極限與索引脈波信息之原點復歸模式

使用此方法，若正極限開關為無效(為低電位)時，初始方向為向右進行位移運動。此原點位置為當觸碰正極限開關時即往左方之索引脈衝位移。

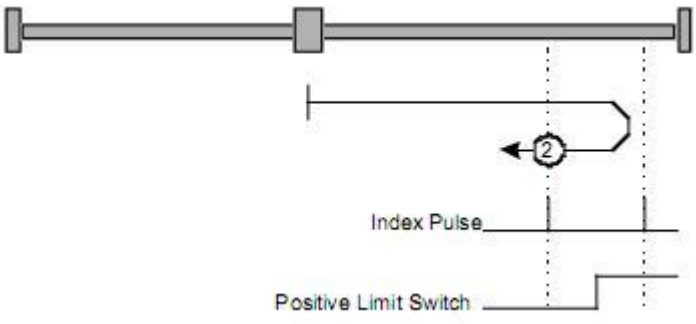


圖 10.2

#3 and 4. 以原點開關正向電位與索引脈波信息之原復歸模式

使用方法 3 或方法 4，其初始位移的方向是依循原點開關當前的狀態。此原點位置取決於該原點開關改變狀態時，向左或向右任一方的索引脈衝。若進行原點復歸的初始位置為索引脈衝，將必須迴轉位移的方向。該點在迴轉位移方向後發生的任何變化，取決於原點開關當前的狀態。

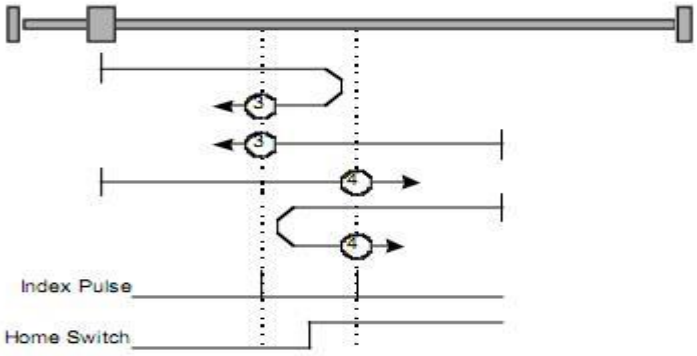


圖 10.3

#5 and 6. 以原點開關負向電位與索引脈波信息之原復歸模式

使用方法 5 或方法 6，其初始位移的方向是依循原點開關當前的狀態。此原點位置取決於該原點開關改變狀態時，向左或向右任一方的索引脈衝。若進行原點復歸的初始位置為索引脈衝，將必須迴轉位移的方向。該點在迴轉位移方向後發生的任何變化，取決於原點開關當前的狀態。

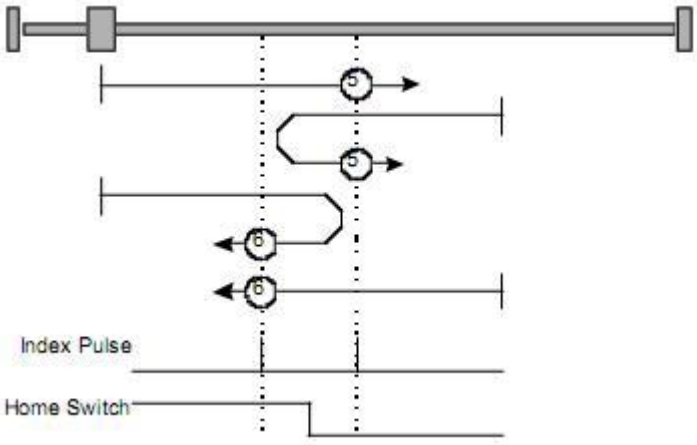


圖 10.4

#7 to 14.基於索引脈波與原點開關之原點復歸模式

這些方法使用一個僅在位移中的一部分啟用的原點開關。事實上，在該軸 posistion 通過此開關，其可'瞬間'被啟用。

使用方法 7 到方法 10，其初始位移的方向是向右，使用方法 11 到方法 14，其初始位移的方向是向左。此外， 若原點開關在位移起始處已為啟用狀態。此時其初始位移的方向是依找尋到的邊緣(edge)而定。此歸回原點的位置為在正緣或負緣任一方的原點開關上的索引脈衝信號。於下方兩張圖顯示，若初始位移方向並未觸碰到原點開關時，此作動需觸碰到極限開關時，方才迴轉位移方向。

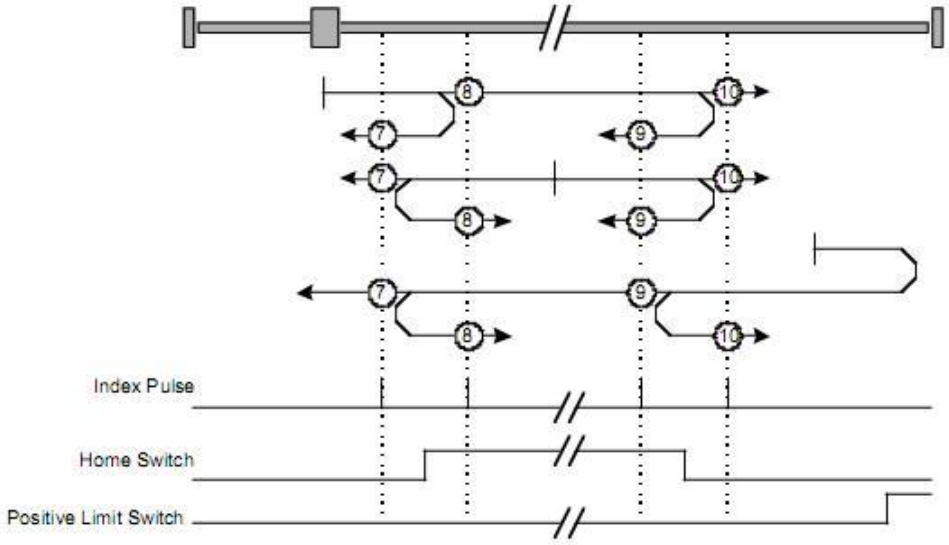


圖 10.5

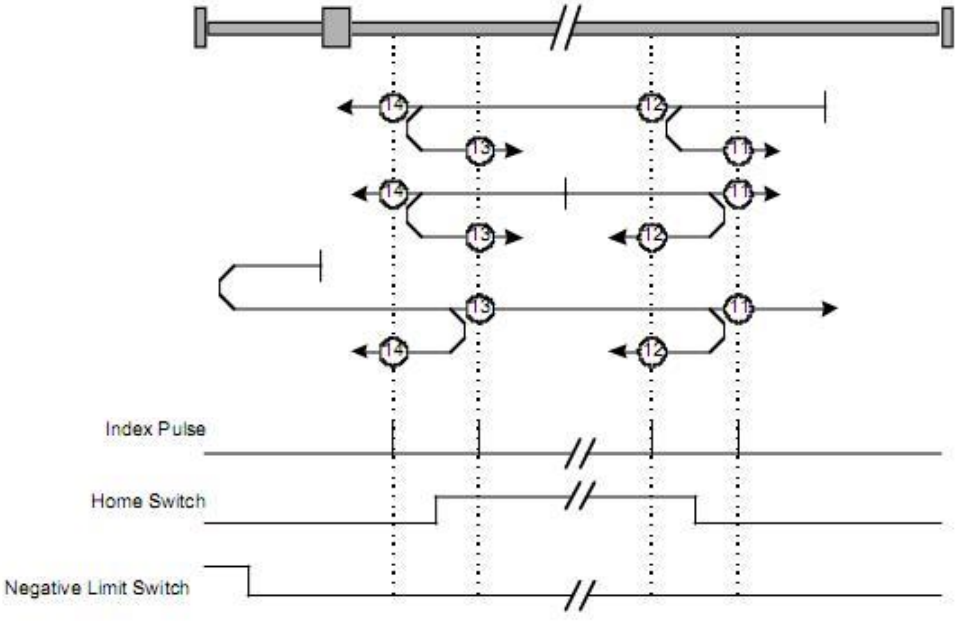


圖 10.6

#15 和 16. 保留

此為保留給原點復歸模式未來擴充使用。

#17 to 30. 與索引脈波無關之原點復歸模式

這些方法類似方法 1 到 14，兩造的區別在於此類歸回原點位置不需依賴索引脈衝。其僅依賴與原點或極限開關的切換來進行原復歸。例：方法 19 與方法 20 類似方法 3 和方法 4 之原點復歸模式，如下圖 11.7 所示。

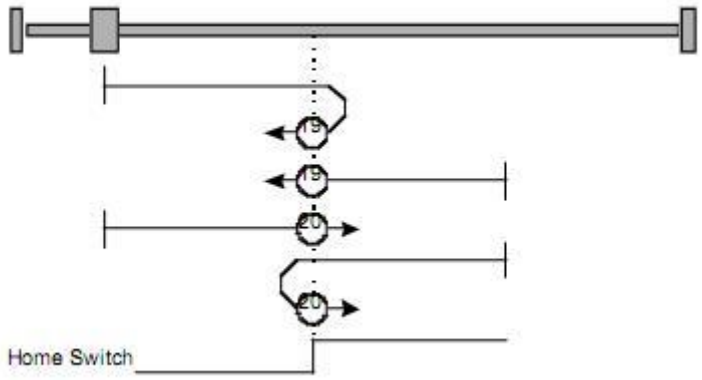


圖 10.7

#31 和 32. 保留

此為保留給原點復歸模式未來擴充使用。

#33 and 34. 與索引脈波相關之原點復歸模式

使用方法 33 或方法 34，其歸回原點正向或負向的區別，在於其歸原回原點選擇的方向位置取決於查找到的索引脈衝。

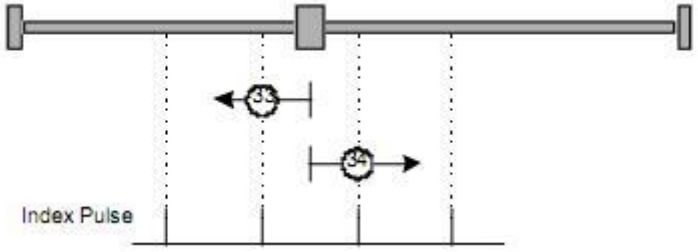


圖 10.8

#35. 依當前 positin 位置進行原點復歸的模式

模式 35 是將目前位置作為原點復歸的位置。

10.2　_DMC_01_set_home_move

■　**格式**

I16 PASCAL _DMC_01_set_home_move (U16 CardNo, U16 NodeID, U16 SlotID)

■　**目的**

設置開始運行原點復歸運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;

I16 status = _DMC_01_set_home_move (CardNo, NodeID, SlotID);
```

NOTE

※如何判斷回 Home 已經完成:
在下達 _DMC_01_set_home_move 後，可運用 _DMC_01_motion_done 的 MC_Status =0 (代表回 Home 動作已停止)且 _DMC_01_motion_status 回傳的 MC_status 之 MDS0=1、Target=1、Mode1=1,Mode2=1(回 Home 模式)等條件來判斷回 Home 是否已經完成，詳細說明可參考 15.1 及 15.2 章節說明。

10.3 _DMC_01_set_home_config2

■ **格式**

I16 PASCAL _DMC_01_set_home_config2 (U16 CardNo,U16 NodeID,U16 SlotID ,
U16 Mode,I32 offset,U16 lowSpeed,U16 highSpeed,F64 acc)

■ **目的**

設置原點復歸各項配置(回 Home 前，請關閉軟體極限設定)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	原點復歸模式：1~35 (參考後頁 note 說明)
offset	I32	脈波數	原點復歸偏移量
lowSpeed	U16	轉速/分	歸回零所使用的速度參數
highSpeed	U16	轉速/分	尋找各極限位置所使用的速度參數
Acc	F64	秒	原點復歸參照的加速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;
I16 status = _DMC_01_set_home_config2 (CardNo, NodeID, SlotID, Mode, offset, lowSpeed,
        highSpeed, acc);
```

NOTE

此 API 與_DMC01_set_home_config 的不同在於，_DMC_01_set_home_config2 的命令是直接下到 Server，_DMC_01_set_home_config 是下到 Memory 他會等_DMC01_set_home_move 的指令下達後，一起進到 Server 並執行所以使用 _DMC_01_set_home_config2 與 _DMC_01_set_home_move2 的搭配，可以省去一些時間；但請注意，_DMC_01_set_home_config 必須搭配 _DMC_01_set_home_move 而 _DMC_01_set_home_config2 必須搭配_DMC_01_set_home_move2，如果混搭，將會發生錯亂現象。

10.4　_DMC_01_set_home_config_offset2

■　**格式**

I16 PASCAL _DMC_01_set_home_config_offset2 (U16 CardNo,U16 NodeID,U16 SlotID , ,I32 offset)

■　**目的**

重新設置原點復歸 offset 值(回 Home 前，請關閉軟體極限設定)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
offset	I32	脈波數	原點復歸偏移量

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;

I16 status = _DMC_01_set_home_config_offset2 (CardNo, NodeID, SlotID, offset);

NOTE

如果在下達_DMC_01_set_home_config2 的命令後，在_DMC_01_set_home_move2 指令未下達前，有執行到_DMC_01_set_position 這個指令，那麼之前下達的 offset 值會被清除，必須在執行_DMC_01_set_home_move2 之前重新使用_DMC_01_set_home_config_offset2 這個 API 給 offset 值。

10.5　_DMC_01_set_home_move2

■　**格式**

I16 PASCAL _DMC_01_set_home_move (U16 CardNo, U16 NodeID, U16 SlotID)

■　**目的**

設置開始運行原點復歸運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	原點復歸模式：1~35 (參考 10.1 節 note 說明)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 , Mode=1;

I16 status = _DMC_01_set_home_move2 (CardNo, NodeID, SlotID, Mode);

10.6　_DMC_01_escape_home_move

■　**格式**

I16 PASCAL _DMC_01_escape_home_move (U16 CardNo, U16 NodeID, U16 SlotID)

■　**目的**

停止原點復歸運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;

I16 status = _DMC_01_escape_home_move (CardNo, NodeID, SlotID);

（此頁有意留為空白）

第十一章　速度運動控制使用封包協定 API

表 11.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_velocity_mode	速度運動控制參數配置
A/B/F	_DMC_01_set_velocity	設置開始運行速度運動
A/B/F	_DMC_01_set_velocity_stop	停止速度運動控制
A/B/F	_DMC_01_get_rpm	取回目前運動的 rpm 值，實際轉速為此變數(rpm)值的 1/10
A/B/F	_DMC_01_set_velocity_torque_limit	在 velocity 模式時設定最大 torque 限制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

11.1 _DMC_01_set_velocity_mode

■ **格式**

I16 PASCAL _DMC_01_set_velocity_mode (U16 CardNo, U16 NodeID, U16 SlotID, F64 Tacc,F64 Tdec)

■ **目的**

速度運動控制參數配置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
F64 Tacc=0.1, Tdec=0.1;

/*  設定速度模式參數(加速時間與減速時間的值) */
  I16 status = _DMC_01_set_velocity_mode (CardNo, NodeID, SlotID, Tacc, Tdec);
```

11.2 _DMC_01_set_velocity

■ **格式**

I16 PASCAL _DMC_01_set_velocity (U16 CardNo, U16 NodeID, U16 SlotID,I32 rpm)

■ **目的**

設置開始運行速度運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
rpm	I32	數值	實際轉速為此變數(rpm)值的 1/10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I32 rpm =-1000; //此例速度運動將會是反轉運動，該實際轉速的值為|-1000| / 10 => 100RPM

I16 status = _DMC_01_set_velocity (CardNo, NodeID, SlotID, rpm);

11.3 _DMC_01_set_velocity_stop

■ **格式**

I16 PASCAL _DMC_01_set_velocity_stop (U16 CardNo,U16 NodeID,U16 SlotID,U16 stop)

■ **目的**

停止速度運動控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
stop	U16	選項單位	0：速度運動操作保持目前的運動狀態 1：停止速度運動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 stop=1;
I16 rpm=2000; //  此例速度運動將會是正轉運動，該實際轉速的值為|2000| / 10 => 200RPM

/*  啟動速度運動的操作  */
I16 status = _DMC_01_set_velocity (CardNo, NodeID, SlotID, rpm);

/*  停止目前運行的速度運動  */
status = _DMC_01_set_velocity_stop (CardNo, NodeID, SlotID, stop);
```

11.4 _DMC_01_get_rpm

■ **格式**

I16 PASCAL _DMC_01_get_rpm (U16 CardNo, U16 NodeID, U16 SlotID, I16 *rpm)

■ **目的**

取回目前運動的 rpm 值，實際轉速為此變數(rpm)值的 1/10。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
rpm	I16	數值	實際轉速為此變數(rpm)值的 1/10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 rpm;

I16 status = _DMC_01_get_rpm (CardNo, NodeID, SlotID, &rpm);
```

11.5 _DMC_01_set_velocity_torque_limit

■ **格式**

I16 PASCAL _DMC_01_set_velocity_torque_limit (U16 CardNo, U16 NodeID, U16 SlotID, U32 torque_limit)

■ **目的**

在 velocity 模式時設定最大 torque 限制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
torque_limit	U32	數值	轉矩最大限制量

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 torque_limit =50; //ratio 值為 50, 50 除以 1000，表示額定扭力值為 5%

I16 status = _DMC_01_set_velocity_torque_limit (CardNo, NodeID, SlotID, torque_limit);

第十二章　轉矩運動控制使用封包協定 API

表 12.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_torque_mode	轉矩運動控制參數配置
A/B/F	_DMC_01_set_torque	設置開始運行轉矩運動
A/B/F	_DMC_01_set_torque_stop	停止轉矩運動
A/B/F	_DMC_01_set_torque_velocity_limit	在 torque 模式時設定最大 velocity 限制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

12.1 _DMC_01_set_torque_mode

■ **格式**

I16 PASCAL _DMC_01_set_torque_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 slope)

■ **目的**

轉矩運動控制參數配置(slope 的值)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
slope	U16	毫秒	0 至 100%額定扭力所需時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 slope=500; // 500 毫秒

I16 status = _DMC_01_set_torque_mode (CardNo, NodeID, SlotID, slope);
```

12.2 _DMC_01_set_torque

■ **格式**

I16 PASCAL _DMC_01_set_torque (U16 CardNo, U16 NodeID, U16 SlotID, I16 ratio)

■ **目的**

設置開始運行轉矩運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	Number Uint	插槽 ID
ratio	I16	數值	額定扭力值千分比(值小於 0 反轉，大於 0 正轉)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
I16 ratio=-50; // ratio 值為-50, -50 除以 1000，表示額定扭力值為 5%且馬達反轉

/* 設置 ratio 值並開始進行轉矩運動 */
I16 status =_DMC_01_set_torque(CardNo, NodeID, SlotID, ratio);
// 若 ratio 的值小於 0 時，馬達反轉；值大於 0 時，馬達正轉。
```

12.3 _DMC_01_set_torque_stop

■ **格式**

I16 PASCAL _DMC_01_set_torque_stop (U16 CardNo, U16 NodeID, U16 SlotID, U16 stop)

■ **目的**

停止轉矩運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
stop	U16	選項單位	0：目前的運動狀態 1：停止轉矩運動

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 stop=1;
I16 ratio=100; // ratio 值為 100，100 除以 1000，表示額定扭力值為 10%且馬達正轉

/*設置 ratio 值並開始進行轉矩運動 */
I16 status =_DMC_01_set_torque(CardNo, NodeID, SlotID, ratio);
/* 停止轉矩運動 */
status = _DMC_01_set_torque_stop(CardNo, NodeID, SlotID, stop);
```

12.4 _DMC_01_set_torque_velocity_limit

■ **格式**

I16 PASCAL _DMC_01_set_torque_velocity_limit (U16 CardNo, U16 NodeID, U16 SlotID, U32 velocity_limit)

■ **目的**

在 torque 模式時設定最大 velocity 限制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
velocity_limit	U32	數值	速度最大限制量

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 velocity_limit =2000;

/*  設置 Velocity 最大限制量, velocity_limit 的值不得大於 30000 */
 I16 status =_DMC_01_set_torque_velocity_limit (CardNo, NodeID, SlotID, velocity_limit);
```

（此頁有意留為空白）

第十三章　使用 PDO 協定 API

表 13.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_ipo_set_svon	對所應對之裝置下達 PowerOn/Off(Servo ON/OFF)—適用於所有 Slave 裝置
A/B/F	_DMC_01_get_buffer_length	取得仍未實行的運動命令
A/B/F	_DMC_01_command_buf_clear	清除間隔時間(buffer dwell 計數值)
A/B	_DMC_01_buf_dwell	兩個運動命令之間的間隔時間
A/B	_DMC_01_set_group	設定關連性群組
A/B/F	_DMC_01_set_dmc_mode	將模式切成 DMC_NET 模式

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

13.1 _DMC_01_ipo_set_svon

■ **格式**

I16 PASCAL _DMC_01_ipo_set_svon (U16 CardNo, U16 NodeID, U16 SlotID, U16 ON_OFF)

■ **目的**

在 PDO 協定下 Servo ON/OFF 設置(所有模組的 Servon On 都可以運用此 API 下達)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ON_OFF	U16	選項單位	0：Servo OFF 1：Servo ON

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
U16 ON_OFF=1;

I16 status = _DMC_01_ipo_set_svon (CardNo, NodeID, SlotID, ON_OFF);
```

13.2 _DMC_01_get_buffer_length

■ **格式**

I16 PASCAL _DMC_01_get_buffer_length (U16 CardNo, U16 NodeID, U16 SlotID, U16* bufferLength)

■ **目的**

取得仍未實行的運動命令(Buffer 大小為 20)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
bufferLength	U16*	個數	未實行的運動命令

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
U16 bufferLength;
I16 status;

/* 啟用同步運動命令 */
status = _DMC_01_sync_move(CardNo);
/* 取得各個站號中尚未執行的運動命令 */
status = _DMC_01_get_buffer_length (CardNo, NodeID, SlotID, &bufferLength);
```

13.3 _DMC_01_command_buf_clear

■ **格式**

I16 PASCAL _DMC_01_command_buf_clear (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

清除間隔時間(buffer dwell 計數值)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0; U16 NodeID =1; U16 SlotID=0; I32 dwell_cnt=3;

I16 status = _DMC_01_buf_dwell(CardNo, NodeID, SlotID, dwell_cnt); //設定 dwell buffer 間隔
status = _DMC_01_command_buf_clear (CardNo, NodeID,SlotID); //清除 dweel buffer 間隔

13.4 _DMC_01_buf_dwell

■ **格式**

I16 PASCAL _DMC_01_buf_dwell (I16 CardNo, U16 NodeID, U16 SlotID, I32 dwell_cnt)

■ **目的**

兩個運動命令之間的間隔時間。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
dwell_cnt	I32	次數	Dewell buffer 間隔 (延遲時間為 2 x dwell_cnt + 2)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0; U16 NodeID =1; U16 SlotID=0;
I32 dwell_cnt=3; //若 dwell_cnt 值為 1，延遲時間為 4ms；此例的值為 3，延遲時間為 2*3+2=8ms

I16 status =_DMC_01_buf_dwell (CardNo, NodeID, SlotID, dwell_cnt);

13.5 _DMC_01_set_group

■ **格式**

I16 PASCAL _DMC_01_set_group (U16 CardNo, U16* NodeID,U16* SlotID,
U16 NodeID_Num, U16 enable)

■ **目的**

設定關連性群組(群組中若有任一軸發生重大問題而緊急停止時，該群組內所有站號將一同緊急停止)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NodeID_Num	U16	數值單位	群組站數
Enable	U16	編號單位	0：解除群組設定。 1：設定為群組。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID[4] ={1,3,5};
U16 SlotID[4]={0,0,0};
U16 NodeID_Num=3; //設定為同一群組個數為 3 個。
U16 enable=1;

I16 status =_DMC_01_set_group (CardNo, NodeID, SlotID, NodeID_Num, enable);
```

NOTE

1. **預設值為所有站號為同一個 Group。**
2. **例如:一但設定 2、3、4 為同一個 Group，則其餘站號都視為獨立個體。**
3. **所有站號最多可設定 6 個 Group。**
4. **同一站號不要在一個以上的 Group 中設定。**

13.6　_DMC_01_set_dmc_mode

■　**格式**

I16 PASCAL _DMC_01_set_dmc_mode (I16 CardNo, U16 NodeID, U16 SlotID)

■　**目的**

將模式切成 DMC_NET 模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
I16 status;

I16 status = _DMC_01_set_dmc_mode (CardNo, NodeID, SlotID);
```

（此頁有意留為空白）

第十四章　停止運動控制 API

表 14.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_emg_stop	Buffer 內所有運動指令將立即停止
A/B/F	_DMC_01_sd_stop	Buffer 內所有運動指令將依減速時間減速停止
A/B/F	_DMC_01_sd_abort	當前正在執行的運動指令將依減速時間停止
A/B/F	_DMC_01_set_sd_mode	Sd_stop 的模式設定
A/B/F	_DMC_01_set_sld_mode	設定 SLD 訊號觸發，SLD 運動模式。
A/B/F	_DMC_01_set_sld_stop	啟動/關閉 SLD 模式設定
A/B/F	_DMC_01_set_sd_time	設定減速時間。
A/B/F	_DMC_01_set_scurve_rate	設定加、減速段，S_Curve 模式的百分比。

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

14.1 _DMC_01_emg_stop

■ **格式**

I16 PASCAL _DMC_01_emg_stop (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

目前執行的運動指令將立即停止，且 Buffer 內所有未執行的等待指令將被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0;U16 NodeID =1; U16 SlotID=0;

I16 status = _DMC_01_emg_stop (CardNo, NodeID, SlotID);

14.2 _DMC_01_sd_stop

■ **格式**

I16 PASCAL _DMC_01_sd_stop (U16 CardNo, U16 NodeID,U16 SlotID,F64 Tdec)

■ **目的**

目前執行的運動指令將依減速時間減速停止，且 Buffer 內所有未執行的等待指令將被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
F64 Tdec=0.1;

I16 status = _DMC_01_sd_stop (CardNo, NodeID, SlotID, Tdec);

14.3 _DMC_01_sd_abort

■ **格式**

I16 PASCAL _DMC_01_sd_abort (U16 CardNo, U16 NodeID, U16 SlotID, F64 Tdec)

■ **目的**

當前正在執行的運動指令將依減速時間停止，但 Buffer 內所有未執行的等待指令不會被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
F64 Tdec=0.1;

I16 status = _DMC_01_sd_abort (CardNo, NodeID, SlotID, Tdec);

14.4 _DMC_01_set_sd_mode

■ **格式**

I16 PASCAL _DMC_01_set_sd_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

Sd_stop(slow down stop)的模式設定。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：任何情形下，執行 sd_stop 後的最終停止位置不允許超過原運動命令值。 1：任何情形下，執行 sd_stop 後的最終停止位置可依照減速時間的設定而超過原運動命令值。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 Mode=1;
F64 Tdec=0.1;

I16 status = _DMC_01_set_sd_mode (CardNo, NodeID, SlotID, Mode);
```

NOTE

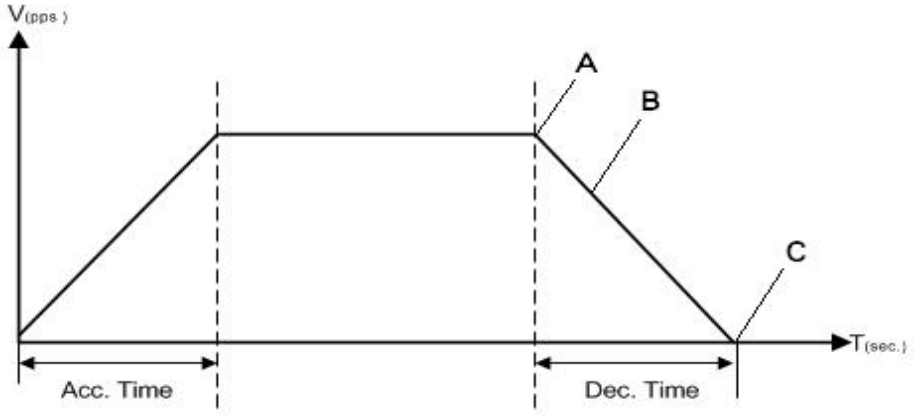


圖 14.1　Sd Mode 設定說明(Mode=0)

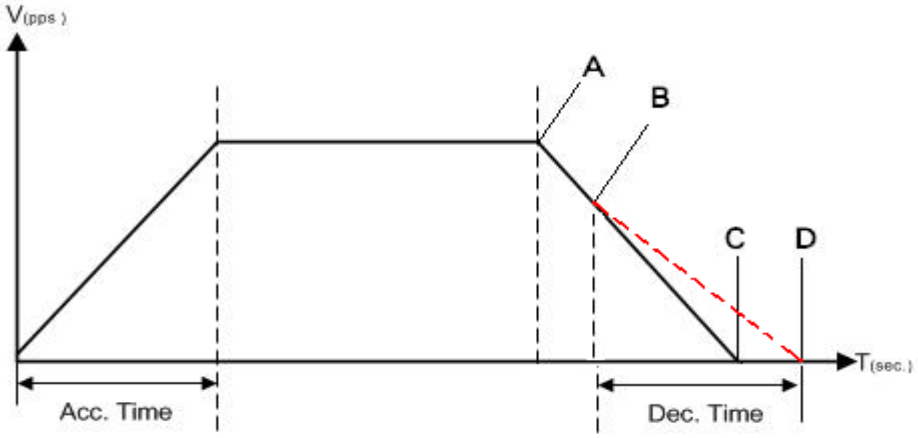


圖 14.2　Sd Mode 設定說明(Mode=1)

14.5　_DMC_01_set_sld_mode

■　**格式**

I16 PASCAL _DMC_01_set_sld_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■　**目的**

設定 SLD 訊號觸發，SLD 運動模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	SLD 模式設定。 0、1：緊急停止。 2：依設定之減速時間停止(DMC_01_set_sd_time) 3：變速度(Change to start Velocity)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
U16 mode=1; //緊急停止。

I16 status = _DMC_01_set_sld_mode (CardNo, NodeID, SlotID, mode);

14.6　_DMC_01_set_sld_stop

■　**格式**

I16 PASCAL _DMC_01_set_sld_stop (U16 CardNo, U16 NodeID, U16 SlotID, U16 enable)

■　**目的**

啟動/關閉 SLD 模式設定。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	U16	選項單位	0：關閉 SLD 模式。 1：啟動 SLD 模式。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
U16 enable=1;　　//啟動 SLD 模式。

I16 status = _DMC_01_set_sld_stop (CardNo, NodeID, SlotID, enable);

14.7　_DMC_01_set_sd_time

■　**格式**

I16 PASCAL _DMC_01_set_sd_time (U16 CardNo, U16 NodeID, U16 SlotID, F64 sd_dec)

■　**目的**

設定減速時間。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
sd_dec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, U16 NodeID =1, SlotID=0;
F64 sd_dec=2

I16 status = _DMC_01_set_sd_time (CardNo, NodeID, SlotID, sd_dec);

14.8　_DMC_01_set_scurve_rate

■　**格式**

I16 PASCAL _DMC_01_set_scurve_rate(U16 CardNo, U16 NodeID, U16 SlotID, U16 scurve_rate)

■　**目的**

設定加、減速段，S_Curve 模式的百分比。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
scurve_rate	U16	數值單位	以加、減速段中心點，往前/往後 T_Curve 模式的百分比(0~100)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo = 0;
U16 NodeID =1, SlotID = 0;
U16 scurve_rate = 60;
I16 status = _DMC_01_set_scurve_rate (CardNo, NodeID, SlotID, scurve_rate);
```

NOTE

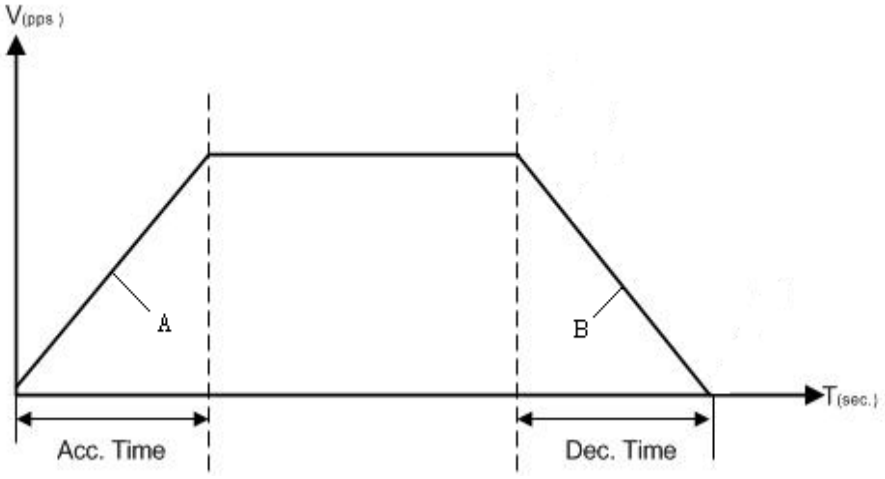


圖 13.3scurve_rate 設定說明

第十五章　運動狀態 API

表 15.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_motion_done	傳回 Master Card 目前實行的運動階段
A/B/F	_DMC_01_motion_status	傳回 Master Card 目前的運動狀態
A/B/F	_DMC_01_set_01_ph_inposition	設定 01PH INP 到位訊號的區間範圍
A/B/F	_DMC_01_get_01_ph_inposition	取得目前 01PH INP 到位訊號設定的區間範圍

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

15.1 _DMC_01_motion_done

■ **格式**

I16 PASCAL _DMC_01_motion_done (U16 CardNo, U16 NodeID, U16 SlotID, U16* MC_status)

■ **目的**

傳回 Master Card 目前實行的運動階段。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
MoSt	U16*	選項單位	0：運動位移動作停止 1：依加速時間進行運動位移 2：依最大速度進行運動位移 3：依減速時間進行運動位移 5：指令在 FIFO，尚未進入 Buffer(伺服或 04PI Md2) 6：Buffer 有運動指令(伺服或 04PI Md2) 12：回 Home 狀態中(伺服或 04PI Md2) 21：PC 與 DSP 的 Counter 沒對準(04PI Md1) 22：FIFO 中有指令(04PI Md1) 23：Buffer Full(04PI Md1)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 MoSt;

I16 status = _DMC_01_motion_done (CardNo, NodeID, SlotID, &MoSt);
```

15.2 _DMC_01_motion_status

■ **格式**

I16 PASCAL _DMC_01_motion_status (U16 CardNo, U16 NodeID, U16 SlotID, U32* MC_status)

■ **目的**

傳回 Master Card 目前的運動狀態。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>插槽 ID</td></tr>
<tr><td>MoSt</td><td>U32*</td><td>選項單位</td><td>運動狀態(bit0~bit15)
<table>
<tr><th>位元</th><th>標誌</th><th>描述</th></tr>
<tr><td>0 ~ 3</td><td>Mode0 ~ Mode3</td><td>啟用選擇模式</td></tr>
<tr><td>4</td><td>DI3</td><td>DI3（SLD）狀態映射</td></tr>
<tr><td>5</td><td>WR</td><td>警報訊息</td></tr>
<tr><td>6</td><td>DR</td><td>資料錯誤訊息</td></tr>
<tr><td>7</td><td>TG</td><td>觸發模式位元</td></tr>
<tr><td>8</td><td>PWRON</td><td>馬達激磁啟動位元</td></tr>
<tr><td>9</td><td>DriverErr</td><td>驅動器錯誤位元</td></tr>
<tr><td>10</td><td>Target</td><td>抵達目標位元</td></tr>
<tr><td>11</td><td>N/A</td><td>運動方向,1:正向 0:負向</td></tr>
<tr><td>12</td><td>MDS0</td><td>Mode specific</td></tr>
<tr><td>13</td><td>MDS1</td><td>Mode specific</td></tr>
<tr><td>14</td><td>PEL</td><td>正極限位元</td></tr>
<tr><td>15</td><td>MEL</td><td>負極限位元</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U32 MoSt=0;

I16 status = _DMC_01_motion_status (CardNo, NodeID, SlotID, &MoSt);
```

NOTE

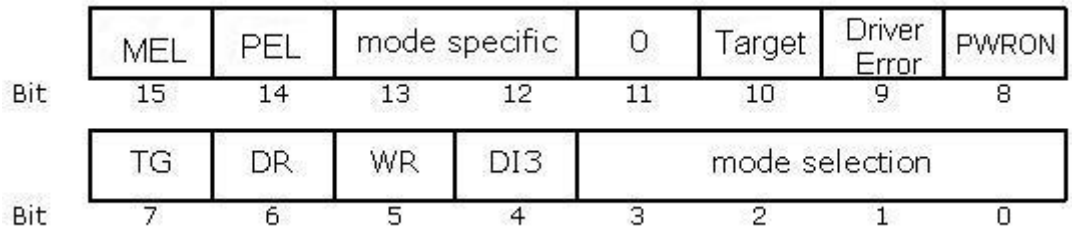


圖 15.1 運動狀態內容

關於「具體模式 (Mode specific)」位元 (位元 12 和位元 13) 相關敘述，請參考表 15.2。在模式選擇位元(位元 0~位元 3)中，僅有 2 種模式供使用者選擇，模式分別為 MODE1（位元 0 為 ON）和 MODE6（位元 1 與位元 2 為 ON）。

表 15.2 運動狀態的具體模式（Mode specific）位元敘述

模式項目	**具體模式**	
	位元 13 為 1	**位元 12 為 1**
原點復歸模式（MODE6）	復歸動作有誤	可實行復歸動作
DMCNET 模式（MODE1）	無定義	模式啟用

表 15.3 運動狀態的 Mode Selection 敘述

位元	0	1	2	3	模式說明
ON/OFF	**ON**	OFF	OFF	OFF	Profile Position mode
ON/OFF	**ON**	**ON**	OFF	OFF	Profile Velocity mode
ON/OFF	OFF	OFF	**ON**	OFF	Profile Torque mode
ON/OFF	OFF	**ON**	**ON**	OFF	Homing mode
ON/OFF	**ON**	**ON**	**ON**	**ON**	DMCNET mode

15.3　_DMC_01_set_01_ph_inposition

■　**格式**

I16 PASCAL _DMC_01_set_01_ph_inposition (U16 CardNo, U16 NodeID,U16 SlotID , U16 poswindow_time,U32 poswindow)

■　**目的**

設定 01PH INP 到位訊號的區間範圍。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
poswindow_time	U16*	數值單位	設定比對次數，比對差值位於設定範圍幾次後，才將 Target 訊號變換
poswindow	U32	數值單位	設定 IN Position 訊號啟動的 Pulse 誤差量

■　**屬性**

軸卡	伺服	04PI Md1	GE01PH	MN	NT	DA&AD	RM64MN
			●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 poswindow_time=10;
U32 poswindow=100;

I16 status = _DMC_01_set_01_ph_inposition (CardNo, NodeID, SlotID, poswindow_time, poswindow);
```

15.4 _DMC_01_get_01_ph_inposition

■ **格式**

I16 PASCAL _DMC_01_get_01_ph_inposition (U16 CardNo, U16 NodeID,U16 SlotID , U16* poswindow_time,U32* poswindow)

■ **目的**

取得目前 01PH INP 到位訊號設定的區間範圍。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
poswindow_time	U16*	數值單位	設定比對次數，比對差值位於設定範圍幾次後，才將 Target 訊號變換
poswindow	U32*	數值單位	設定 IN Position 訊號啟動的 Pulse 誤差量

■ **屬性**

軸卡	伺服	04PI Md1	GE01PH	MN	NT	DA&AD	RM64MN
			●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 poswindow_time;
U32 poswindow;

I16 status = _DMC_01_get_01_ph_inposition (CardNo, NodeID, SlotID, &poswindow_time, &poswindow);
```

第十六章　運動計數值 API

表 16.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_get_command	取得 command 計數值
A/B/F	_DMC_01_set_command	設置新的 command 計數值
A/B/F	_DMC_01_get_position	取得目前 position 計數值
A/B/F	_DMC_01_set_position	設置新的 position 計數值
A/B/F	_DMC_01_get_target_pos	取得目的位置的 position 值
A/B/F	_DMC_01_get_torque	取得傳回目前的轉矩計數值
A/B/F	_DMC_01_get_current_speed	取得目前的運動分量速度
A/B/F	_DMC_01_get_current_speed_rpm	取得目前每分鐘的轉速乘上 10 倍的值

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

16.1 _DMC_01_get_command

■ **格式**

I16 PASCAL _DMC_01_get_command (U16 CardNo, U16 NodeID, U16 SlotID, I32* cmd)

■ **目的**

取得 command 計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
cmd	I32*	脈波數	目前 Command 的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
I32 cmd = 0;

I16 status = _DMC_01_get_command (CardNo, NodeID, SlotID, &cmd);
```

16.2　_DMC_01_set_command

■　**格式**

I16 PASCAL _DMC_01_set_command (U16 CardNo, U16 NodeID, U16 SlotID, I32 cmd)

■　**目的**

設置新的 command 計數值(下此指令之前應先執行 Set_Position 指令)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
cmd	I32	脈波數	欲設置的 Command 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0; I32 cmd=1000000;

I16 status = _DMC_01_set_command (CardNo, NodeID, SlotID, cmd);

16.3 _DMC_01_get_position

■ **格式**

I16 PASCAL _DMC_01_get_position (U16 CardNo, U16 NodeID, U16 SlotID, I32* pos)

■ **目的**

取得目前 position 計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pos	I32*	脈波數	目前反饋的 position 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0; I32 pos;

I16 status = _DMC_01_get_position (CardNo, NodeID, SlotID, &pos);

16.4 _DMC_01_set_position

■ **格式**

I16 PASCAL _DMC_01_set_position (U16 CardNo, U16 NodeID, U16 SlotID, I32 pos)

■ **目的**

設置新的 position 計數值(使用此 API 時，會切換為 DMC_NET 模式)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pos	I32	脈波數	設置 position 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0; I32 pos =500000;

I16 status = _DMC_01_set_position (CardNo, NodeID, SlotID, pos);

16.5　_DMC_01_get_target_pos

■　**格式**

I16 PASCAL _DMC_01_get_target_pos (U16 CardNo, U16 NodeID, U16 SlotID, I32* pos)

■　**目的**

取得當前正在執行的運動指令之目標位置值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
pos	I32*	脈波數	目的位置的 position 計數值(請參閱下方 note 說明)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 pos=0;

I16 status = _DMC_01_get_target_pos (CardNo, NodeID, &pos);
```

NOTE

※下達任一運動指令後，如果須依賴 _DMC_01_get_target_pos 指令的回傳值 pos 來判斷下一個步驟，請於下達運動指令後，延遲 10~100ms(此值依 CPU 性能與程式撰寫方式調整)後，再下達_DMC_01_get_target_pos 指令，以確保資料正確性。

16.6　_DMC_01_get_torque

■　**格式**

I16 PASCAL _DMC_01_get_torque (U16 CardNo, U16 NodeID, U16 SlotID, U16* torque)

■　**目的**

取得傳回目前的轉矩計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
torque	U16*	數值	目前的轉矩值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16 torque;

I16 status = _DMC_01_get_torque (CardNo, NodeID, SlotID, &torque);
```

16.7　_DMC_01_get_current_speed

■　**格式**

I16 PASCAL _DMC_01_get_current_speed (U16 CardNo, U16 NodeID, U16 SlotID, I32* speed)

■　**目的**

取得目前的運動分量速度(只在 DMC_NET 模式有意義)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
speed	I32*	脈波數/秒	取得目前的運動分量速度

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
I32 speed;

I16 status = _DMC_01_get_current_speed (CardNo, NodeID, SlotID, &speed);
```

16.8 _DMC_01_get_current_speed_rpm

■ **格式**

I16 PASCAL _DMC_01_get_current_speed_rpm (U16 CardNo, U16 NodeID, U16 SlotID, I32 *rpm)

■ **目的**

取得目前每分鐘的轉速乘上 10 倍的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
rpm	I32*	轉速/秒	實際轉速值為此變數(rpm)值的 1/10

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
I32 rpm;

I16 status = _DMC_01_get_current_speed_rpm (CardNo, NodeID, SlotID, &rpm);
// 若您取得到的轉速值為 1000，實際轉速值為 100RPM.
```

（此頁有意留為空白）

第十七章　軟體極限 API

表 17.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_soft_limit	設置軟體正/負極限的參考值
A/B/F	_DMC_01_enable_soft_limit	設置啟用極限開啟與否和碰觸極限後停止方式
A/B/F	_DMC_01_disable_soft_limit	關閉軟體極限
A/B/F	_DMC_01_get_soft_limit_status	取得運動期間軟體正/負極限的狀態

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

17.1　_DMC_01_set_soft_limit

■　**格式**

I16 PASCAL _DMC_01_set_soft_limit (U16 CardNo, U16 NodeID, U16 SlotID, I32 PLimit, I32 NLimit)

■　**目的**

設置軟體正/負極限的參考值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
PLimit	I32	數值	軟體正極限的值
NLimit	I32	數值	軟體負極限的值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;
I32 PLimit = 8000;
I32 NLimit = -6000;

I16 status = _DMC_01_set_soft_limit (CardNo, NodeID, SlotID, PLimit, NLimit);
```

17.2　_DMC_01_enable_soft_limit

■　**格式**

I16 PASCAL _DMC_01_enable_soft_limit (U16 CardNo, U16 NodeID, U16 SlotID, I16 Action)

■　**目的**

設置啟用極限開啟與否和碰觸極限後的停止方式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Action	I16	選項單位	1：碰觸極限後立即停止 2：碰觸極限後減速停止*1*2

*1. 減速時間可由 API _DMC_01_set_sd_time 進行設定。

*2. 在 DDA 模式僅顯示軟體正/負極限的狀態，所下達的位置命令仍會執行。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I16   Action=1; //  選擇碰觸極限後立即停止

I16 status = _DMC_01_enable_soft_limit (CardNo, NodeID, SlotID, Action);
```

17.3　_DMC_01_disable_soft_limit

■　**格式**

I16 PASCAL _DMC_01_disable_soft_limit (U16 CardNo, U16 NodeID,U16 SlotID)

■　**目的**

關閉軟體極限。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID=1, SlotID=0;

I16 status = _DMC_01_disable_soft_limit(CardNo, NodeID, SlotID);

17.4 _DMC_01_get_soft_limit_status

■ **格式**

I16 PASCAL _DMC_01_get_soft_limit_status (U16 CardNo, U16 NodeID, U16 SlotID, U16* PLimit_sts,U16* NLimit_sts)

■ **目的**

取得運動期間軟體正/負極限的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
PLimit_sts	U16*	旗標	0：未碰觸到軟體正極限 1：已碰觸到軟體正極限
NLimit_sts	U16*	旗標	0：未碰觸到軟體負極限 1：已碰觸到軟體負極限

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 PLimit_sts, NLimit_sts;

I16 status = _DMC_01_get_soft_limit_status(CardNo, NodeID, SlotID, &PLimit_sts,
&NLimit_sts);
```

（此頁有意留為空白）

第十八章　單軸運動控制 API

表 18.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_move	參照 T-curve 速度剖面下相對座標進行運動位移
A/B/F	_DMC_01_start_sr_move	參照 S-curve 速度剖面下相對座標進行運動位移
A/B/F	_DMC_01_start_ta_move	參照 T-curve 速度剖面下絕對座標進行運動位移
A/B/F	_DMC_01_start_sa_move	參照 S-curve 速度剖面下絕對座標進行運動位移
A/B/F	_DMC_01_p_change	以新的位置值替換目前的位置設置
A/B/F	_DMC_01_v_change	以新的速度值設置替換目前的運動速度
A/B/F	_DMC_01_start_tr_move_2seg	參照 T-curve 速度剖面下的相對座標進行二段速度運動位移
A/B/F	_DMC_01_start_sr_move_2seg	參照 S-curve 速度剖面下的相對座標進行二段速度運動位移
A/B/F	_DMC_01_start_ta_move_2seg	參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移
A/B/F	_DMC_01_start_sa_move_2seg	參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移
A/B/F	_DMC_01_start_tr_move_2seg2	參照 T-curve 速度剖面下的相對座標進行二段速度運動位移
A/B/F	_DMC_01_start_sr_move_2seg2	參照 S-curve 速度剖面下的相對座標進行二段速度運動位移
A/B/F	_DMC_01_start_ta_move_2seg2	參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移
A/B/F	_DMC_01_start_sa_move_2seg2	參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移
A/B/F	_DMC_01_feedrate_overwrite	改變運動向量速度或向量速度比率
A/B/F	_DMC_01_start_v3_move	增加 EndVel 的單軸運動位移

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

18.1　_DMC_01_start_tr_move

■　**格式**

I16 PASCAL _DMC_01_start_tr_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行運動位移，請參閱敘述中圖 18.1 示意。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

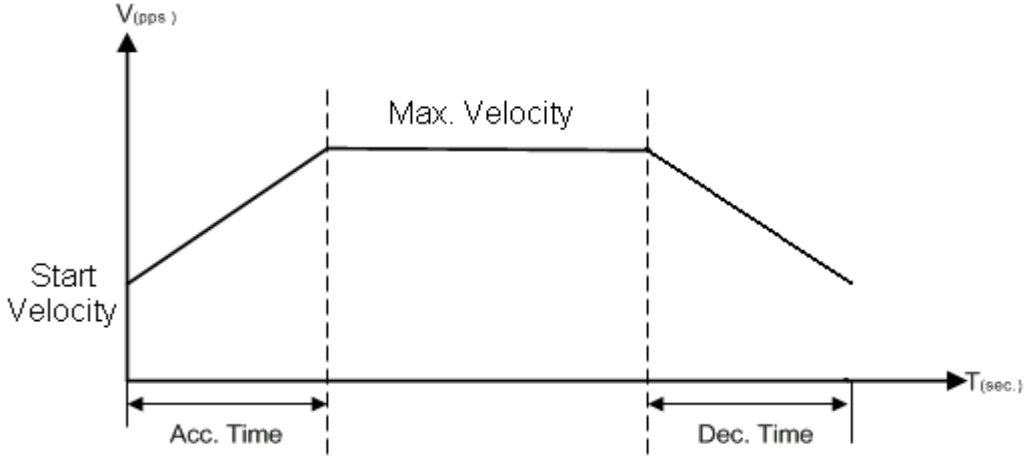


圖 18.1 T-curve 速度剖面下相對座標運動

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=100000, StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_tr_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);

18.2　_DMC_01_start_sr_move

■　**格式**

I16 PASCAL _DMC_01_start_sr_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行運動位移，請參閱敘述中圖 18.2 示意。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

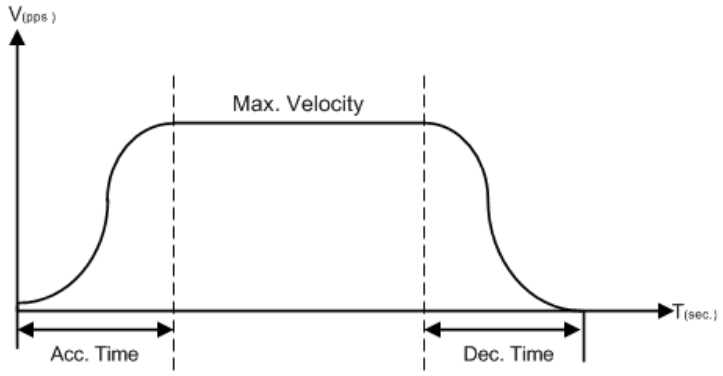


圖 18.2 S-curve 速度剖面下相對座標運動

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;

I32 Dist=100000, StrVel=0, MaxVel=30000;

F64 Tacc=0.1, Tdec=0.1;

16 status = _DMC_01_start_sr _move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);

18.3　_DMC_01_start_ta_move

■　**格式**

I16 PASCAL _DMC_01_start_ta_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=100000;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_ta_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);

18.4　_DMC_01_start_sa_move

■　**格式**

I16 PASCAL _DMC_01_start_sa_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=100000;
I32 StrVel=0, MaxVel=30000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sa_move (CardNo, NodeID, SlotID, Dist, StrVel, MaxVel, Tacc, Tdec);

18.5 _DMC_01_p_change

■ **格式**

I16 PASCAL _DMC_01_p_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewPos)

■ **目的**

以新的位置值替換目前的位置設置(p_change 只能對單軸運動指令下達)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewPos	I32	脈波數	欲替換的位置參數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;U16 NodeID =1, SlotID=0;
I32 NewPos=100000;

I16 status = _DMC_01_p_change (CardNo, NodeID, SlotID, NewPos);
```

18.6　_DMC_01_v_change

■　**格式**

I16 PASCAL _DMC_01_v_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewSpeed, F64 sec)

■　**目的**

以新的速度值設置替換目前的運動速度(v_change 不能在減速段下達即 motion done=3 時)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewSpeed	I32	脈波數/秒	欲替換速度參數
sec	F64	秒	速度轉換時，指定的加/減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0;U16 NodeID =1, SlotID=0;
I32 NewSpeed=3000;
F64 sec=0.1;
I16 status = _DMC_01_v_change (CardNo, NodeID, SlotID, NewSpeed, sec);

NOTE

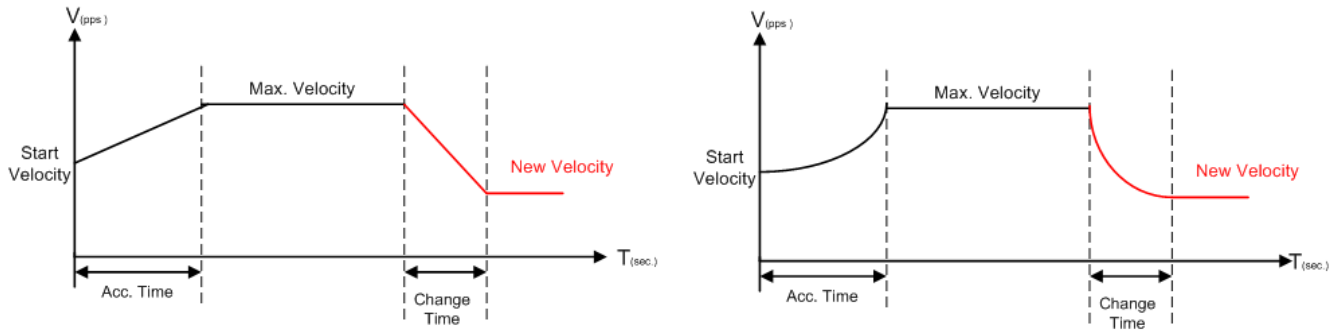


圖 18.3　更換新的速度參數

※在單軸及多軸運動下，都可以進行 V-change 的指令，多軸運動請對主控軸下達指令即可。

18.7　_DMC_01_start_tr_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_tr_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行二段速度運動位移。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**描述**

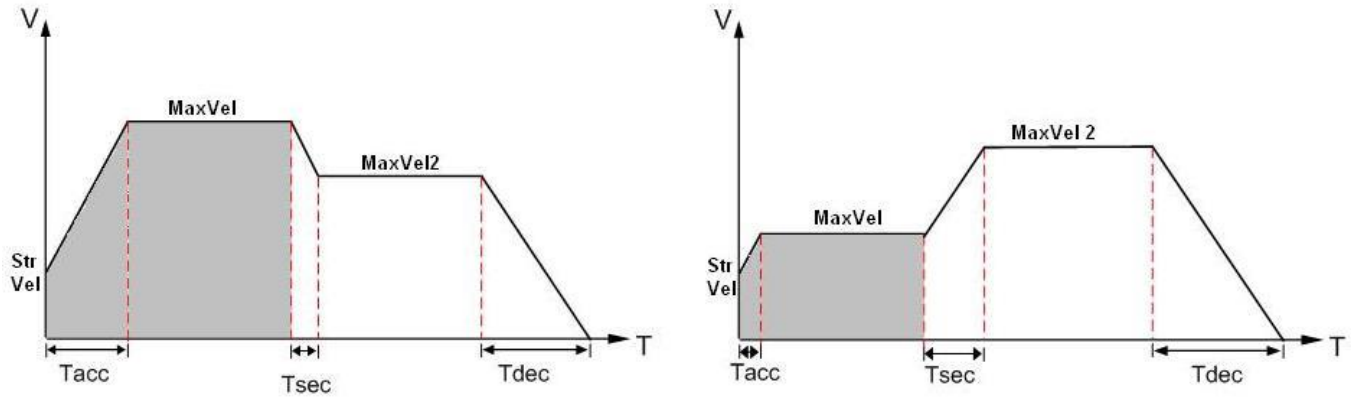


圖 18.4　T-curve 速度剖面下相對座標運動（灰色區塊表示為 Dist, 白色區塊則表示為 Dist2）

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
I32 Dist=500000;
I32 Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000;
I32 MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
```

NOTE

※請務必注意，此 API 參數中的 Dist 與 Dist2 必須「同向」，如圖 18.5 表示一個錯誤的設置。

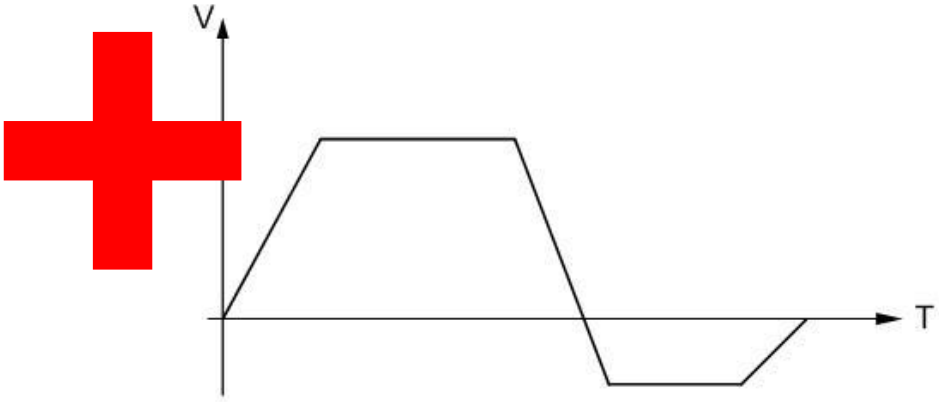


圖 18.5　錯誤圖例 - Dist 與 Dist2 不同向

18.8　_DMC_01_start_sr_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_sr_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

18.9　_DMC_01_start_ta_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_ta_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_ta_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

18.10　_DMC_01_start_sa_move_2seg

■　**格式**

I16 PASCAL _DMC_01_start_sa_move_2seg(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_sa_move_2seg (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

18.11　_DMC_01_start_tr_move_2seg2

■　**格式**

I16 PASCAL _DMC_01_start_tr_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

NOTE

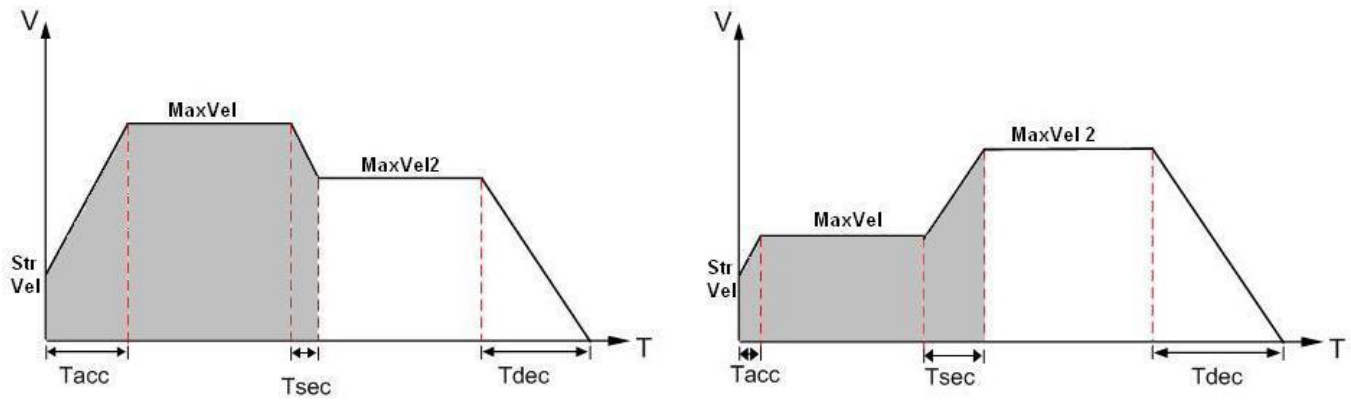


圖 18.5　T-curve 速度剖面下相對座標運動
(灰色區塊表示為 Dist, 白色區塊則表示為 Dist2)

18.12　_DMC_01_start_sr_move_2seg2

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段相對座標行程
Dist2	I32	脈波數	指定的第 2 段相對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_sr_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel, MaxVel, MaxVel2, Tacc, Tsec, Tdec);

18.13　_DMC_01_start_ta_move_2seg2

■ **格式**

I16 PASCAL _DMC_01_start_ta_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;
I16 status = _DMC_01_start_ta_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
```

18.14　_DMC_01_start_sa_move_2seg2

■　**格式**

I16 PASCAL _DMC_01_start_sa_move_2seg2 (U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 Dist2, I32 StrVel, I32 MaxVel, I32 MaxVel2, F64 Tacc, F64 Tsec, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行二段速度運動位移。

※在操作該函式前，Motion Buffer 會先被清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的第 1 段絕對座標行程
Dist2	I32	脈波數	指定的第 2 段絕對座標行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	第 1 段行程中設置的最大速度
MaxVel2	I32	脈波數/秒	第 2 段行程中設置的最大速度
Tacc	F64	秒	指定的加速時間
Tsec	F64	秒	第 1 段行程切換到第 2 段行程的加速/減速時間
Tdec	F64	秒	減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I32 Dist=500000, Dist2=500000
I32 StrVel=1000;
I32 MaxVel=10000, MaxVel2=30000;
F64 Tacc=0.1;
F64 Tsec=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_sa_move_2seg2 (CardNo, NodeID, SlotID, Dist, Dist2, StrVel,
MaxVel, MaxVel2, Tacc, Tsec, Tdec);
```

18.15　_DMC_01_feedrate_overwrite

■　**格式**

I16 PASCAL _DMC_01_feedrate_overwrite (U16 CardNo, U16 NodeID, U16 SlotID, U16 Mode, I32 New_Speed, F64 sec)

■　**目的**

改變運動向量速度或向量速度比率。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：與 V_Change 相同，在有運動執行下，改變當前運動的向量速度。 1：不論當下有無運動都可執行，可改變當下運動及後續所有運動的向量速度。 2：不論當下有無運動都可執行，可改變當下運動及後續所有運動的向量速度比例，範圍是 0% ~ 1000%。
NewSpeed	I32	脈波數/秒	Mode=0,1 時:欲替換速度參數 Mode=2 時:向量速度比例值範圍是 0% ~ 1000%
sec	F64	秒	速度轉換時, 指定的加/減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**

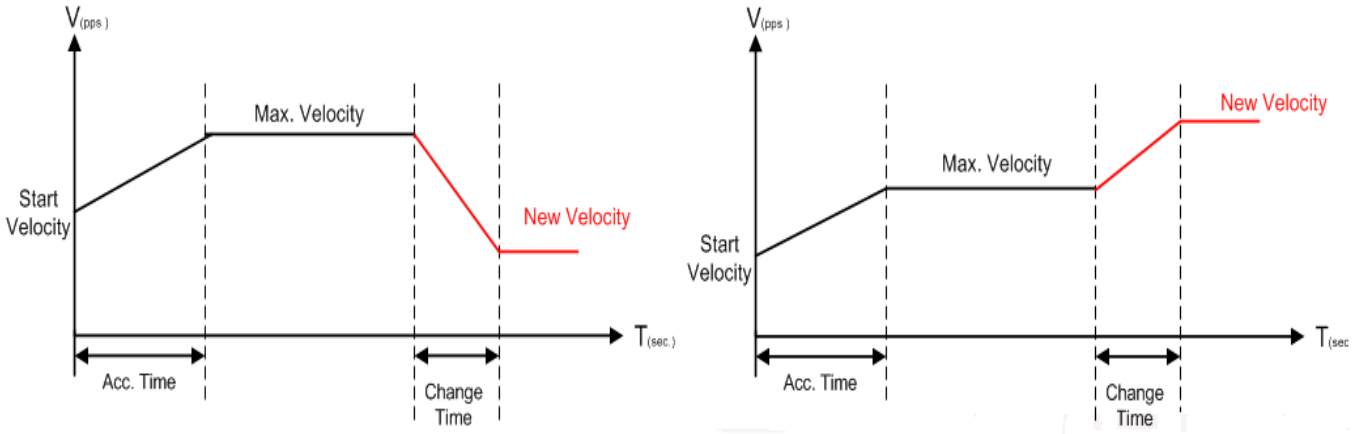


圖 18.7　Mode = 0 更換新的向量速度

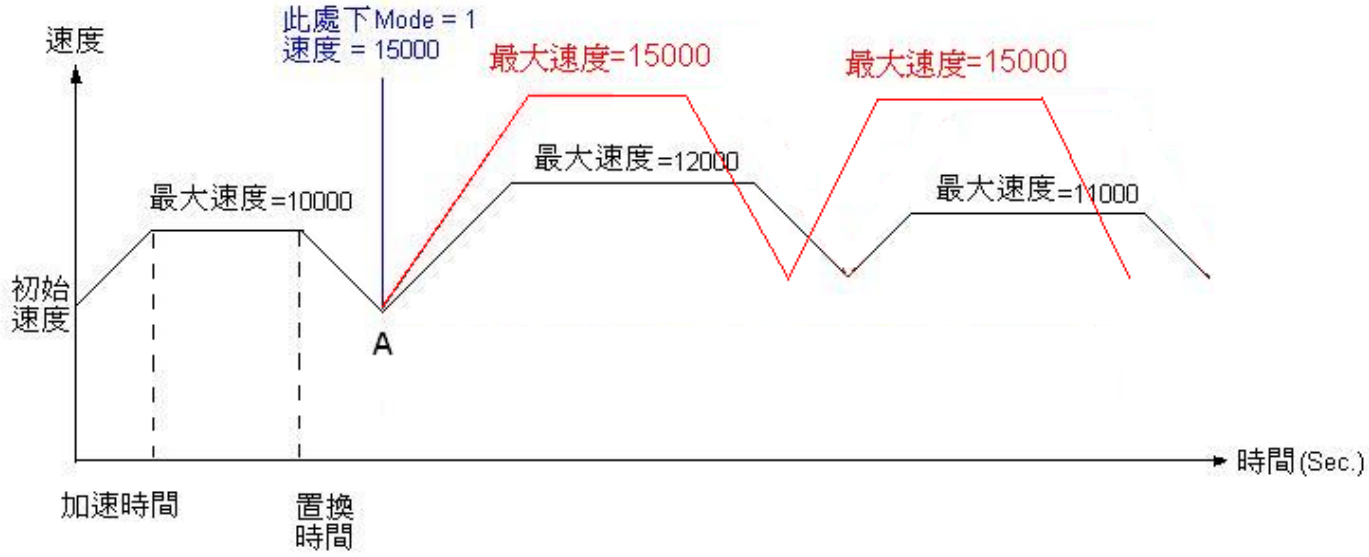


圖 18.8　Mode = 1 更換新的向量速度

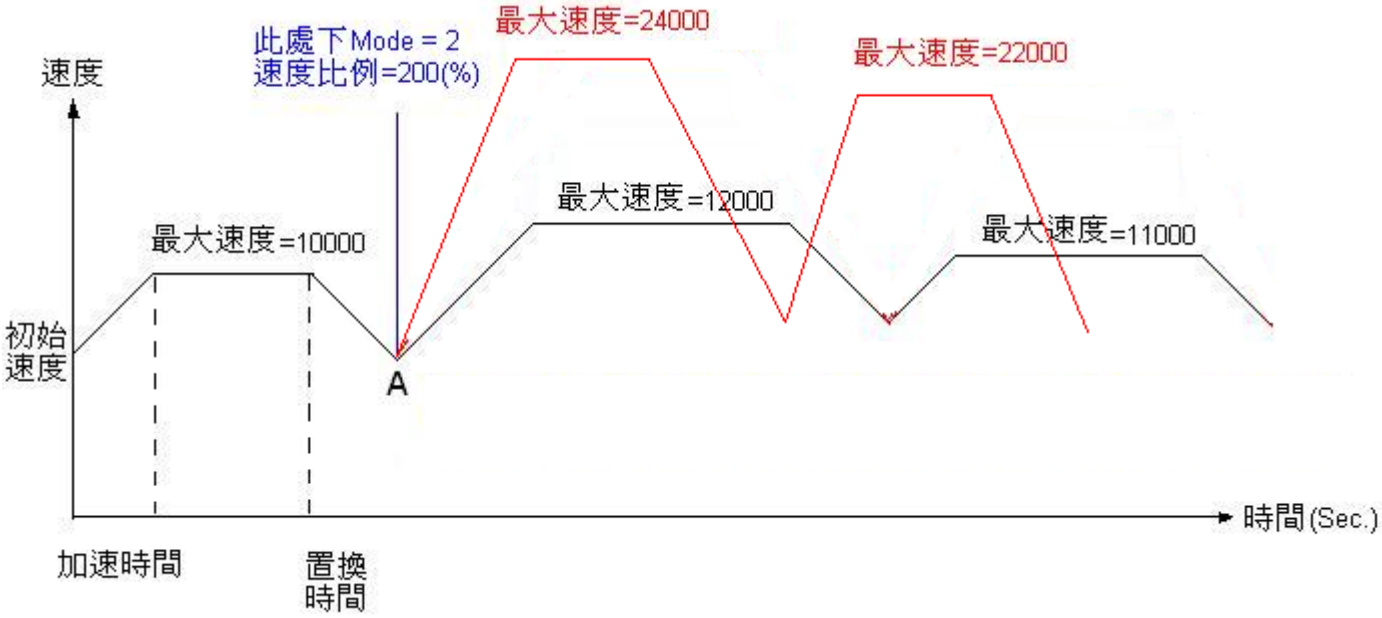


圖 18.9　Mode = 2 更換新的向量速度比例

■ **範例**

U16 CardNo=0; U16 NodeID =1, SlotID=0, Mode=0;
I32 NewSpeed=3000;
F64 sec=0.1;

I16 status = _DMC_01_feedrate_overwrite (CardNo, NodeID, SlotID, Mode, NewSpeed, sec);

NOTE

使用 DMC_01_feedrate_overwrite　API 有幾點事項需注意。

1. **若下 feedrate_overwrite 的時機剛好落在運動的加速段及最大速度段，那麼加、減速度及最大速度將依照您下的 NewSpeed 比例變化。**
2. **若下 feedrate_overwrite 的時機剛好落在運動的減速段，那麼 API 將回應錯誤值(9)，API 下達失敗，需要有 Check API Return Code 及重下機制。**

18.16　_DMC_01_start_v3_move

■　**格式**

I16 PASCAL _DMC_01_start_v3_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■　**目的**

增加 EndVel 的單軸運動位移。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32	脈波數	指定的運動行程
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
M_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
M_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

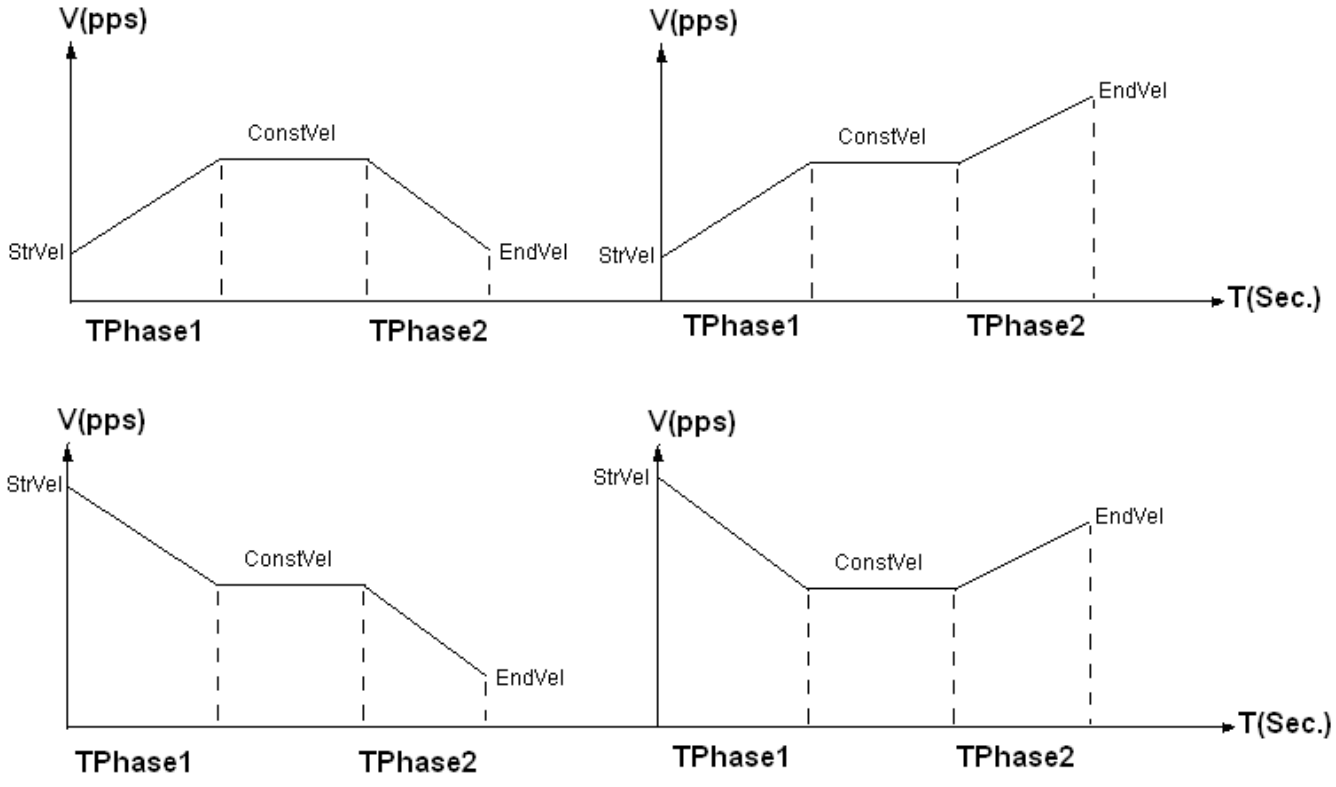
軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0; U16 NodeID =1, SlotID=0;
I32 Dist=100000, StrVel=0, ConstVel=50000, EndVel=20000;
F64 TPhase1=0.2, TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_move (CardNo, NodeID, SlotID, Dist, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);

NOTE



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 18.10 TPhase1 與 TPhase2 說明

18.17　_DMC_01_multi_table_V3_move

■　**格式**

I16 PASCAL _DMC_01_multi_table_V3_move(U16 CardNo, U16 MotionNum, U16 NodeID, U16 SlotID, I32 *Dist,I32 StrVel, I32 *ConstVel, I32 EndVel , F64 *Tphase, F64 Tdec, U16 m_curve, U16 m_r_a);

■　**目的**

單軸使用 Table 的方式連續移動至不同位置。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	群組卡號 GroupNo 為 1~4
MotionNum	U16	編號單位	運動指令數量(最大為 20 筆)
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dist	I32 *	脈波數陣列	對應運動指令數量的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32 *	脈波數/秒 陣列	對應運動指令數量的常態速度
EndVel	I32	脈波數/秒	運動結束速度
Tphase	F64	秒　陣列	對應運動指令數量的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo = 0, MotionNum = 4, NodeID = 1, SlotID = 0;
I32Dist[4] = {5000,6000,4000,7000};
I32 StrVel = 2000, EndVel = 2000;
I32ConstVel[4] = {3000,2000,1000,5000};
F64Tphase[4] = {0.5,0.1,0.1,0.1};
F64 Tdec = 0.5;
U16 m_curve = 1, m_r_a = 0;

I16 status = _DMC_01_multi_table_V3_move(CardNo,MotionNum,NodeID,SlotID,Dist, StrVel, ConstVel, EndVel, Tphase, Tdec, m_curve, m_r_a);

NOTE

假設使用 4 筆運動指令，須設定 4 筆運動目標路徑，Tphase 為每筆運動指令的加速時間，且只有最後一筆指令有 Tdec 減速時間。
使用限制：指令是會近入 BUFFER，所以要連續使用此指令時，需確認 BUFFER 數量。

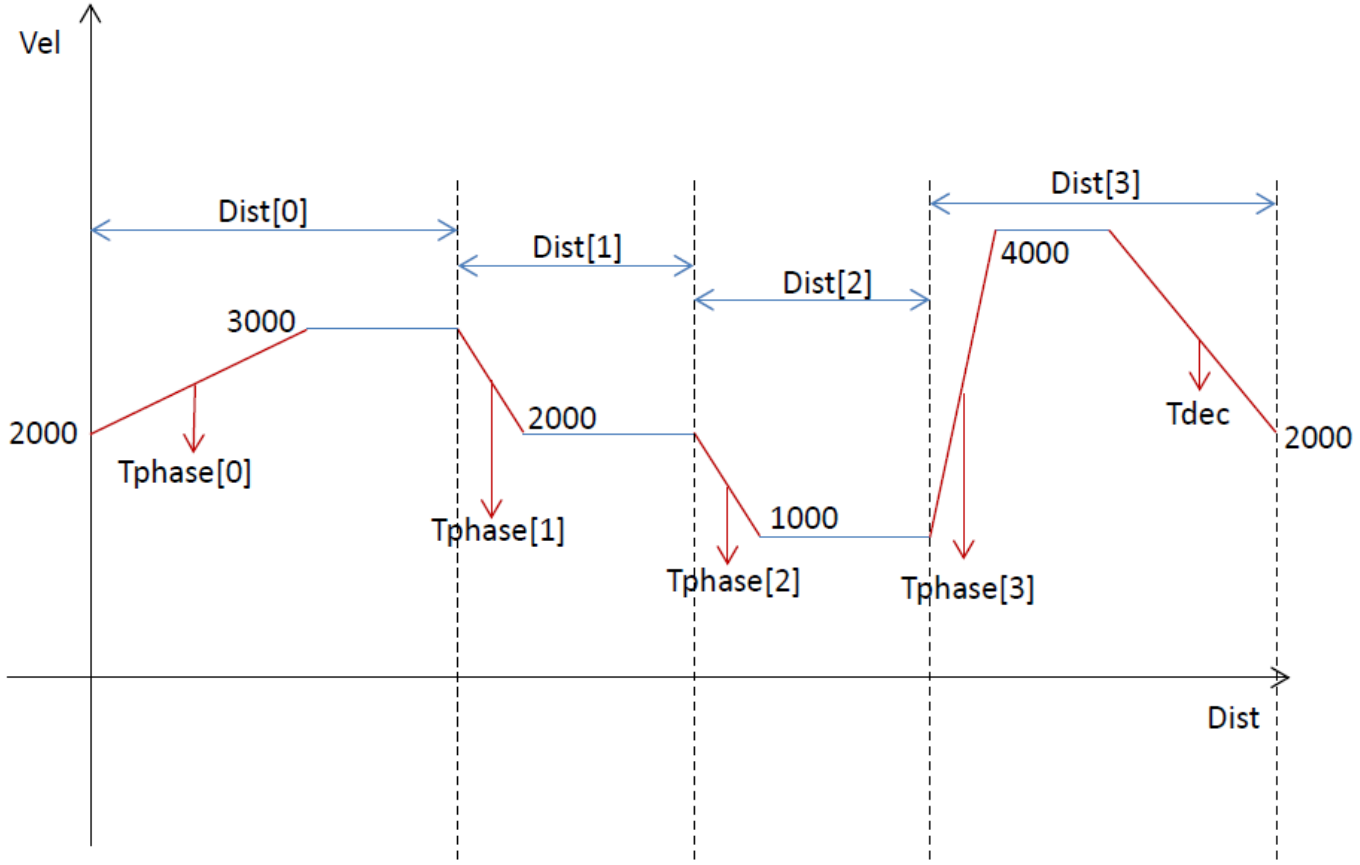


圖 18.10 TPhase1 與 TPhase2 說明

第十九章　二軸線性補間運動控制 API

表 19.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_move_xy	參照 T-curve 速度剖面相對座標進行兩軸線性補間運動
A/B/F	_DMC_01_start_sr_move_xy	參照 S-curve 速度剖面相對座標進行兩軸線性補間運動
A/B/F	_DMC_01_start_ta_move_xy	參照 T-curve 速度剖面絕對座標進行兩軸線性補間運動
A/B/F	_DMC_01_start_sa_move_xy	參照 S-curve 速度剖面絕對座標進行兩軸線性補間運動
A/B/F	_DMC_01_start_v3_move_xy	增加 EndVel 的兩軸線性補間運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

19.1 _DMC_01_start_tr_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸線性補間運動。

※設置 StrVel 的值時，該值務必設定小於 MaxVel 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2] ={0, 0};
I32 DisX=30000;
I32 DisY=40000;
I32 StrVel=0;
I32 MaxVel=3000;
F64 Tacc=0.1;
F64 Tdec=0.1;
```

I16 status = _DMC_01_start_tr_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);

// 當 MaxVel 設置為 3000RPM，則 X 軸速度為 1800RPM；Y 軸速度為 2400RPM

NOTE

如圖 19.1, 兩軸直線補間意指從 P0 到 P1 運動位移 XY 的位置。該兩軸會同時起始與終止，並且運動位移的路徑會在同一直線上。

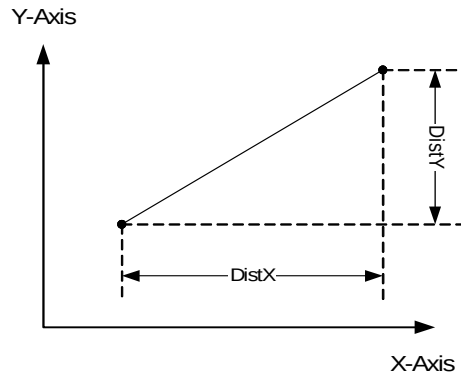


圖 19.1

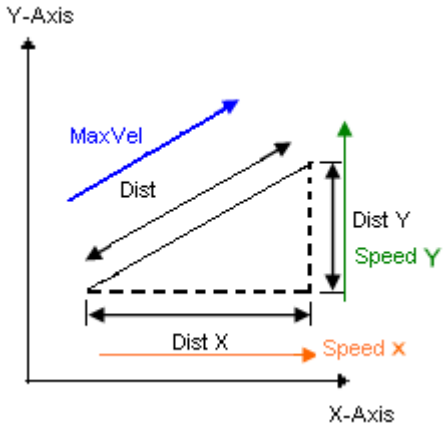


圖 19.2

沿著 X 軸與 Y 軸(DistX :DistY)分別的速率比與運動速度的關係為：



承上，從上圖 19.2 表示，您將可以得知最大速度(MaxVel)與 X 軸的速度(speed X)和 Y 軸的速度(speed Y)的關係如下：

$$\text{Speed X} = \frac{\text{MaxVel} \times \text{Dist X}}{\text{Dist}} \quad , \quad \text{Speed Y} = \frac{\text{MaxVel} \times \text{Dist Y}}{\text{Dist}}$$

19.2　_DMC_01_start_sr_move_xy

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行兩軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2] ={0, 0};
I32 DisX=50000;
I32 DisY 100000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;F64 Tdec=0.1;
I16 status = _DMC_01_start_sr_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);
```

19.3 _DMC_01_start_ta_move_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2] ={0, 0};
I32 DisX=50000;
I32 DisY 100000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;F64 Tdec=0.1;

I16 status = _DMC_01_start_ta_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);
```

19.4　_DMC_01_start_sa_move_xy

■　**格式**

I16 PASCAL _DMC_01_start_sa_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={0,1};
U16 SlotID[2]={0, 0};
I32 DisX=50000;
I32 DisY 100000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;F64 Tdec=0.1;

I16 status = _DMC_01_start_sa_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel, MaxVel, Tacc, Tdec);
```

19.5　_DMC_01_start_v3_move_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_move_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■　**目的**

增加 EndVel 的兩軸線性補間運動。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的行程
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的行程
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={0,1};
U16 SlotID[2]={0, 0};
I32 DisX=50000;
I32 DisY= 100000;
I32 StrVel=0;
I32 ConstVel=50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_move_xy (CardNo, NodeIDArray, SlotID, DistX, DistY, StrVel,
ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

第二十章　二軸圓弧補間運動控制 API

表 20.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_arc_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
A/B/F	_DMC_01_start_sr_arc_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
A/B/F	_DMC_01_start_ta_arc_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
A/B/F	_DMC_01_start_sa_arc_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)
A/B/F	_DMC_01_start_tr_arc2_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
A/B/F	_DMC_01_start_sr_arc2_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
A/B/F	_DMC_01_start_ta_arc2_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
A/B/F	_DMC_01_start_sa_arc2_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)
A/B/F	_DMC_01_start_tr_arc3_xy	參照 T-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
A/B/F	_DMC_01_start_sr_arc3_xy	參照 S-curve 速度剖面相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
A/B/F	_DMC_01_start_ta_arc3_xy	參照 T-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
A/B/F	_DMC_01_start_sa_arc3_xy	參照 S-curve 速度剖面絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)
A/B/F	_DMC_01_start_spiral_xy	兩軸螺旋運動(已知條件：X、Y 軸中心座標)
A/B/F	_DMC_01_start_spiral2_xy	兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)
A/B/F	_DMC_01_start_v3_arc_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、角度)
A/B/F	_DMC_01_start_v3_arc2_xy	增加 EndVel 的兩軸圓弧補間運動(已知條件：終點座標、角度)
A/B/F	_DMC_01_start_v3_arc3_xy	增加EndVel的兩軸圓弧補間運動(已知條件：中心座標、終點座標)
A/B/F	_DMC_01_start_v3_spiral_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標)
A/B/F	_DMC_01_start_v3_spiral2_xy	增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)
A/B/F	_DMC_01_start_v3_ellipse_xy	增加 EndVel 的兩軸橢圓運動(已知條件：X、Y 軸中心座標)

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

20.1 _DMC_01_start_tr_arc_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000;
I32 Center_Y =50000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

NOTE

_DMC_01_start_xx_arc_xy 可操作 360 度(倍數)的圓。

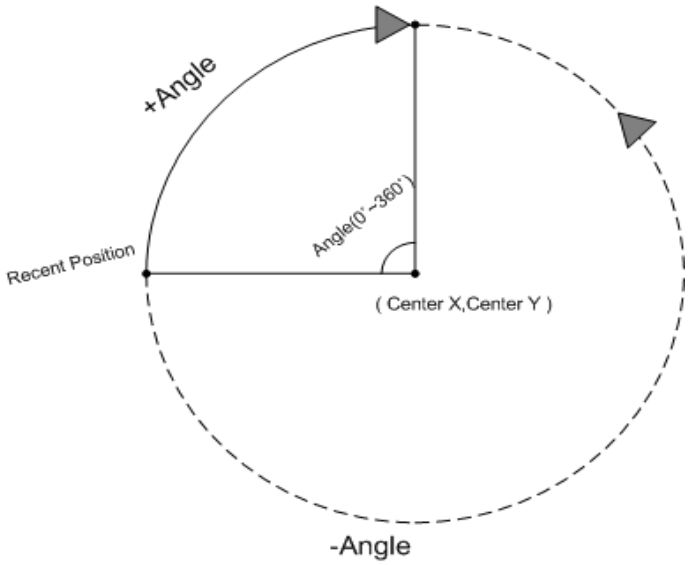


圖 20.1

20.2　_DMC_01_start_sr_arc_xy

■　**格式**

I16 PASCAL _DMC_01_start_sr_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000, Center_Y =50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sr_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

20.3　_DMC_01_start_ta_arc_xy

■　**格式**

I16 PASCAL _DMC_01_start_ta_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000, Center_Y =50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_ta_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

20.4　_DMC_01_start_sa_arc_xy

■　**格式**

I16 PASCAL _DMC_01_start_sa_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、角度)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =50000, Center_Y =50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sa_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

20.5 _DMC_01_start_tr_arc2_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之相對性終點位置 X
End_Y	I32	脈波數	指定軸對應之相對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000;
I32 End_Y =-50000;
I32 StrVel=0;
I32 MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y, Angle,
StrVel, MaxVel, Tacc, Tdec);
```

NOTE

_DMC_01_start_xx_arc2_xy 無法操作 360 度(倍數)的圓。

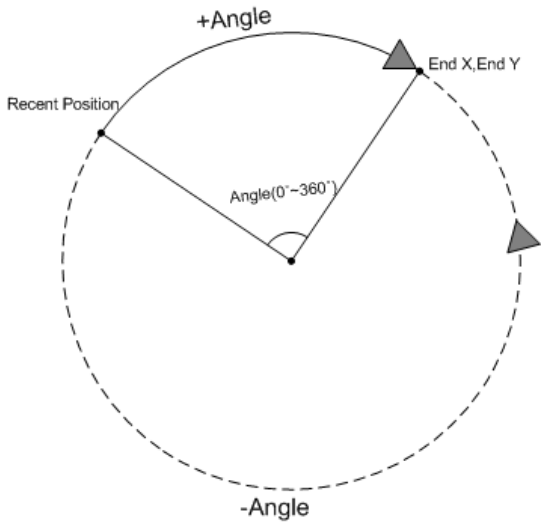


圖 20.2

20.6　_DMC_01_start_sr_arc2_xy

■　**格式**

I16 PASCAL _DMC_01_start_sr_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之相對性終點位置 X
End_Y	I32	脈波數	指定軸對應之相對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000, End_Y =-50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sr_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
```

20.7　_DMC_01_start_ta_arc2_xy

■　**格式**

I16 PASCAL _DMC_01_start_ta_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之絕對性終點位置 X
End_Y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000, End_Y =-50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_ta_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y, Angle, StrVel, MaxVel, Tacc, Tdec);
```

20.8　_DMC_01_start_sa_arc2_xy

■　**格式**

I16 PASCAL _DMC_01_start_sa_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：終點座標、角度)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之絕對性終點位置 X
End_Y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000, End_Y =-50000;
I32 StrVel=0, MaxVel=50000;
F64 Angle=180;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sa_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y,
Angle, StrVel, MaxVel, Tacc, Tdec);
```

20.9　_DMC_01_start_tr_arc3_xy

■　**格式**

I16 PASCAL _DMC_01_start_tr_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性中心位置 Y
End_x	I32	脈波數	指定軸對應之相對性終點位置 X
End_y	I32	脈波數	指定軸對應之相對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000;
I32 Center_Y =25000;
I32 StrVel=0;
I32 MaxVel=50000;
I32 End_x=50000;
I32 End_y=50000;
I16 Dir=1; //此值為 1，表示此為逆時針方向圓弧補間
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

NOTE

_DMC_01_start_xx_arc3_xy 只能操作 360 度以下的圓。

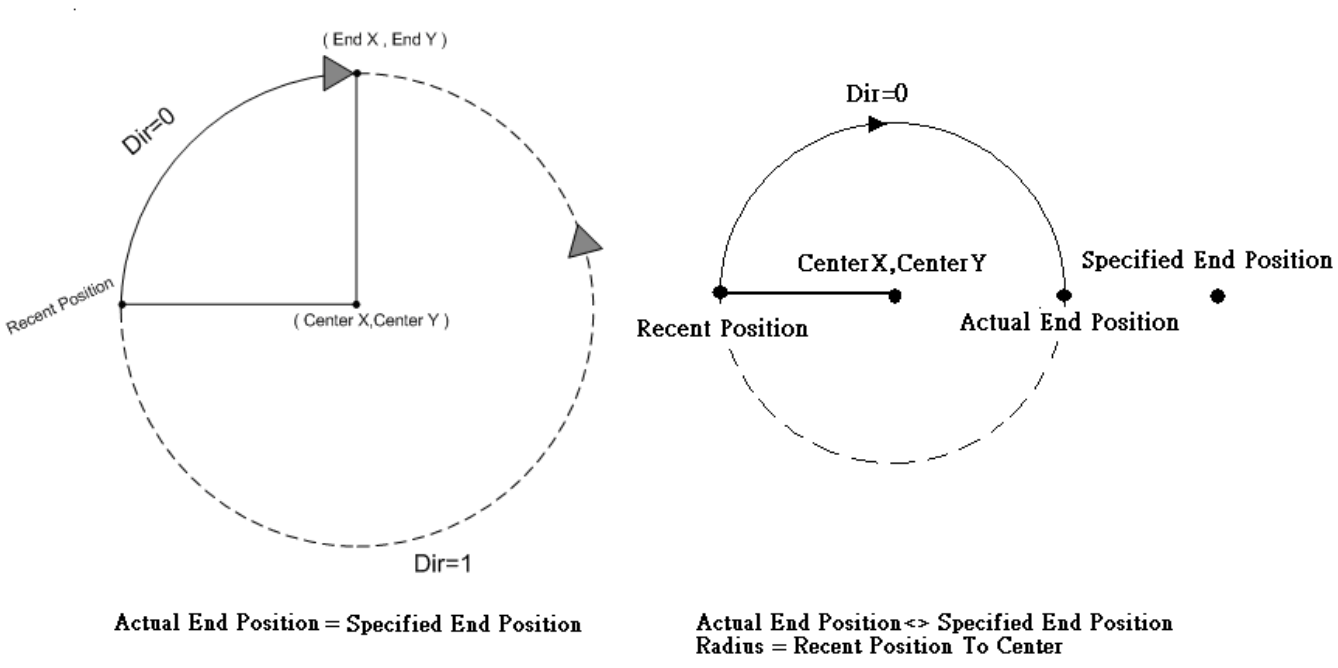


圖 20.3

20.10　_DMC_01_start_sr_arc3_xy

■　**格式**

I16 PASCAL _DMC_01_start_sr_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性中心位置 Y
End_x	I32	脈波數	指定軸對應之相對性終點位置 X
End_y	I32	脈波數	指定軸對應之相對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 StrVel=0, MaxVel=50000;
I32 End_x=50000, End_y=50000;
I16 Dir=1; //此值為 1，表示此為逆時針方向圓弧補間
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sr_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

20.11　_DMC_01_start_ta_arc3_xy

■　**格式**

I16 PASCAL _DMC_01_start_ta_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 T-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性中心位置 Y
End_x	I32	脈波數	指定軸對應之絕對性終點位置 X
End_y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 StrVel=0, MaxVel=50000;
I32 End_x=50000, End_y=50000;
I16 Dir=1;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_ta_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

20.12　_DMC_01_start_sa_arc3_xy

■　**格式**

I16 PASCAL _DMC_01_start_sa_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的絕對座標進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性中心位置 Y
End_x	I32	脈波數	指定軸對應之絕對性終點位置 X
End_y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 StrVel=0, MaxVel=50000;
I32 End_x=50000, End_y=50000;
I16 Dir=1;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sa_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, MaxVel, Tacc, Tdec);
```

20.13　_DMC_01_start_spiral_xy

■　**格式**

I16 PASCAL _DMC_01_start_spiral_xy (U16 CardNo, U16* NodeID, U16* SlotID,
I32 Center_X, I32 Center_Y, I32 spiral_interval, I32 spiral_angle, I32 StrVel, I32 MaxVel,
F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

進行兩軸螺旋運動(已知條件：X、Y 軸中心座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應中心位置 X
Center_Y	I32	脈波數	指定軸對應中心位置 Y
spiral_interval	I32	脈波數	螺旋之間相對性位置距離
spiral_angle	I32	數值單位	螺旋運動之總角度(360 度一圈)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000,
I32 Spiral_interval= 20000,Spiral_angel=1800;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1;
U16 m_r_a=0;

I16 status = _DMC_01_start_spiral_xy (CardNo, NodeID, SlotID, Center_X, Center_Y,
Spiral_interval, Spiral_angel, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

NOTE

每 6 軸(1~6 及 7~12)只能同時下達一個 Spiral 指令，其他軸可下達如:直線插補運動 (1、2、3 軸都可以)。

20.14　_DMC_01_start_spiral2_xy

■　**格式**

I16 PASCAL _DMC_01_start_spiral2_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 end_x, I32 end_y, U16 dir, U16 circlenum, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec, U16 m_curve, U16 m_r_a)

■　**目的**

進行兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
center_X	I32	脈波數	指定軸對應中心位置 X
center_Y	I32	脈波數	指定軸對應中心位置 Y
end_X	I32	脈波數	指定軸對應終點位置 X
end_Y	I32	脈波數	指定軸對應終點位置 Y
dir	U16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
circlenum	U16	數值單位	螺旋運動之圈數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 center_x =25000, center_y=25000;
I32 end_x =25000, end_y =25000;
U16 dir= 1,circelnum=5;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1;
U16 m_r_a=0;

I16 status = _DMC_01_start_spiral2_xy (CardNo, NodeID, SlotID, center_x, center_y,
end_x, end_y, dir, circelnuml, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

20.15　_DMC_01_start_v3_arc_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_arc_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y, F64 Angle, I32 StrVel, I32ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■　**目的**

增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、角度)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**

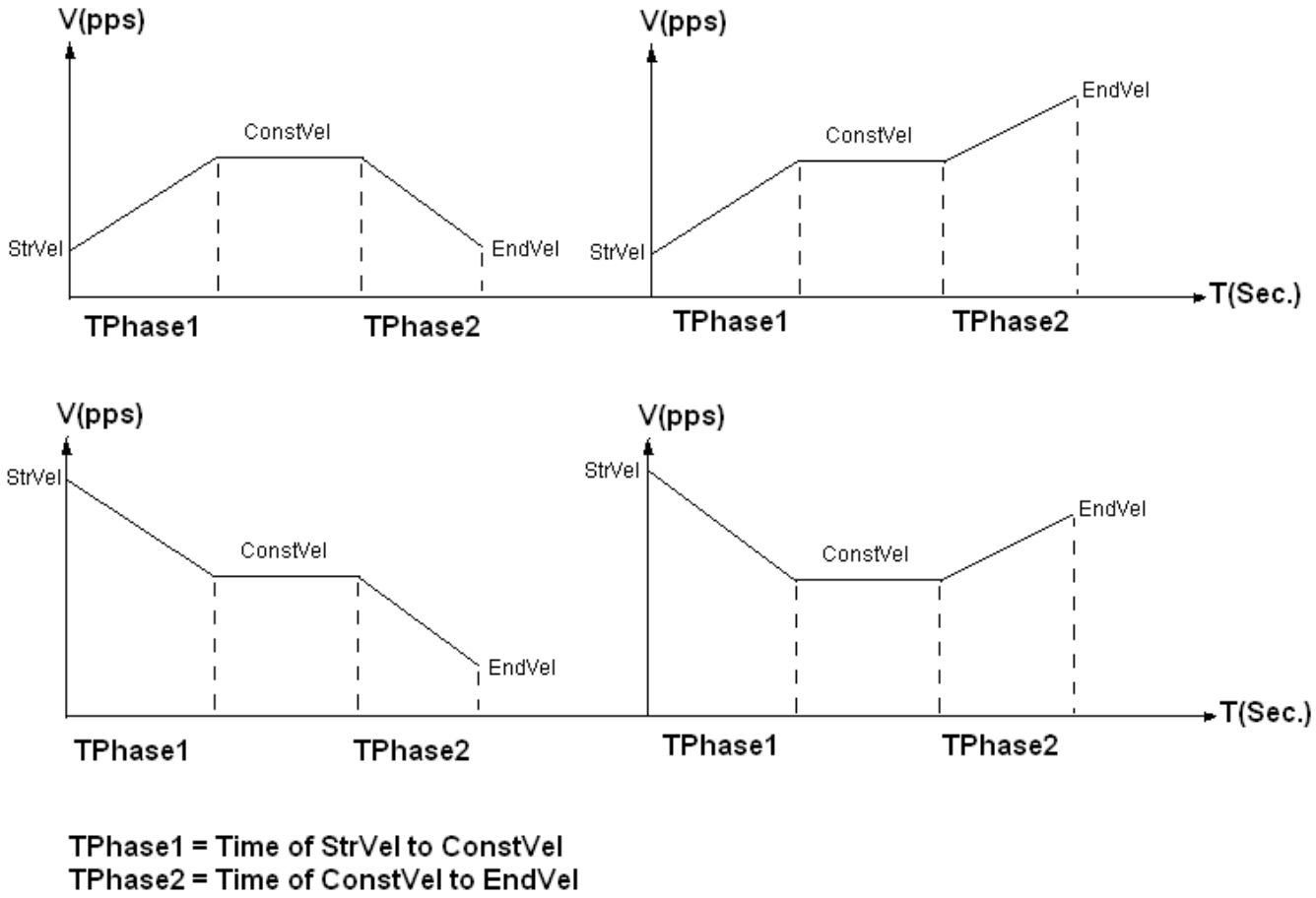


圖 20.4 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0, NodeIDArray[2]={1,2}, SlotID[2]={0, 0};
I32 Center_X =50000;
I32 Center_Y =50000;
F64 Angle=180;
I32 StrVel=0;
I32 ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_arc_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

20.16　_DMC_01_start_v3_arc2_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_arc2_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 End_X, I32 End_Y, F64 Angle, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的兩軸圓弧補間運動(已知條件：終點座標、角度)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
End_X	I32	脈波數	指定軸對應之絕對性終點位置 X
End_Y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Angle	F64	度(°)	一圓弧的角度設置。
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 End_X =-50000;
I32 End_Y =-50000;
I32 StrVel=0;
I32 ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_arc2_xy (CardNo, NodeIDArray, SlotID, End_X, End_Y,
Angle, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

20.17　_DMC_01_start_v3_arc3_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_arc3_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y,I32 End_X,I32 End_Y, I16 Dir, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的兩軸圓弧補間運動(已知條件：中心座標、終點座標)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取各種用來進行圓弧補間的站號 ID 數組 NodeIDArray[0] 存放第一組站號 NodeIDArray[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性中心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性中心位置 Y
End_x	I32	脈波數	指定軸對應之絕對性終點位置 X
End_y	I32	脈波數	指定軸對應之絕對性終點位置 Y
Dir	I16	選項單位	指定的方向(值為 0:順時針方向 1:逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeIDArray[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000;
I32 End_x=50000, End_y=50000;
I16 Dir=1;
I32 StrVel=0, ConstVel=50000 , EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_arc3_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
End_x, End_y,Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

20.18　_DMC_01_start_v3_spiral_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_spiral_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 Center_X, I32 Center_Y, I32 spiral_interval, I32 spiral_angle, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應中心位置 X
Center_Y	I32	脈波數	指定軸對應中心位置 Y
spiral_interval	I32	脈波數	螺旋之間相對性位置距離
spiral_angle	I32	數值單位	螺旋運動之總角度(360 度一圈)
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y =25000,
I32 Spiral_interval= 20000, Spiral_angel=1800;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_spiral_xy (CardNo, NodeID, SlotID, Center_X, Center_Y,
Spiral_interval, Spiral_angel, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

20.19　_DMC_01_start_v3_spiral2_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_spiral2_xy (U16 CardNo, U16* NodeID, U16* SlotID, I32 center_x, I32 center_y, I32 end_x, I32 end_y, U16 dir, U16 circlenum, I32 StrVel, 32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的兩軸螺旋運動(已知條件：X、Y 軸中心座標，X、Y 軸終點座標)。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeID	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
center_X	I32	脈波數	指定軸對應中心位置 X
center_Y	I32	脈波數	指定軸對應中心位置 Y
end_X	I32	脈波數	指定軸對應終點位置 X
end_Y	I32	脈波數	指定軸對應終點位置 Y
dir	U16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
circlenum	U16	數值單位	螺旋運動之圈數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0
U16 NodeID[2]={1,2};
U16 SlotID[2]={0, 0};
I32 center_x =25000, center_y=25000;
I32 end_x =25000, end_y =25000;
U16 dir= 1, circelnum=5;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_spiral2_xy (CardNo, NodeID, SlotID, center_x, center_y,
end_x, end_y, dir, circelnuml, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

20.20　_DMC_01_start_v3_ellipse_xy

■　**格式**

I16 PASCAL _DMC_01_start_v3_ellipse_xy(U16 CardNo, U16* NodeIDArray,U16* SlotID I32 Center_X, I32 Center_Y, I32 majorRadius, I32 minorRadius, F64 Angle, I16 mode, I32 StrVel, I32 MaxVel, I32 EndVel, F64 Tacc, F64 Tdec,U16 m_curve, U16 m_r_a)

■　**目的**

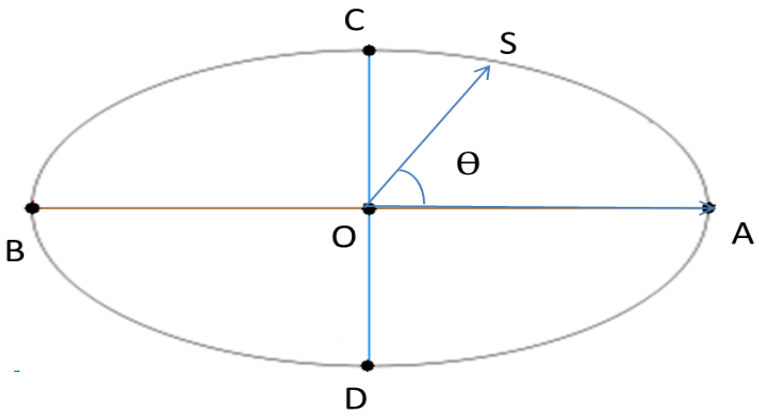
增加 EndVel 的兩軸橢圓運動(已知條件：X、Y 軸中心座標)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	用來作為 X 軸與 Y 軸位移之 Node 的介面卡號
NodeIDArray	U16*	編號單位	存取用來進行螺旋運動的站號 ID 數組 NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應中心位置 X
Center_Y	I32	脈波數	指定軸對應中心位置 Y
majorRadius	I32	脈波數	橢圓半長軸長
minorRadius	I32	脈波數	橢圓半短軸長
Angle	F64	數值單位	度(°) 一圓弧的角度設置。
mode	U16	選項單位	0: 接近正交座標系 X 軸的橢圓(default), 1: 接近正交座標系 Y 軸的橢圓
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
EndVel	I32	脈波數/秒	運動結束速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default，8 軸以下使用) 1：絕對位移，8 軸以下使用 2：絕對位移，12 軸以下(含)使用

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				



A: 橢圓起點
O: 橢圓圓心
OB: 半長軸長(majorRadius)
OD: 半短軸長(majorRadius)
θ: 角度

圖 20.5 橢圓參數說明

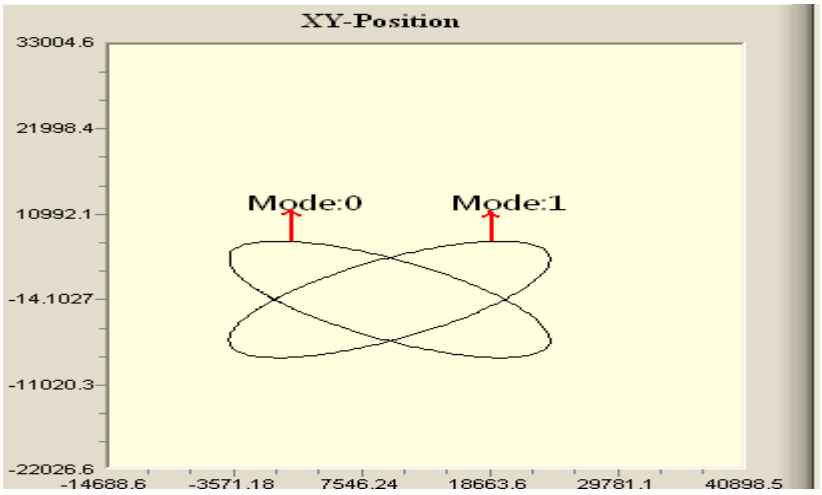


圖 20.6 橢圓 Mode 參數說明

■ **範例**

U16 CardNo=1
U16 NodeIDArray [2]={1,2};
U16 SlotID[2]={0, 0};
I32 Center_X =25000, Center_Y=25000;
I32 majorRadius =25000, minorRadius =15000;
I16 mode= 0;
I32 StrVel=0, ConstVel =50000;I32 EndVel=20000;
F64 Angle=180, Tacc=0.1,Tdec=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_ellipse_xy(CardNo, NodeIDArray, SlotID Center_X, Center_Y, majorRadius, minorRadius, Angle, mode, StrVel, MaxVel, EndVel, Tacc, Tdec, m_curve, m_r_a);

（此頁有意留為空白）

第二十一章　三軸線性補間運動控制 API

表 21.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_move_xyz	參照 T-curve 速度剖面相對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_sr_move_xyz	參照 S-curve 速度剖面相對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_ta_move_xyz	參照 T-curve 速度剖面絕對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_sa_move_xyz	參照 S-curve 速度剖面絕對座標進行三軸線性補間運動
A/B/F	_DMC_01_start_v3_move_xyz	增加 EndVel 的三軸線性補間運動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

21.1 _DMC_01_start_tr_move_xyz

■ **格式**

I16 PASCAL _DMC_01_start_tr_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行三軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_tr_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

21.2　_DMC_01_start_sr_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_sr_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■　**目的**

參照 S-curve 速度剖面下的相對座標進行三軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的相對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的相對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的相對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_sr_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

21.3 _DMC_01_start_ta_move_xyz

■ **格式**

I16 PASCAL _DMC_01_start_ta_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行三軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
I16 status = _DMC_01_start_ta_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

21.4 _DMC_01_start_sa_move_xyz

■ **格式**

I16 PASCAL _DMC_01_start_sa_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的絕對座標進行三軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1,Tdec=0.1;
I16 status = _DMC_01_start_sa_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, MaxVel, Tacc, Tdec);
```

21.5　_DMC_01_start_v3_move_xyz

■　**格式**

I16 PASCAL _DMC_01_start_v3_move_xyz(U16 CardNo, U16* NodeID,U16* SlotID, I32 DisX, I32 DisY, I32 DisZ, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■　**目的**

增加 EndVel 的三軸線性補間運動。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

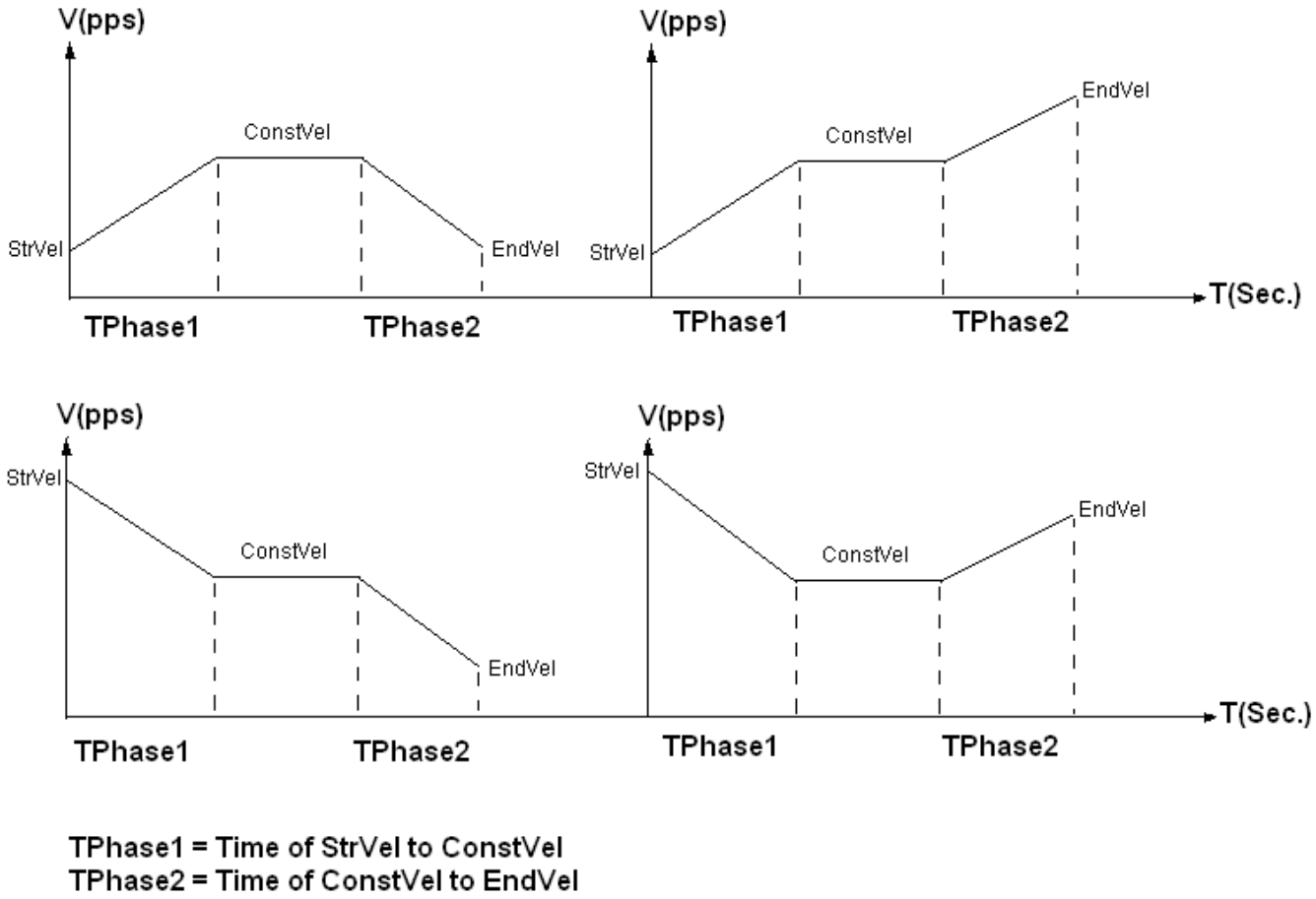
■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行線性補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
DisX	I32	脈波數	對應 X 軸之站號 ID 運動的絕對性路徑參數
DisY	I32	脈波數	對應 Y 軸之站號 ID 運動的絕對性路徑參數
DisZ	I32	脈波數	對應 Z 軸之站號 ID 運動的絕對性路徑參數
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 21.4 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 DisX =25000, DisY =50000 , DisZ =75000 ;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_move_xyz (CardNo, NodeIDArray, SlotID, DisX, DisY, DisZ,
StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

（此頁有意留為空白）

第二十二章　三軸螺旋補間運動控制 API

表 22.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_start_tr_heli_xy	參照 T-curve 速度剖面相對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_sr_heli_xy	參照 S-curve 速度剖面相對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_ta_heli_xy	參照 T-curve 速度剖面絕對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_sa_heli_xy	參照 S-curve 速度剖面絕對座標進行三軸螺旋補間運動
A/B/F	_DMC_01_start_v3_heli_xy	增加 EndVel 的三軸螺旋補間運動
A/B/F	_DMC_01_start_v3_sphere_xyz	三點成圓

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

22.1 _DMC_01_start_tr_heli_xy

■ **格式**

I16 PASCAL _DMC_01_start_tr_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的相對座標進行三軸螺旋補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之相對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之相對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

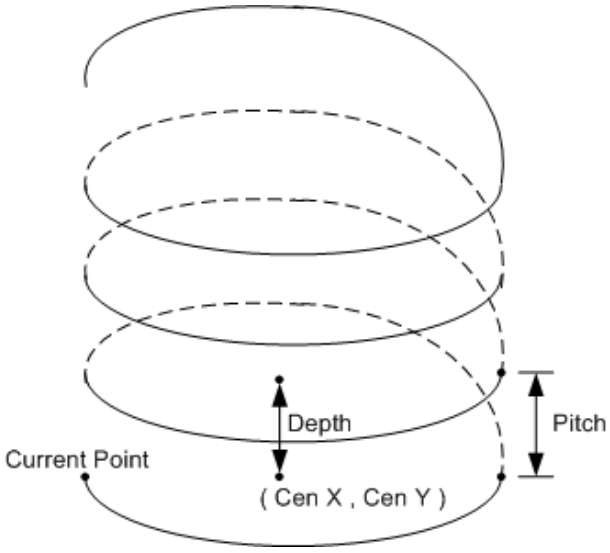


圖 22.1

■　**範例**

U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000;
I32 Center_Y =50000;
I32 Depth =10000;
I32 Pitch = 20000;
I16 Dir=1; //值為 1，表示此為逆時針方向的螺旋圓弧運動
I32 StrVel=0;
I32 MaxVel=50000;
F64 Tacc=0.1;
F64 Tdec=0.1;

I16 status = _DMC_01_start_tr_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y, Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);

NOTE

這裡所下的最大速度是 X、Y 軸向量速度的依據，但 Z 軸的速度是依照 Depth 及 Pitch 的值所推算出來的。

Z 的速度推算公式如下: (Pitch * Maxvel) /(Radius * 2 * π**)。**

22.2 _DMC_01_start_sr_heli_xy

■ **格式**

I16 PASCAL _DMC_01_start_sr_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的相對座標進行三軸螺旋補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之相對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之相對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之相對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之相對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sr_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

22.3 _DMC_01_start_ta_heli_xy

■ **格式**

I16 PASCAL _DMC_01_start_ta_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 T-curve 速度剖面下的絕對座標進行三軸螺旋補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X,	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_ta_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

22.4 _DMC_01_start_sa_heli_xy

■ **格式**

I16 PASCAL _DMC_01_start_sa_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID, I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec)

■ **目的**

參照 S-curve 速度剖面下的絕對座標進行三軸螺旋補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_start_sa_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, MaxVel, Tacc, Tdec);
```

22.5 _DMC_01_start_v3_heli_xy

■ **格式**

I16 PASCAL _DMC_01_start_v3_heli_xy(U16 CardNo, U16* NodeID,U16* SlotID,
I32 Center_X, I32 Center_Y, I32 Depth, I32 Pitch, I16 Dir, I32 StrVel, 32 ConstVel, I32 EndVel,
F64 TPhase1, F64 TPhase2, U16 m_curve, U16 m_r_a)

■ **目的**

增加 EndVel 的三軸螺旋補間運動。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Center_X	I32	脈波數	指定軸對應之絕對性圓心位置 X
Center_Y	I32	脈波數	指定軸對應之絕對性圓心位置 Y
Depth	I32	脈波數	指定軸對應之絕對性位置深度(整個 Z 方向高度)
Pitch	I32	脈波數	指定之絕對性的兩螺旋之間的高度
Dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**

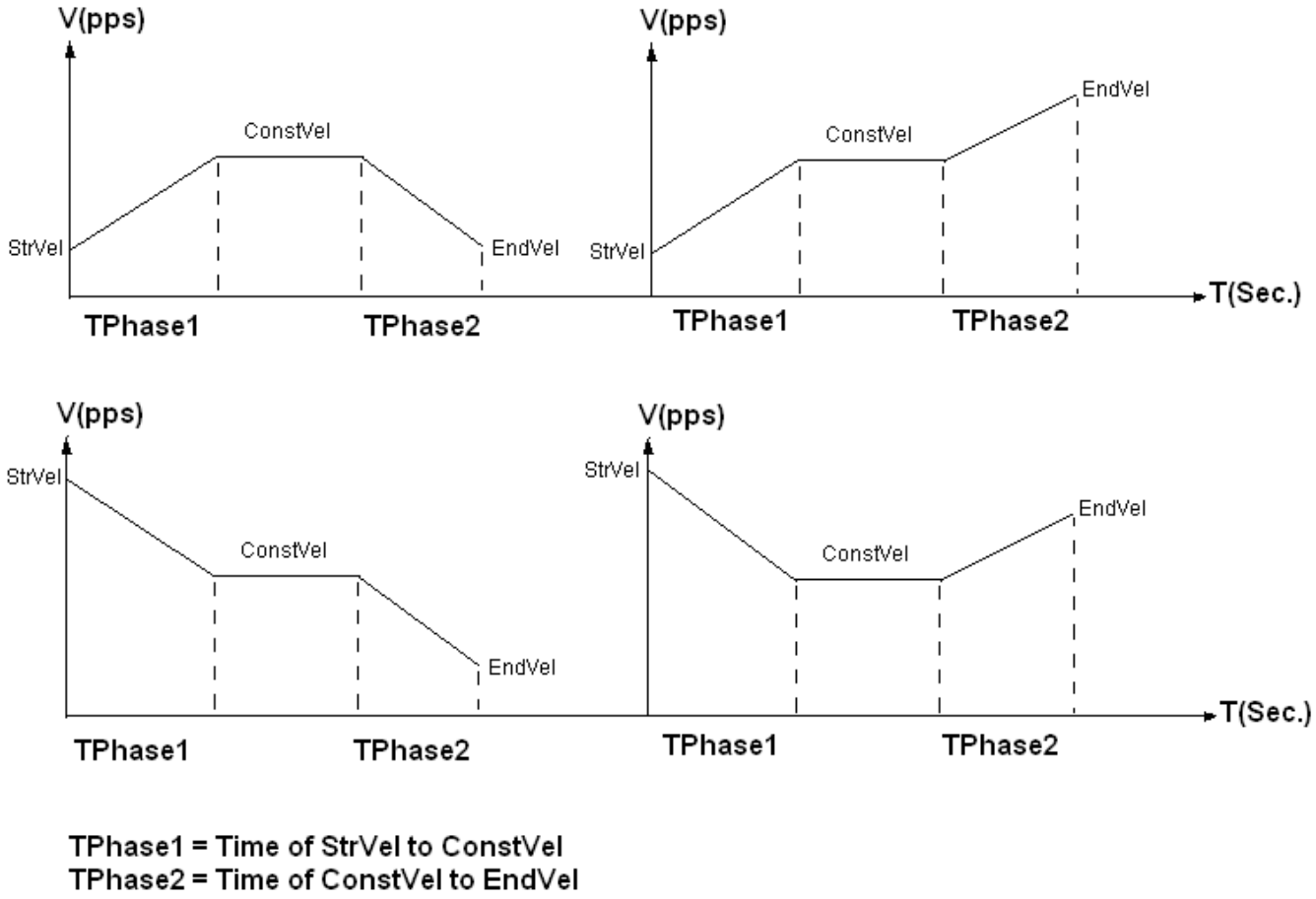


圖 22.2 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 Center_X =25000, Center_Y =50000, Depth =10000, Pitch = 20000;
I16 Dir=1;
I32 StrVel=0, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_heli_xy (CardNo, NodeIDArray, SlotID, Center_X, Center_Y,
Depth, Pitch, Dir, StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

22.6 _DMC_01_start_v3_sphere_xyz

■ **格式**

I16 PASCAL _DMC_01_start_v3_sphere_xyz (U16 CardNo, U16* NodeID, U16* SlotID, I32 pos1_x, I32 pos1_y, I32 pos1_z, I32 pos2_x, I32 pos2_y, I32 pos2_z, I32 StrVel, I32 ConstVel, I32 EndVel, F64 Tacc, F64 Tdec, U16 m_curve, U16 m_r_a)

■ **目的**

三點成圓(起始點與第一指定點與第二指定點，三點不能在同一直線上)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeIDArray	U16*	編號單位	存取各種用來進行螺旋補間的站號 ID 數組 NodeIDArray[0] = Node_1 NodeIDArray[1] = Node_2 NodeIDArray[2] = Node_3
SlotID	U16*	編號單位	插槽 ID
Pos1_x	I32	脈波數	第一指定點 X 軸位置
Pos1_y	I32	脈波數	第一指定點 Y 軸位置
Pos1_z	I32	脈波數	第一指定點 Z 軸位置
Pos2_x	I32	脈波數	第二指定點 X 軸位置
Pos2_y	I32	脈波數	第二指定點 Y 軸位置
Pos2_z	I32	脈波數	第二指定點 Z 軸位置
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0;
U16 NodeIDArray[3]={1,2,3};
U16 SlotID[3]={0, 0, 0};
I32 pos1_x =25000, pos1_y =50000, pos1_z =20000;
I32 pos2_x =95000, pos2_y =110000, pos2_z =60000;
I32 StrVel=0, ConstVel=50000, EndVel=20000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_sphere_xyz (CardNo, NodeIDArray, SlotID, pos1_x, pos1_y,
pos1_z, pos2_x, pos2_y, pos2_z, StrVel, ConstVel, EndVel,Tacc, Tdec, m_curve, m_r_a);
```

（此頁有意留為空白）

第二十三章　速度運動控制 API

表 23.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_tv_move	參照 T-curve 速度剖面下的速度運動控制
A/B/F	_DMC_01_sv_move	參照 S-curve 速度剖面下的速度運動控制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

23.1　_DMC_01_tv_move

■　**格式**

I16 PASCAL _DMC_01_tv_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 StrVel, I32 MaxVel, F64 Tacc,I16 Dir)

■　**目的**

參照 T-curve 速度剖面下的速度運動控制。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數(最小值 5pps)
Tacc	F64	秒	指定的加速時間
Dir	I16	選項單位	0：往正方向 1：往負方向

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **敘述**

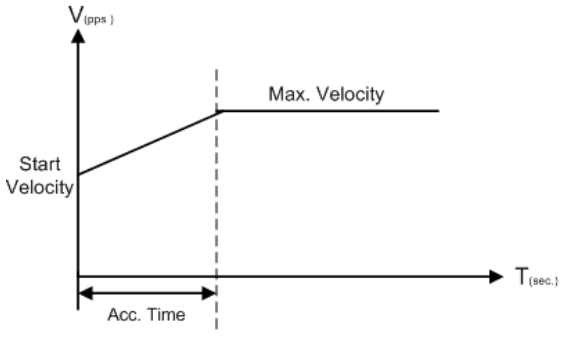


圖. 23.1　trapezoidal 運動模式速度常量表示

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
I32 StrVel=100, MaxVel=5000;
F64 Tacc=0.1;
I16 Dir =1;

I16 status = _DMC_01_tv_move(CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
```

23.2　_DMC_01_sv_move

■　**格式**

I16 PASCAL _DMC_01_sv_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 StrVel, I32 MaxVel, F64 Tacc,I16 Dir)

■　**目的**

參照 S-curve 速度剖面下的速度運動控制。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數(最小值 5pps)
Tacc	F64	秒	指定的加速時間
Dir	I16	選項單位	0：往正方向 1：往負方向

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **敘述**

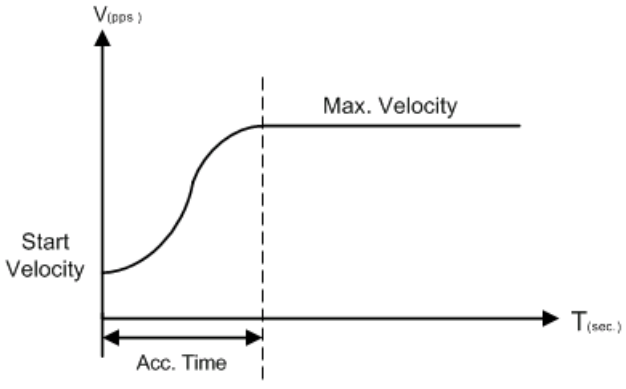


圖 23.2 S-Curve 運動模式速度常量表示

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
I32 StrVel=100, MaxVel=5000;
F64 Tacc=0.1;
I16 Dir =1;

I16 status = _DMC_01_sv_move(CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc, Dir);
```

（此頁有意留為空白）

第二十四章　同步運動控制 API

表 24.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_sync_move	設置開始運行同步運動
A/B/F	_DMC_01_sync_move_config	設置同步運動功能是否啟動

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

24.1　_DMC_01_sync_move

■　**格式**

I16 PASCAL _DMC_01_sync_move(I16 CardNo)

■　**目的**

設置開始運行同步運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;

I16 status = DMC_01_sync_move(CardNo);
```

24.2　_DMC_01_sync_move_config

■　**格式**

I16 PASCAL _DMC_01_sync_move_config(I16 CardNo, U16 NodeID,U16 SlotID,I16 enable)

■　**目的**

設置同步運動功能是否啟動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	I16	選項單位	0：關閉同動 1：啟動同動

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, NodeID=1, SlotID=0;
I16 enable=1;

I16 status = _DMC_01_sync_move_config(CardNo,NodeID, SlotID,enable);

（此頁有意留為空白）

第二十五章　遠端模組控制 API

表 25.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_get_rm_input_value	取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值
A/B/F	_DMC_01_set_rm_input_filter	設定遠端擴充模組輸入埠的軟體濾波等級
A/B/F	_DMC_01_set_rm_input_filter_enable	設定遠端擴充模組輸入埠 bit 0 到 bit 15 軟體遮罩開啟與否
A/B/F	_DMC_01_set_rm_output_value	設定遠端擴充模組輸出埠 bit 0 到 bit 15 的值
A/B/F	_DMC_01_set_rm_output_value_error_handle	設定遠端擴充模組的發生錯誤時獲取的輸出值
A/B/F	_DMC_01_get_rm_output_value	取得遠端擴充模組的輸出值
A/B/F	_DMC_01_get_rm_output_value_error_handle	取得遠端擴充模組的輸出值，在發生錯誤時，系統是否將該值保留或重置
A/B/F	_DMC_01_set_rm_output_active	設定遠端擴充模組輸出是否開啟

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

25.1 _DMC_01_get_rm_input_value

■ **格式**

I16 PASCAL _DMC_01_get_rm_input_value(U16 CardNo, U16 NodeID,U16 SlotID, U16 port,U16 *value)

■ **目的**

取得遠端擴充模組輸入埠 bit 0 到 bit 15 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
Value	U16*	數值	接收的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value;

/* Remote 數位輸入模組  Port 0 所接收的資料  */
  I16 status = _DMC_01_get_rm_input_value(CardNo, NodeID, SlotID, Port, &Value);
```

25.2　_DMC_01_set_rm_input_filter

■　**格式**

I16 PASCAL _DMC_01_set_rm_input_filter(U16 CardNo, U16 NodeID,U16 SlotID , U16 port,U16 value)

■　**目的**

設定遠端擴充模組輸入埠的軟體濾波等級。(當 value 變數的值為 1 時，軟體濾波時間為 1ms，值為 2 時，軟體濾波時間為 2ms，依此類推)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16	數值	軟體濾波時間(1~255ms)，輸入 0 會視為 1. 值為 1 時，濾波時間為 1ms

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			

■ **敘述**

Mode 參數設定示意

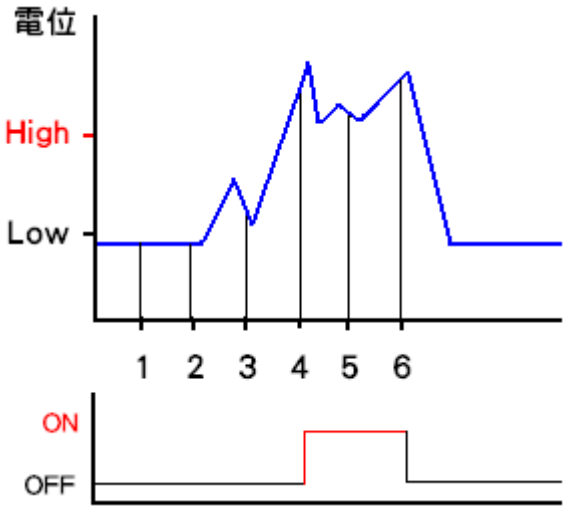


圖 25.1 Filter = 1 這個設定下，電位超過 High 連續 1 個 ms 就 ON 起來了，低於 High 連續 1 個 ms 就 OFF。

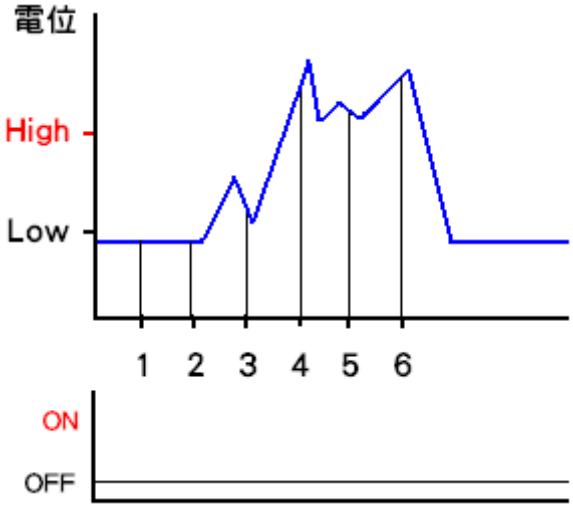


圖 25.2 Filter = 4 這個設定下，電位超過 High 連續 4 個 ms 才會 ON 起來，所以上圖都是 OFF 呈現。

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value=2; //  此例值為 2，表示軟體濾波時間為 2ms

/*  設定 Remote 數位輸入模組 Port 0 的軟體濾波時間為 2ms */
I16 status = _DMC_01_set_rm_input_filter(CardNo, NodeID, SlotID, Port, Value);
```

25.3　_DMC_01_set_rm_input_filter_enable

■　**格式**

I16 PASCAL _DMC_01_set_rm_input_filter_enable(U16 CardNo, U16 NodeID, U16 SlotID,U16 port,U16 enable)

■　**目的**

設定遠端擴充模組輸入埠 bit 0 到 bit 15 軟體遮罩開啟與否。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
enable	U16	數值	Port 0/1/2/3 bit0~bit15 濾波器遮罩(值：0~0xFFFF)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
				●			

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Enable=0xFF;

/*  設定 Remote 數位輸入模組 Port 0 濾波遮罩的 bit0~bit7 設置為 ON */
  I16 status = _DMC_01_set_rm_input_filter_enable(CardNo, NodeID, SlotID, Port, Enable);
```

25.4 _DMC_01_set_rm_output_value

■ **格式**

I16 PASCAL _DMC_01_set_rm_output_value(U16 CardNo, U16 NodeID,U16 SlotID, U16 port,U16 value)

■ **目的**

設定遠端擴充模組輸出埠 bit 0 到 bit 15 的值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16	數值	Remote 數位輸出模組 Port 0/1/2/3 的 bit 0~bit 15 欲設定的值(ON/OFF)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value=0xF;

/*  設定 Remote 數位輸入模組 Port 0 的 bit0~bit3 設置為 ON */
  I16 status = _DMC_01_set_rm_output_value(CardNo, NodeID, SlotID, Port, Value)
```

25.5　_DMC_01_set_rm_output_value_error_handle

■　**格式**

I16 PASCAL _DMC_01_set_rm_output_value_error_handle(U16 CardNo, U16 NodeID, U16 SlotID,U16 port,U16 value)

■　**目的**

保存遠端擴充模組發生錯誤(離線)時的輸出值，直到解除 error_handle 或系統斷電。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16	選項單位	0：錯誤發生時(離線)，輸出值將重置為 0 1：錯誤發生時(離線)，輸出值將被保存直到系統 斷電

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value=1;

I16 status = _DMC_01_set_rm_output_value_error_handle(CardNo, NodeID, SlotID, port, Value);
```

25.6　_DMC_01_get_rm_output_value

■　**格式**

I16 PASCAL _DMC_01_get_rm_output_value (U16 CardNo, U16 NodeID, U16 SlotID, U16 port, U16* value)

■　**目的**

取得遠端擴充模組的輸出值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16*	數值	取得 Remote 數位輸出模組 Port 0/1/2/3 的 bit 0~bit 15 設定的值(ON/OFF)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value;

I16 status = _DMC_01_get_rm_output_value (CardNo, NodeID, SlotID, Port, &Value);
```

25.7　_DMC_01_get_rm_output_value_error_handle

■　**格式**

I16 PASCAL _DMC_01_get_rm_output_value_error_handle (U16 CardNo, U16 NodeID, U16 SlotID,U16 port,U16* value)

■　**目的**

取得設定保存遠端擴充模組發生錯誤(離線)時的輸出值，直到解除 error_handle 或系統斷電。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
port	U16	選項單位	0：Port 0 1：Port 1 2：Port 2 3：Port 3
value	U16*	選項單位	取得 value 的值，當值為 0：錯誤發生時(離線)，輸出值將重置為 0 1：錯誤發生時(離線)，輸出值將被保存直到系統 斷電

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●		

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Port=0;
U16 Value;

I16 status = _DMC_01_get_rm_output_value_error_handle(CardNo, NodeID, SlotID, port, &Value);
```

25.8 _DMC_01_set_rm_output_active

■ **格式**

I16 PASCAL _DMC_01_set_rm_output_active (U16 CardNo, U16 NodeID, U16 SlotID, U16 Enable)

■ **目的**

設定遠端擴充模組輸出是否開啟。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Enable	U16	選項單位	0：關閉輸出 1：啟動輸出，將設定的輸出值發送到目的端

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
					●	●	

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Enable=1;

I16 status = _DMC_01_set_rm_output_active (CardNo, NodeID, SlotID, Enable);
```

第二十六章　電子手輪與 JOG 操作 API

表 26.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_rm_mpg_axes_enable	設定手輪運動控制
A/B/F	_DMC_01_set_rm_mpg_axes_enable2	手輪運動控制（可設定馬達圈數比例分母值）
A/B/F	_DMC_01_set_rm_jog_axes_enable	設定 JOG 運動控制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

26.1 _DMC_01_set_rm_mpg_axes_enable

■ **格式**

I16 PASCAL _DMC_01_set_rm_mpg_axes_enable (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable, U16 pulser_ratio, U32 *ratio, U32 *slop)

■ **目的**

設定手輪運動控制(手輪致能時伺服 P1-68 會調整成最大值 100，並記錄原值，等手輪除能時寫回伺服)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	選項單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值。
slop	U32*	數值單位	設定接受手輪之最大速度斜率 (PPS/秒，Max=1000)。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **範例**

I16 rt = 0;
U16 CardNo = 0, MasterNodeID = 1, MasterSlotID = 0; //RM-64 站號為 1
U16 NodeID[4] = {2, 3, 0, 0}, SlotID[4] = {0}; //使用兩個 ASD-A2F 伺服馬達, 站號分別為 2, 3
U16 enable = 1, pulser_ratio = 4; //設定手輪每格輸出 pulse 比例
U32 ratio[4] = {1, 1, 0, 0}; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值
U32 slop[4] = {100,100,0,0}; //設定接受手輪之最大速度斜率為 100

rt = _DMC_01_set_rm_mpg_axes_enable(CardNo, MasterNodeID, MasterSlotID, NodeID, SlotID, enable, pulser_ratio, ratio, slop);

若同樣條件在步進馬達與 04PI，則結果如下：
手輪轉動一圈(100 格) = 脈波輸出值為 100*10=1000

例二：(ratio 的設定)
條件設定：假設手輪每轉 100 格， 手輪倍數 x10，pulser_ratio = 4，ratio=2， slop=1000
結果: 手輪轉動一圈(100 格) = 馬達轉動二圈(128000*10*2=2560000)
若同樣條件在步進馬達與 04PI，則結果如下：
手輪轉動一圈(100 格) = 脈波輸出值為 100*10=1000，ratio 值的設定與輸出脈波無關，只與手輪設定倍數有關。

例三：(slop 的設定)
條件設定：假設手輪每轉 100 格， 手輪倍數 x100，pulser_ratio = 4，ratio=1， slop=100
正確結果應該: 手輪轉動一圈(100 格) = 馬達轉動一圈(128000 * 100 * 1=12800000)
實際結果為: 手輪轉動一圈(100 格) = 馬達實際轉動(12531200)，由於設定接受手輪的最大速度斜率為 100(PPS/秒)，所以手輪搖太快超出的部分被濾掉了。

例四：在步進馬達與 04PI，則結果如下：
條件設定：假設手輪每轉 100 格， 手輪倍數 X100，pulser_ratio = 4，ratio=1， slop=100
正確結果應該: 手輪轉動一圈(100 格) = 脈波輸出值應為 100 *100=10000
實際結果為: 手輪轉動一圈(100 格) = 脈波輸出為 3560，跟伺服一樣，速度斜率超出的部分被濾掉了。

26.2　_DMC_01_set_rm_mpg_axes_enable2

■　**格式**

I16 PASCAL _DMC_01_set_rm_mpg_axes_enable2 (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable,U16 pulser_ratio, U32 *ratio, U32 *slop, U16 *denominator)

■　**目的**

設定手輪運動控制（可設定馬達圈數比例之分母值）。
(手輪致能時伺服 P1-68 會調整成最大值 100，並記錄原值，等手輪除能時寫回伺服)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	選項單位	0：關閉手輪功能 1：啟動手輪功能
pulser_ratio	U16	選項單位	手輪轉動每格與輸出脈波數比例值。 1：手輪轉動 4 格，脈波輸出值 1 。 4：手輪轉動 1 格，脈波輸出值 1 。
Ratio	U32*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分子)。
Slop	U32*	數值單位	設定接受手輪之最大速度斜率 (PPS/秒，Max=1000)。
Denominator	U16*	數值單位	調整手輪每轉與脈波輸出(馬達圈數)之倍數值(分母)。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **範例**

```
I16 rt = 0;
U16 CardNo = 0, MasterNodeID = 1, MasterSlotID = 0; //RM-64  站號為 1
U16 NodeID[4] = {2, 3, 0, 0}, SlotID[4] = {0}; //使用兩個 ASD-A2F  伺服馬達,  站號分別為 2, 3
U16 enable = 1, pulser_ratio = 4; //設定手輪每格輸出 pulse  比例
U32 ratio[4] = {10, 10, 0, 0}; //設定手輪每轉與脈波輸出(馬達圈數)之倍數值
U32 slop[4] = {1000,1000,0,0}; //設定接受手輪之最大速度斜率為  1000
U16 denominator [4] = {36,36, 36, 36}; //設定手輪每轉輸出比率(分母)

rt = _DMC_01_set_rm_mpg_axes_enable2(CardNo, MasterNodeID, MasterSlotID, NodeID,
SlotID, enable, pulser_ratio, ratio, slop, denominator);
```

NOTE

經實際測試，若使用每轉一圈為 100pulse 的手輪，在高速轉動時，因受限於 RM64MN 的接收頻率，會發生無法接收之現象;建議客戶使用 25pulse/每轉，且有 4 倍頻的手輪，經測試，在高速轉動的情形下，不會發生 RM64MN 無法接收之現象。

以上述範例說明：

條件設定：

假設手輪轉 100 格, 手輪倍數 X10, ratio=10, Denominator=36, pulser_ratio =4, slop=1000, 電子齒輪比 P1.44/P1.45 = 1

手輪轉動 1 格 = 馬達轉動脈波數(**128000 * 10 / 100 * 10 / 36=355.55**) * **P1.45/P1.44** * 正好可以控制馬達帶動的圓盤轉動 1 度(**1/360 * 1280000=355.55**)

公式:

手輪轉動一圈(100 格) =
馬達轉動脈波數(**128000 * 手輪倍數 * ratio / Denominator**) * **P1.45 / P1.44**。

換算比率參考:

手輪倍數	Ratio	Denominator	Slop	手輪轉數 (轉一圈)	伺服轉數
X1	1	1	1000	100	128000
X10	1	1	1000	1000	1280000
X100	1	1	1000	10000	12800000
X1	2	1	1000	100	256000
X1	1	2	1000	100	64000

與台達伺服 P1.44 及 P1.45 搭配範例: MPG 轉 X 格，伺服動 Y 個 Pulse

A: 基礎 P1.44 / P1.45 調整的比率值=640(1280000 / 2000)。

B: 基礎 et_rm_mpg_axes_enable2 中的 Ratio / Denominator 比率值=1。

C: 是目前 P1.44/p1.45 調整的比率值。

D: 待求值 set_rm_mpg_axes_enable2 中的 Ratio/Denominator 比率值。

演算式為: **D = B * C / A * Y / X** → **D = 0.5 * C / 640 * Y / X**

注意: 當您使用 Ratio/Denominator 比例值來調整模組輸出值時，
(手輪輸出一格 * 手輪倍數 * ratio / Denominator)得出模組輸出變化量，
其值將無條件刪去小數，僅得整數值輸出。

26.3　_DMC_01_set_rm_jog_axes_enable

■　**格式**

I16 PASCAL _DMC_01_set_rm_jog_axes_enable (U16 CardNo, U16 MasterNodeID, U16 MasterSlotID, U16* NodeID, U16* SlotID, U16 enable, U16 jog_mode, I32 *jog_speed, F64 *sec)

■　**目的**

設定 JOG 運動控制。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
MasterNodeID	U16	編號單位	使用的 RM-MN64 站號
MasterSlotID	U16	編號單位	使用的 RM-MN64 插槽編號
NodeID	U16*	編號單位	操作的伺服站號(最多 4 軸)
SlotID	U16*	編號單位	操作的伺服插槽編號(最多 4 軸)
enable	U16	選項單位	0：關閉 JOG 功能 1：啟動 JOG 功能
jog_mode	U16	選項單位	JOG 軸選擇方式 0：RM 模式 1：MPG 模式
jog_speed	I32*	脈波數/秒	JOG 速度
sec	F64	脈波數/秒	JOG 加速時間

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
							●

■ **範例**

```
I16 rt;
U16 CardNo = 0, MasterNodeID = 1, MasterSlotID = 0; //RM-64  站號為 1
U16 NodeID[4] = {2, 3, 4, 0}, SlotID[4] = {0}; //使用 3 個 ASD-A2F 伺服馬達，站號分別為 2, 3, 4
U16 enable = 1;
U16 jog_mode = 0; //使用 RM 模組 IO 點選擇 X, Y, Z 軸
I32 jog_speed[4] = {128000, 128000, 128000, 0}; //設定 JOG 速度為  128000 pps.
F64 sec = 0.1;

//馬達站號 2 為 X 軸，站號 3 為 Y 軸，站號 4 為 Z 軸，共計 3 軸
rt = _DMC_01_set_rm_jog_axes_enable (CardNo, MasterNodeID, MasterSlotID, NodeID,
SlotID, enable, jog_mode, jog_speed, sec);
```

第二十七章　四組脈波介面專用 API

表 27.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_rm_04pi_ipulser_mode	設定脈波介面模組輸入相位模式
A/B/F	_DMC_01_set_rm_04pi_opulser_mode	設定脈波介面模組輸出相位模式
A/B/F	_DMC_01_set_rm_04pi_svon_polarity	設定 PWR ON（SVON）的準位
A/B/F	_DMC_01_set_rm_04pi_DO2	啟動設定 DO2 埠
A/B/F	_DMC_01_set_rm_04pi_homing_ratio	設置原點復歸速度增量比
A/B/F	_DMC_01_rm_04PI_get_buffer	取得仍未執行的運動命令

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

27.1 _DMC_01_set_rm_04pi_ipulser_mode

■ **格式**

I16 PASCAL _DMC_01_set_rm_04pi_ipulser_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

設定脈波介面模組輸入相位模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：AB phase 1：CW / CCW

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //設置輸入相位為 AB phase

I16 status = _DMC_01_set_rm_04pi_ipulser_mode (CardNo, NodeID, SlotID, mode);
```

27.2　_DMC_01_set_rm_04pi_opulser_mode

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_opulser_mode (U16 CardNo, U16 NodeID, U16 SlotID ,U16 mode)

■　**目的**

設定脈波介面模組輸出相位模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：AB phase 1：CW / CCW 2：Pulse Direction DIR+ in low level 3：Pulse Direction DIR+ in high level

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**敘述**

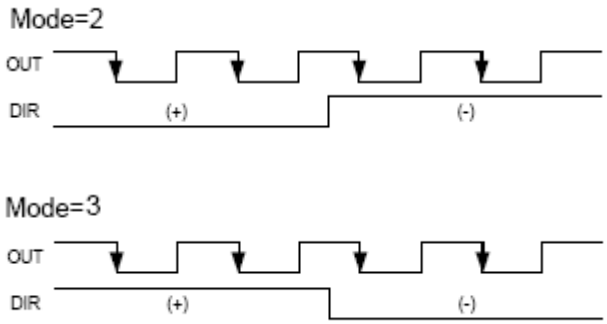


圖 27.1 參數 mode 功能示意

■　**範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //設置輸出相位為 AB phase
I16 status = _DMC_01_set_rm_04pi_opulser_mode (CardNo, NodeID, SlotID, mode);

27.3　_DMC_01_set_rm_04pi_svon_polarity

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_svon_polarity (U16 CardNo, U16 NodeID, U16 SlotID, U16 polarity)

■　**目的**

設定 POWER ON（SVON）的準位。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
polarity	U16	選項單位	0：Normal high 1：Normal low

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 ON_OFF= 1;
U16 polarity = 0; //設置為 Normal high

/* 啟用 POWER ON（Servo ON）功能 */
I16 status = _DMC_01_04pi_set_poweron (CardNo, NodeID, SlotID, polarity);

/* 當驅動器為 Low active 時，此函式之 polarity 的值須設置為值 0（Normal high） */
status = _DMC_01_set_rm_04pi_svon_polarity(CardNo, NodeID, SlotID, polarity);
```

27.4　_DMC_01_set_rm_04pi_DO2

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_DO2 (U16 CardNo, U16 NodeID, U16 SlotID, U16 ON_OFF)

■　**目的**

啟動設定 DO2 埠。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ON_OFF	U16	選項單位	0：關閉 DO2 埠 1：啟動 DO2 埠

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 ON_OFF=1; //  開啟 DO2 埠

I16 status = _DMC_01_set_rm_04pi_DO2 (CardNo, NodeID, SlotID, ON_OFF);
```

27.5　_DMC_01_set_rm_04pi_homing_ratio

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_homing_ratio (U16 CardNo, U16 NodeID, U16 SlotID, U16 ratio)

■　**目的**

設置原點復歸速度增量比。

※使用 RM04PI 操作原點復歸，操作「_DMC_01_set_home_config」函式，該函式速度參數為 unsigned short 型別(16 bit)，最大速度僅能設定至 65535；故若欲使用大於 65535 速度值時，則必須使用此「_DMC_01_set_rm_04pi_homing_ratio」函式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ratio	U16	數值單位	回 Home 速度增量比

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 ratio=100;

I16 status = _DMC_01_set_rm_04pi_homing_ratio( CardNo, NodeID, SlotID, ratio);
```

NOTE

欲運行原點復歸運動，操作「_DMC_01_set_home_config」函式，函式參數 lowSpeed 值為 200，highSpeed 值為 2000，此處 ratio 值為 100，則該 RM04PI 進行原復歸運動所使用的實際歸回原點的速度為：

lowSpeed * ratio = 200 * 100 = 2000

highSpeed * ratio = 2000 * 100 = 200000

27.6　_DMC_01_rm_04PI_get_buffer

■　**格式**

I16 PASCAL _DMC_01_rm_04PI_get_buffer(U16 CardNo,U16 NodeID, U16 SlotID, U16 *bufferLength)

■　**目的**

取得仍未執行的運動命令。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
bufferLength	U16*	選項單位	Buffer 中尚未執行的運動命令

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 bufferLength ;

I16 status = _DMC_01_rm_04PI_get_buffer( CardNo, NodeID, SlotID, &bufferLength);
```

27.7　_DMC_01_set_rm_04pi_md2_feedback_mode

■　**格式**

I16 _stdcall _DMC_01_set_rm_04pi_md2_feedback_mode(U16 CardNo, U16 NodeID,U16 SlotID ,U16 mode)

■　**目的**

當 RM 模組 04PI md2 或是 01PH 無連接 Feedback 訊號時,模組發生警報時會將位置命令設為 0，直到警報解除。使用此 API 將 mode 設為 1 可使該站警報時將位置命令保持固定。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	數值單位	Feedback 狀態 0:有連接　1:無連接

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 mode =50;
I16 status = _DMC_01_set_stepping_move_rate (CardNo, NodeID, SlotID, mode);
```

27.8　_DMC_01_set_rm_04pi_home_config_Tdec

■　**格式**

I16 _stdcall _DMC_01_set_rm_04pi_home_config_Tdec(U16 CardNo,U16 NodeID ,U16 SlotID ,U32 Tdec, U32 enable)

■　**目的**

當 RM 脈波模組原點復歸模式是選擇使用原點開關(ORG)作為觸發訊號時，在 DMC_01_set_home_move 指令下達前可使用此 API 設定由 High_Speed 切換至 Low_Speed 的減速時間。可減少復歸運動速度切換過於劇烈所產生的機構抖動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Tdec	U32	數值單位	ORG 觸發後的減速時間, 單位:ms
enable	U32	數值單位	enable 狀態 0: 無減速　1:有減速

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;
U32 Tdec =1000, enable=1;
/*  設定原點復歸模式*/
I16 status = _DMC_01_set_home_config (CardNo, NodeID, SlotID, Mode, offset, lowSpeed,
highSpeed, acc);
I16 status = _DMC_01_set_rm_04pi_home_config_Tdec(CardNo, NodeID , SlotID , Tdec,
enable);
/*啟用原點復歸*/
I16 status = _DMC_01_set_home_move (CardNo, NodeID, SlotID);
```

27.9　_DMC_01_set_rm_04pi_Home_counter_Delay_time

■　**格式**

I16 _stdcall _DMC_01_set_rm_04pi_Home_counter_Delay_time(U16 CardNo, U16 NodeID,U16 SlotID ,U16 Delay_time)

■　**目的**

當 RM 脈波模組原點復歸動作時，可由此 API 設定由 High_Speed 切換至 Low_Speed 且運動方向相反時的等待時間。觸發訊號為原點(ORG)或極限(PEL,MEL)皆有此功能。可減少復歸運動方向切換過於劇烈所產生的機構抖動。

(此為 SDO 指令寫入模組 0X622B 位置)

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Delay_time	U16	數值單位	ORG 觸發後的減速時間 Default:10 數值範圍: 0~255 單位:ms

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0, Mode=1,
I32 offset =200;
U16 lowSpeed=200, highSpeed=2000;
F64 acc=0.1;
U26 Delay_time =250;
/*  設定原點復歸模式*/
I16 status = _DMC_01_set_home_config (CardNo, NodeID, SlotID, Mode, offset, lowSpeed, highSpeed, acc);
I16 status = _DMC_01_set_rm_04pi_Home_counter_Delay_time(CardNo, NodeID, SlotID , Delay_time)
/*啟用原點復歸*/
I16 status = _DMC_01_set_home_move (CardNo, NodeID, SlotID);
```

（此頁有意留為空白）

第二十八章　四組脈波介面（模式 1）運動控制 API

表 28.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_rm_04pi_md1_start_move	RM04PI 在模式 1 下，進行單軸運動位移
A/B/F	_DMC_01_rm_04pi_md1_v_move	RM04PI 在模式 1 下，進行速度運動控制
A/B/F	_DMC_01_rm_04pi_md1_start_line2	RM04PI 在模式 1 下，進行兩軸線性補間運動
A/B/F	_DMC_01_rm_04pi_md1_start_line3	RM04PI 在模式 1 下，進行三軸線性補間運動
A/B/F	_DMC_01_rm_04pi_md1_start_line4	RM04PI 在模式 1 下，進行四軸線性補間運動
A/B/F	_DMC_01_rm_04pi_md1_start_arc	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：中心座標、角度)
A/B/F	_DMC_01_rm_04pi_md1_start_arc2	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：終點座標、角度)
A/B/F	_DMC_01_rm_04pi_md1_start_arc3	RM04PI 在模式 1 下，進行兩軸圓弧補間運動 (已知條件：中心座標、終點座標)
A/B/F	_DMC_01_rm_04pi_md1_start_heli	RM04PI 在模式 1 下，進行三軸螺旋補間運動
A/B/F	_DMC_01_rm_04pi_md1_p_change	RM04PI 在模式 1 下，將設定的新位置值替換目前的位置
A/B/F	_DMC_01_rm_04pi_md1_v_change	RM04PI 在模式 1 下，將設定的新速度值替換目前的速度
A/B/F	_DMC_01_rm_04pi_md1_set_gear	RM04PI 在模式 1 下，啟動設定 Gear 功能與其參數設置
A/B/F	_DMC_01_rm_04pi_md1_set_soft_limit	RM04PI 在模式 1 下，設定是否啟動軟體極限
A/B/F	_DMC_01_rm_04pi_md1_get_soft_limit_status	RM04PI 在模式 1 下，取得目前 4 軸碰撞軟體極限的狀態
A/B/F	_DMC_01_rm_04pi_md1_set_sld	設定 SLD 埠（DI3）啟動與其相關配置
A/B/F	_DMC_01_rm_04pi_md1_get_mc_error_code	當 alarm code 為 299 時，取得該 RM04PI 在模式 1 下，發生運動控制錯誤的訊息
A/B/F	_DMC_01_set_rm_04pi_ref_counter	RM04PI 在模式 1 下，選擇重新連線參考的位置

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

28.1　_DMC_01_rm_04pi_md1_start_move

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行單軸運動位移。

※設置 StrVel 的值時，該值務必設定小於 MaxVel 的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16*	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
Dist	I32	脈波數	指定的行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 Dist = 1280000;
U16 StrVel = 1000, MaxVel =12800;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve = 1;U16 m_r_a = 0;

/* RM04PI MODE1 進行 T-curve 速度剖面相對座標位移運動 */
I16 status = _DMC_01_rm_04pi_md1_start_move (CardNo, NodeID, SlotID, Dist, StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);
```



RM04PI Md1 是一個站號下可以使用四個模組，通道只有一個，下指令時請特別檢查指令 _DMC_01_get_alm_code 的 alm_code 回傳值是否為 299， 如果回傳值為 299，請使用指令 _DMC_01_rm_04pi_md1_get_mc_error_code 檢查其回傳值 error_code 的值，error_code 回應值=127(代表 Buffer Full)。

28.2　_DMC_01_rm_04pi_md1_v_move

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_v_move(U16 CardNo, U16 NodeID,U16 SlotID, I32 StrVel, I32 MaxVel, F64 Tacc,I16 dir,U16 m_curve)

■　**目的**

RM04PI 在模式 1 下，進行速度運動控制。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	插槽 ID
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度(最小值 5pps)
Tacc	F64	秒	指定的加速時間
dir	I16	選項單位	0：進行正向速度位移 1：進行負向速度位移
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID = 0;
I32 StrVel = 1000, MaxVel =12800;
F64 Tacc=0.1;
I16 dir = 0; //  方向為正向
U16 m_curve = 1; //參照 T-curve 速度剖面

/* RM04PI MODE1  進行正向速度位移運動  */
I16 status = _DMC_01_rm_04pi_md1_v_move (CardNo, NodeID, SlotID, StrVel, MaxVel, Tacc,
dir, m_curve);
```

28.3　_DMC_01_rm_04pi_md1_start_line2

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_line2(U16 CardNo, U16 NodeID,U16 *SlotID, I32 *Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行兩軸線性補間運動。

※相關動作敘述說明，請參閱第 19 章「二軸線性補間運動控制 API」說明。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
Dist	I32*	脈波數	對應 SlotID[0]運動的路徑參數 對應 SlotID[1]運動的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID[2] ={0, 1};
I32 Dist[2]={30000, 40000};
I32 StrVel=0, MaxVel=3000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=2; //參照 S-curve 速度剖面
U16 m_r_a=1; //使用絕對座標位移

I16 status = _DMC_01_rm_04pi_md1_start_line2(CardNo, NodeID, SlotID, Dist, StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

28.4　_DMC_01_rm_04pi_md1_start_line3

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_line3(U16 CardNo, U16 NodeID,U16 *SlotID, I32 *Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行三軸線性補間運動。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID SlotID[2]存放第三組插槽 ID
Dist	I32*	脈波數	對應 SlotID[0]運動的路徑參數 對應 SlotID[1]運動的路徑參數 對應 SlotID[2]運動的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID[3] ={0,1,2};
I32 Dist[3]={10000, 20000, 30000};
I32 StrVel=0, MaxVel=3000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1; //參照 T-curve 速度剖面
U16 m_r_a=1; //使用絕對座標位移

I16 status = _DMC_01_rm_04pi_md1_start_line3 (CardNo, NodeID, SlotID, Dist,StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

28.5 _DMC_01_rm_04pi_md1_start_line4

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_line4(U16 CardNo, U16 NodeID,U16 *SlotID, I32 *Dist, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■ **目的**

RM04PI 在模式 1 下，進行四軸線性補間運動。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	存取各種用來進行線性補間的站號 ID 數組
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID SlotID[2]存放第三組插槽 ID SlotID[3]存放第四組插槽 ID
Dist	I32*	脈波數	對應 SlotID[0]運動的路徑參數 對應 SlotID[1]運動的路徑參數 對應 SlotID[2]運動的路徑參數 對應 SlotID[3]運動的路徑參數
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID[4] ={0,1,2,3};
I32 Dist[4]={10000, 20000, 30000, 40000};
I32 StrVel=0, MaxVel=3000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=1; //參照 T-curve 速度剖面進行絕對座標運動位移

I16 status = _DMC_01_rm_04pi_md1_start_line4 (CardNo, NodeID, SlotID, Dist, StrVel,
MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

28.6　_DMC_01_rm_04pi_md1_start_arc

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_arc(U16 CardNo, U16 NodeID,U16* SlotID, I32* Center, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行兩軸圓弧補間運動。（已知條件：中心座標、角度）

※相關動作敘述說明，請參閱第 20 章「二軸圓弧補間運動控制 API」說明。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
Center	I32*	脈波數	Center[0]存放第一組圓心位置 Center[1]存放第二組圓心位置
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360°
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[2]={0,1};
I32 Center[2]={5000,5000}, StrVel=0, MaxVel=1000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=1; //參照 T-curve 速度剖面進行絕對座標運動位移

I16 status = _DMC_01_rm_04pi_md1_start_arc(CardNo, NodeID, SlotID, Center, Angle, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

28.7　_DMC_01_rm_04pi_md1_start_arc2

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_arc2(U16 CardNo, U16 NodeID,U16* SlotID, I32* End, F64 Angle, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行兩軸圓弧補間運動。(已知條件：終點座標、角度)

※相關動作敘述說明，請參閱第 20 章「二軸圓弧補間運動控制 API」說明。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
End	I32*	脈波數	End[0]存放第一組終點位置 End[1]存放第二組終點位置
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360°
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	最大速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[2]={0,1};
I32 End[2]={5000,5000}, StrVel=0, MaxVel=1000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1, m_r_a=1; //參照 T-curve 速度剖面進行絕對座標運動位移

I16 status = _DMC_01_rm_04pi_md1_start_arc2(CardNo, NodeID, SlotID, End, Angle, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

28.8　_DMC_01_rm_04pi_md1_start_arc3

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_arc3(U16 CardNo, U16 NodeID,U16* SlotID, I32* Center,I32* End, I16 dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve, U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行兩軸圓弧補間運動(已知條件：中心座標、終點座標)

※相關動作敘述說明，請參閱第 20 章「二軸圓弧補間運動控制 API」說明。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID
Center	I32*	脈波數	Center[0]存放第一組圓心位置 Center[1]存放第二組圓心位置
End	I32*	脈波數	End[0]存放第一組終點位置 End[1]存放第二組終點位置
dir	I16	選項單位	指定的方向(值為 0：順時針方向 1：逆時針方向)
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0, NodeID=1, SlotID[2]={0,1}, m_curve=1, m_r_a=1;
I32 Center[2]={5000,2500}, End[2]={10000,5000}, StrVel=0, MaxVel=1000;
I16 dir=1; //表示往逆時針方向
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_rm_04pi_md1_start_arc3(CardNo, NodeID, SlotID, Center, End, dir,
StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

28.9　_DMC_01_rm_04pi_md1_start_heli

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_start_heli(U16 CardNo, U16 NodeID,U16* SlotID, I32* Center,I32 Depth, I32 Pitch, I16 dir, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec, U16 m_curve,U16 m_r_a)

■　**目的**

RM04PI 在模式 1 下，進行三軸螺旋補間運動。

※相關動作敘述說明，請參閱第 22 章「三軸螺旋補間運動控制 API」說明。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16*	編號單位	SlotID[0]存放第一組插槽 ID SlotID[1]存放第二組插槽 ID SlotID[2]存放第三組插槽 ID
Center	I32*	脈波數	Center[0]存放第一組圓心位置 Center[1]存放第二組圓心位置 Center[2]存放第三組圓心位置
Depth	I32	脈波數	指定軸對應之相對性位置深度(整個 Z 方向的高度)
Pitch	I32	脈波數	指定之相對性的兩螺旋之間的高度
dir	I16	選項單位	螺旋圓弧運動的方向。 0：順時針方向 1：逆時針方向
StrVel	I32	脈波數/秒	初始速度參數
MaxVel	I32	脈波數/秒	切線速度參數
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：參照 T-curve 速度剖面(Default) 2：參照 S-curve 速度剖面
m_r_a	U16	選項單位	0：相對座標位移(Default) 1：絕對座標位移

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID=1, SlotID[3]={0,1,2}, m_curve=1, m_r_a=1;
I32 Center[3]={5000,2500,2500}, Depth = 2500, Pitch = 500, StrVel = 0, MaxVel = 1000;
I16 dir=1; //表示往逆時針方向
F64 Tacc=0.1, Tdec=0.1;

I16 status = _DMC_01_rm_04pi_md1_start_heli(CardNo, NodeID, SlotID, Center, Depth, Pitch, dir, StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);

28.10　_DMC_01_rm_04pi_md1_p_change

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_p_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewPos)

■　**目的**

RM04PI 在模式 1 下，將設定的新位置值替換目前的位置(p_change 只能對單軸運動指令下達)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewPos	I32	脈波數	欲替換的位置參數

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I32 NewPos=100000;

I16 status = _DMC_01_rm_04pi_md1_p_change (CardNo, NodeID, SlotID, NewPos);
```

28.11　_DMC_01_rm_04pi_md1_v_change

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_v_change (U16 CardNo, U16 NodeID, U16 SlotID, I32 NewSpeed, F64 sec)

■ **目的**

RM04PI 在模式 1 下，將設定的新速度值替換目前的速度(v_change 不能在減速段下達即 motion done=3 時)。

※關於其相關描述，請參閱第 18.6 節「_DMC_01_v_change」的描述項目。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
NewSpeed	I32	脈波數/秒	欲替換速度參數
sec	F64	秒	速度轉換時，指定的加/減速時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I32 NewSpeed=3000;
F64 sec=0.1;

I16 status = _DMC_01_rm_04pi_md1_v_change (CardNo, NodeID, SlotID, NewSpeed, sec)

28.12　_DMC_01_rm_04pi_md1_set_gear

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_set_gear (U16 CardNo, U16 NodeID, U16 SlotID, I16 numerator,I16 denominator,U16 Enable)

■　**目的**

RM04PI 在模式 1 下，啟動設定 Gear 功能與其參數設置。

※由於步進驅動器無電子齒輪比功能，故可以此函式達到此功能需求。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
numberator	I16	數值單位	電子齒輪比分子
denominator	I16	數值單位	電子齒輪比分母
Enable	U16	選項單位	0：關閉 Gear 功能 1：啟動 Gear 功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I16 numberator=1, denominator=2;
U16 Enable=1;

I16 status = _DMC_01_rm_04pi_md1_set_gear(CardNo, NodeID, SlotID, numerator, denominator, Enable);

28.13　_DMC_01_rm_04pi_md1_set_soft_limit

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_set_soft_limit(U16 CardNo, U16 NodeID,U16 SlotID, I32 PLimit, I32 NLimit,U16 Enable)

■ **目的**

RM04PI 在模式 1 下，設定是否啟動軟體極限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
PLimit	I32	數值單位	軟體正極限設定值
NLimit	I32	數值單位	軟體負極限設定值
Enable	U16	選項單位	0：關閉軟體極限 1：啟動軟體極限

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

U16 CardNo=0; U16 NodeID =1, SlotID=0;
I32 PLimit=5000, NLimit=-5000;
U16 Enable=1;

I16 status = _DMC_01_rm_04pi_md1_set_soft_limit (CardNo, NodeID, SlotID, PLimit, NLimit, Enable);

28.14　_DMC_01_rm_04pi_md1_get_soft_limit_status

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_md1_get_soft_limit_status (U16 CardNo, U16 NodeID, U16 SlotID ,U16* NLimit_status,U16* PLimit_status)

■ **目的**

RM04PI 在模式 1 下，取得目前 4 軸碰撞軟體極限的狀態。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>插槽 ID</td></tr>
<tr><td>NLimit_status</td><td>U16*</td><td>數值單位</td><td>傳回碰撞軟體負極限的狀態
<table><tr><th>數值</th><th>定義</th></tr><tr><td>1</td><td>Slot 0 碰撞軟體負極限</td></tr><tr><td>2</td><td>Slot 1 碰撞軟體負極限</td></tr><tr><td>4</td><td>Slot 2 碰撞軟體負極限</td></tr><tr><td>8</td><td>Slot 3 碰撞軟體負極限</td></tr><tr><td>….</td><td></td></tr><tr><td>15</td><td>Slot 0~3 都碰撞軟體負極限</td></tr></table></td></tr>
<tr><td>PLimit_status</td><td>U16*</td><td>數值單位</td><td>傳回碰撞軟體正極限的狀態
<table><tr><th>數值</th><th>定義</th></tr><tr><td>1</td><td>Slot 0 碰撞軟體正極限</td></tr><tr><td>2</td><td>Slot 1 碰撞軟體正極限</td></tr><tr><td>4</td><td>Slot 2 碰撞軟體正極限</td></tr><tr><td>8</td><td>Slot 3 碰撞軟體正極限</td></tr><tr><td>….</td><td></td></tr><tr><td>15</td><td>Slot 0~3 都碰撞軟體正極限</td></tr></table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■ **範例**

```
U16 CardNo=0; U16 NodeID =1, SlotID=0;
U16 NLimit_status, PLimit_status;
I16 status = _DMC_01_rm_04pi_md1_get_soft_limit_status(CardNo, NodeID, SlotID ,
```

```
&NLimit_status, &PLimit_status);
```

28.15　_DMC_01_rm_04pi_md1_set_sld

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_set_sld (U16 CardNo, U16 NodeID, U16 SlotID, I16 enable, I16 sd_logic,I16 mode)

■　**目的**

設定 SLD 埠（DI3）啟動與其相關配置。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	I16	選項單位	0：關閉 SLD 埠功能 1：啟動 SLD 埠功能
sd_logic	I16	選項單位	0：Normal high 1：Normal low
mode	I16	選項單位	1：使用 EMG stop（緊急停止） 2：使用 Slow down stop（減速停止）

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I16 enable=1;
I16 sd_logic=1;//埠操作極性為 Normal high
I16 mode=1; //使用的停止模式為 EMG stop

I16 status = _DMC_01_rm_04pi_md1_set_sld(CardNo, NodeID, SlotID, enable, sd_logic, mode);

28.16　_DMC_01_rm_04pi_md1_get_mc_error_code

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_md1_get_mc_error_code (U16 CardNo, U16 NodeID, U16 SlotID, U16 error_code)

■　**目的**

當 alarm code 為 299 時，取得該 RM04PI 在模式 1 下，發生運動控制錯誤的訊息。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
error_code	U16*	數值單位	運動控制錯誤訊息代碼

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID=0;
U16* error_code;
U32* alm_code;

/* 取得 Slave 的錯誤訊息 */
I16 status = _DMC_01_get_alm_code (CardNo, NodeID, SlotID, &alm_code);
```

NOTE

當 alm_code 的值為 299 時，使用此函式。以取得 RM04PI 目前在運動控制位移中發生的錯誤訊息代碼。

_DMC_01_rm_04pi_md1_get_mc_error_code (CardNo, NodeID, SlotID, &error_code);

例：error_code 值為 127，表示” 運動指令 Buffer 已滿”（相關 Error Code 的詳細說明，請參閱 PCI_DMC_01_Err.h）

28.17　_DMC_01_set_rm_04pi_ref_counter

■　**格式**

I16 PASCAL _DMC_01_set_rm_04pi_ref_counter (U16 CardNo, U16 NodeID,U16 SlotID , U16 mode)

■　**目的**

M04PI 在模式 1 下，當模組重新連線後參考位置的選擇。

※使用此函式時，請確認是否有連結回授（Feedback）。

若有，mode 參數請設置為 1；反之如無接回授，則將該參數設置為 0。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
mode	U16	選項單位	0：參考 command 計數值 1：參考 position 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 mode=0; //參考 command 計數值

I16 status =_DMC_01_set_rm_04pi_ref_counter (CardNo, NodeID, SlotID, mode);
```

第二十九章　四組類比輸出遠端擴充模組 API

表 29.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_rm_04da_set_output_value	設定 DA 輸出數值
A/B/F	_DMC_01_rm_04da_get_output_value	讀取 DA 輸出數值
A/B/F	_DMC_01_rm_04da_get_return_code	讀取 DA 狀態值
A/B/F	_DMC_01_rm_04da_set_output_range	設定 DA 輸出範圍
A/B/F	_DMC_01_rm_04da_set_output_enable	設定腳位輸出開啟/關閉
A/B/F	_DMC_01_rm_04da_set_output_overrange	設定輸出範圍增加 10%
A/B/F	_DMC_01_rm_04da_set_output_error_clear	清除錯誤狀態
A/B/F	_DMC_01_rm_04da_set_output_error_handle	斷線保持 DA 原本設定狀態
A/B/F	_DMC_01_rm_04da_set_output_offset_value	設定 DA offset 數值
A/B/F	_DMC_01_rm_04da_get_output_offset_value	讀取 DA offset 數值
A/B/F	_DMC_01_rm_04da_read_data	讀取 DA 輸出數值

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

29.1 _DMC_01_rm_04da_set_output_value

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_value (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 Value)

■ **目的**

設定 DA 輸出數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16*	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Value	U16	數值單位	輸出值為 0 ~ 65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 Value = 0x5ff;

I16 status = _DMC_01_rm_04da_set_output_value (CardNo, NodeID, SlotID, ChannelNo, Value);
```

29.2 _DMC_01_rm_04da_get_output_value

■ **格式**

I16 PASCAL _DMC_01_rm_04da_get_output_value(U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 *Value)

■ **目的**

讀取 DA 輸出數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16*	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Value	U16	數值單位	輸出值為 0 ~ 65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 Value = 0;

I16 status = _DMC_01_rm_04da_get_output_value(CardNo, NodeID, SlotID, ChannelNo, &Value) ;
```

29.3 _DMC_01_rm_04da_get_return_code

■ **格式**

I16 PASCAL _DMC_01_rm_04da_get_return_code (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, U16* ReturnCode)

■ **目的**

讀取 DA 狀態值。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~63</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>Data</td><td>U16</td><td>數值單位</td><td>取得傳回狀態參數
<table>
<tr><th>D3</th><th>D2</th><th>D1</th><th>D0</th></tr>
<tr><td>out enable</td><td colspan="3">out range mode</td></tr>
<tr><th>D7</th><th>D6</th><th>D5</th><th>D4</th></tr>
<tr><td colspan="2">return error code</td><td>error handle</td><td>over range</td></tr>
<tr><th>D11</th><th>D10</th><th>D9</th><th>D8</th></tr>
<tr><td colspan="4">return error code</td></tr>
<tr><th>D15</th><th>D14</th><th>D13</th><th>D12</th></tr>
<tr><td colspan="4">return error code</td></tr>
<tr><td colspan="4">Return Error Code:
0x2fe :HWFault　　0x2fc:OffsetOverRange
0x2fd:DAValueOverMax　　0x2fb: DAValueOverMin
0x2ef:OverTemp　　0x2ee:Slowactive
0x2ed:IOUTfault　　0x2e0:Multfault</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 1;
U16 ReturnCode = 0;

I16 status = _DMC_01_rm_04da_get_return_code (CardNo, NodeID, SlotID, ChannelNo,
&ReturnCode);
```

29.4　_DMC_01_rm_04da_set_output_range

■　**格式**

I16 PASCAL _DMC_01_rm_04da_set_output_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, U16 Range)

■　**目的**

設定 DA 輸出範圍。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~63
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Range	U16	數值單位	增益值為 0 ~ 7 數值 / 定義 0 / 輸出範圍: 0-5V (預設值) 1 / 輸出範圍: 0-10V 2 / 輸出範圍: ±5V 3 / 輸出範圍: ±10V 5 / 輸出範圍: 4-20mA 6 / 輸出範圍: 0-20mA 7 / 輸出範圍: 0-24mA

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 2;
U16 Range = 3; //選擇模式 3
I16 status = _DMC_01_rm_04da_set_output_range (CardNo, NodeID, SlotID, ChannelNo, range);
```

29.5 _DMC_01_rm_04da_set_output_enable

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_enable (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 Enable)

■ **目的**

設定腳位輸出開啟/關閉。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 (0~12)
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Enable	U16	選項單位	0：輸出禁能 1：輸出致能。將設定的輸出值發送到目的端

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 3;
U16 Enable = 1; //致能

I16 status = _DMC_01_rm_04da_set_output_enable (CardNo, NodeID, SlotID, ChannelNo, Enable);
```

29.6　_DMC_01_rm_04da_set_output_overrange

■　**格式**

I16 PASCAL _DMC_01_rm_04da_set_output_overrange (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 On_Off)

■　**目的**

設定輸出範圍增加 10%。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 (0~12)
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	選項單位	0：關閉 Overrange 1：啟動 Overrange

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 On_Off = 1; //啟動 OverRange。
// 輸出電壓範圍設定為-5V~5V 時，若啟動 OverRange，則輸出範圍擴大為 -5.5V~5.5V。

I16 status = _DMC_01_rm_04da_set_output_overrange (CardNo, NodeID, SlotID, ChannelNo, On_Off);

29.7　_DMC_01_rm_04da_set_output_error_clear

■　**格式**

I16 PASCAL _DMC_01_rm_04da_set_output_error_clear (U16 CardNo, U16 NodeID,U16 SlotID, U16 ChannelNo, U16 On_Off)

■　**目的**

清除錯誤狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	選項單位	1：啟動 Clear Error

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 On_Off = 1; //Clear Error

I16 status = _DMC_01_rm_04da_set_output_error_clear (CardNo, NodeID, SlotID,
ChannelNo, On_Off);
```

29.8　_DMC_01_rm_04da_set_output_error_handle

■　**格式**

I16 PASCAL _DMC_01_rm_04da_set_output_error_handle (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, U16 On_Off)

■　**目的**

斷線保持 DA 原本設定狀態。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	選項單位	0：關閉 ErrorHandle 1：啟動 ErrorHandle

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 ChannelNo = 3;
U16 On_Off = 1; //啟用 ErrorHandle

I16 status = _DMC_01_rm_04da_set_output_error_handle (CardNo, NodeID, SlotID,
ChannelNo, On_Off);
```

29.9 _DMC_01_rm_04da_set_output_offset_value

■ **格式**

I16 PASCAL _DMC_01_rm_04da_set_output_offset_value (U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, I16 Value)

■ **目的**

設定 DA offset 數值。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~63</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>SetValue</td><td>I16</td><td>選項單位</td><td>數值為 +127 ~ -128
■ 1 Step of Value = 38.14μv
■ Adjust the offset of the channel by -16 LSBs to +15.875 LSBs in increments of 1/8 LSB
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>127</td><td>Offset Adjustment: +15.875 LSBs (4.844mv)</td></tr>
<tr><td>126</td><td>Offset Adjustment: +15.75 LSBs (4.806mv)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>0</td><td>No Adjustment (default)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>-127</td><td>Offset Adjustment: -15.875 LSBs(- 4.844mv)</td></tr>
<tr><td>-128</td><td>Offset Adjustment: -16 LSBs(- 4.882mv)</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 ChannelNo = 0;
I16 Value = 0xf; //offset 設置為 0xf

I16 status = _DMC_01_rm_04da_set_output_offset_value (CardNo, NodeID, SlotID,
ChannelNo, Value);
```

29.10　_DMC_01_rm_04da_get_output_offset_value

■　**格式**

I16 PASCAL _DMC_01_rm_04da_get_output_offset_value(U16 CardNo, U16 NodeID, U16 SlotID, U16 ChannelNo, I16 *Value)

■　**目的**

讀取 DA offset 數值。

■　**參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~63</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>SetValue</td><td>I16</td><td>選項單位</td><td>數值為 +127 ~ -128
■ 1 Step of Value = 38.14μv
■ Adjust the offset of the channel by -16 LSBs to +15.875 LSBs in increments of 1/8 LSB
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>127</td><td>Offset Adjustment: +15.875 LSBs (4.844mv)</td></tr>
<tr><td>126</td><td>Offset Adjustment: +15.75 LSBs (4.806mv)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>0</td><td>No Adjustment (default)</td></tr>
<tr><td>…</td><td>……</td></tr>
<tr><td>-127</td><td>Offset Adjustment: -15.875 LSBs(- 4.844mv)</td></tr>
<tr><td>-128</td><td>Offset Adjustment: -16 LSBs(- 4.882mv)</td></tr>
</table></td></tr>
</table>

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

U16 CardNo = 0, NodeID = 1, SlotId = 0;
U16 ChannelNo = 0;
I16 Value = 0;

I16 status = _DMC_01_rm_04da_get_output_offset_value (CardNo, NodeID, SlotID, ChannelNo, & Value);

29.11　_DMC_01_rm_04da_read_data

■　**格式**

I16 PASCAL _DMC_01_rm_04da_read_data(U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16* data)

■　**目的**

讀取 DA 輸出數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~12
SlotID	U16*	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
data	U16*	數值單位	輸出值為 0 ~ 65535

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 ChannelNo = 0;
U16 data = 0;

I16 status = _DMC_01_rm_04da_read_data(CardNo, NodeID, SlotID, channelno, &data);
```

（此頁有意留為空白）

第三十章　四組類比輸入遠端擴充模組 API

表 30.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_04ad_input_range	設定 AD 的 Input 範圍
A/B/F	_DMC_01_get_04ad_input_range	取得目前 AD 的 Input 設定範圍
A/B/F	_DMC_01_set_04ad_conversion_time	設定 AD 轉換速度
A/B/F	_DMC_01_get_04ad_conversion_time	取得目前 AD 所設定的轉換速度
A/B/F	_DMC_01_get_04ad_data	讀取輸入之電壓數值
A/B/F	_DMC_01_set_04ad_average_mode	設定 AD 平均功能之模式
A/B/F	_DMC_01_get_04ad_average_mode	讀取 AD 平均功能之模式
A/B/F	_DMC_01_set_04ad_input_enable	設定 AD Channel 是否致能 Input

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

30.1 _DMC_01_set_04ad_input_range

■ **格式**

I16 _DMC_01_set_04ad_input_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 range)

■ **目的**

設定 AD 的 Input 範圍。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~12</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>Range</td><td>U16</td><td>數值單位</td><td>輸入值為 0 ~ 5
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>0</td><td>輸入範圍:-10V~10V</td></tr>
<tr><td>1</td><td>輸入範圍:0V~10V</td></tr>
<tr><td>2</td><td>輸入範圍:-5V~5V</td></tr>
<tr><td>3</td><td>輸入範圍:0V~5V</td></tr>
<tr><td>4</td><td>NULL</td></tr>
<tr><td>5</td><td>輸入範圍:0~20mA</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 0;
U16 range = 1; //選擇 Range -10 ~ 10V

I16 status = _DMC_01_set_04ad_input_range (CardNo, NodeID, SlotID, channelno, range);
```

30.2　_DMC_01_get_04ad_input_range

■　**格式**

I16 _DMC_01_get_04ad_input_range (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 *range)

■　**目的**

取得目前 AD 的 Input 設定範圍。

■　**參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 0~63</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>ChannelNo</td><td>U16</td><td>編號單位</td><td>頻道編號為 0~3</td></tr>
<tr><td>Range</td><td>U16</td><td>數值單位</td><td>取得回傳參數，值為 0 ~ 5
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>0</td><td>輸入範圍:-10V~10V</td></tr>
<tr><td>1</td><td>輸入範圍:0V~10V</td></tr>
<tr><td>2</td><td>輸入範圍:-5V~5V</td></tr>
<tr><td>3</td><td>輸入範圍:0V~5V</td></tr>
<tr><td>4</td><td>NULL</td></tr>
<tr><td>5</td><td>輸入範圍:0~20mA</td></tr>
</table></td></tr>
</table>

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 range;

I16 status = _DMC_01_get_04ad_input_range (CardNo, NodeID, SlotID, channelno, &range);
```

30.3 _DMC_01_set_04ad_conversion_time

■ **格式**

I16 _DMC_01_set_04ad_conversion_time (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■ **目的**

設定 AD 轉換速度。

■ **參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 (0~12)</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>Mode</td><td>U16</td><td>選項單位</td><td>輸入值為 0 ~ 6
<table>
<tr><th>數值</th><th>輸出比率(Hz)</th><th>-3 dB 頻率 (Hz)</th><th>RMS 雜訊 (μv)</th></tr>
<tr><td>0</td><td>372</td><td>200</td><td>9.6</td></tr>
<tr><td>1</td><td>1001</td><td>520</td><td>15.5</td></tr>
<tr><td>2</td><td>2005</td><td>1040</td><td>22.7</td></tr>
<tr><td>3</td><td>2534</td><td>1300</td><td>26.1</td></tr>
<tr><td>4</td><td>4826</td><td>2500</td><td>39.2</td></tr>
<tr><td>5</td><td>6041</td><td>3100</td><td>46.0</td></tr>
<tr><td>6</td><td>12166</td><td>6300</td><td>120.0</td></tr>
</table></td></tr>
</table>

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 mode = 1;
I16 status = _DMC_01_set_04ad_conversion_time(CardNo, NodeID, SlotID, mode);
```

30.4　_DMC_01_get_04ad_conversion_time

■　**格式**

I16 _DMC_01_get_04ad_conversion_time (U16 CardNo, U16 NodeID, U16 SlotID, U16* mode)

■　**目的**

取得目前 AD 所設定的轉換速度。

■　**參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>NodeID</td><td>U16</td><td>編號單位</td><td>站號 ID 為 (0~12)</td></tr>
<tr><td>SlotID</td><td>U16</td><td>編號單位</td><td>0</td></tr>
<tr><td>Mode</td><td>U16</td><td>選項單位</td><td>輸出值為 0 ~ 6
<table>
<tr><th>數值</th><th>輸出比率(Hz)</th><th>-3 dB 頻率 (Hz)</th><th>RMS 雜訊 (μv)</th></tr>
<tr><td>0</td><td>372</td><td>200</td><td>9.6</td></tr>
<tr><td>1</td><td>1001</td><td>520</td><td>15.5</td></tr>
<tr><td>2</td><td>2005</td><td>1040</td><td>22.7</td></tr>
<tr><td>3</td><td>2534</td><td>1300</td><td>26.1</td></tr>
<tr><td>4</td><td>4826</td><td>2500</td><td>39.2</td></tr>
<tr><td>5</td><td>6041</td><td>3100</td><td>46.0</td></tr>
<tr><td>6</td><td>12166</td><td>6300</td><td>120.0</td></tr>
</table></td></tr>
</table>

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 mode;
I16 status = _DMC_01_get_04ad_conversion_time(CardNo, NodeID, SlotID, &mode);
```

30.5　_DMC_01_get_04ad_data

■ **格式**

I16 _DMC_01_get_04ad_data (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16* value)

■ **目的**

讀取輸入之電壓數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~63
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
ReturnData	U16	數值單位	0~65535

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■ **範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotId = 0;
U16 channelno = 2;
U16 value;

I16 status = _DMC_01_get_04ad_data (CardNo, NodeID, SlotID, channelno, &value);
```

30.6　_DMC_01_set_04ad_average_mode

■　**格式**

I16 _DMC_01_set_04ad_average_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 mode)

■　**目的**

設定 AD 平均功能之模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~63
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Mode	U16	數值單位	輸入數值 0~5

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 mode = 1;

I16 status = _DMC_01_set_04ad_average_mode(CardNo, NodeID, SlotID, channelno, mode);
```

30.7　_DMC_01_get_04ad_average_mode

■　**格式**

I16 _DMC_01_get_04ad_average_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16* mode)

■　**目的**

讀取 AD 平均功能之模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~63
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
Mode	U16	數值單位	讀取數值 0~5

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1, mode;

I16 status = _DMC_01_get_04ad_average_mode (CardNo, NodeID, SlotID, channelno, &mode);

30.8　_DMC_01_set_04ad_input_enable

■　**格式**

I16 _DMC_01_set_04ad_input_enable (U16 CardNo, U16 NodeID, U16 SlotID, U16 channelno, U16 ON_OFF)

■　**目的**

設定 AD Channel 是否致能 Input。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID 為 0~63
SlotID	U16	編號單位	0
ChannelNo	U16	編號單位	頻道編號為 0~3
On_Off	U16	數值單位	0:Disable　1:Enable

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
						●	

■　**範例**

```
U16 CardNo = 0;
U16 NodeID = 1;
U16 SlotID = 0;
U16 channelno = 1;
U16 ON_OFF=1;

I16 status = _DMC_01_set_04ad_input_enable (CardNo, NodeID, SlotID, channelno,
ON_OFF);
```

(此頁有意留為空白)

第三十一章 Slave 資訊 API

表 31.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_get_devicetype	獲取 slave 裝置類型
A/B/F	_DMC_01_get_slave_version	獲取 slave 裝置韌體版本
A/B/F	_DMC_F01_get_slave_subversion	獲取 Salve 韌體版本次版號

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

31.1 _DMC_01_get_devicetype

■ **格式**

I16 PASCAL _DMC_01_get_devicetype (I16 CardNo, U16 NodeID, U16 SlotID, U32 *DeviceType, U32 *IdentityObject)

■ **目的**

取得 slave 裝置類型。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
DeviceType	U32*	數值	Salve 裝置類型 （參考後頁 note 說明）
IdentityObject	U32*	數值	裝置的物件字典代碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

I16 CardNo=0, NodeID=1, SlotID=0;
U32 DeviceType, IdentityObject;

I16 status = _DMC_01_get_devicetype (CardNo, NodeID, SlotID, &DeviceType, &IdentityObject);

NOTE

※關於 DeviceType 變數代表裝置類型的代碼，請參考下表：

Device Type	Code	Device Type	Code
伺服驅動器 A2F 系列	0x04020192	伺服驅動器 M 系列	0x06020192
伺服驅動器 A2R 系列	0x08020192	伺服驅動器 S 系列	0x09020192
伺服驅動器 B2F 系列	0x05020192		
ASD-DMC-RM32MN	0x04110191	ASD-DMC-RM64MN	0x08110191
ASD-DMC-RM32NT	0x04120191	ASD-DMC-RM64NT	0x08120191
ASD-DMC-RM04PI(MD1)	0x1C100191	ASD-DMC-RM04AD	0x08140191
ASD-DMC-RM04PI(MD2)	0x14100191	ASD-DMC-RM04DA	0x08180191
ASD-DMC-RM32PT	0x04130191	ASD-DMC-GE01PH	0x11210191
ASD-DMC-GE16MN	0x08230191 【註 1】	ASD-DMC-GE16NT	0x08230191 【註 1】

【註 1】ASD-DMC-GE 的 IO 模組(GE16MN/GE16NT)，其為 64 點 IO 計為 1 個 Device type，即 1 個 Node。

※關於 IdentityObject 變數代表各類裝置物件字典代碼，請參考下表：

Object Dictionary	Code
伺服驅動器 A2F 系列	0x6000
伺服驅動器 M 系列	0x6030
伺服驅動器 A2R 系列	0x6040
伺服驅動器 S 系列	0x6050
伺服驅動器 B2F 系列	0x6070
Remote 模組	0x1000
Gateway 模組	0x2000

31.2 _DMC_01_get_slave_version

■ **格式**

I16 PASCAL _DMC_01_get_slave_version(I16 CardNo, U16 NodeID,U16 SlotID, U16* version)

■ **目的**

取得 slave(模組或伺服)裝置韌體版本。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
version	U16*	版本編號	Salve 韌體版本

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
I16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 version;

I16 status = _DMC_01_get_slave_version(CardNo, NodeID, SlotID, &version);
```

31.3 _DMC_01_get_slave_subversion

■ **格式**

I16 PASCAL _DMC_01_get_slave_subversion (U16 CardNo, U16 NodeID, U16 SlotID, U16 *SubVersion)

■ **目的**

取得 slave(模組或伺服)裝置版本次版號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
SubVersion	U16*	版本編號	Salve 韌體版本次版號

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
I16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 SubVersion;

I16 status = _DMC_01_get_slave_subversion (CardNo, NodeID, SlotID ,& SubVersion);
```

（此頁有意留為空白）

第三十二章　監控參數 API

表 32.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_monitor	設定欲監控的參數
A/B/F	_DMC_01_get_monitor	取得監控的參數獲取的值
A/B/F	_DMC_01_get_servo_command	取得伺服驅動器發送的命令值
A/B/F	_DMC_01_get_servo_DI	取得伺服驅動器 DI 訊號的值
A/B/F	_DMC_01_get_servo_DO	取得伺服驅動器 DO 訊號的值

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

32.1　_DMC_01_set_monitor

■　**格式**

I16 PASCAL _DMC_01_set_monitor (U16 CardNo, U16 NodeID, U16 SlotID, U16 monitorw)

■　**目的**

設定欲監控的參數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
monitorw	U16	數值	欲監控的參數值(U8+U8)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

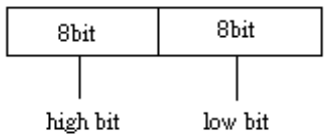
■　**範例**

U16 CardNo=0, NodeID=1, SlotID=0;
U16 monitorw=0x0101; //脈波命令輸入脈波數(電子齒輪比之後)—資料長度(32bit)

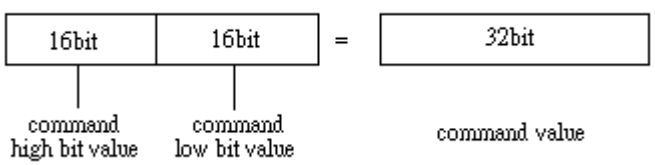
I16 status = _DMC_01_set_monitor (CardNo, NodeID, SlotID, monitorw);

NOTE

在_DMC_01_set_monitor monitorw=0x0101 (脈波命令輸入脈波數**)** monitorw **說明如下:**



DMC_01_get_monitor value(32bit)說明如下



※monitorw 的值請參閱下表 32.2

表 32.2

監控項目索引	監控項目	單位	資料長度	04PI	Servo
0	馬達迴授脈波數(電子齒輪比之後)	使用者單位	32-bit[有號數]	■	■
1	脈波命令輸入脈波數(電子齒輪比之後)	使用者單位	32-bit[有號數]	■	■
2	控制命令脈波與迴授脈波誤差數	使用者單位	32-bit[有號數]	□	■
3	馬達迴授脈波數	128 萬 Pulse/rev	32-bit[有號數]	□	■
4	脈波命令輸入脈波數(電子齒輪比之前)	脈波數	32-bit[有號數]	□	■
5	誤差脈波數(電子齒輪之後)	脈波數	32-bit[有號數]	□	■
6	脈波命令輸入頻率	轉發封包數 K/秒	16-bit[有號數]	□	■
7	馬達轉速	0.1 轉/分	32-bit[有號數]	□	■
8	速度輸入命令	0.01 伏特	16-bit[有號數]	□	■
9	速度輸入命令	0.1 轉/分	32-bit[有號數]	□	■
10	扭矩輸入命令	0.01 伏特	16-bit[有號數]	□	■
11	扭矩輸入命令	百分比(%)	16-bit[有號數]	□	■
12	平均轉矩	百分比(%)	16-bit[有號數]	□	■
13	峰值轉矩	百分比(%)	16-bit[有號數]	□	■
14	主迴路電壓 (BUS 電壓)	伏特	16-bit[無號數]	□	■
15	負載/馬達慣性比	0.1 倍	16-bit[無號數]	□	■
16	IGBT 溫度	^{o}C	16-bit[有號數]	□	■
17	共振頻率	赫茲	16-bit[無號數]	□	■
18	Z 相原點處的數值(脈波數)	-5000 ~ +5000	16-bit[有號數]	□	■
19	映射參數#1：P0 - 25	數值	32-bit[無號數]	□	■
20	映射參數#2：P0 - 26	數值	32-bit[無號數]	□	■
21	映射參數#3：P0 - 27	數值	32-bit[無號數]	□	■
22	映射參數#4：P0 - 28	數值	32-bit[無號數]	□	■
23	監視變數#1：P0 - 09	數值	32-bit[有號數]	□	■
24	監視變數#2：P0 - 10	數值	32-bit[有號數]	□	■
25	監視變數#3：P0 - 11	數值	32-bit[有號數]	□	■
26	監視變數#4：P0 – 12	數值	32-bit[有號數]	□	■
27	Z 相原點處的數值(電子齒輪比之後)	-半圈 ~ +半圈	32-bit[有號數]	□	■
28	異警碼	數值	32-bit[無號數]	■	■
39	DI 狀態		16-bit[無號數]	■	■
40	DO 狀態		16-bit[無號數]	■	■
120	DMCNET 通訊狀態		16-bit[無號數]	□	■
121	DMCNET 運轉時封包錯誤計數		32-bit[無號數]	□	■
122	DMCNET 運轉時間	秒	16-bit[無號數]	□	■

32.2　_DMC_01_get_monitor

■　**格式**

I16 PASCAL _DMC_01_get_monitor (U16 CardNo, U16 NodeID, U16 SlotID, U32 *value)

■　**目的**

取得監控的參數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
value	U32*	數值	監控參數目前的數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
U16 monitorw=0x2728; //DI/DO 狀態—資料長度(DI-16bit-Hi Bit,DO-16bit-Low Bit)
U32 value;
I16 status = _DMC_01_set_monitor (CardNo, NodeID, SlotID, monitorw);
status = _DMC_01_get_monitor (CardNo, NodeID, SlotID, &value);
```

NOTE

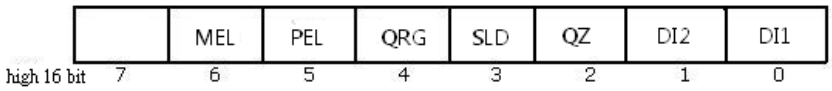
※在伺服下: _DMC_01_set_monitor　monitorw=0x27(DI 狀態)或 0x28(DO 狀態) get_monitor Value 就會是伺服的 DI 或 DO 的值。

※在 RM04PI Model1 下：_DMC_01_set_monitor，monitorw=0x27(DI 狀態)或 0x28(DO 狀態) get_monitor Value 值說明如下(都會回 32bit Data):

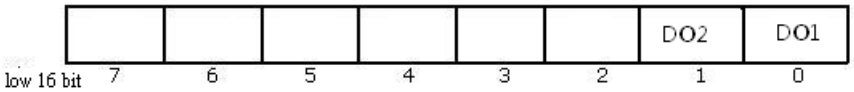


※在 RM04PI Model2(GE01PH)下

在_DMC_01_set_monitor，monitorw=0x27(DI 狀態) get_monitor Value 值說明如下:



在_DMC_01_set_monitor，monitorw=0x28(DO 狀態) get_monitor Value 值說明如下:



32.3　_DMC_01_get_servo_command

■　**格式**

I16 PASCAL _DMC_01_get_servo_command (U16 CardNo, U16 NodeID, U16 SlotID, U32 *servo_cmd)

■　**目的**

取得伺服驅動器發送的命令值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
servo_cmd	U32*	數值	驅動器命令值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 servo_cmd;

/* servo_cmd 的值為取回 servo 位置 command 值 */
I16 status = _DMC_01_get_servo_command (CardNo, NodeID, SlotID, &servo_cmd);
```

32.4　_DMC_01_get_servo_DI

■　**格式**

I16 PASCAL _DMC_01_get_servo_DI (U16 CardNo, U16 NodeID, U16 SlotID,
U16 *servo_DI)

■　**目的**

取得伺服驅動器(RM04PI_Md2)DI 訊號的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
servo_DI	U16*	數值	驅動器上(RM04PI_Md2)DI1~DI8 訊號的值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 servo_DI;

I16 status = _DMC_01_get_servo_DI (CardNo, NodeID, SlotID, &servo_DI);
```

NOTE

04PI_Md2(01PH)、Servo、Servo_DI 說明

		MEL	PEL	QRG	SLD	QZ	DI2	DI1
Bit	7	6	5	4	3	2	1	0

32.5　_DMC_01_get_servo_DO

■　**格式**

I16 PASCAL _DMC_01_get_servo_DO (U16 CardNo, U16 NodeID, U16 SlotID, U16 *servo_DO)

■　**目的**

取得伺服驅動器(04PI_Md2)DO 訊號的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
servo_DO	U16*	數值	驅動器上(RM04PI_Md2)DO1~DO5 訊號的值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 servo_DO;

I16 status = _DMC_01_get_servo_DO (CardNo, NodeID, SlotID, &servo_DO);
```

NOTE

04PI_Md2(01PH)、Servo　Servo_DO 說明



（此頁有意留為空白）

第三十三章　警報訊息 API

表 33.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_set_ralm	輸出的伺服驅動器警示訊息重置
A/B/F	_DMC_01_get_alm_code	取得 Slave 的錯誤資訊
A/B/F	_DMC_01_master_alm_code	取得 Master Card 連線的錯誤資訊
A/B/F	_DMC_01_slave_error	取得 slave 通訊中連續錯誤次數
A/B/F	_DMC_01_set_ralm_servo_all	重置該卡串接伺服驅動器之警示訊息

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

33.1 _DMC_01_set_ralm

■ **格式**

I16 PASCAL _DMC_01_set_ralm (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

輸出的伺服驅動器(包含 RM04PI)警示訊息重置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID =1;
U16 SlotID =0;

I16 status = _DMC_01_set_ralm (CardNo, NodeID, SlotID);
```

NOTE

由於_DMC_01_set_ralm 這個指令執行，需要一些時間，如果習慣在下運動指令之前都去執行_DMC_01_set_ralm，那麼常會造成系統反應不及時，請特別注意用法。

33.2　_DMC_01_get_alm_code

■　**格式**

I16 PASCAL _DMC_01_get_alm_code (U16 CardNo, U16 NodeID, U16 SlotID, U32 *alm_code)

■　**目的**

取得 Slave 的錯誤資訊。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
alm_code	U32*	數值單位	Slave 的錯誤代碼

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo=0, NodeID=1, SlotID=0;
U32 alm_code;

/* alm_code 的值，表示 Slave 的錯誤資訊，關於驅動器的錯誤碼，請參閱驅動器相關手冊 */
I16 status = _DMC_01_get_alm_code (CardNo, NodeID, SlotID, &alm_code);
```

NOTE

非驅動器部分，Error Code 說明如下：

錯誤代碼	錯誤敘述
0x09	**位置控制誤差量大於設定容許值時**
0x14	**負極限開關訊號觸發**
0x15	**正極限開關訊號觸發**
0x97	**外部 ALARM 訊號觸發**
0x283	**軟體正極限訊號**
0x285	**軟體負極限訊號**
0x299	**請參照** 28.16 節 _DMC_01_rm_04pi_md1_get_mc_error_code **使用**

33.3 _DMC_01_master_alm_code

■ **格式**

I16 PASCAL _DMC_01_master_alm_code(U16 CardNo,U16* alm_code)

■ **目的**

取得 Master Card 連線的錯誤資訊。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
alm_code	U16*	數值	Master Card 連線失敗的錯誤代碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 alm_code;

I16 status = _DMC_01_master_alm_code (CardNo , &alm_code);
```

NOTE

錯誤代碼一覽

錯誤代碼	錯誤敘述
0	正常
0X185 == 389	通訊不穩定

33.4 _DMC_01_slave_error

■ **格式**

I16 PASCAL _DMC_01_slave_error (U16 CardNo, U16 NodeID,U16 SlotID,U16* alm_cnt)

■ **目的**

取得 slave 通訊中連續錯誤次數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
alm_cnt	U16*	次數	通訊中 slave 連續錯誤的次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●	●	●	●	●

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 alm_cnt;

I16 status = _DMC_01_slave_error (CardNo, NodeID, SlotID, &alm_cnt);
```

33.5　_DMC_01_set_ralm_servo_all

■　**格式**

I16 PASCAL _DMC_01_set_ralm_servo_all (U16 CardNo)

■　**目的**

重置該卡串接伺服驅動器之警示訊息

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■　**範例**

```
U16 CardNo=1;

I16 status = _DMC_01_set_ralm_servo_all (CardNo);
```

33.6　_DMC_01_set_limit_auto_ralm

■ **格式**

I16 _ PASCAL _DMC_01_set_limit_auto_ralm(U16 CardNo,U16 enable)

■ **目的**

在運動指令下達前,自動重置該卡串接之伺服驅動器(包含 RM04PI)由正反極限產生的警示訊息

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
enable	U16	編號單位	1: 啟動, 0:除能

:

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=1;

I16 status =_DMC_01_set_limit_auto_ralm(CardNo, 1)

第三十四章　多軸運動控制 API

表 34.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_multi_axes_move	設置 2 軸以上的運動控制
A/B/F	_DMC_01_liner_speed_master	多軸直線運動運動速度設定
A/B/F	_DMC_01_start_v3_multi_axes	增加 EndVel 的多軸(2 軸以上)運動控制

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

34.1 _DMC_01_multi_axes_move

■ **格式**

I16 PASCAL _DMC_01_multi_axes_move(U16 CardNo,U16 AxisNum, U16* NodeID, U16* SlotID, I32 *DistArrary, I32 StrVel, I32 MaxVel, F64 Tacc, F64 Tdec,U16 m_curve, U16 m_r_a)

■ **目的**

設置 2 軸以上的運動控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
AxisNum	U16	編號單位	多軸行程數量
NodeID	U16*	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	U16*	編號單位	插槽 ID SlotID[0] 存放第一組插槽 ID SlotID[1] 存放第二組插槽 ID SlotID[2] 存放第三組插槽 ID
DistArrary	I32*	脈波數陣列	各軸所需執行的行程 DistArrary [0] 存放第一組運動的行程 DistArrary [1] 存放第二組運動的行程 DistArrary [2] 存放第三組運動的行程
StrVel	I32	脈波數/秒	初始速度
MaxVel	I32	脈波數/秒	最大速度
Tacc	F64	秒	指定的加速時間
Tdec	F64	秒	指定的減速時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
U16 CardNo=0, AxisNum=4;
U16 NodeID[4]={1,2,3,4}, SlotID[4]={0,0,0,0};
I32 DistArrary[4]={1000, 2000, 3000, 6000};
I32 StrVel=1000, MaxVel=50000;
F64 Tacc=0.1, Tdec=0.1;
U16 m_curve=1;
U16 m_r_a=0;

/*設置為參照 T-curve 速度剖面下的相對座標進行多軸運動控制。*/
I16 status = _DMC_01_multi_axes_move(CardNo, AxisNum, NodeID, SlotID, DistArrary,
StrVel, MaxVel, Tacc, Tdec, m_curve, m_r_a);
```

34.2　_DMC_01_liner_speed_master

■　**格式**

I16 PASCAL _DMC_01_liner_speed_master (U16 CardNo, U16 NodeID, U16 SlotID, U16 Mode)

■　**目的**

多軸直線運動(Line2、Line3、Multi_Axis)運動速度設定，原本(Mode=0)運動速度的設定是向量速度，(Mode=1)的運動速度設定為行走距離最遠軸的最大分量速度(只對 Master 軸下達即可，其它軸速度會根據 Master 軸的值自動算出)。

■　**參數**

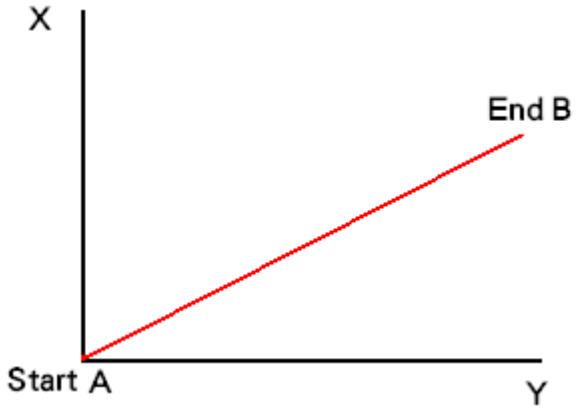
名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：向量速度(Default 設定)。 1：行走距離最遠軸的最大分量速度。

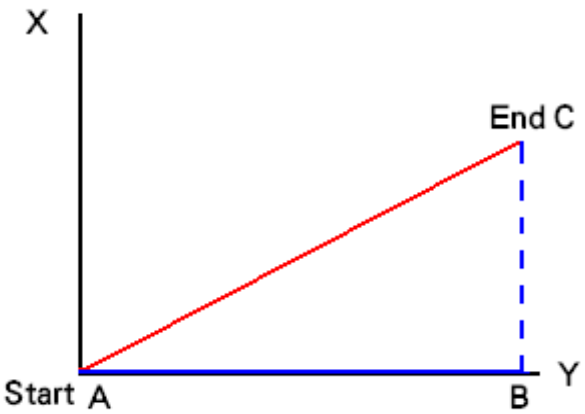
■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**敘述**

Mode 參數設定示意





■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Mode=1;

I16 status = _DMC_01_liner_speed_master (CardNo, NodeID, SlotID, Mode);
```

34.3 _DMC_01_start_v3_multi_axes

■ **格式**

I16 PASCAL _DMC_01_start_v3_multi_axes(U16 CardNo,U16 AxisNum, U16* NodeID, U16* SlotID, I32 *DistArrary, I32 StrVel, I32 ConstVel, I32 EndVel, F64 TPhase1, F64 TPhase2, U16 m_curve,U16 m_r_a)

■ **目的**

增加 EndVel 的多軸(2 軸以上)運動控制。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

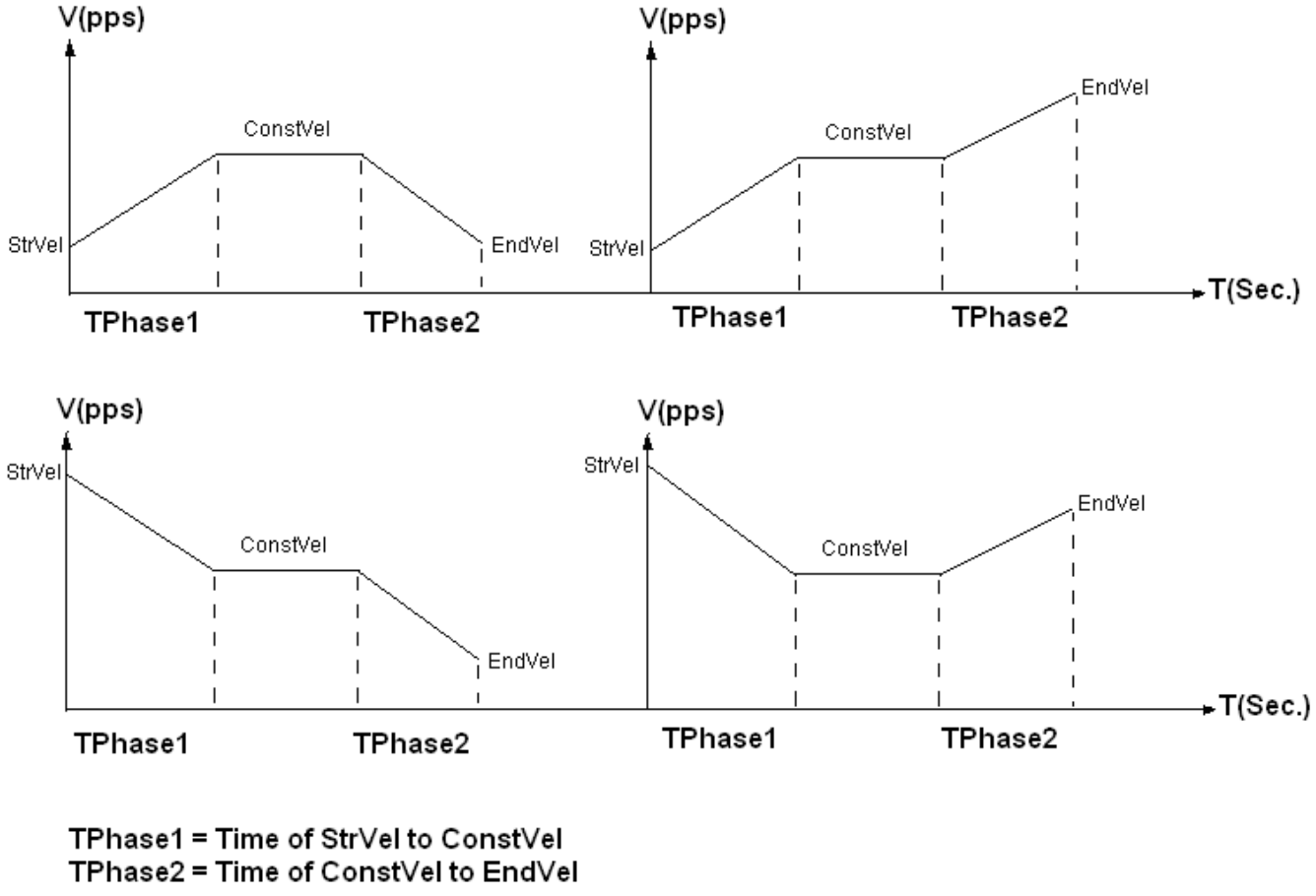
■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
AxisNum	U16	編號單位	多軸行程數量
NodeID	U16*	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	U16*	編號單位	插槽 ID SlotID[0] 存放第一組插槽 ID SlotID[1] 存放第二組插槽 ID SlotID[2] 存放第三組插槽 ID
DistArrary	I32*	脈波數陣列	各軸所需執行的行程 DistArrary [0] 存放第一組運動的行程 DistArrary [1] 存放第二組運動的行程 DistArrary [2] 存放第三組運動的行程
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**



TPhase1 = Time of StrVel to ConstVel
TPhase2 = Time of ConstVel to EndVel

圖 34.3 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0, AxisNum=4;
U16 NodeID[4]={1,2,3,4}, SlotID[4]={0,0,0,0};
I32 DistArrary[4]={1000, 2000, 3000, 6000};
I32 StrVel=1000, ConstVel =50000;
I32 EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=0;

I16 status = _DMC_01_start_v3_multi_axes (CardNo, AxisNum, NodeID, SlotID, DistArrary,
                StrVel, ConstVel, EndVel, TPhase1, TPhase2, m_curve, m_r_a);
```

34.4 _DMC_01_start_v3_msbrline_move

■ **格式**

I16 PASCAL _DMC_01_start_v3_msbrline_move(U16 CardNo,U16 AxisNum, U16* NodeID,U16* SlotID,U16 ArcNodeBit,I32* DistArray,I32* DistArray2,U16 Mode,F64 Parameter,F64 SpdRatio,F64 Angle1,F64 Angle2,I32 StrVel,I32 ConstVel,I32 EndVel,F64 Tphase1,F64 Tphase2,U16 m_curve,U16 m_r_a);

■ **目的**

指定 EndVel 的多軸(2 軸以上)運動控制。移動的路徑是由目前位置經過第一組目標位置到達第二組目標位置所形成的弧線。

※StrVel 與 EndVel 的值，容許比 ConstVel 的值大。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
AxisNum	U16	編號單位	多軸行程數量
NodeID	U16*	編號單位	站號 ID NodeID[0] 存放第一組站號 NodeID[1] 存放第二組站號 NodeID[2] 存放第三組站號
SlotID	U16*	編號單位	插槽 ID SlotID[0] 存放第一組插槽 ID SlotID[1] 存放第二組插槽 ID SlotID[2] 存放第三組插槽 ID
ArcNodeBit	U16*	編號單位	判斷銜接的運動是否為 Arc 預設值為 0 Bit0~7：代表 M1 的站號 1~8，如果是 Arc 軸則值為 1，否則 0，Bit8~15 代表 M2 同義
DistArrary	I32*	脈波數陣列	各軸所需執行的行程 DistArrary [0] 存放第一組運動的行程 DistArrary [1] 存放第二組運動的行程 DistArrary [2] 存放第三組運動的行程
DistArrary2	I32*	脈波數陣列	DistArrary2 [0] 存放第一組運動的行程 DistArrary2 [1] 存放第二組運動的行程 DistArrary2 [2] 存放第三組運動的行程
Mode	U16	選項單位	0:M1、M2 均為直線, (Default) 1:M1 為直線、M2 為圓弧, 2:M1 為圓弧、M2 為直線, 3:M1、M2 為圓弧
Parameter	F64	選項單位	行程誤差量，誤差定義為與第一目標點的最短

			距離。此值過大時，將會自動調整至合理範圍(固定起始點以及第二目標點)
SpdRatio	F64	選項單位	M1 與 M2 的速度比率，相當於 SpdM2 / SpdM1。調整兩段運動上面的速度變化。 預設值為 0
Angle1	F64	選項單位	當 M1 為 Arc 時的角度， M1 為 Line 此值不會被使用。預設值為 0
Angle2	F64	選項單位	當 M2 為 Arc 時的角度， M2 Line，此值不會被使用。預設值為 0
StrVel	I32	脈波數/秒	初始速度
ConstVel	I32	脈波數/秒	常態速度
EndVel	I32	脈波數/秒	運動結束速度
TPhase1	F64	秒	StartVel 到 ConstVel 的時間
TPhase2	F64	秒	ConstVel 到 EndVel 的時間
m_curve	U16	選項單位	1：T-curve(Default) 2：S-curve
m_r_a	U16	選項單位	0：相對位移(Default) 1：絕對位移

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **描述**

可將兩個有折角的運動(Motion1(簡稱 m1) & Motion2(簡稱 m2))(無論圓或直線)合併成為一個運動(MSBR)的函式

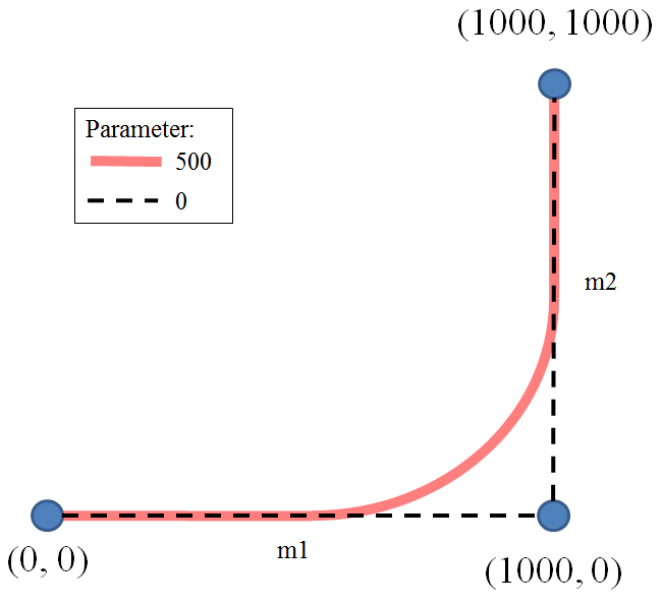


圖 34.40 Parameter 說明

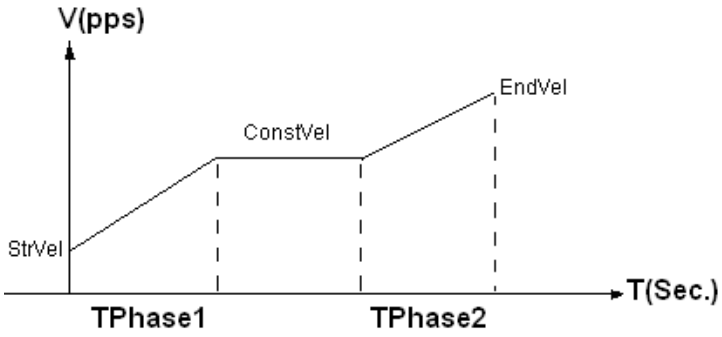


圖 34.41 TPhase1 與 TPhase2 說明

■ **範例**

```
U16 CardNo=0, AxisNum=3;
U16 NodeID[3]={1,2,3}, SlotID[3]={0};
I32 DistArrary[12]={1000, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
I32 DistArrary2[12]={1000, 1000, 1000, 0, 0, 0, 0, 0, 0, 0, 0, 0};
U16 Mode=0;
F64 Parameter=500, SpdRatio=0, Angle1=0, Angle2=0;
I32 StrVel=1000, ConstVel =50000, EndVel=20000;
F64 TPhase1=0.2;
F64 TPhase2=0.1;
U16 m_curve=1, m_r_a=1;

I16 status = _DMC_01_start_v3_msbrline_move(CardNo, AxisNum, NodeID, SlotID,
                ArcNodeBit, DistArray, DistArray2, Mode, Parameter, SpdRatio, Angle1, Angle2,
                StrVel, ConstVel, EndVel, Tphase1, Tphase2, m_curve, m_r_a);
```

（此頁有意留為空白）

第三十五章　緩衝區操作 API

表 35.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_trigger_buf_function	使用伺服驅動器 DI3(SLD)觸發 Motion 指令

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

35.1　_DMC_01_set_trigger_buf_function

■　**格式**

I16 PASCAL _DMC_01_set_trigger_buf_function (I16 CardNo, U16 NodeID, U16 SlotID, U16 enable)

■　**目的**

使用伺服驅動器 DI3（SLD）觸發 Motion 指令。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	U16	選項單位	0：buffer 只要有 motion 指令就執行 1：利用 DI 觸發 buffer 裏的 motion 指令

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●						

■　**範例**

I16 CardNo=0

U16 NodeID=1, SlotID=0;

U16 enable=1; //啟動此函式功能。將 DI3（SLD）觸發取得 Buffer 裏的 motion 指令

I16 status = _DMC_01_set_trigger_buf_function (CardNo, NodeID, SlotID, enable);

第三十六章 Interrupt API

表 36.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_set_int_factor	設定中斷模式，共有 8 種模式。
A/B	_DMC_01_int_enable	中斷設定致能。
A/B	_DMC_01_int_disable	中斷設定解除。
A/B	_DMC_01_get_int_count	中斷累計次數
A/B	_DMC_01_get_int_status	讀取目前中斷觸發的事件值
A/B	_DMC_01_Link_interrupt	連結一處理程序，當設定之中斷發生就導入處理。

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

36.1 _DMC_01_set_int_factor

■ **格式**

I16 PASCAL _DMC_01_set_int_factor (U16 CardNo, U16 NodeID, U16 int_factor)

■ **目的**

設定中斷的模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
int_factor	U16	選項單位	共有 8 種模式: 1. Normal Stop. 2. Next Buffer. 4. Acceleration End. 8. Deceleration Start. 16. Sdo Finish.(目前未開放) 32. DMC Cycle Start. (目前未開放) 64. RM04PI-FIFO. (目前未開放) 128. User Define.(目前未開放)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 int_facter=5; //Normal Stop + Acceleration End.

I16 status = _DMC_01_set_int_factor (CardNo, NodeID, int_factor);
```

36.2 _DMC_01_int_enable

■ **格式**

I16 PASCAL _DMC_01_int_enable (U16 CardNo, U16 NodeID)

■ **目的**

設定中斷致能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;

I16 status = _DMC_01_int_enable (CardNo, NodeID);
```

36.3 _DMC_01_int_disable

■ **格式**

I16 PASCAL _DMC_01_int_disable (U16 CardNo, U16 NodeID)

■ **目的**

設定中斷禁能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;

I16 status = _DMC_01_int_disable (CardNo, NodeID);
```

36.4 _DMC_01_get_int_count

■ **格式**

I16 PASCAL _DMC_01_get_int_count (U16 CardNo, U16 NodeID, U16 count)

■ **目的**

讀取中斷達成次數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
count	U16	中斷次數	達成中斷的次數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16*count;

I16 status =_DMC_01_get_int_count (CardNo, NodeID, &count);
```

36.5 _DMC_01_get_int_status

■ **格式**

I16 PASCAL _DMC_01_get_int_status (U16 CardNo, U16 NodeID, U16 event_int_status)

■ **目的**

讀取目前中斷觸發的事件值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
Event_int_status	U16	選項單位	目前中斷觸發的事件值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●	●	●				

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 event_int_status;//參考 36.1 模式設定。

I16 status = _DMC_01_get_int_status (CardNo, NodeID, & event_int_status);

36.6 _DMC_01_link_interrupt

■ **格式**

I16 PASCAL _DMC_01_link_interrupt (U16 CardNo, void (__stdcall *callbackAddr)
(U16 CardNo, U16 NodeID))

■ **目的**

設定一處理程序，當中斷條件發生就進入此程序處理。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
callbackAddr	U16	處理程序	預設之中斷處理程序

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
__stdcall * CallBack;

void _stdcall CallBack(U16 CardNo, U16 NodeID)
{
    U16 i;
    i = CardNo + NodeID;
}

I16 status = _DMC_01_link_interrupt (CardNo, CallBack);
```

（此頁有意留為空白）

第三十七章 Security API

表 37.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_read_security	Master Card 讀取指定記憶體區塊中的安全驗證資料
A/B	_DMC_01_read_security_status	Master Card 獲取目前的記憶體是處於讀/寫的狀態
A/B	_DMC_01_write_security	Master Card 將指定的安全資料寫入指定的記憶體區塊
A/B	_DMC_01_write_security_status	Master Card 寫入安全資料前，將記憶體寫入功能致能
A/B/F	_DMC_01_check_userpassword	Master Card 確認使用者對 Memory 讀/寫的權限
A/B/F	_DMC_01_write_userpassword	Master Card 變更密碼
A/B/F	_DMC_01_check_verifykey	Master Card 驗證碼驗證
A/B/F	_DMC_01_write_verifykey	Master Card 寫入驗證碼
A/B/F	_DMC_01_read_serialno	Master Card 讀取產品序號
A/B/F	_misc_slave_check_userpassword	Slave(04PI)確認使用者對 Memory 讀/寫的權限
A/B/F	_misc_slave_write_userpassword	Slave(04PI)變更密碼
A/B/F	_misc_slave_get_serialno	Slave(04PI)讀取產品序號。
A/B/F	_misc_security	使用者 Key+SerialNo 經加密產生驗證碼
A/B/F	_misc_slave_write_verifykey	Slave(04PI)寫入驗證碼
A/B/F	_misc_slave_check_verifykey	Slave(04PI)驗證碼驗證
A/B/F	_misc_slave_user_data_buffer_read	Slave(04PI)讀取指定記憶體區塊中的資料
A/B/F	_misc_slave_user_data_buffer_write	Slave(04PI)將指定的資料寫入指定的記憶體區塊
A/B/F	_misc_slave_user_data_to_flash	Slave(04PI)將資料由 Bufeer 寫入 Flash 中

註 A:PCI_DMC_A01 B:PCI_DMC_B01 F:PCI_DMC_F01

37.1 _DMC_01_read_security

■ **格式**

I16 PASCAL _DMC_01_read_security (U16 CardNo, U16 page, U16 *array)

■ **目的**

讀取 Master Card 指定記憶體中的安全性資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Page	U16	編號單位	指定讀取的記憶體頁數編號
array	U16*	數值	由指定讀取的記憶體中讀回的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 Page=0;     // 0~9 共 10 頁
U16 array;

I16 status = _DMC_01_read_security ( CardNo, page ,&array);
```

37.2 _DMC_01_read_security_status

■ **格式**

I16 PASCAL _DMC_01_read_security_status (U16 CardNo, U16 *status)

■ **目的**

讀取 Master Card 目前記憶體是處於讀或寫的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
status	U16*	旗標	4(Read Only) 或 6(Read/Write)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 status;

I16 status = _DMC_01_read_seacurity_status (CardNo, &status );
```

37.3 _DMC_01_write_security

■ **格式**

I16 PASCAL _DMC_01_write_security (U16 CardNo, U16 page, U16 array)

■ **目的**

將安全資料寫入 Master Card 指定的記憶體區塊。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Page	U16	編號單位	指定寫入的記憶體頁數編號
array	U16	數值	要寫入指定記憶體的安全資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 page=0; // 0~9 共 10 頁
U16 array={1,4,7,11,0a,ff,12,8,0b,10,3,5,c1,14,0d,6};

//執行此 API 前，須先執行_DMC_01_write_security_status (CardNo, 1)將讀/寫狀態變更為
  寫入狀態。
I16 status =_DMC_01_wtite_security( CardNo, page, array );
```

NOTE

注意:執行完_DMC_01_wtite_security 這個 API 後，Dll 會自動將讀/寫狀態變更為 Read Only 狀態，所以要再寫入資料，須重新使用_DMC_01_write_security_status (CardNo, 1)將讀/寫狀態變更為寫入狀態，然後再執行_DMC_01_wtite_security 寫入資料。

37.4 _DMC_01_write_security_status

■ **格式**

I16 PASCAL _DMC_01_write_security_status (U16 CardNo, U16 status)

■ **目的**

Master Card 在寫入安全資料前，將記憶體寫入功能致能。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
status	U16	旗標	將記憶體寫入功能致能(0:disable,1:enable)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U16 status =1;

I16 status = _DMC_01_write_security_status (CardNo, status);
```

37.5 _DMC_01_check_userpassword

■ **格式**

I16 PASCAL _DMC_01_check_userpassword (U16 CardNo, U32 *password_data, U16 *password_state)

■ **目的**

Master Card 在讀/寫資料前，確認使用者對 Memory 讀/寫的權限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Password_data	U32*	數值	輸入 Check 的密碼
Password_state	U16*	旗標	Check 密碼後的回覆，0→Fail 1→OK

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 password_data[2]={0x12345678,0xabcdef};    //64bit data
U16 password_state;

I16 status = _DMC_01_check_userpassword ( CardNo, password_data, &password_state );
```

37.6 _DMC_01_write_userpassword

■ **格式**

I16 PASCAL _DMC_01_write_userpassword (U16 CardNo, U32 password_data)

■ **目的**

Master Card 變更使用者密碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Password_data	U32	數值	輸入要變更的密碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 password_data[2]={0x12345678,0xabcdef};    //64bit data

I16 status = _DMC_01_write_userpassword ( CardNo, password_data );
```

37.7 _DMC_01_check_verifykey

■ **格式**

I16 PASCAL _DMC_01_check_verifykey (U16 CardNo, U32 *Verifykey, U16*state)

■ **目的**

Master Card 將驗證碼帶入驗證是否相符。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Verifykey	U32*	數值	要驗證的驗證碼資料
State	U16*	旗標	Check 驗證碼後的回覆，0→Lock 1→Pass

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef};      //128bit data
U16 sate;

I16 status = _DMC_01_check_verifykey ( CardNo, Verifykey, &state );
```

37.8 _DMC_01_write_verifykey

■ **格式**

I16 PASCAL _DMC_01_write_verifykey (U16 CardNo, U32 *Verifykey)

■ **目的**

將驗證碼寫入 Master Card。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Verifykey	U32*	數值	要寫入的驗證碼資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef};     //128bit data

I16 status = _DMC_01_write_verifykey (CardNo, Verifykey);
```

37.9 _DMC_01_read_serialno

■ **格式**

I16 PASCAL _DMC_01_read_serialno (U16 CardNo, U32 *Serialno)

■ **目的**

讀取 Master Card 記憶體中的產品序號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Serialno	U32*	數值	存於記憶體中的產品序號資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 Serialno[2];

I16 status = _DMC_01_read_serialno ( CardNo, &Serialno[0] );
```

37.10 _misc_slave_check_userpassword

■ **格式**

I16 PASCAL _misc_slave_check_userpassword (U16 CardNo, U16 NodeID, U16 SlotID , U32 Password_data, U16 *Password_state)

■ **目的**

Slave(04PI)在讀/寫資料前，確認使用者對記憶體讀/寫的權限。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Password_data	U32	數值	輸入 Check 的密碼
Password_state	U16*	旗標	Check 密碼後的回覆，0→Fail 1→OK

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Password_data[2]={0x12345678,0xabcdef};      //64bit data
U16 Password_state;

I16 status = _misc_slave_check_userpassword (CardNo, NodeID, SlotID, Password_data,
             &Password_state);
```

37.11 _misc_slave_write_userpassword

■ **格式**

I16 PASCAL _misc_slave_write_userpassword (U16 CardNo, U16 NodeID, U16 SlotID ,U32 Password_data)

■ **目的**

Slave(04PI)寫入使用者密碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Password_data	U32	數值	要寫入記憶體的密碼變更資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Password_data[2]={0x12345678,0xabcdef};

I16 status = _misc_slave_write_userpasswaord (CardNo, NodeID, SlotID, Password_data);

37.12 _misc_slave_get_serialno

■ **格式**

I16 PASCAL _misc_slave_get_serialno (U16 CardNo, U16 NodeID, U16 SlotID ,U32 *Serialno)

■ **目的**

讀取 Slave(04PI)產品序號。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Serialno	U32*	數值	讀取記憶體中的產品序號資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Serialno[2];

I16 status = _misc_slave_get_serialno (CardNo, NodeID, SlotID, &Serialno);
```

37.13 _misc_security

■ **格式**

I16 PASCAL _misc_security (U32 OtherWord0, U32 OtherWord1, U32 SyntekWord0, U32 SyntekWord1, U32 *Password0, U32 *Password1, U32 *Password2, U32 *Password3)

■ **目的**

Slave(04PI)由使用者自訂 64bit key 值與 64bit Serialno 帶入加密函數中得出 128bit 的驗證碼。

■ **參數**

名稱	資料型別	單位	描述
OtherWord0	U32	數值	32bit(0)使用者自訂 key 值
OtherWord1	U32	數值	32bit(1)使用者自訂 key 值
SyntekWord0	U32	數值	32bit (0)Serialno
SyntekWord1	U32	數值	32bit (1)Serialno
Password0	U32*	數值	加密函數得出 32bit(0)的驗證碼
Password1	U32*	數值	加密函數得出 32bit(1)的驗證碼
Password2	U32*	數值	加密函數得出 32bit(2)的驗證碼
Password3	U32*	數值	加密函數得出 32bit(3)的驗證碼

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U32 OtherWord0; //32bit
U32 OtherWord1; //32bit
U32 SyntekWord0; //32bit
U32 SyntekWord1; //32bit
U32 Password0[2];
U32 Password1[2];
U32 Password2[2];
U32 Password3[2];

I16 status = _misc_security (OtherWord0, OtherWord1, SyntekWord0, SyntekWord1,
        &Password0, &Password1, &Password2, &Password3);
```

37.14 _misc_slave_write_verifykey

■ **格式**

I16 PASCAL _misc_slave_write_verifykey (U16 CardNo, U16 NodeID, U16 SlotID ,U32 Verifykey)

■ **目的**

Slave(04PI)寫入驗證碼。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Verifykey	U32	數值	將驗證碼寫入記憶體

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef };    //128bit data

I16 status = _misc_slave_write_verifykey (CardNo, NodeID, SlotID, Verifykey);
```

37.15 _misc_slave_check_verifykey

■ **格式**

I16 PASCAL _misc_slave_check_verifykey (U16 CardNo, U16 NodeID, U16 SlotID , U32 Verifykey, U16 *Lock_state)

■ **目的**

將驗證碼帶入 Slave(04PI)驗證是否相符。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Verifykey	U32	數值	要驗證的驗證碼資料
Lock_state	U16*	旗標	Check 驗證碼後的回覆 0：LOCK 1：PASS

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U32 Verifykey[4]={0x12345678,0xabcdef, 0x12345678,0xabcdef };      //128bit data
U16 Lock_state;

I16 status =_misc_slave_check_verifykey (CardNo, NodeID, SlotID, Verifykey, &Lock_state);
```

37.16 _misc_slave_user_data_buffer_read

■ **格式**

I16 PASCAL _misc_slave_user_data_buffer_read (U16 CardNo, U16 NodeID, U16 SlotID U16 Address, U32* Data)

■ **目的**

讀取 Slave(04PI)指定 Page 的記憶體資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Address	U16	數值	要讀取的位置
Data	U32*	數值	記憶體儲存的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Address;
U32 Data[2];

I16 status = _misc_slave_user_data_buffer_read (CardNo, NodeID, SlotID, Address, &Data);
```

37.17 _misc_slave_user_data_buffer_write

■ **格式**

I16 PASCAL _misc_slave_user_data_buffer_write (U16 CardNo, U16 NodeID, U16 SlotID U16 Address, U32 Data)

■ **目的**

將資料寫入暫存區中。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Address	U16	數值	要寫入的位置資料
Data	U32	數值	要寫入的資料

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 Address;
U32 Data;

I16 status = _misc_slave_user_data_buffer_write (CardNo, NodeID, SlotID, Address, Data);
```

37.18 _misc_slave_user_data_to_flash

■ **格式**

I16 PASCAL _misc_slave_user_data_to_flash (U16 CardNo, U16 NodeID, U16 SlotID)

■ **目的**

將資料由暫存區寫入 Slave(04PI)指定位置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;

I16 status = _misc_slave_user_data_to_flash (CardNo, NodeID, SlotID);
```

（此頁有意留為空白）

第三十八章　極限反向 API

表 38.1

API 歸屬	函數名稱	描述
A/B/F	_DMC_01_rm_04pi_set_MEL_polarity	設定 RM04PI 負極限正向/反向
A/B/F	_DMC_01_rm_04pi_get_MEL_polarity	取得 RM04PI 負極限目前的狀態
A/B/F	_DMC_01_rm_04pi_set_PEL_polarity	設定 RM04PI 正極限正向/反向
A/B/F	_DMC_01_rm_04pi_get_PEL_polarity	取得 RM04PI 正極限目前的狀態
A/B/F	_DMC_01_set_01ph_alm_polarity	設定 GE01PH ALM 準位
A/B/F	_DMC_01_set_01ph_org_polarity	設定 RM04PI/GE01PH ORG 準位

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

38.1　_DMC_01_rm_04pi_set_MEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_set_MEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 inverse)

■ **目的**

將 RM04PI 負極限反向，此設定在 RM04PI 模組斷電重開後將回復為預設值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Inverse	U16	選項資料	0：正向 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 inverse=1; //設定負極限反向

I16 status = _DMC_01_rm_04pi_set_MEL_polarity (CardNo, NodeID, SlotID, inverse);
```

38.2　_DMC_01_rm_04pi_get_MEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_get_MEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 *data)

■ **目的**

取得 RM04PI 負極限目前的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Data	U16	選項資料	0：正向 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 data;

I16 status = _DMC_01_rm_04pi_get_MEL_polarity (CardNo, NodeID, SlotID, &data);
```

38.3　_DMC_01_rm_04pi_set_PEL_polarity

■　**格式**

I16 PASCAL _DMC_01_rm_04pi_set_PEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 inverse)

■　**目的**

將 RM04PI 正極限反向，此設定在 RM04PI 模組斷電重開後將回復為預設值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Inverse	U16	選項資料	0：正向 1：反向

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■　**範例**

```
U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 inverse=1;   //設定正極限反向

I16 status = _DMC_01_rm_04pi_set_PEL_polarity (CardNo, NodeID, SlotID, inverse);
```

38.4　_DMC_01_rm_04pi_get_PEL_polarity

■ **格式**

I16 PASCAL _DMC_01_rm_04pi_get_PEL_polarity (U16 CardNo, U16 NodeID, U16 SlotID U16 *data)

■ **目的**

取得 RM04PI 正極限目前的狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	選項單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Data	U16	旗標	0：正向; 1：反向

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●	●				

■ **範例**

U16 CardNo=0;
U16 NodeID=1;
U16 SlotID=0;
U16 data;

I16 status = _DMC_01_rm_04pi_get_PEL_polarity (CardNo, NodeID, SlotID, &data);

38.5　_DMC_01_set_01ph_alm_polarity

■　**格式**

I16 PASCAL _DMC_01_set_01ph_alm_polarity(I16 CardNo, U16 NodeID, U16 SlotID, U16 inverse)

■　**目的**

設定 GE01PH ALM 準位，此設定在 GE01PH 模組斷電重開後將回復為預設值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
inverse	U16	選項單位	0：High 電位觸發 1：Low 電位觸發

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●GE01PH Only				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 inverse=0 ;// High 電位觸發。

I16 status = _DMC_01_set_01ph_alm_polarity(CardNo, NodeID, SlotID, inverse);
```

38.6　_DMC_01_set_01ph_org_polarity

■　**格式**

I16 PASCAL _DMC_01_set_01ph_org_polarity(I16 CardNo, U16 NodeID, U16 SlotID, U16 inverse)

■　**目的**

設定 RM04PI/GE01PH ORG 準位，此設定在 RM04PI/GE01PH 模組斷電重開後將回復為預設值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
inverse	U16	選項單位	0：High 電位觸發 1：Low 電位觸發

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
			●				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 inverse=0 ;// High 電位觸發。

I16 status = _DMC_01_set_01ph_org_polarity(CardNo, NodeID, SlotID, inverse);
```

（此頁有意留為空白）

第三十九章 Compare API

表 39.1

函數名稱	描述
_DMC_01_set_compare_channel_position	設定該 Channel 新的 Position 計數值
_DMC_01_get_compare_channel_position	讀取該 Channel 目前的 Position 計數值
_DMC_01_set_compare_ipulser_mode	設定脈波介面模組輸入相位模式
_DMC_01_set_compare_channel_direction	設定該 Channel 脈波方向
_DMC_01_set_compare_channel_trigger_time	設定 Trigger 致能持續時間
_DMC_01_set_compare_channel_one_shot	設定為執行一次 Trigger 致能狀態
_DMC_01_set_compare_channel_source	設定其比較來源
_DMC_01_channel0_position_cmp	執行 Compare Type 為 Compare0
_DMC_01_channel1_output_enable	設定 Compare1 輸出開啟/關閉
_DMC_01_channel1_output_mode	Compare1 設定輸出模式
_DMC_01_channel1_get_io_status	讀取 Compare1 狀態
_DMC_01_channel1_set_gpio_out	設置 GPIO 輸出腳位狀態
_DMC_01_channel1_position_compare_table	Compare1 設定一般 Compare 相關資料
_DMC_01_channel1_position_compare_table_level	Compare1 設定自訂 Compare 相關資料
_DMC_01_channel1_position_compare_table_cnt	讀取已 Compare 計數次數
_DMC_01_set_compare_channel_polarity	設定 Compare 的準位
_DMC_01_channel0_position_cmp_by_gpio	設定 Compare 的觸發條件由 GPIO 控制
_DMC_01_channel1_position_re_compare_table	用上次的 Compare 條件，重新執行一次 Channel1 的 Compare
_DMC_01_channel1_position_re_compare_table_level	用上次的 Compare 條件，重新執行一次 Channel1(Level 模式)的 Compare
_DMC_01_set_compare_channel_enable	設定 Compare 功能開啟/關閉
_DMC_01_channel1_get_fifo_status	取得目前 Compare Table 內資料的儲存狀態
_DMC_01_channel1_get_fifo_counter	取得目前 Compare Table 內資料的筆數

此章節 API 僅 PCI_DMC_B01 可使用

說明: QEP1 代表 編碼器來源 1　　QEP2 代表 編碼器來源 2

API名稱 xxxxxx _channel0_xxxxx 用於 等距 Compare 模式

xxxxxx _channel1_xxxxx 用於非等距 Compare 模式

39.1 _DMC_01_set_compare_channel_position

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_position (U16 CardNo,
U16 compare_channel, I32 position)

■ **目的**

設定該 QEP Channel 新的計數值。(此數值與模組之 Position 意義並不相等)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_Channel	U16	編號單位	Channel No 為 0~1
Position	I32	數值單位	欲設定之 QEP Channel 新的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Compare_Channel = 0;
I32 position = 0;

I16 status = _DMC_01_set_compare_channel_position (CardNo, compare_channel, position);
```

39.2 _DMC_01_get_compare_channel_position

■ **格式**

I16 PASCAL _DMC_01_get_compare_channel_position (U16 CardNo,
U16 compare_Channel, I32 *position)

■ **目的**

讀取該 QEP Channel 目前的計數值。(此數值與模組之 Position 意義並不相等)

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_Channel	U16	編號單位	Channel No 為 0~1
Position	I32*	脈波數	讀取該 QEP Channel 目前的計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Compare_channel = 0;
I32 position;

I16 status = _DMC_01_get_compare_channel_position (CardNo, compare_channel,
&position);
```

39.3 _DMC_01_set_compare_ipulser_mode

■ **格式**

I16 PASCAL _DMC_01_set_compare_ipulser_mode (U16 CardNo, U16 mode)

■ **目的**

設定脈波介面模組輸入相位模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Mode	U16	數值單位	0：AB Phase 1：CW/CCW

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 mode = 0;       //AB Phase

I16 status = _DMC_01_set_compare_ipulser_mode (CardNo, mode);
```

39.4 _DMC_01_set_compare_channel_direction

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_direction (U16 CardNo,
U16 compare_channel, U16 dir)

■ **目的**

設定該 QEP Channel 脈波方向。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
Dir	U16	數值單位	0：Normal 1：Inverse

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel = 0;
U16 dir = 1; //Inverse

I16 status = _DMC_01_set_compare_channel_direction (CardNo, compare_channel, dir);
```

39.5 _DMC_01_set_compare_channel_trigger_time

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_trigger_time (U16 CardNo, U16 compare_channel, U32 time_us)

■ **目的**

設定 Trigger 致能持續時間。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
Time_us	U32	時間單位	輸入每個 Trigger 致能持續時間

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel = 0;
U16 time_us = 20; //20us

I16 status = _DMC_01_set_compare_channel_trigger_time(CardNo, compare_channel, time_us);
```

NOTE

Compare Type0 狀態下，Trigger Time 設定值最小為 1us，若輸入值為 0 則顯示 0.8us。
Compare Type1 狀態下 Trigger Time 最小為 3us，若輸入值小於 3 則顯示 3us。

39.6 _DMC_01_set_compare_channel_one_shot

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_one_shot (U16 CardNo,
U16 compare_channel)

■ **目的**

設定為執行一次 Trigger 致能狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel=0;

I16 status = _DMC_01_set_compare_channel_one_shot (CardNo, compare_channel);
```

39.7 _DMC_01_set_compare_channel_source

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_source (U16 CardNo,
U16 compare_channel, U16 source)

■ **目的**

設定其比較來源。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
Source	U16	編號單位	0：QEP1 1：QEP2

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 compare_channel = 0;
U16 source = 0;

I16 status = _DMC_01_set_compare_channel_source (CardNo, compare_channel, source);
```

39.8 _DMC_01_channel0_position_cmp

■ **格式**

I16 PASCAL _DMC_01_channel0_position_cmp (U16 CardNo, I32 satrt, U16 dir, U16 interval, U32 trigger_cnt)

■ **目的**

執行 Compare Type0。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Start	I32	數值單位	Compare 起始位置
Dir	U16	數值單位	0：正向 1：反向
Interval	U16	脈波數	脈波 Compare 間隔數
Trigger_cnt	U32	數值單位	Trigger Compare 總數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 start = 100000;
U16 dir = 0;
U16 Interval = 10; //10 pulse
U32 trigger_cnt = 50000;

I16 status = _DMC_01_channel0_position_cmp (CardNo, start, dir, interval, trigger_cnt);
```

39.9 _DMC_01_channel1_output_enable

■ **格式**

I16 PASCAL _DMC_01_channel1_output_enable (U16 CardNo, U16 on_off)

■ **目的**

將 Compare Position 的存放陣列啟用，FPGA 內部的 Compare 功能才會開始以實體位置跟陣列的存放位置依序做比較。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
On_off	U16	數值單位	0：Off 1：On

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;
U16 on_off = 1;

I16 status = _DMC_01_channel1_output_enable (CardNo, on_off);

39.10 _DMC_01_channel1_output_mode

■ **格式**

I16 PASCAL _DMC_01_channel1_output_mode (U16 CardNo, U16 Mode)

■ **目的**

Compare Type1 設定輸出模式。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Mode	U16	數值單位	0：Normal 模式 1：自訂模式

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 mode = 1;

I16 status = _DMC_01_channel1_output_mode (CardNo, mode);
```

NOTE

Normal 模式

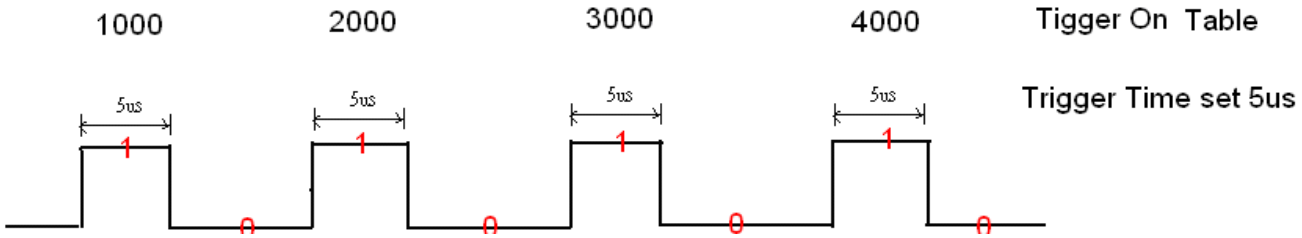


圖 39.1 Compare Output Normal 模式

※Trigger On 的位置表設定，是由 API「_DMC_01_channel1_position_compare_table」的"table_size"參數進行設定。

※Trigger 時間的設置，是由 API「_DMC_01_set_compare_channel_trigger_time」的"time_us"參數進行設定。

自訂模式

當設定為自訂模式，level table 的值設置為 0x88880000，如圖 39.2 所示。

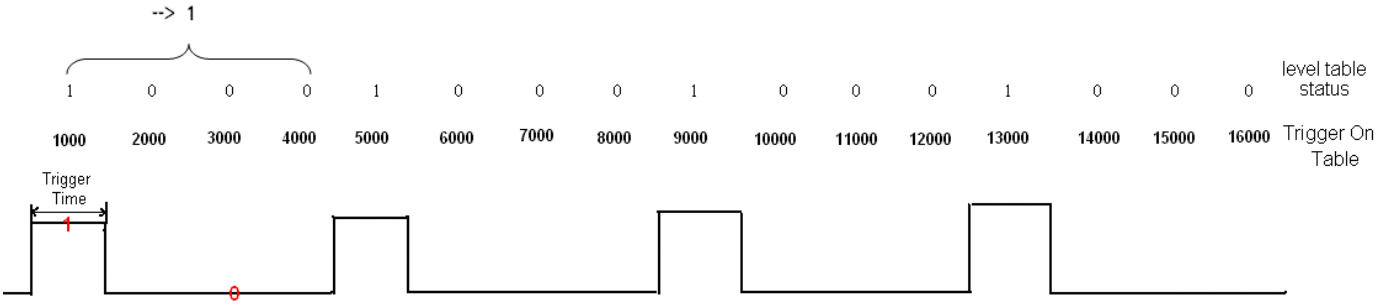


圖 39.2 Compare Output 使用者自訂模式

※自訂模式下 Trigger On 的位置表設定，

是由 API「_DMC_01_channel1_position_compare_table_level」的” level_table”參數進行設定。

※Trigger 時間則為到下一個位置所花費的時間（ex. 位置 1000 到位置 2000 花費的時間）。

39.11 _DMC_01_channel1_get_io_status

■ **格式**

I16 PASCAL _DMC_01_channel1_get_io_status (U16 CardNo, U16* io_status)

■ **目的**

讀取 Compare Type1 狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Io_status	U16*	數值單位	Compare Type1 狀態

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 io_status;

I16 status = _DMC_01_channel1_get_io_status (CardNo,& io_status);
```

39.12 _DMC_01_channel1_set_gpio_out

■ **格式**

I16 PASCAL _DMC_01_channel1_set_gpio_out (U16 CardNo, U16 on_off)

■ **目的**

設置 GPIO 輸出腳位狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
On_off	U16	數值單位	0：Off 1：On

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 on_off = 1;

I16 status = _DMC_01_channel1_set_gpio_out (CardNo, on_off);
```

39.13 _DMC_01_channel1_position_compare_table

■ **格式**

I16 PASCAL _DMC_01_channel1_position_compare_table (U16 CardNo, I32* pos_table, U32 table_size)

■ **目的**

設定 Compare Type1 的相關資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Pos_table	I32*	數值單位	依照資料計算出需 Compare 的 Pos_Table
Table_size	U32	數值單位	Pos_table 需 Compare 的大小

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 pos_table[4] = {1000,2000,3000,4000};
U32 table_size =4;

I16 status = _DMC_01_channel1_position_compare_table (CardNo, pos_table, table_size);
```

NOTE

Table_Size 最大設定值為 100000。

39.14 _DMC_01_channel1_position_compare_table_level

■ **格式**

I16 PASCAL _DMC_01_channel1_position_compare_table_level (U16 CardNo,
I32* pos_table, U32* level_table, U32 table_size)

■ **目的**

設定 Compare Type1 的 Level 相關資料。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Pos_table	I32*	數值單位	依照資料計算出需 Compare 的 Pos_Table
Level_table	U32*	數值單位	設定自訂 Compare 的 Level_Table
Table_size	U32	數值單位	Pos_table 需 Compare 的大小

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
I32 pos_table[4] ={1000,2000,3000,4000};
U32 level_table[4] ={1,0,0,0};
U32 table_size =4;

I16 status = _DMC_01_channel1_position_compare_table_level (CardNo, pos_table,
             level_table,table_size);
```

NOTE

Table_Size 最大設定值為 100000。

39.15 _DMC_01_channel1_position_compare_table_cnt

■ **格式**

I16 PASCAL _DMC_01_channel1_position_compare_table_cnt (U16 CardNo, U32* cnt)

■ **目的**

讀取已 Trigger 的次數(Compare Type1 使用)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Cnt	U32*	數值單位	讀取已 Trigger 的次數(Compare Type1 使用)

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U32 cnt;

I16 status = _DMC_01_channel1_position_compare_table_cnt (CardNo,& cnt);
```

39.16 _DMC_01_set_compare_channel_polarity

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_polarity (U16 CardNo, U16 inverse)

■ **目的**

設定 Compare Trigger 的準位。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Inverse	U16	數值單位	0：Normal 1：Inverse

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 inverse = 1;

I16 status = _DMC_01_set_compare_channel_polarity (CardNo, inverse);
```

39.17 _DMC_01_channel0_position_cmp_by_gpio

■ **格式**

I16 PASCAL _DMC_01_channel0_position_cmp_by_gpio (U16 CardNo, U16 dir, U16 interval, I32 trigger_cnt)

■ **目的**

設定 CompareType0 的觸發條件由 GPIO 控制。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Dir	U16	數值單位	0：正向 1：反向
Interval	U16	數值單位	脈波 Compare 間隔數
Trigger _cnt	I32	數值單位	Trigger Compare 總數

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0;
U16 Dir = 1;
U16 Interval = 10;
I32 trigger_cnt = 50000;

I16 status = _DMC_01_channel0_position_cmp_by_gpio (CardNo, Dir, Interval, trigger_cnt);
```

39.18 _DMC_01_channel1_position_re_compare_table

■ **格式**

I16 PASCAL _DMC_01_channel1_position_re_compare_table (U16 CardNo)

■ **目的**

用上次設定的 Compare 條件，重新執行一次 Compare Type1。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;

I16 status = _DMC_01_channel1_position_re_compare_table (CardNo);

39.19 _DMC_01_channel1_position_re_compare_table_level

■ **格式**

I16 PASCAL _DMC_01_channel1_position_re_compare_table_level (U16 CardNo)

■ **目的**

用上次設定的 Compare 條件，重新執行一次 Compare Type1(Level 模式)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0;

I16 status = _DMC_01_channel1_position_re_compare_table_level (CardNo);

39.20 _DMC_01_set_compare_channel_enable

■ **格式**

I16 PASCAL _DMC_01_set_compare_channel_enable (U16 CardNo,
U16 compare_channel,U16 enable)

■ **目的**

設定 FPGA 內的 Compare 功能開啟/關閉，若未啟用比較陣列(Channel1_output_enable)，實體 Compare 因為沒有位置可以比較，所以依然不會觸發。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Compare_channel	U16	編號單位	Channel No 為 0~1
enable	U16	數值單位	0：關閉 Compare 功能。 1：開啟 Compare 功能。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0, ;
U16 compare_channel = 0;
U16 enable = 1;//開啟 Compare 功能

I16 status = _DMC_01_set_compare_channel_enable (CardNo, compare_channel, enable);

39.21 _DMC_01_channel1_get_fifo_status

■ **格式**

I16 PASCAL _DMC_01_channel1_position_re_compare_table_level (U16 CardNo, U16* fifo_status)

■ **目的**

取得目前 Compare Table 內資料的存放狀態。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
fifo_status	U16*	數值單位	Compare Table 內資料的存放狀態

數值	說明
0	Empty
1	Almost empty
2	Not empty or full
3	Almost full
4	Full

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0, fifo_status = 0;

I16 status = _DMC_01_channel1_position_re_compare_table_level (CardNo, &fifo_status);

39.22 _DMC_01_channel1_get_fifo_counter

■ **格式**

I16 PASCAL _DMC_01_channel1_get_fifo_counter (U16 CardNo, U16* fifo_cnt)

■ **目的**

取得目前 CompareTable 內存放比較資料的筆數。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
fifo_cnt	U16*	數值單位	目前 CompareTable 內可填入剩餘筆數。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 CardNo = 0, fifo_cnt = 0;

I16 status = _DMC_01_channel1_get_fifo_counter (CardNo, &fifo_cnt);

第四十章　連續速度 API

表 40.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_speed_continue	連續速度功能是否致能
A/B	_DMC_01_speed_continue_mode	連續速度模式
A/B	_DMC_01_speed_continue_combine_ratio	連續速度合成比率

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

40.1　_DMC_01_speed_continue

■　**格式**

I16 PASCAL _DMC_01_speed_continue (U16 CardNo, U16 NodeID, U16 SlotID, U16 enable)

■　**目的**

啟用/關閉連續速度功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Enable	U16	選項單位	0：Disable 連續速度功能 1：Enable 連續速度功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
U16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 enable=1;

I16 status = _DMC_01_speed_continue (CardNo, NodeID, SlotID, enable);
```

NOTE

當您啟用 _DMC_01_speed_continue　參數 enable=1，接下來所下的任何運動指令，其中的 StrVel 參數必須設定為 0，才能達成 Speed Continue 的效果。

加減速時間必須大於 2ms。

若您要在多軸指令中使用_DMC_01_speed_continue 指令，您必須逐軸下達，參數 enable=1

40.2　_DMC_01_speed_continue_mode

■　**格式**

I16 PASCAL _DMC_01_speed_continue_mode (U16 CardNo, U16 NodeID, U16 SlotID, U16 mode)

■　**目的**

設定連續速度模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	選項單位	0：等加速模式 1：加、減速模式 2：最大速度模式

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, U16 NodeID=1, U16 SlotID=0;
U16 mode=1;

I16 status = _DMC_01_speed_continue_mode (CardNo, NodeID, SlotID, mode);

NOTE

Mode 值為 0

當設定條件為 Dist 為 1000、MaxVel 為 20000、Tacc 與 Tdec 均為 0.1。
在原定的加減速時間下，上述條件表示在進行此 Dist 移動過程中，不可能達到 MaxVel 的值 20000。故當 mode 值設定為 0 時，將會等比率縮減 MaxVel、Tacc 與 Tdec。以相等加速度完成。圖 41.1 所示，黑色為原規劃路徑，而紅色為實際路徑。

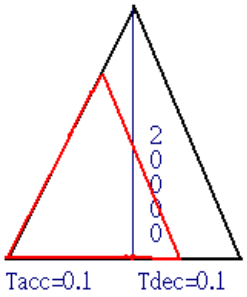


圖 40.1

Mode 值為 1

當設定條件為 Dist 為 1000、MaxVel 為 20000、Tacc 與 Tdec 均為 0.1。
在原定的加減速時間下，上述條件表示在進行此 Dist 移動過程中，不可能達到 MaxVel 的值 20000。故當 mode 值設定為 1 時，Tacc 與 Tdec 仍會維持在 0.1，但 MaxVel 值會自動縮減。圖 41.2 所示，黑色為原規劃路徑，而紅色為實際路徑。

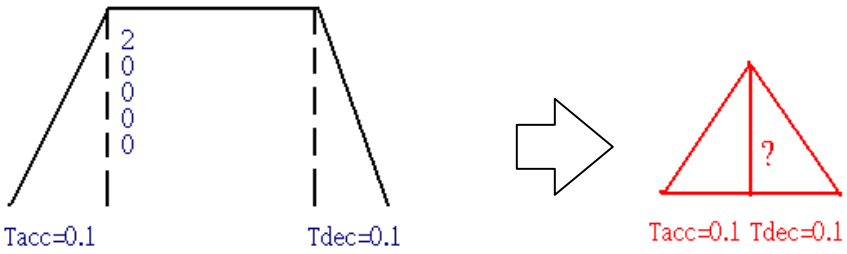


圖 40.2

Mode 值為 2

當設定條件為 Dist 為 1000、MaxVel 為 20000、Tacc 與 Tdec 均為 0.1。
在原定的加減速時間下，上述條件表示在進行此 Dist 移動過程中，不可能達到 MaxVel 的值 20000。故當 mode 值設定為 2 時，MaxVel 的值將維持不變，但 Tacc 與 Tdec 會依實際狀況變動。圖 41.3 所示，黑色為原規劃路徑，而紅色為實際路徑。

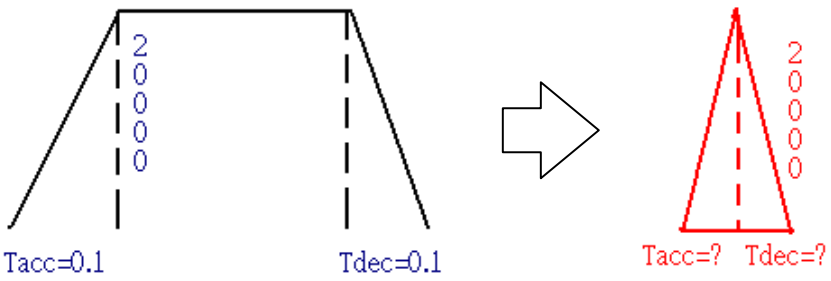


圖 40.3

40.3　_DMC_01_speed_continue_combine_ratio

■　**格式**

I16 PASCAL _DMC_01_speed_continue_combine_ratio (U16 CardNo, U16 NodeID, U16 SlotID, U16 ratio)

■　**目的**

設定連續速度合成百分比。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Ratio	U16	數值單位	合成百分比(0~100)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

U16 CardNo=0, U16 NodeID=1, U16 SlotID=0;
U16 ratio=100;

I16 status = _DMC_01_speed_continue_combine_ratio (CardNo, NodeID, SlotID, ratio);

NOTE

Ratio 的值為 100

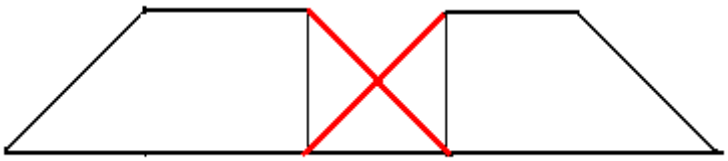


圖 40.4

Ratio 的值為 50

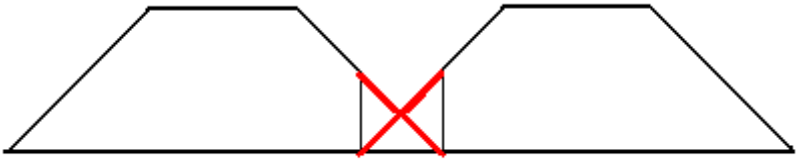


圖 40.5

※1~6 軸執行 Speed Continue 時，會運用到 7~12 軸的資源。

所以當 1~6 軸執行 Speed Continue Enable 時，7~12 軸不能執行 P2P 及 Continue 等動作。

必須將 Speed Continue Disable 後，7~12 軸方可進行後續動作。

第四十一章　特殊運用 API

表 41.1

API 歸屬	函數名稱	描述
A/B	_DMC_01_enable_electcam	設定開啟/關閉電子凸輪功能
A/B	_DMC_01_set_cam_enable	開啟/關閉間隙補償功能
A/B	_DMC_01_set_cam_pitch	設定間隙補償間隔(Pulse)
A/B	_DMC_01_set_cam_org	設定間隙補償開始原點位置
A/B	_DMC_01_set_cam_mode	設定間隙補償模式(單向/雙向)
A/B	_DMC_01_set_cam_table	建立間隙補償參數表資料
A/B	_DMC_01_set_cam_table2	建立間隙補償參數表資料並設定補償數量
A/B	_DMC_01_set_backlash	開啟/關閉背隙補償功能
A/B	_DMC_01_set_backlash_info	設定背隙補償參數
A/B	_DMC_01_set_ecam_repeat	開啟/關閉電子凸輪重複功能

註 A:PCI_DMC_A01　　B:PCI_DMC_B01　　F:PCI_DMC_F01

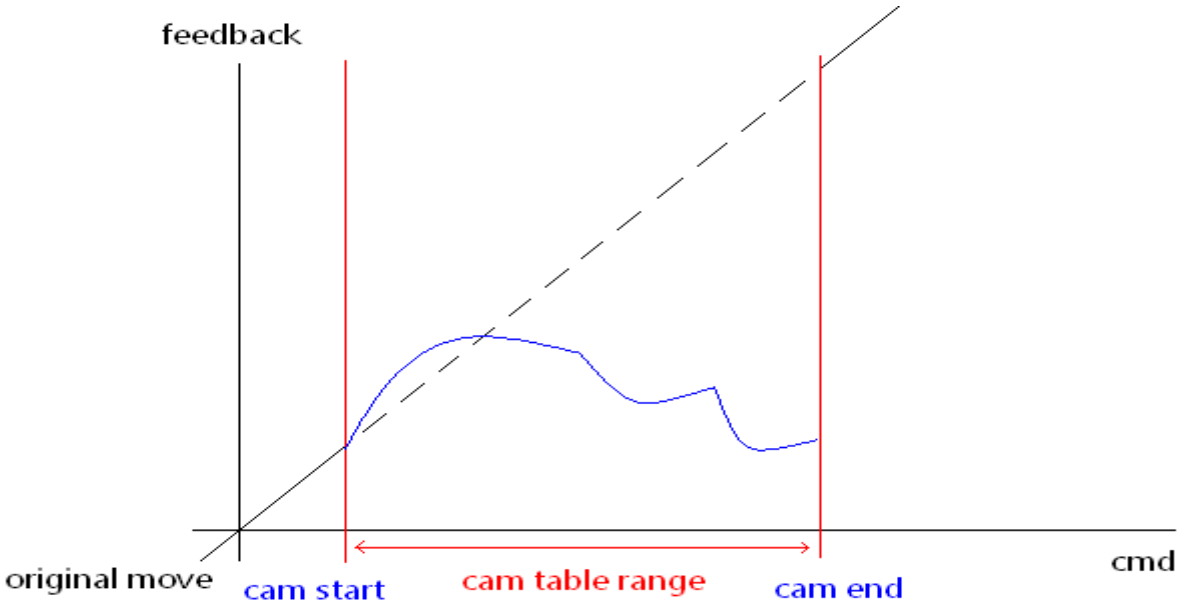


圖 41.1

說明:未使用電子凸輪前，其運動軌跡(cmd 與 feedback 相等)，進入電子凸輪後(cam table)，將依照上圖藍色路線進行，於 cam table 結束後，將恢復原運動軌跡，請注意由藍色路線銜接原運動軌跡處，會因為追隨問題而導致暴衝現象，故 cam table 最好包含整個運動行

程。

41.1　_DMC_01_enable_electcam

■　**格式**

I16 PASCAL _DMC_01_enable_electcam(I16 CardNo, U16 NodeID, U16 SlotID, U16 enable,U16 axisbit,U16 mode)

■　**目的**

設定開啟/關閉電子凸輪功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID(設定為主軸站號)
SlotID	U16	編號單位	插槽 ID
enable	U16	數值單位	0：關閉電子凸輪功能 1：開啟電子凸輪功能
axisbit	U16	數值單位	二進制值，設定為主軸的 bit 不能致能 Bit0:Node1 Bit1:Node2 Bit2:Node3 Bit3:Node4 Bit4:Node5 Bit5:Node6 Bit6:Node7 Bit7:Node8 Bit8:Node9 Bit9:Node10 Bit10:Node11 Bit11:Node12
mode	U16	數值單位	0：跟隨主軸 Command 1：跟隨主軸 Position

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

I 16 CardNo=0, U16 NodeID=1, SlotID=0;　//設定 Node1 為主軸
U16 enable =1; //開啟電子凸輪功能
U16 axisbit =14; //Node1 為主軸，Node2,3,4 設定為從軸 axisbit=0(2^0)+2(2^1)+4(2^2)+8(2^3)=14
U16 mode=0; //跟隨主軸 Command

I16 status = _DMC_01_enable_electcam(CardNo, NodeID, SlotID, enable, axisbit, mode);

41.2　_DMC_01_set_cam_enable

■　**格式**

I16 PASCAL _DMC_01_set_cam_enable(I16 CardNo, U16 NodeID, U16 SlotID,U16 on_off)

■　**目的**

開啟/關閉間隙補償功能(不使用此功能時，請將間隙態補償 Disable)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
on_off	U16	數值單位	0：Disable 間隙態補償功能 1：Enable 間隙補償功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

I 16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 enable=1; //開啟間隙補償功能

I16 status = _DMC_01_set_cam_enable(CardNo, NodeID, SlotID, on_off);

41.3 _DMC_01_set_cam_pitch

■ 格式

I16 PASCAL _DMC_01_set_cam_pitch (U16 CardNo, U16 NodeID, U16 SlotID,I32 pitch)

■ 目的

設定間隙補償間隔(Pulse)。

■ 參數

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Pitch	I32	數值單位	設定間隙補償間隔(Pulse)

■ 屬性

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ 範例

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0;
I32 pitch =100000; //間隙補償間隔(Pulse)

//每 10000Pulse 間隔補償
I16 status = DMC_01_set_cam_pitch(CardNo, NodeID, SlotID, pitch);
```

41.4　_DMC_01_set_cam_org

■　**格式**

I16 PASCAL _DMC_01_set_cam_org(I16 CardNo, U16 NodeID, U16 SlotID, I16 Dir, I32 orgpos)

■　**目的**

設定間隙補償開始原點位置。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dir	U16	數值單位	0：設定正向間隙補償原點 1：設定負向間隙補償原點
orgpos	I32	數值單位	間隙補償原點起始位置

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0, status;
U16 Dir0=0; //正向間隙補償
U16 Dir1=1; //負向間隙態補償
I32 orgpos0 =100000; //正向間隙補償起始位置
I32 orgpos1 =900000; //負向間隙補償起始位置

status = _DMC_01_set_cam_enable( CardNo, NodeID, SlotID, 1);   //開啟間隙補償
status = _DMC_01_set_cam_pitch(CardNo, NodeID, SlotID,10000);//每 10000Pulse 間隔補償
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir0, orgpos0);
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir1, orgpos1);
```

NOTE

若整個行程由座標原點 0 開始，在 1000000 結束，則正向間隙**補償由 100000 這個位置開始，而負向的**間隙**補償，由 900000 這個位置開始。**

41.5　_DMC_01_set_cam_mode

■　**格式**

I16 PASCAL _DMC_01_set_cam_mode (I16 CardNo, U16 NodeID, U16 SlotID,U16 Mode)

■　**目的**

設定間隙補償模式(單向/雙向)。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Mode	U16	數值單位	0：設定雙向/雙表格間隙補償(來回用不同的參數表格資料做間隙補償) 1：設定雙向/單表格間隙補償(來回都用同一個參數表格資料做間隙補償)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 Mode=0; //設定為雙向/雙表格間隙補償

I16 status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, Mode);
```

41.6　_DMC_01_set_cam_table

■　**格式**

I16 PASCAL _DMC_01_set_cam_table2(I16 CardNo, U16 NodeID, U16 SlotID, I16 Dir, I32* table)

■　**目的**

建立間隙補償參數表資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dir	U16	數值單位	0：設定正向間隙補償 1：設定負向間隙補償
Table	I32*	數值單位	間隙補償參數資料

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0, status;
U16 Dir0=0; //正向間隙補償
U16 Dir1=1; //負向間隙補償
I32 orgpos0 =100000; //正向間隙補償起始位置
I32 orgpos1 =900000; //負向間隙補償起始位置
//設定正向間隙補償表資料,最大筆數 10000
I32 table0={10050,20030,30045,40040,50025,60020,70010,80045,90030};
//設定負向間隙補償表資料,最大筆數 10000
I32 table1={90032,80043,70015,60028,50020,40035,30040,20032,10045};

status = DMC_01_set_cam_enable( CardNo, NodeID, SlotID, 1);   //開啟間隙補償
status = DMC_01_set_cam_pitch(CardNo, NodeID, SlotID,10000);//每 10000Pulse 間隔補償

status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir0, orgpos0);
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir1, orgpos1);
```

```
status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 0);//設定為雙向/雙表格模式
status = _DMC_01_set_cam_table( CardNo, NodeID, SlotID, Dir0, table0);
status = _DMC_01_set_cam_table( CardNo, NodeID, SlotID, Dir1, table1);

status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 1);//設定為雙向/單表格模式
status = _DMC_01_set_cam_table( CardNo, NodeID, SlotID, Dir0, table0);
```

NOTE

因為是雙向都用同一個間隙補償資料表，所以只設定正向間隙補償資料表即可，負向部分會自動依正向動間隙資料表反算做補償

若整個行程由座標原點 0 開始，在 1000000 結束，則正向間隙補償由 100000 這個位置開始，而負向的間隙補償，由 900000 這個位置開始。

41.7　_DMC_01_set_cam_table2

■　**格式**

I16 PASCAL _DMC_01_set_cam_table2(I16 CardNo, U16 NodeID, U16 SlotID, I16 Dir, I32* table,I32 Num)

■　**目的**

建立間隙補償參數表資料並設定間隙補償補償數量。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Dir	U16	數值單位	0：設定正向間隙補償 1：設定負向間隙補償
table	I32*	數值單位	間隙補償參數資料
Num	I32	數值單位	間隙補償數量

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
I 16 CardNo=0, U16 NodeID=1, SlotID=0, status = 0;
U16 Dir0=0; //正向間隙補償
U16 Dir1=1; //負向間隙補償
I32 orgpos0 =100000; //正向間隙補償起始位置
I32 orgpos1 =900000; //負向間隙補償起始位置
I32 table0={10050,20030,30045,40040,50025,60020,70010,80045,90030};
    //設定正向間隙補償表資料,最大筆數 10000
I32 table1={90032,80043,70015,60028,50020,40035,30040,20032,10045};
    //設定負向間隙補償表資料,最大筆數 10000
I32 Num0=9, Num1=8;

status = DMC_01_set_cam_enable( CardNo, NodeID, SlotID, 1);   //開啟間隙補償
status = DMC_01_set_cam_pitch(CardNo, NodeID, SlotID,10000);//每 10000Pulse 間隔補償
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir0, orgpos0);
status = _DMC_01_set_cam_org( CardNo, NodeID, SlotID, Dir1, orgpos1);
```

```
status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 0);//設定為雙向/雙表格模式
status = _DMC_01_set_cam_table2( CardNo, NodeID, SlotID, Dir0, table0, Num0);
status = _DMC_01_set_cam_table2( CardNo, NodeID, SlotID, Dir1, table1, Num1);

status = _DMC_01_set_cam_mode ( CardNo, NodeID, SlotID, 1);//設定為雙向/單表格模式
status = _DMC_01_set_cam_table2( CardNo, NodeID, SlotID, Dir0, table0, Num0);
```

NOTE

因為是雙向都用同一個間隙補償資料表，所以只設定正向間隙補償資料表即可，負向部分會自動依正向間隙資料表反算做補償

若整個行程由座標原點 0 開始，在 1000000 結束，則正向間隙補償由 100000 這個位置開始，而負向的間隙補償，由 900000 這個位置開始。

41.8　_DMC_01_set_backlash

■　**格式**

I16 PASCAL _DMC_01_set_backlash(I16 CardNo, U16 NodeID, U16 SlotID,I16 enable)

■　**目的**

開啟/關閉背隙補償功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
enable	I16	數值單位	0：關閉背隙補償功能 1：開啟背隙補償功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

I 16 CardNo=0, U16 NodeID=1, SlotID=0;
U16 enable =1; /開啟背隙補償功能

I16 status = _DMC_01_set_backlash(CardNo, NodeID, SlotID, enable);

41.9　_DMC_01_set_backlash_info

■　**格式**

I16 PASCAL _DMC_01_set_backlash_info(I16 CardNo, U16 NodeID, U16 SlotID, I16 backlash,U16 accstep,U16 contstep,U16 decstep)

■　**目的**

設定背隙補償參數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
backlash	I16	數值單位	背隙補償值(Pulse)，設定 accstep,contstep, decstep 並依照圖 41.1 的方式完成背隙補償
accstep	U16	數值單位	設定圖 41.1 加速時間(ms)
contstep	U16	數值單位	設定圖 41.1 常速時間(ms)
decstep	U16	數值單位	設定圖 41.1 減速時間(ms)

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

backlash 的值為 100

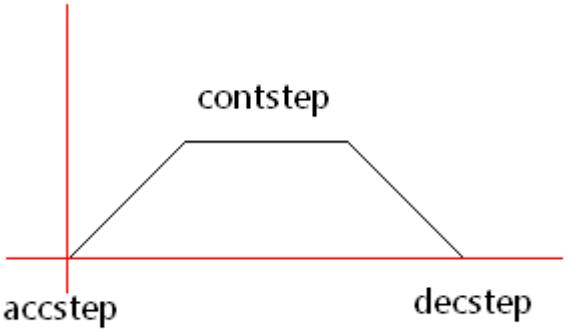


圖 41.1

I 16 CardNo=0, U16 NodeID=1, SlotID=0, status = 0;
U16 enable =1; //開啟背隙補償功能

I 16 backlash =100;
U16 accstep =10, contstep=10, decstep=10;
//在加速時間=10ms，常速時間=10ms，減速時間=10ms 的條件下算出以何種速度在上述條件內完成 100Pulse 的背隙補償;至於這三個參數的設定值，要如何不造成速度突增，需要嘗試。

```
status = _DMC_01_set_backlash_info(CardNo, NodeID, SlotID, backlash, accstep, contstep,
         decstep);
status = _DMC_01_set_backlash(CardNo, NodeID, SlotID, enable);
```

NOTE

背隙補償的 backlash 設定值若為正值，表示往正方向移動的背隙補償值，裝置若往負方向移動，則不做背隙補償; 反之 backlash 設定值若為負值，表示往負方向移動的背隙補償值，裝置若往正方向移動，則不做背隙補償。

Accstep+contstep+decstep 總和，不能超過 1000ms。

41.10　_DMC_01_set_ecam_repeat

■　**格式**

I16 PASCAL _DMC_01_set_ecam_repeat (U16 CardNo, U16 NodeID, U16 SlotID,
I32 start_pos,I32 end_pos, U16 enable)

■　**目的**

開啟/關閉電子凸輪重複功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
start_pos	I32	數值單位	設定起始位置
end_pos	I32	數值單位	設定結束位置
enable	U16	選項單位	0：關閉電子凸輪重複功能 1：開啟電子凸輪重複功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

U16 CardNo=0, NodeID=1, SlotID=0;
I32 start_pos =100000, end_pos =800000;
U16 enable =1; //開啟電子凸輪重複功能

I16 status = _DMC_01_set_ecam_repeat (CardNo, NodeID, SlotID, start_pos, end_pos, enable);

（此頁有意留為空白）

第四十二章　其它 API

表 42.1

API 歸屬	函數名稱	描述
A/B/F	_misc_app_get_circle_endpoint	取得圓弧補間所需之終點座標(X, Y)
A/B/F	_misc_app_get_circle_center_point	取得圓弧補間所需之中心座標(X, Y)
A/B/F	_misc_set_record_debuging	Debug 記錄功能是否啟動
A/B/F	_misc_open_record_debuging_file	設置輸出 Debug 記錄存放位置
A/B/F	_DMC_01_enable_dda_mode	啟動 DDA 填表功能
A/B/F	_DMC_01_set_dda_data	填入 DDA Table 資料
A/B/F	_DMC_01_get_dda_cnt	取得目前 DDA Table 尚存的資料筆數
A/B/F	_DMC_01_clear_dda_data	將 DDA Table 中尚存的資料清除。
A/B/F	_DMC_01_save_device_info	將模組資料存為文字檔。
A/B/F	_DMC_01_compare_device_info	將目前連線之模組資料與已儲存之資料檔進行比對，並輸出比對結果的紀錄檔。
A/B/F	_DMC_01_enable_record_mode	開啟/關閉以毫秒為單位擷取 Monitor 所下參數的功能。
A/B/F	_DMC_01_get_record_data	讀取以毫秒為單位擷取 Monitor 所下參數的功能
A/B	_DMC_01_set_mc_fifo_mode	FIFO 參數設定。
A/B	_DMC_01_set_function_time	設定每 1ms 最小的通訊時間以確保通訊不會異常。(詳細請參照 API 內說明)
A/B/F	_DMC_01_get_fifo_cnt	取得 FIFO 暫存區資料筆數。
A/B/F	_DMC_01_set_gear	啟動設定 Gear 功能與其參數設置
A/B/F	_DMC_01_get_gear	讀取 Gear 設定值。
A/B/F	_DMC_01_set_position_ref_source	設定座標的參考來源模式
A/B/F	_DMC_01_get_position_ref_source	取得目前座標的參考來源模式
A/B/F	_DMC_01_set_virtual_command	Set position 的快速模式
A/B/F	_DMC_01_get_virtual_command	取得實際 command、position 與 virtual_command 的差值
A/B/F	_DMC_01_set_virtual_position	將實際 Position 值設定為 Virtual Position 的值
A/B/F	_DMC_01_disable_virtual_pos	選擇重置 Command 與 Position 的連動性
A/B/F	_DMC_01_set_stepping_move_rate	設定解析放大的比率
MH1/MP1	_DMC_01_MRAM_read_word	讀取 MH1(MP1)記憶體 MRAM 的值 (單位:word)

MH1/MP1	_DMC_01_MRAM_write_word	寫值到 MH1(MP1)的記憶體 MRAM (單位:word)
MH1/MP1	_DMC_01_MRAM_read_dword	讀取 MH1(MP1)記憶體 MRAM 的值 (單位:double word)
MH1/MP1	_DMC_01_MRAM_write_dword	寫值到 MH1(MP1)的記憶體 MRAM (單位:double word)
A/B	_DMC_01_get_watchdog_cnt	讀取 A/B01 軸卡 WatchDog 功能目前的累積計數值 (單位:mSec)
A/B	_DMC_01_set_watchdog_cnt	修改 A/B01 軸卡 WatchDog 功能目前的累積計數值 (單位:mSec)
A/B	_DMC_01_get_watchdog_value	讀取 A/B01 軸卡 WatchDog 功能的計數目標值。(單位:mSec)
A/B	_DMC_01_set_watchdog_value	修改 A/B01 軸卡 WatchDog 功能的計數目標值。(單位:mSec)
A/B	_DMC_01_set_rm_04pi_md2_feedback_mode	設定脈波模組 Feedback 的連接狀態

註 A:PCI_DMC_A01　B:PCI_DMC_B01　F:PCI_DMC_F01

42.1 _misc_app_get_circle_endpoint

■ **格式**

I16 PASCAL _misc_app_get_circle_endpoint (I32 Start_X, I32 Start_Y, I32 Center_X, I32 Center_Y, F64 Angle, I32* End_x, I32* End_y)

■ **目的**

取得圓弧補間所需之終點座標(X, Y)。

■ **參數**

名稱	資料型別	單位	描述
Start_X	I32	脈波數	起始座標 X 軸
Start_Y	I32	脈波數	起始座標 Y 軸
Center_X	I32	脈波數	中心座標 X 軸
Center_Y	I32	脈波數	中心座標 Y 軸
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360°
End_x	I32*	脈波數	終點座標 X 軸
End_y	I32*	脈波數	終點座標 Y 軸

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I32 Start_X=0, Start_Y=0;
I32 Center_X=100000, Center_Y=0;
I32 End_x, End_y;
F64 Angle=180;

I16 status = _misc_app_get_circle_endpoint (Start_X, Start_Y, Center_X, Center_Y, Angle,
&End_x, &End_y);
```

42.2 _misc_app_get_circle_center_point

■ **格式**

I16 PASCAL _misc_app_get_circle_center_point (I32 Start_X, I32 Start_Y, I32 End_x, I32 End_y, F64 Angle, I32* Center_X, I32* Center_Y)

■ **目的**

取得圓弧補間所需之中心座標(X, Y)。

■ **參數**

名稱	資料型別	單位	描述
Start_X	I32	脈波數	起始座標 X 軸
Start_Y	I32	脈波數	起始座標 Y 軸
End_x	I32	脈波數	終點座標 X 軸
End_y	I32	脈波數	終點座標 Y 軸
Angle	F64	度(°)	一圓弧的角度設置。一個圓弧為 360°
Center_X	I32*	脈波數	中心座標 X 軸
Center_Y	I32*	脈波數	中心座標 Y 軸

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
I32 Start_X=0, Start_Y=0;
I32 End_x=100000, End_y0;
I32 Center_X, Center_Y;
F64 Angle=180;

I16 status = _misc_app_get_circle_center_point(Start_X, Start_Y , End_x, End_y, Angle,
&Center_X, &Center_Y);
```

42.3 _misc_set_record_debuging

■ **格式**

I16 PASCAL _misc_set_record_debuging (U16 enable)

■ **目的**

Debug 記錄功能是否啟動。

■ **參數**

名稱	資料型別	單位	描述
Enable	U16	選項單位	0：關閉 Debug 記錄 1：啟動 Debug 記錄

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

U16 Enable = 1;

I16 status = _misc_set_record_debuging (Enable);

42.4 _misc_open_record_debuging_file

■ **格式**

I16 PASCAL _misc_open_record_debuging_file (I16 CardNo,Char* file_name, U16 open)

■ **目的**

設置輸出 Debug 記錄存放位置。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
file_name	Char*	字元陣列單位	Debug 記錄文件存放路徑
open	U16	選項單位	0：存放在 VC6 集成開發環境 debug window 1：記錄存放在 file_name 設定的路徑文件中

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
Char file_name[20]=”log_output.txt”;
I16 CardNo=0,
U16 open=1;

I16 status = _misc_open_record_debuging_file(CardNo ,file_name, open);
```

42.5　_DMC_01_enable_dda_mode

■　**格式**

I16 PASCAL _DMC_01_enable_dda_mode (U16 CardNo, U16 enable)

■　**目的**

啟動 DDA 填表功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15。
Enable	U16	選項單位	0：關閉 DDA 填表功能。 1：啟動 DDA 填表功能。 Bit0:Node1 Bit1:Node2 Bit2:Node3 Bit3:Node4 Bit4:Node5 Bit5:Node6 Bit6:Node7 Bit7:Node8 Bit8:Node9 Bit9:Node10 Bit10:Node11 Bit11:Node12

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 Enable = 5;   //啟用 Node1,3

I16 status = _DMC_01_enable_dda_mode (CardNo, Enable);
```

42.6　_DMC_01_set_dda_data

■　**格式**

I16 PASCAL _DMC_01_set_dda_data (U16 CardNo, U32* abs_pos)

■　**目的**

填入 DDA Table 資料。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
Abs_Pos	U32*	數值資料	此欄位為一 12 軸資料的陣列

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;

//Abs_Pos 為一 12 軸的資料陣列，最大 Buffer 為 1000 筆，每一 ms 會執行一筆 DDA Table 的資料，要注意的是在 Buffer 執行完畢之前，不接受其他指令(例如:Sd_Stop 等)。
U32 Abs_Pos[0][11] = {1000,2000,0,0,0,0,0,0,0,0,0,0};

I16 status = _DMC_01_set_dda_data (CardNo, Abs_Pos);
//其他未啟用 DDA Table 功能的 Node 可以正常執行其他動作。
```

42.7　_DMC_01_get_dda_cnt

■　**格式**

I16 PASCAL _DMC_01_get_dda_cnt (U16 CardNo, U16* dda_cnt)

■　**目的**

取得目前 DDA Table 尚存的資料筆數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
dda_cnt	U16*	數值資料	目前 DDA Table 尚存的資料筆數

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U16 dda_cnt;

I16 status = _DMC_01_get_dda_cnt (CardNo, & dda_cnt);
```

42.8　_DMC_01_clear_dda_data

■　**格式**

I16 PASCAL _DMC_01_clear_dda_data (U16 CardNo)

■　**目的**

將 DDA Table 中尚存的資料清除。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;

I16 status = _DMC_01_clear_dda_data (CardNo);
```

42.9　_DMC_01_save_device_info

■　**格式**

I16 PASCAL _DMC_01_save_device_info (U16 CardNo, char * path)

■　**目的**

將模組資料存為文字檔。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
path	Char*	字串	存檔路徑及檔名。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
char path = “C:\log\log.txt”;

16 status = _DMC_01_save_device_info (CardNo, path);
```

42.10 _DMC_01_compare_device_info

■ **格式**

I16 PASCAL _DMC_01_compare_device_info (U16 CardNo, char *path, char *pathLog)

■ **目的**

將目前連線之模組資料與已儲存之資料檔進行比對，並輸出比對結果的紀錄檔。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
path	char*	字串	比對模組的檔案路徑及檔名。
path_log	char *	字串	與模組資料比對結果存檔路徑及檔名。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
char path[50] = “C:\log\DMC_NET.txt”;
char path_log=“C:\log\log.txt”;

I16 status = _DMC_01_get_dda_cnt (CardNo, path, path_log);
```

42.11　_DMC_01_enable_record_mode

■　**格式**

I16 PASCAL _DMC_01_enable_record_mode (U16 CardNo, U16 enable, U16 mode)

■　**目的**

開啟/關閉以毫秒為單位擷取 Monitor 所下參數的功能。

■　**參數**

<table>
<tr><th>名稱</th><th>資料型別</th><th>單位</th><th>描述</th></tr>
<tr><td>CardNo</td><td>U16</td><td>編號單位</td><td>介面卡號 CardNo 為 0~15</td></tr>
<tr><td>enable</td><td>U16</td><td>選項單位</td><td>0：Default
1：所下 Monitor 參數有動作時記錄。
2：所下 Monitor 參數有、無動作皆會記錄。</td></tr>
<tr><td>mode</td><td>U16</td><td>數值單位</td><td>
<table>
<tr><th>數值</th><th>定義</th></tr>
<tr><td>1</td><td>Node1 開啟/關閉。</td></tr>
<tr><td>2</td><td>Node2 開啟/關閉。</td></tr>
<tr><td>4</td><td>Node3 開啟/關閉。</td></tr>
<tr><td>8</td><td>Node4 開啟/關閉。</td></tr>
<tr><td>….</td><td></td></tr>
<tr><td>2048</td><td>Node12 開啟/關閉。</td></tr>
</table>
</td></tr>
</table>

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0, status;
U16 NodeID[11]={1,2,3,4,5,0,0,0,0,0,0,0}, SlotID=0;
    U16 monitorw=0; //監控馬達回授脈波數　請參照手冊表 32.2
U16 mode=2;   //馬達不動作時也記錄其回授波數

status = _DMC_01_set_monitor(CardNo, NodeID, 0, monitorw);

status = _DMC_01_enable_record_mode (CardNo, mode);
```

42.12　_DMC_01_get_record_data

■　**格式**

I16 PASCAL _DMC_01_get_record_data (U16 CardNo, U32* record_data)

■　**目的**

讀取以毫秒為單位擷取 Monitor 所下參數的功能。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
record_data	U32*	數值單位	讀取以毫秒為單位，Monitor 所下參數的 12 軸資料。

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0;
U32 PPos[12] = {0};

I16 status = _DMC_01_get_record_data (CardNo, &PPos [0]);
```

42.13　_DMC_01_set_mc_fifo_mode

■　**格式**

I16 PASCAL _DMC_01_set_mc_fifo_mode (U16 CardNo, U16 mode)

■　**目的**

FIFO 參數設定。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
mode	U16	選項單位	0：Default 1：For PCB 2：set_function_time will active

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo = 0, status = 0;
U16 mode=2;
U32 ms1000=480;

status = _DMC_01_set_mc_fifo_mode ( CardNo, mode);

status = _DMC_01_set_function_time(CardNo, ms1000);
```

42.14 _DMC_01_set_function_time

■ **格式**

I16 PASCAL _DMC_01_set_function_time (U16 CardNo,U32 ms1000)

■ **目的**

設定每 1ms 最小的通訊時間以確保通訊不會異常。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
ms1000	U32	數值單位	Fifo mode=2 開啟 Default 通訊時間為 600us 可調整值區間為 390~800us。

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo = 0, status = 0;
U16 mode=2;
U32 ms1000=480;

status = _DMC_01_set_mc_fifo_mode ( CardNo, mode);

status = _DMC_01_set_function_time(CardNo, ms1000);
```

NOTE

此 API 所設定的時間為填入通訊封包所需的時間，通訊封包所需時間分為兩部分，一部分是基本時間(30us/站)，若有 12 站，基本上需要 360us，另一部分則視所執行運動的性質。

42.15　_DMC_01_get_fifo_cnt

■　**格式**

I16 PASCAL _DMC_01_get_fifo_cnt(U16 CardNo, U16* fifo_cnt)

■　**目的**

取得 FIFO 暫存區資料筆數。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
fifo_cnt	U16*	數值單位	Fifo Counter

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0, fifo_cnt;

I16 status = _DMC_01_get_fifo_cnt( CardNo, &fifo_cnt);
```

42.16　_DMC_01_set_gear

■　**格式**

I16 PASCAL _DMC_01_set_gear(U16 CardNo, U16 NodeID, U16 SlotID, I16 Numerator, I16 Denominator, U16 On_Off)

■　**目的**

啟動設定 Gear 功能與其參數設置。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Numberator	I16	數值單位	電子齒輪比分子
Denominator	I16	數值單位	電子齒輪比分母
On_Off	U16	選項單位	0：關閉 Gear 功能 1：啟動 Gear 功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

U16 CardNo=0, NodeID =1, SlotID=0;
I16 Numberator=1, Denominator=2;
U16 On_Off =1;

I16 status = _DMC_01_set_gear(CardNo, NodeID, SlotID, Numerator, Denominator, On_Off);

42.17　_DMC_01_get_gear

■　**格式**

I16 PASCAL _DMC_01_get_gear(U16 CardNo, U16 NodeID, U16 SlotID, I16* Numerator, I16* Denominator, U16* On_Off)

■　**目的**

讀取 Gear 設定值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
Numberator	*I16	數值單位	電子齒輪比分子
Denominator	*I16	數值單位	電子齒輪比分母
On_Off	*U16	選項單位	0：關閉 Gear 功能 1：啟動 Gear 功能

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
		●					

■　**範例**

```
U16 CardNo=0, NodeID =1, SlotID=0;
I16 Numberator, Denominator;
U16 On_Off;

I16 status = _DMC_01_get_gear(CardNo, NodeID, SlotID, &Numerator, &Denominator,
             &On_Off);
```

42.18　_DMC_01_set_position_ref_source

■　**格式**

I16 PASCAL _DMC_01_set_position_ref_source (I16 CardNo, U16 NodeID, U16 SlotID, U16 ref_source)

■　**目的**

設定座標的參考來源模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ref_source	U16	選項單位	0：參考 Command 位置 1：參考 Position 位置 2：參考 Command + Offset 位置 3：參考 Position + Offset 位置

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 ref_source =0;

I16 status = _DMC_01_set_position_ref_source (CardNo, NodeID, SlotID, ref_source);
```

42.19　_DMC_01_get_position_ref_source

■　**格式**

I16 PASCAL _DMC_01_get_position_ref_source (I16 CardNo, U16 NodeID, U16 SlotID, U16 *ref_source)

■　**目的**

取得目前座標的參考來源模式。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
ref_source	U16*	選項單位	0：參考 Command 位置 1：參考 Position 位置 2：參考 Command + Offset 位置 3：參考 Position + Offset 位置

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 ref_source;

I16 status = _DMC_01_get_position_ref_source (CardNo, NodeID, SlotID, &ref_source);
```

42.20 _DMC_01_set_virtual_command

■ **格式**

I16 PASCAL _DMC_01_set_virtual_command (I16 CardNo, U16 NodeID, U16 SlotID, I32 virtual_command)

■ **目的**

Set position 的快速模式(會將實際 command 與 position 改成 virtual_command 的值)。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
virtual_command	I32	數值單位	欲設置的 virtual command 計數值

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
I16 CardNo = 0, status = 0;
U16 NodeID = 1, SlotID = 0, on_off = 0;

//0: 將實際 command 與 position 都改成 virtual_command 的值
//1: 只將實際 command 改成 virtual_command 的值
I32 virtual_command = 0;

status = _DMC_01_disable_virtual_pos (CardNo, NodeID, SlotID, on_off);

status = _DMC_01_set_virtual_command (CardNo, NodeID, SlotID, virtual_command);
```

42.21　_DMC_01_get_virtual_command

■　**格式**

I16 PASCAL _DMC_01_get_virtual_command (I16 CardNo, U16 NodeID, U16 SlotID, I32 *virtual_command)

■　**目的**

取得實際 command、position 與 virtual_command 的差值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
virtual_command	I32*	數值單位	取得實際 command、position 與 virtual_command 的差值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
I32 virtual_command ;

I16 status = _DMC_01_get_virtual_command (CardNo, NodeID, SlotID, &virtual_command);
```

42.22　_DMC_01_set_virtual_position

■　**格式**

I16 PASCAL _DMC_01_set_virtual_position (I16 CardNo, U16 NodeID, U16 SlotID, I32 virtual_Position)

■　**目的**

將實際 position 值設定為 virtual_position 的值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
virtual_Position	I32	數值單位	欲設置的 virtual position 計數值

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
I32 virtual_Position =0;

I16 status = _DMC_01_set_virtual_position (CardNo, NodeID, SlotID, virtual_Position);
```

42.23　_DMC_01_disable_virtual_pos

■　**格式**

I16 PASCAL _DMC_01_disable_virtual_pos (I16 CardNo, U16 NodeID, U16 SlotID, U16 on_off)

■　**目的**

選擇重置 Command 與 Position 的連動性。

■　**參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
on_off	U16	選項單位	使用_DMC_01_set_virtual_command 時 0：Command 與 Position 都清除 1：只清除 Command

■　**屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■　**範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 on_off =1;

I16 status = _DMC_01_disable_virtual_pos (CardNo, NodeID, SlotID, on_off);
```

42.24 _DMC_01_set_stepping_move_rate

■ **格式**

I16 PASCAL _DMC_01_set_stepping_move_rate (I16 CardNo, U16 NodeID, U16 SlotID, U16 MoveRatio)

■ **目的**

設定解析放大的比率。

■ **參數**

名稱	資料型別	單位	描述
CardNo	I16	編號單位	介面卡號 CardNo 為 0~15
NodeID	U16	編號單位	站號 ID
SlotID	U16	編號單位	插槽 ID
MoveRatio	U16	數值單位	解析放大的比率

■ **屬性**

軸卡	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
	●		●				

■ **範例**

```
I16 CardNo=0;
U16 NodeID =1, SlotID=0;
U16 MoveRatio =50;
//例如:RM04PI Mode2 CW/CCW 模式下，最大的速度限制為 200K，若 MoveRatio 設定為
  50，則最大速度限制為 4K，速度設定超過 4K 會出現 ALM59，與 ALM9 以資區別。

I16 status = _DMC_01_set_stepping_move_rate (CardNo, NodeID, SlotID, MoveRatio);
```

NOTE

※注意：

RM04PI Mode1，只須設定 NodeID 站號，所有的 Slot 就會跟著調整 MoveRatio

RM04PI Mode2，根據設定的 NodeID 站號，來設定 MoveRatio。

42.25　_DMC_01_MRAM_read_word

■　**格式**

I16 PASCAL _DMC_01_MRAM_read_word (U16 offset, U16* word_value)

■　**目的**

讀取 MH1(MP1)記憶體 MRAM 的值(單位:word，MRAM 範圍:0x0000~0XFFFE)。

■　**參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFE(MRAM 相對位址)
word_value	U16*	數值單位	讀取位於 MRAM 相對位址的資料(單位:word)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 offset =0x0001;
U16 word_value;

I16 status = _DMC_01_MRAM_read_word (offset, &word_value);
```

42.26 _DMC_01_MRAM_write_word

■ **格式**

I16 PASCAL _DMC_01_MRAM_write_word (U16 offset, U16 word_value)

■ **目的**

寫值到 MH1(MP1)的記憶體 MRAM(單位:word，MRAM 範圍:0x0000~0XFFFE)。

■ **參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFE(MRAM 相對位址)
word_value	U16	數值單位	寫值到位於 MRAM 的相對位址(單位:word)

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 offset =0x0001;
U16 word_value=0x3f5d;

I16 status = _DMC_01_MRAM_write_word (offset, word_value);
```

42.27　_DMC_01_MRAM_read_dword

■　**格式**

I16 PASCAL _DMC_01_MRAM_read_dword (U16 offset, U32* dword_value)

■　**目的**

讀取 MH1(MP1)記憶體 MRAM 的值(單位:double word，MRAM 範圍:0x0000~0XFFFC)。

■　**參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFC(MRAM 相對位址)
dword_value	U32*	數值單位	讀取位於 MRAM 相對位址的資料 (單位:double word)

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 offset =0x0001;
U32 dword_value;

I16 status = _DMC_01_MRAM_read_dword (offset, &dword_value);
```

42.28 _DMC_01_MRAM_write_dword

■ **格式**

I16 PASCAL _DMC_01_MRAM_write_dword (U16 offset, U32 dword_value)

■ **目的**

寫值到 MH1(MP1)的記憶體 MRAM(單位:double word，MRAM 範圍:0x0000~0XFFFC)。

■ **參數**

名稱	資料型別	單位	描述
offset	U16	數值單位	0~FFFC(MRAM 相對位址)
dword_value	U32	數值單位	寫值到位於 MRAM 的相對位址 (單位:double word)

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 offset =0x0001;
U32 dword_value=0x3f5d16d2;

I16 status = _DMC_01_MRAM_write_dword (offset, dword_value);
```

NOTE

範例中寫入 0x3f5d16d2，用 read_word 讀取位址 0x0001 位址，得到 16d2 資料，讀取位址 0x0003 位址，得到 3f5d 資料。

42.29　_DMC_01_get_watchdog_cnt

■　**格式**

I16 PASCAL _DMC_01_get_watchdog_cnt (U16 CardNo, U32 *watchdog_cnt)

■　**目的**

讀取軸卡 WatchDog 功能目前的累積計數值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
watchdog_cnt	U32*	數值單位	WatchDog 目前的累積計數值(單位:ms)*1

*1. 當 watchdog_value 非 0 時，watchdog_cnt 每 1ms 會進行加 1。

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_cnt;

I16 status = _DMC_01_get_watchdog_cnt (CardNo, & watchdog_cnt);
```

42.30 _DMC_01_set_watchdog_cnt

■ **格式**

I16 PASCAL _DMC_01_set_watchdog_cnt (U16 CardNo, U32 watchdog_cnt)

■ **目的**

修改軸卡 WatchDog 功能目前的累積計數值。

■ **參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
watchdog_cnt	U32	數值單位	欲設置的 watchdog_cnt 計數值(單位:ms)

■ **屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■ **範例**

```
U16 CardNo=0;
U32 watchdog_cnt;

I16 status = _DMC_01_set_watchdog_cnt (CardNo, watchdog_cnt);
```

42.31　_DMC_01_get_watchdog_value

■　**格式**

I16 PASCAL _DMC_01_get_watchdog_value(U16 CardNo, U32 *watchdog_value)

■　**目的**

讀取軸卡 WatchDog 功能的計數目標值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
watchdog_value	U32*	數值單位	WatchDog 目前的計數目標值(單位:ms)*1

*1. watchdog_value 為 0 時，watchdog_cnt 不會進行累積計數。

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_value ;

I16 status = _DMC_01_get_watchdog_value (CardNo, &watchdog_value);
```

42.32　_DMC_01_set_watchdog_value

■　**格式**

I16 PASCAL _DMC_01_set_watchdog_value(U16 CardNo, U32 watchdog_value)

■　**目的**

設定軸卡 WatchDog 功能的計數目標值。

■　**參數**

名稱	資料型別	單位	描述
CardNo	U16	編號單位	介面卡號 CardNo 為 0~15
watchdog_value	U32	數值單位	WatchDog 的計數目標值(單位:ms)*1

*1. watchdog_value 為 0 時，watchdog_cnt 不會進行累積計數。

■　**屬性**

主機板	伺服	04PI Md1	04PI Md2(GE01PH)	MN	NT	DA&AD	RM64MN
●							

■　**範例**

```
U16 CardNo=0;
U32 watchdog_value ;

I16 status = _DMC_01_set_watchdog_value (CardNo, watchdog_value);
```

NOTE

當 watchdog_value >0 且 watchdog_cnt = watchdog_value 時軸卡會觸發異警.

此時 master_alm_code 取回值為 0X187.　動作邏輯如下:

